

Joaquín Ataz-López
(等)

ConTeXt LMTX 简明教程 (社区版)

ConTeXt LMTX 简明教程(社区版)

版本 1.1 [2026 年 03 月 09 日]

「ConTeXt LMTX 简明教程」© 2020–2021, 2026, Joaquín Ataz-López 等贡献者采用 CC BY-NC-SA 4.0 许可证。如需查看该许可证的副本,请访问 <https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本作品可从 <https://codeberg.org/jd/anss-ce> 下载和使用,须遵守上述许可证条件。

版本历史:

- 2026 年 03 月 06 日: 版本 1.0。初版,基于 Joaquín Ataz-López 所著《ConTeXt Mark IV 简明教程》1.6 版本的英文翻译(译者匿名)。
- 2026 年 04 月 21 日: 版本 1.1。重大内容更新,反映 ConTeXt LMTX 的特性。移除了部分 MkII 和 MkIV 信息,新增了部分内容,修正了一些排版错误,更新了一些翻译等。

以下是原版版权页
ConTeXt Mark IV 简明教程

ConTeXt Mark IV 简明教程
版本 1.6 [2021 年 01 月 02 日]

© 2020–2021, Joaquín Ataz-López

原标题: Una introducción (no demasiado breve) a ConT_EXt Mark IV
英文翻译: 一位希望保持匿名的朋友。

本文档的作者(及其英文翻译者)授权其自由分发和使用,包括在以下条件下以数字格式复制和重新分发本文档的权利:须注明其作者身份,且不得将其包含在任何软件包或套件中,或包含在使用或分发条件不包括接收者自由复制和重新分发权利的文档中。本文档的翻译同样获得授权,但须注明原始文本的作者身份,且翻译文本须在 *Free Software Foundation* 的 FDL 许可证、*Creative Commons* 授权复制和重新分发的许可证或类似许可证下分发。

上述规定并不妨碍以书面形式获得作者明确授权后方可出版、营销或翻译本文档纸质版。

版本历史:

- 2020 年 08 月 18 日: 版本 1.0(仅西班牙语)原始文档。
- 2020 年 08 月 23 日: 版本 1.1(仅西班牙语):作者修正了 minor 错误 拼写误解。
- 2020 年 09 月 03 日: 版本 1.15(仅西班牙语):更多错误 拼写误解。
- 2020 年 09 月 05 日: 版本 1.16(仅西班牙语):更多错误 拼写误解,以及一些非常细微的更改,使文本更加清晰(我希望如此)。
- 2020 年 09 月 06 日: 版本 1.17(仅西班牙语):我发现的 minor 错误数量令人难以置信。我只需要停止重新阅读文档,就找不到更多错误了!
- 2020 年 10 月 21 日: 版本 1.5(仅西班牙语)纳入了建议并修正了 NTG-context 用户报告的错误。
- 2021 年 01 月 02 日: 版本 1.6:在仔细重新阅读文档后提出的修正建议,恰逢其被翻译成英文。这是第一个英文版本。

目录

前言	vi
贡献者列表	viii
《一本不算太短的 ConTeXt Mark IV 入门》前言	ix
What is ConTeXt and how do we work with it	1
第 1 章 ConTeXt: 概览	2
第 1 节 那么,什么是 ConTeXt?	2
第 2 节 排版文本	3
第 3 节 标记语言	4
第 4 节 T _E X 及其衍生品	5
第 5 节 ConTeXt	7
第 2 章 我们的第一个源文件	13
第 1 节 准备实验:必备工具	13
第 2 节 实验本身	15
第 3 节 我们示例文件的结构	19
第 4 节 关于如何运行「context」的一些额外细节	19
第 5 节 错误管理	20
第 3 章 ConTeXt 的命令与其他基本概念	23
第 1 节 ConTeXt 的保留字符	23
第 2 节 命令本身	26
第 3 节 命令的作用范围	28
第 4 节 命令的操作选项	31
第 5 节 命令语法和选项概要,以及调用时方括号和花括号的使用	33
第 6 节 ConTeXt 命令的官方列表	34
第 7 节 定义新命令	35
第 8 节 其他基本概念	38
第 9 节 ConTeXt 的自学方法	42
第 4 章 源文件与项目	44
第 1 节 源文件的编码	44

第 2 节 源文件中 ConTeXt 以特殊方式处理的字符	45
第 3 节 简单项目与多文件项目	49
第 4 节 简单项目中源文件的结构	50
第 5 节 T _E X 风格的多文件管理	51
第 6 节 ConTeXt 项目本身	53
Global aspects of the document	58
第 5 章 页面与文档分页	59
第 1 节 页面尺寸	59
第 2 节 页面上的元素	63
第 3 节 页面布局(<code>\setuplayout</code>)	65
第 4 节 页码编号	69
第 5 节 强制或建议的分页	71
第 6 节 页眉和页脚	72
第 7 节 在页面边缘和边距中插入文本元素	75
第 6 章 ConTeXt 中的字体与颜色	78
第 1 节 「ConTeXt Standalone」中包含的排版字体	78
第 2 节 字体特性	79
第 3 节 设置文档的主字体	81
第 4 节 更改字体或某些字体特性	83
第 5 节 与使用某些替代形式相关的其他事项	88
第 6 节 颜色的使用与配置	90
第 7 章 文档结构	95
第 1 节 文档中的结构划分	95
第 2 节 节的类型及其层级	96
第 3 节 节命令的通用语法	97
第 4 节 节及其标题的格式与配置	99
第 5 节 定义新的节命令	106
第 6 节 文档的宏观结构	107
第 8 章 目录与索引	109
第 1 节 目录	109
第 2 节 列表、组合列表以及基于列表的目录	118
第 3 节 索引	121
第 9 章 引用与超链接	126
第 1 节 引用类型	126
第 2 节 内部引用	127
第 3 节 交互式电子文档	132
第 4 节 指向外部文档的超链接	134
第 5 节 在最终 PDF 中创建书签	138

Particular issues	140
第 10 章 字符、单词、文本与水平间距	141
第 1 节 获取键盘上通常无法直接输入的字符	141
第 2 节 特殊字符格式	148
第 3 节 字符间距与词间距	152
第 4 节 复合词	155
第 5 节 文本的语言	156
第 11 章 段落、行与垂直间距	163
第 1 节 段落及其特性	163
第 2 节 段落之间的垂直间距	165
第 3 节 ConTeXt 如何构建构成段落的行	168
第 4 节 行间距	173
第 5 节 与行相关的其他事项	174
第 6 节 水平与垂直对齐	176
第 12 章 特殊构造与段落	179
第 1 节 脚注与尾注	179
第 2 节 多列段落	185
第 3 节 结构化列表	190
第 4 节 描述与枚举	196
第 5 节 线条与框架	199
第 6 节 其他感兴趣的环境和构造	202
第 13 章 图像、表格及其他浮动对象	205
第 1 节 什么是浮动对象,它们有什么作用?	205
第 2 节 外部图像	206
第 3 节 表格	213
第 4 节 图像、表格及其他浮动对象的共同方面	218
第 5 节 定义额外的浮动对象	223
Appendices	226
第 A 章 安装、配置和更新 ConTeXt	227
第 1 节 安装和配置「ConTeXt Standalone」	227
第 2 节 安装 LMTX	228
第 3 节 在同一系统上使用多个版本的 ConTeXt(仅限 Unix 类型系统)	233
第 B 章 生成数学和非数学符号的命令	234
第 C 章 命令索引	238

前言

一本不算太短的 *ConT_EXt Mark IV* 入门 (有时也被称为 ANSS, 至少在英语 ConT_EXt 用户中如此) 由 Joaquín Ataz-López 所著, 是备受 ConT_EXt 用户推崇的参考文档, 对新用户尤其有价值, 无论他们之前是否有过 plain T_EX、L^AT_EX 的经验, 还是对 T_EX 家族完全陌生。该书已被从西班牙语原版翻译成其他五种语言, 足见其广泛吸引力。

时代在前进, ConT_EXt 也在发展。我敢大胆推测, 到 2026 年, 大多数 ConT_EXt 用户都在使用 ConT_EXt LMTX。尽管 Mark IV 和 LMTX 版本的 ConT_EXt 非常相似, 但 ANSS 中的一些信息已不能反映 ConT_EXt LMTX 的当前行为。此外, LMTX 中还有值得提及的新功能。

Joaquín 慷慨地支持为其著作创建新「社区版」的想法。同时, 对标题和署名的 (微小) 调整是合适的, 以反映目标 ConT_EXt 版本以及更广泛的作者群体 (尽管 Joaquín 的工作始终是这本新书的基础)。

本书的 ConT_EXt 源代码当前位于 <https://codeberg.org/jd/anss-ce>^{*}。欢迎所有 ConT_EXt 用户贡献。对于熟悉 git 的用户, 欢迎提交拉取请求。在 Codeberg 网站上开启一个「议题」是另一种贡献方式。第三种选择是将补丁通过电子邮件发送给其中一位维护者。最后, 对本书的重大更改 (例如关于轻微或重大重组的想法) 最好通过 ConT_EXt 邮件列表处理。

现有书籍的改编值得一篇新的前言。我决定这样做, 以推动项目前进。然而, 我必须明确无误地强调, 我在贡献者中并无特殊地位; 我不是 ConT_EXt 专家, 也没有撰写如此规模技术文档的经验。我希望 ConT_EXt 社区的许多成员能根据自己的技能和空余时间做出贡献, 为所有 ConT_EXt 用户提供一份有用且最新的文档。

ANSS 的前言被收录到本书中, 原因有几个, 这里我只提四个。首先, 它解释了 ANSS 开发背后的原理; 这些原理依然存在, 并部分解释了为什么这本「ANSS-CE」书被认为有价值。其次, 它解释了左边距中看到的两幅图像的含义, 读者肯定想知道这些信息。第三, 那篇前言包含了一些关于 T_EX 和 ConT_EXt 的历史背景, 我相信值得花几分钟阅读。第四, 解释了 ANSS (以及 ANSS-CE, 除非未来某个时候进行重大更改) 的整体结构, 这有助于读者更好地理解本书试图做什么, 以及它试图如何做。

最后, 除本前言外, 所有使用“我”的地方均指 Joaquín Ataz-López。随着本书的发展, “我”的数量可能会减少, 但本书的起源将始终与 ANSS 同在。



Jim Diamond
2026 年 3 月

贡献者列表

本节列出已知为本书做出贡献(无论大小)的人员;无疑遗漏了许多人。如果您的名字应在此列,请按照前言中的说明报告遗漏。(如果您的名字在此,而您谦虚地希望将其改为「匿名 #<n>」,请告知维护者。)

Joaquín Ataz-López(ANSS 作者)

匿名 #1(西班牙语原版书的英文译者)

Aditya Mahajan

Florent Michel

Jim Diamond

Hans Hagen

Henning Hraban Ramm

Max Chernoff

Pablo Rodriguez

Wolfgang Schuster

《一本不算太短的 ConT_EXt Mark IV 入门》前言^{*}

亲爱的读者,这是一份关于 ConT_EXt 的文档,ConT_EXt 是一个源自 T_EX 的排版系统,而 T_EX 又是另一个由 DONALD E. KNUTH 于 1977 年至 1982 年间在斯坦福大学创建的排版系统。

ConT_EXt 旨在创建具有极高排版质量的文档——无论是纸质文档还是设计用于计算设备屏幕上显示的文档。它不是一个文字处理器或文本编辑器,而是如前所述,一个系统,或者换句话说,一套旨在排版文档的工具,排版被理解为文档不同元素在页面或屏幕上的图形布局和可视化。总之,ConT_EXt 旨在提供所有必要的工具,以使文档具有最佳的外观。其理念是能够生成不仅写得好,而且「优美」的文档。在这方面,我们可以引用 DONALD E. KNUTH 在介绍 T_EX (ConT_EXt 所基于的系统)时所写的话:

如果你只是想制作一份勉强过得去的文档——可以接受、基本可读但并不真正优美——一个更简单的系统通常就足够了。使用 T_EX 的目标是制作出最优的质量;这需要更注意细节,但你不会觉得多花点功夫有多难,而且你会对成品感到特别自豪。

当我们用 ConT_EXt 准备稿件时,我们会精确指示如何将其转换为页面(或屏幕),其排版质量可与世界上最好的印刷厂相媲美。要做到这一点,一旦我们学会了这个系统,我们所需的工作量只比在任何文字处理器或文本编辑器中正常录入文档稍微多一点。事实上,一旦我们在处理 ConT_EXt 上获得了一定的熟练度,考虑到文档的主要格式细节是在 ConT_EXt 中全局描述的,并且我们处理的是文本文件——一旦习惯了,这是一种更自然的创建和编辑文档的方式,我们的总工作量可能会更少;此外,这类文件比文字处理器那种庞大的二进制文件要轻得多,也更容易处理。

关于 ConT_EXt 有相当多的文档,几乎全是英文。我们可能认为是 ConT_EXt 官方发行版的东西——称为「ConT_EXt Standalone」²——例如,包含大约 180 个 PDF 文

^{*} 本前言起初打算翻译/改编自「《T_EX 书》」的前言,该文档解释了关于 T_EX 你需要知道的一切。最终,我不得不偏离这一初衷;然而,我保留了一些片段,希望对于那些了解它的人,能唤起一些回响。

² 在本文第一版起草时,那里的陈述是符合事实的;但在 2020 年春天,ConT_EXt wiki 进行了更新,从那时起,我们必须认为 ConT_EXt 的「官方」发行版已经变成了 LMTX。然而,对于第一次接触 ConT_EXt 世界的人来说,我仍然建议使用「ConT_EXt Standalone」,因为它是一个更稳定的发行版。[附录 A](#) 解释了如何安装任一发行版。

档(大部分是英文,但也有一些荷兰语和德语),包括手册、示例和技术文章;在 Pragma ADE 网站(孕育了 ConTeXt 的公司)上,(在我 2020 年 5 月统计的那天)有 224 个可下载的文档,其中大部分随「ConTeXt Standalone」一起分发,但也有一些是其他的。尽管如此,这些庞大的文档对于学习 ConTeXt 并不是特别有用,因为通常这些文档的目标读者并不是对系统一无所知但想学习它的人。在「ConTeXt Standalone」称为「手册」的 56 个 PDF 文件中,只有一个假设读者对 ConTeXt 一无所知。这份文档名为「ConTeXt Mark IV, 一瞥」。然而,这份文档,正如其名称所示,仅限于介绍系统并解释如何使用 ConTeXt 做某些事情。如果后续有一本结构更清晰、更系统的参考手册,它会是一个很好的入门介绍。这本参考手册并不存在,而且与 ConTeXt「一瞥」相关的文档与其他文档之间的差距太大了。

2001 年,有一本参考手册被编写出来,可以在 [Pragma ADE 网站](#)上找到;但尽管有这个名字,一方面它并非旨在成为一本完整的手册,另一方面它针对的是 ConTeXt 的旧版本(称为 Mark II),因此已经相当过时。

2013 年,该手册被部分更新,但许多章节没有被重写,它包含的信息同时涉及 ConTeXt Mark II 和 ConTeXt Mark IV(当前版本),但并非总能完全清楚哪些信息指的是哪个版本。也许这就是为什么这本手册没有出现在「ConTeXt Standalone」包含的文档中。然而,尽管存在这些缺陷,在我们阅读了介绍性的「ConTeXt Mark IV, 一瞥」之后,该手册仍然是开始学习 ConTeXt 的最佳文档。对于 ConTeXt 入门同样非常有用的是其 [wiki](#) 上的信息,在撰写本文时,该 wiki 正在重新设计,结构更加清晰,尽管它也将仅适用于 Mark II 的解释与其他适用于 Mark IV 或两个版本的解释混合在一起。这种缺乏区分的情况也出现在 ConTeXt 命令的官方列表中¹,该列表没有指明哪些命令仅适用于两个版本之一。

基本上,本入门书是根据这里列出的四个信息来源编写的:ConTeXt「一瞥」、2013 年手册、wiki 的内容以及命令的官方列表(其中包含每个命令允许的配置选项);此外,当然还有我自己的测试和结论。所以,事实上这本入门书是研究工作的结果,有一段时间我倾向于将其命名为「我所知道的 ConTeXt Mark IV」或「我学到的关于 ConTeXt Mark IV 的知识」。最终,我放弃了这些标题,因为尽管它们可能很真实,但我觉得它们可能会阻止某些人进入 ConTeXt;可以肯定的是,尽管文档(在我看来)存在一些缺陷,但这确实是一个有用且多功能的工具,学习它所付出的努力无疑是值得的。通过使用 ConTeXt,我们可以操作和配置文本文档,以实现不了解该系统的人根本无法想象的事情。

由于我的身份,我无法避免在整个文档中不时出现我对信息缺乏的抱怨。我不希望这被误解:我非常感激 ConTeXt 的创造者们设计了如此强大的工具并将其提供给公众。只是我忍不住认为,如果它的文档得到改进,这个工具会受欢迎得多:人们需要投入大量时间来学习它,这与其内在难度(它确实有难度,但并不比其他类似工具更大——事实上反而更小)关系不大,而是由于缺乏清晰、完整和组织良好的信息来区分 ConTeXt 的两个版本,解释每个版本中的功能,最重要的是,阐明每个命令、参数和选项的作用。

诚然,这类信息需要大量的时间投入。但由于许多命令共享名称相似的选项,或许可以提供一种选项词汇表,这也有助于发现一些不一致之处,例如两个名称相同的选项做不同的事情,或者为了做同一件事,在不同的命令中使用不同名称的选项。

¹ 列表见 [节 6](#)

至于第一次接触 ConT_EXt 的读者,请不要让我的抱怨打消您的积极性,因为尽管信息不足确实会增加学习所需的时间,但至少对于本入门书所涉及的内容,我已经投入了这些时间,读者无需再投入。仅凭从本入门书中学到的知识,读者将拥有一个工具,使他们能够以前所未有的轻松方式制作文档。

由于本文档中解释的内容很大程度上来自我自己的结论,因此很可能,尽管我已经亲自测试了所解释的大部分内容,但某些陈述或观点可能既不正确也不够正统。当然,我会感激读者能提供的任何修正、细微差别或澄清,这些可以发送到 joaquin@ataz.org。然而,为了减少我可能出错的情况,我尽量不涉及那些我没有找到任何信息且无法(或不想)亲自尝试的问题。有时这是因为我的测试结果不具结论性,有时则是因为我并非总是测试所有内容:ConT_EXt 拥有的命令和选项数量惊人,如果我必须测试所有内容,我将永远无法完成这本入门书。然而,有些情况下我无法避免假设某事,即做出我认为可能但并非完全确定的陈述。在这些情况下,段落左边距中会放置一个『推测』图像。图像 旨在图形化地表示假设。¹ 在其他时候,我别无选择,只能承认我不知道某事,甚至对其没有合理的假设:在这种情况下,左边距中可见的图像 旨在表示的不仅仅是推测或无知。² 但由于我向来不擅长图形表示,我不确定我选择的图像是否真的能传达这么多细微差别。



另一方面,这本入门书是从一个对 T_EX 和 ConT_EXt 都一无所知的读者的角度编写的,尽管我希望它也能对那些从 T_EX 或 L^AT_EX (最流行的 T_EX 衍生品) 转来、第一次接触 ConT_EXt 的人有所帮助。尽管如此,我意识到试图取悦如此多不同类型的读者,我有可能无法让任何人满意。因此,在不确定的情况下,我始终明确,本文档的主要受众是 ConT_EXt 的新手,刚刚进入这个迷人生态系统的新手。

成为 ConT_EXt 的新手并不意味着也是计算机工具使用的新手;尽管在本入门书中我不假设读者具有任何特定水平的计算机素养,但我确实假定了一定的「合理素养」,这意味着,例如,对文字处理器和文本编辑器之间的区别有大致了解,知道如何创建、打开和操作文本文件,知道如何安装程序,知道如何打开终端并执行命令.....仅此而已。

在我撰写这些文字时,重读本入门书的前面部分,我意识到有时我会跑题,深入到学习 ConT_EXt 所不需要的计算机问题中,这可能会吓跑新手,而其他时候我又忙于解释一些相当明显的事情,这可能会让有经验的读者感到无聊。我恳求两者的宽容。理性上,我知道一个完全初涉计算机化文本管理的人甚至几乎不知道 ConT_EXt 的存在,但从另一个角度来看,在我的专业环境中,我周围都是那些在使用文字处理器时不断与文本作斗争的人,他们做得相当好,但从未处理过文本文件本身,因此他们忽略了诸如文本文件使用什么编码或文字处理器和文本编辑器之间的区别等基本问题。

本手册是为对 ConT_EXt 或 T_EX 一无所知的人设计的,这意味着我包含了明显不是关于 ConT_EXt 而是关于 T_EX 的信息;但我认为没有必要用对读者不相关的信息来加重他们的负担,例如某个命令实际上是有效的,但它到底是 ConT_EXt 命令还是属于 T_EX;因此,只有在某些时候,当我认为有用时,我才会澄清某些命令实际上属于 T_EX。

关于本文档的组织结构,材料分为三个部分:

¹ 我不是画的这幅图,而是从网上下载的(<https://es.dreamstime.com/>),上面说它是免版税图像。

² 也是在互联网上找到的(<https://www.freepik.es/>),那里授权免费使用。

- **第一部分**, 包括前四章, 提供了 ConT_EXt 的全局概述, 解释它是什么以及我们如何使用它, 展示了如何转换文档的第一个示例, 以便随后解释 ConT_EXt 的一些基本概念以及与 ConT_EXt 源文件相关的某些问题。

总的来说, 这些章节面向的是迄今为止只知道如何使用文字处理器的读者。已经了解标记语言的读者可以跳过前面的这些章节; 如果读者已经了解 T_EX 或 L^AT_EX, 他们也可以跳过第 3 章和第 4 章的大部分内容。尽管如此, 我建议至少阅读:

- ◊ 关于 ConT_EXt 命令的信息(第 3 章), 特别是它的功能、配置方式, 因为这是 L^AT_EX 和 ConT_EXt 在构思和语法上的主要区别所在。由于本入门书仅涉及后者, 这些区别并未明确指明, 但是阅读本章的读者如果了解 L^AT_EX 的工作原理, 将立即理解这两种语言的语法差异, 以及 ConT_EXt 允许我们配置和定制几乎所有命令工作方式的方法。
- ◊ 关于多文件 ConT_EXt 项目的信息(第 4 章), 这与使用其他基于 T_EX 系统的工作方式不太相似。
- **第二部分**, 包括第 5 章到第 9 章, 重点关注我们认为的 ConT_EXt 文档的主要全局方面:
 - ◊ 主要影响文档外观的两个方面是页面尺寸和布局以及所用字体。第 5 章和第 6 章专门讨论这些问题。
 - * 第一章聚焦于页面: 尺寸、构成页面的元素、布局(即页面元素如何分布)等。出于系统性原因, 与分页和允许我们影响分页的机制相关的更具体方面也在此讨论。
 - * 第 6 章解释与字体及其处理相关的命令。这里还包括对颜色使用和管理的基本解释, 因为尽管颜色严格来说不是字体的特征, 但它同样影响文档的外部外观。
 - ◊ 第 7 章和第 8 章聚焦于文档的结构以及 ConT_EXt 为作者提供的编写结构良好的文档的工具。第 7 章侧重于严格意义上的结构(文档的结构划分), 第 8 章侧重于这如何反映在目录中; 不过, 在解释这一点时, 我们借此机会也解释了如何使用 ConT_EXt 生成各种索引, 因为对于 ConT_EXt 来说, 这些都归在「列表」的概念下。
 - ◊ 最后, 第 9 章聚焦于引用, 这是任何文档在需要引用文档其他部分(内部引用)或其他文档(外部引用)时的一个重要全局方面。对于后者, 我们目前只对指向外部文档的引用(链接)感兴趣。这些链接(也可能出现在内部引用中)使我们的文档具有交互性, 在本章中我们将解释 ConT_EXt 用于创建此类文档的一些特性。

这些章节不需要按特定顺序阅读, 除了第 8 章, 如果先阅读第 7 章会更容易理解。无论如何, 我已尽力确保当某个章节或小节中出现的问题在本文档的其他

地方有讨论时,文本会包含提及并附上指向该讨论点的超链接。然而,我无法保证总是如此。

- 最后是**第三部分**(第 10 章及以后),侧重于更详细的方面。它们不仅彼此独立,甚至其小节也是独立的(除了最后一章可能例外)。鉴于 ConT_EXt 包含的大量实用工具,这部分可能会非常广泛;但由于我的理解是,当读者到达这里时,他们已经准备好主动深入研究 ConT_EXt 的文档,所以我只包含了以下几章:
 - ◊ 第 10 章和第 11 章讨论我们可以称之为任何文本文档的核心元素:文本由字符组成,字符组成单词,单词排列成行,行又组成段落,段落之间由垂直间距分隔.....显然,所有这些问题本可以放在一章中,但由于那样会太长,我将这个问题分成两章,一章讨论字符、单词和水平间距,另一章讨论行、段落和垂直间距。
 - ◊ 第 12 章是一个大杂烩,处理文档中常见的元素和结构,主要是学术、科学或技术文档:脚注、结构化列表、描述、编号等。
 - ◊ 最后,第 13 章聚焦于浮动对象,尤其是其中最具代表性的:插入文档的图像和表格。
- 入门书以三个**附录**结束。一个是关于安装 ConT_EXt,第二个附录包含几十个允许生成各种符号的命令——主要但不仅限于数学用途,第三个附录包含本文档中解释或提及的 ConT_EXt 命令的字母顺序列表。

还有许多问题有待解释:处理引文和参考文献、撰写专业文本(数学、化学.....)、与 XML 的连接、与 Lua 代码的接口、模式和基于模式的处理、使用 MetaPost 设计图形等等。这就是为什么,由于我没有包含对 ConT_EXt 的完整解释,也不打算这样做,我将本文档称为「ConT_EXt Mark IV 入门」;并且我补充说这个入门书不算太短,因为显然情况如此:一份留下这么多未完成事项但已超过 300 页的文本绝不是简短的入门。这是因为我希望读者理解 ConT_EXt 的逻辑,或者至少是我所理解的逻辑。它并不声称是一本参考手册,而是一本自学指南,让读者准备好制作中等复杂度的文档(这包括大多数可能遇到的文档),并且最重要的是教会读者想象可以用这个强大的工具做什么,并在现有文档中找到如何做的方法。本文档也不是教程。教程旨在逐步增加难度,以便逐步教授要学的内容;在这方面,我选择从第二部分开始,而不是按照难度顺序排列材料,以便更系统。但尽管它不是教程,我还是包含了非常多的示例。

对于某些读者来说,本文档的标题可能会让他们想起由 OETIKER, PARTL, HYNAL 和 SCHLEGL 编写的一份互联网上的文本,那是介绍 L^AT_EX 世界的最佳文档之一。我指的是「一本不算太短的 L^AT_EX 2_ε 入门」。这并非巧合,而是一种致敬和感激:感谢那些撰写像那样的文本的人的慷慨工作,使得许多人能够开始使用像 L^AT_EX 和 ConT_EXt 这样有用且强大的工具。这些作者帮助我开始了 L^AT_EX;我希望为某个想开始使用 ConT_EXt 的人做同样的事情,尽管在本文档的西班牙语原版中,我只针对

缺乏自己语言文档的西班牙语世界。我希望这份文档能满足这一期望,与此同时,其他人已慷慨地提出将其翻译成其他语言,于是有了这个英文版。谢谢。

Joaquín Ataz-López
2020 年夏季

What is ConT_EXt and how do
we work
with it

第1章 ConT_EXt: 概览

Table of Contents: 第1节 那么,什么是 ConT_EXt? ; 第2节 排版文本; 第3节 标记语言; 第4节 T_EX 及其衍生品; 4.1 T_EX 引擎; 4.2 从 T_EX 衍生出的格式; 第5节 ConT_EXt; 5.1 ConT_EXt 简史; 5.2 ConT_EXt 与 L^AT_EX 的比较; 5.3 理解使用 ConT_EXt 的工作流程; 5.4 获取 ConT_EXt 的帮助;

第1节 那么,什么是 ConT_EXt?

ConT_EXt 是一个排版系统,或者换句话说:一套广泛的工具集,旨在让用户能够绝对且完全地控制特定电子文档(用于纸上打印或屏幕显示)的外观和呈现。本章将解释这意味着什么。但首先,让我们强调一下 ConT_EXt 的一些特点。

- ConT_EXt 有三种风味:Mark II、Mark IV 和 LMTX。ConT_EXt Mark II 已冻结;即,它不再处于开发中(并且已经很多年了),并且不太可能有进一步的更改或添加新功能。只有在需要更正某些错误时才会出现新版本。ConT_EXt Mark IV 更新得多,但现在也或多或少处于冻结状态。而 ConT_EXt LMTX 则仍在开发中,并且不时会出现引入一些改进或额外实用程序的新版本。尽管 LMTX 仍在开发中,但它已经相当成熟,新版本引入的更改不太可能改变现有功能的工作方式;一些更改提供了新功能,而其他更改则非常微妙,仅影响系统的底层操作。对于普通用户来说,这些更改通常是完全透明的;就好像它们没有发生一样。尽管所有三种风味有很多共同点,但它们之间也存在一些不兼容的特性。因此,本入门只关注 ConT_EXt LMTX,从现在开始,不带「Mark」的「ConT_EXt」指的就是 ConT_EXt LMTX。
- ConT_EXt 是自由软件(或免费软件,但不仅仅是免费的意思)。程序本身(即构成 ConT_EXt 的计算机工具集)在 GNU 通用公共许可证下分发。文档则在允许自由复制和分发的「知识共享」许可证下提供。
- ConT_EXt 既不是文字处理器程序,也不是文本编辑程序,而是一套旨在转换我们之前用喜欢的文本编辑器写好的文本的工具。因此,当我们使用 ConT_EXt 工作时:
 - ◊ 我们首先使用任何「纯文本」文本编辑器编写一个或多个文本文件。具体来说,这不包括文字处理器。
 - ◊ 在这些文件中,除了构成文档内容的文本之外,还有一系列指令,告诉 ConT_EXt 从原始文本文件生成的最终文档必须具有的外观。完整的 ConT_EXt

指令集实际上是一种语言;由于这种语言允许人们编程实现文本的排版转换,我们可以说 ConT_EXt 是一种排版编程语言。

- 一旦我们编写了源文件,它们将由一个程序(也称为「context」¹)处理,该程序将根据它们生成一个 PDF 文件,准备好发送到打印机、印刷厂或在屏幕上查看。
- 因此,在 ConT_EXt 中,我们必须区分我们正在编写的文档和 ConT_EXt 生成的文档。为避免任何疑问,在本入门中,我将包含格式化指令的文本文档称为源文件,而将 ConT_EXt 根据源文件生成的 PDF 文档称为最终文档。

以上基本要点将在下面进一步展开。

第2节 排版文本

编写文档(书籍、文章、章节、传单、打印输出、论文...)和对其进行排版是两种截然不同的活动。编写文档在很大程度上等同于起草;这由决定其内容和结构的作者完成。作者直接创建的文档,正如他或她所写的那样,被称为手稿。就其本质而言,只有作者或被允许阅读的人才能接触到手稿。要超越这个私密群体进行传播,手稿需要被出版。今天,出版某物——在其「向公众开放」的词源学意义上——就像把它放在互联网上一样简单,任何找到它并想阅读的人都可以访问。但直到相对最近,出版还是一个成本高昂的过程,依赖于某些专业化的专业人士,只有那些因其内容或作者而被认为特别有趣的手稿才能进入这个过程。即使在今天,我们也倾向于将出版一词保留用于这种专业出版,其中手稿的外观经历一系列旨在提高文档可读性的转换。这一系列转换就是我们所说的排版。

排版的目标通常是——撇开旨在吸引读者注意力的广告类文本不谈——制作具有最大可读性的文档,即印刷文本的质量能够吸引或促进阅读,并确保读者感到舒适。许多因素促成了这一点;有些当然与文档的内容有关(质量、清晰度、组织...),但其他的则取决于所用字体的类型和尺寸、文档中空白的使用、段落之间的视觉分隔等。此外,还有其他类型的资源,不完全是图形或视觉类的,例如文档中是否存在对读者的特定辅助,如页眉和页脚、索引、词汇表、粗体的使用、边注等。了解并正确处理排版者可用的所有资源的知识可以称为「排版艺术」或「印刷艺术」。

历史上,直到计算机出现之前,作者和排版员的职责是相当分开的。作者用手写或称为打字机的 19 世纪机器写作,后者的排版资源甚至比手写还要有限;然后作者将原稿交给出版商或印刷商,他们将其转换以获得印刷文档。

今天,计算机科学使作者更容易决定排版的每一个细节。然而,这并不能消除这样一个事实:一个好作者所需的品质与一个好排版员所需的品质并不相同。根据所

¹ ConT_EXt 同时是一种语言和一个程序(除此之外还有其他东西)。这一事实,在像当前这样的文本中,带来了一个问题:我们有时必须区分这两个方面。这就是为什么我采用了一个排版约定:当我想专门指代语言,或者同时指代语言和程序时,我使用带有其标志的写法「ConT_EXt」(ConT_EXt)。然而,当我想专门指代程序时,我将全部使用小写字母并用等宽字体(典型于计算机终端和打字机)写成「context」。对于属于这种语言的命令和示例,我也会使用这种字体。

处理文档的类型,作者需要理解所写主题、清晰的阐述、结构良好的思维以创建组织有序的文本、创造力、节奏感等。但排版员必须结合对所掌握的概念和图形资源的良好了解,以及足够好的品味来和谐地使用它们。

使用一个好的文字处理程序¹有可能获得一个排版上相当不错的文档。但一般来说,文字处理器并非为排版而设计,其结果虽然可能正确,但无法与专门为控制文档排版而设计的其他工具所获得的结果相媲美。事实上,文字处理器是打字机的演变方式,并且由于这些工具掩盖了文本创作与其排版之间的区别,它们的使用往往会产生结构不良且排版不充分的文本。相反,像 ConT_EXt 这样的工具是从印刷机演变而来的;它们提供了更多的排版可能性,最重要的是,不可能在学习使用它们的同时不沿途获得许多与排版相关的概念。这就是与文字处理器的区别,有些人可以使用文字处理器多年却不学一点排版知识。

第3节 标记语言

在计算机出现之前,如前所述,作者用手写或打字机准备手稿,然后交给出版商或印刷商,后者负责将手稿转换为最终的印刷文本。尽管作者在转换过程中的参与相对较少,但他或她确实通过一些方式进行干预,例如,指出手稿的某些行是其不同部分(章节、小节...)的标题,或者指示某些内容应以某种方式在排版上加以强调。这些指示由作者直接在手稿上做出,有时是明确地,有时是通过随时间不断发展的某些惯例。例如,章节总是在新页开始,通过在标题前插入几个空行、在其下划线、用大写字母书写,或者将要强调的文本放在两个下划线之间、增加段落的缩进等。

简而言之,作者标记文本以提供有关应如何排版的指示。然后,编辑会在文本上手写其他指示给印刷商,例如,要使用的字体及其大小。

今天,在计算机化的世界中,我们可以通过所谓的标记语言继续这样做以生成电子文档。这类语言使用一系列标记或指示,处理包含它们的文件的程序知道如何解释这些标记。如今最著名的标记语言可能是 HTML,因为今天大多数网页都基于它。一个 HTML 页面包含网页的文本,以及一系列告诉加载页面的浏览器程序应如何显示它的标记。Web 浏览器理解的 HTML 标记,连同关于如何以及在何处使用它们的指令,被称为「HTML 语言」,这是一种标记语言。但除了 HTML,还有许多其他标记语言;事实上它们正在蓬勃发展,因此 XML,作为首屈一指的标记语言,如今随处可见,并且几乎用于所有事情:用于数据库设计、创建特定语言、传输结构化数据、应用程序配置文件等。还有用于图形设计(SVG、TikZ 或 MetaPost)、数学公式(MathML)、音乐(Lilypond 和 MusicXML)、金融、地理信息学等的标记语言。当然,也有旨在进行文本排版转换的标记语言,其中 T_EX 及其衍生品最为突出。

关于指示文本外观的排版标记,有两种类型可以参考:纯排版标记和概念标记,或者如果你愿意,也可以说是逻辑标记。纯排版标记仅限于精确指示应使用哪种排版资源来显示特定文本;例如,当我们指示某些文本应为粗体或斜体时。另一方面,

¹ 根据一个相当古老的惯例,我们区分文本编辑器和文字处理器。早期的文本编辑程序处理未格式化的文本文件,而另一种则处理格式化文本的二进制文件。

概念标记指示文档中某段文本在整体中 fulfills 什么功能,例如当我们指示某物是标题、副标题或引用时。通常,倾向于使用第二种标记的文档更加一致且更容易排版,因为它们再次指出了创作与排版之间的区别:作者指示某一行是标题,或者某个片段是警告或引用;而排版员决定如何在排版上强调所有标题、警告或引用。因此,一方面保证了一致性,因为所有 fulfills 相同功能的片段看起来都一样,另一方面节省了时间,因为每种片段的格式只需指定一次。

第4节 T_EX 及其衍生品

T_EX 是在 1970 年代末由 DONALD E. KNUTH 开发的,他是斯坦福大学理论计算机编程教授(现为荣誉退休教授),他实现该程序是为了制作自己的出版物,并作为系统开发和注释的示例。与 T_EX 一起,KNUTH 还开发了一种名为 MetaFont 的附加编程语言,用于设计排版字体,并用它设计了一种他命名为 *Computer Modern* 的字体,该字体除了任何字体通常的字符外,还包括一套完整的用于书写数学的「字形」¹。除此之外,他还添加了一些额外的实用程序,于是称为 T_EX 的排版系统诞生了,由于其强大、结果质量高、使用灵活和可能性广泛,它被认为是用于文本排版的最佳计算机化系统之一。它被设计用于包含大量数学公式的文本,但很快人们就发现该系统的可能性使其适用于所有类型的文本。

在内部,T_EX 的工作方式与以前印刷厂的排版工相同。对 T_EX 来说,一切都是盒子:字母包含在盒子中,空格也是盒子,几个字母(包含几个字母的盒子)形成一个包含单词的新盒子,几个单词以及它们之间的空格形成一个包含一行的盒子,几行变成一个包含段落的盒子... 依此类推。此外,所有这些都具有非凡的测量精度。考虑到 T_EX 处理的最小单位比通常用于测量字符和行(这是大多数文字处理程序处理的最小单位)的排版点还要小 65,536 倍。这意味着 T_EX 处理的最小单位大约为 0.000005356 毫米。KNUTH 认为这对于所有实际目的来说都足够精确。

T_EX 这个名字来源于希腊单词 τεχνη 的词根,用大写字母书写(ΤΕΧΝΗ)。因此,单词 T_EX 的最后一个字母不是拉丁字母『X』,而是希腊字母『χ』,其发音显然像苏格兰语 *loch* 中的『ch』。所以 T_EX 应该发音为 *Tech*。另一方面,这个希腊词同时意味着「艺术」和「技术」,这就是 KNUTH 选择它来命名他的系统的原因。他写道,这个名字的目的是「提醒你 T_EX 主要关注高质量的技术手稿。它强调的是艺术和技术,正如其希腊词根所蕴含的意义」。

按照 KNUTH 建立的惯例,T_EX 应该写成:

- 在像本文这样的排版格式化文本中,使用我一直以来使用的标志:三个大写字母,中间的『E』稍微向下偏移,以使『T』和『X』对齐更紧密;换句话说:「T_EX」。

为了便于书写这个标志,Knuth 在 T_EX 中包含了一个用于在最终文档中书写该标志的指令:`\TeX`。

¹ 在排版中,字形是一个字符、多个字符或部分字符的图形表示,相当于今天的字母类型(刻有字母或活字的 bits)

- 在未格式化的文本中(例如电子邮件或文本文件),使用大写的『T』和『X』,以及小写的中间『e』;因此:「TeX」。

这个惯例继续用于所有包含其专有名称的 T_EX 衍生品中,ConT_EXt 就是这种情况。在未格式化的文本中书写时,我们需要写成「ConTeXt」。

4.1 T_EX 引擎

T_EX 程序是自由的自由软件:其源代码对公众开放,任何人都可以随意使用或修改它,唯一的条件是,如果进行了修改,则结果不能称为「T_EX」。这就是为什么随着时间的推移,出现了该程序的某些改编版本,对其进行了不同的改进,这些通常被称为 T_EX 引擎。除了原始的 T_EX 程序,按出现时间顺序,主要的引擎有 pdfT_EX、e-T_EX、X_YT_EX、LuaT_EX 和 luametaT_EX。它们每个都应包含前一个的改进。另一方面,这些改进,在 LuaT_EX 出现之前,并没有影响语言本身,而只影响输入文件、输出文件、源处理以及宏的底层操作。

使用哪个 T_EX 引擎是 T_EX 社区内一个争论很多的问题。我在这里不展开这个问题,因为 ConT_EXt LMTX 只与 luametaT_EX 一起工作。实际上,在 ConT_EXt 世界中,关于 T_EX 引擎的讨论变成了关于使用 Mark II(与 pdfT_EX 和 X_YT_EX 一起工作)、Mark IV(仅与 LuaT_EX 一起工作)还是 LMTX(仅与 luametaT_EX 一起工作)的讨论。

4.2 从 T_EX 衍生出的格式

T_EX 的核心只理解大约 300 条非常基本的指令,称为原语,适用于排版操作和编程功能。这些指令中的绝大多数都是非常低级的,用计算机术语来说,这意味着它们对计算机来说比人类更容易理解,因为它们涉及非常基本的操作,比如「将此字符向上移动 0.000725 毫米」这类。因此,KNUTH 认为 T_EX 应该是可扩展的,意思是应该有一种机制允许在更高级别定义指令,这些指令更容易被人类理解。这些指令在执行时被分解为其他更简单的指令,被称为宏。例如,打印 (`\TeX`) 标志的 T_EX 指令,在执行时分解如下:

```
T
\kern -.1667em
\lower .5ex
\hbox {E}
\kern -.125em
X
```

但对于人类来说,理解和记住简单的命令「`\TeX`」会执行打印标志所需的排版操作要容易得多。

什么是宏和什么是原语之间的区别,实际上仅从 T_EX 开发者的角度来看才重要。从用户的角度来看,它们是指令,或者如果你愿意,也可以说是命令。Knuth 称它们为控制序列。

这种通过宏扩展语言的可能性是使 T_EX 成为如此强大工具的特性之一。事实上,KNUTH 自己创建了大约 600 个宏,与 300 个原语一起构成了称为「Plain T_EX」

的系统。将 $\text{T}_\text{E}\text{X}$ 本身与 Plain $\text{T}_\text{E}\text{X}$ 混淆是很常见的,事实上,通常关于 $\text{T}_\text{E}\text{X}$ 的几乎所有书面或口头内容,实际上都是指 Plain $\text{T}_\text{E}\text{X}$ 。声称是关于 $\text{T}_\text{E}\text{X}$ 的书籍(包括奠基性的「*The $\text{T}_\text{E}\text{X}$ Book*」)实际上指的是 Plain $\text{T}_\text{E}\text{X}$;而那些认为他们直接在使用 $\text{T}_\text{E}\text{X}$ 的人实际上是在使用 Plain $\text{T}_\text{E}\text{X}$ 。

Plain $\text{T}_\text{E}\text{X}$ 在 $\text{T}_\text{E}\text{X}$ 术语中被称为格式,由一套广泛的宏以及关于如何使用它们的方式和方法的某些语法规则组成。除了 Plain $\text{T}_\text{E}\text{X}$,随着时间的推移,其他格式也被开发出来,其中值得一提的是 $\text{L}^{\text{A}}\text{T}_\text{E}\text{X}$,这是由 LESLIE LAMPORT 于 1985 年创建的用于 $\text{T}_\text{E}\text{X}$ 的一套广泛宏,它可能是学术、技术和数学世界中使用最多的 $\text{T}_\text{E}\text{X}$ 衍生品。 $\text{ConT}_\text{E}\text{Xt}$ 作为从 $\text{T}_\text{E}\text{X}$ 衍生出的格式,与 $\text{L}^{\text{A}}\text{T}_\text{E}\text{X}$ 是同一级别的。

通常,这些格式伴随有一个程序,该程序在调用「`tex`」(或实际用于处理的引擎)处理源文件之前,将构成它们的宏加载到内存中。但是,尽管所有这些格式实际上都在运行 $\text{T}_\text{E}\text{X}$,但从用户的角度来看,由于每种格式都有不同的指令和不同的语法规则,我们可以将它们视为不同的语言。它们都从 $\text{T}_\text{E}\text{X}$ 中汲取灵感,但不同于 $\text{T}_\text{E}\text{X}$,也彼此不同。

第 5 节 $\text{ConT}_\text{E}\text{Xt}$

实际上, $\text{ConT}_\text{E}\text{Xt}$ 最初是 $\text{T}_\text{E}\text{X}$ 的一种格式,但今天远不止于此。 $\text{ConT}_\text{E}\text{Xt}$ 包括:

1. 一套非常广泛的 $\text{T}_\text{E}\text{X}$ 宏。如果 Plain $\text{T}_\text{E}\text{X}$ 有大约 900 条指令, $\text{ConT}_\text{E}\text{Xt}$ 则有大约 3500 条;如果算上这些命令支持的不同选项的名称,我们谈论的是大约 4000 个单词的词汇量。词汇量如此之大是因为 $\text{ConT}_\text{E}\text{Xt}$ 促进学习的策略,该策略意味着包含任意数量的命令和选项同义词。

其意图是,如果要实现某种效果,那么对于英语使用者称呼该效果的每种方式,都有一个命令或选项可以实现它——这应该使语言的使用更容易。例如,为了同时获得粗体和斜体字母, $\text{ConT}_\text{E}\text{Xt}$ 有三个指令都可以达到相同的结果:`\bi`、`\italicbold` 和 `\bolditalic`。

2. 同样广泛的用于 MetaPost 的宏集,MetaPost 是一种源自 MetaFont 的图形编程语言,而 MetaFont 又是 KNUTH 与 $\text{T}_\text{E}\text{X}$ 联合开发的字体设计语言。
3. 用 PERL(最古老的)、RUBY(有些也老了,有些没那么老)和 LUA(最新的)开发的各种脚本。
4. 一个集成 $\text{T}_\text{E}\text{X}$ 、MetaPost、LUA 和 XML 的接口,允许用这些语言中的任何一种编写和处理文档,或者混合使用其中一些语言的元素。

也许你对上面的解释不太理解?别担心!我用了许多计算机术语,提到了许多程序和语言。使用 $\text{ConT}_\text{E}\text{Xt}$ 并不需要了解所有这些不同的组件。重要的是,在这个学习阶段,要记住 $\text{ConT}_\text{E}\text{Xt}$ 集成了许多来自不同来源的工具,这些工具共同构成了一个排版系统。

正是由于这后一种集成不同来源工具的特性,我们说 $\text{ConT}_\text{E}\text{Xt}$ 是一种旨在排版文档的「混合技术」。我的理解是,这使 $\text{ConT}_\text{E}\text{Xt}$ 成为一个非常先进和强大的系统。

尽管 ConT_EXt 远不止是一套用于 T_EX 的宏集合,但它仍然基于 T_EX,这就是为什么这份我声称不过是入门的文档侧重于这一点。

另一方面,ConT_EXt 比 T_EX 更现代。当 T_EX 被创建时,计算机向大众普及才刚刚开始,我们远未看到互联网和多媒体世界将会是什么样子。在这方面,ConT_EXt 自然地集成了一些在 T_EX 中一直有点格格不入的东西,例如包含外部图形、处理颜色、电子文档中的超链接、采用适合屏幕显示文档的页面尺寸等。

5.1 ConT_EXt 简史

ConT_EXt 大约诞生于 1991 年。它是由 HANS HAGEN 和 TON OTTEN 在一家名为「*Pragma Advanced Document Engineering*」的荷兰文档设计和处理公司创建的,通常缩写为 Pragma ADE。它最初是一套带有荷兰语名称的 T_EX 宏集合,非正式地称为 *Pragmatex*,面向公司中必须处理排版文档编辑的许多细节、不习惯使用标记语言或荷兰语以外的接口的非技术员工。因此,ConT_EXt 的第一个版本是用荷兰语编写的。其想法是创建足够数量的具有统一且一致接口的宏。大约在 1994 年,该包足够稳定,可以用荷兰语编写用户手册,并在 1996 年,通过 HANS HAGEN 的倡议,对它的引用开始采用「ConT_EXt」这个名称。这个名字声称的意思是「用 T_EX 处理文本」(使用拉丁语介词『con』意为『与』),但同时也是一个关于英语(和荷兰语)单词「context」的文字游戏。因此,在这个名字背后,隐藏着一个涉及「T_EX」、「text」和「context」的三重文字游戏。

因此,由于这个名字基于文字游戏,ConT_EXt 应该发音为『context』而不是『contech』,因为后者会失去这个文字游戏。

接口大约在 2005 年开始被翻译成英语,从而产生了被称为 ConT_EXt Mark II 的版本,其中『II』的解释是,在开发者的心目中,之前的荷兰语版本是版本『I』,尽管它从未被正式这样称呼。在接口被翻译成英语后,该系统的使用开始传播到荷兰以外,接口也被翻译成其他欧洲语言,如法语、德语、意大利语和罗马尼亚语。尽管如此,ConT_EXt 的「官方」文档通常基于英语版本,本文档使用的就是这个版本。

在其初始版本中,ConT_EXt Mark II 与 pdfT_EX 引擎一起工作。但后来,随着 X_YT_EX 引擎的出现,ConT_EXt Mark II 被修改以允许使用这个新引擎,与 pdfT_EX 相比,它带来了许多优势。但是当几年后 LuaT_EX 出现时,决定在内部重新配置 ConT_EXt 的工作方式,以集成这个新引擎提供的所有新可能性。于是,ConT_EXt Mark IV 诞生了,它于 2007 年推出,紧随 LuaT_EX 推出之后。很可能,影响重新配置 ConT_EXt 以适应 LuaT_EX 的决定的因素之一是,ConT_EXt 的三位主要开发者中的两位,HANS HAGEN 和 TACO HOEKWATER,同时也是开发 LuaT_EX 的主要团队的一部分。这就是为什么 ConT_EXt Mark IV 和 LuaT_EX 同时诞生并协同发展。ConT_EXt 和 LuaT_EX 之间存在一种在 T_EX 的任何其他衍生品中都不存在的协同作用;我怀疑是否有任何其他衍生品能像 ConT_EXt 那样利用 LuaT_EX 的优势。

Mark II、Mark IV 和 LMTX 之间有许多差异,尽管其中大部分是内部的,也就是说,它们与宏在更低级别实际工作的方式有关,因此从用户的角度来看,差异并不

明显:许多宏的名称和参数保持不变。然而,确实存在一些影响接口的差异,迫使人们根据所使用的版本以不同的方式做事。这些差异相对较少,但它们确实影响到非常重要的方面,例如输入文件的编码或系统安装字体的处理。话虽如此,在 2026 年,Mark II 和 Mark IV 应被视为历史工具,不应用于开发新文档。

然而,如果某处有一份文档解释(或列出)Mark II、Mark IV 和 LMTX 之间明显的差异,那将是非常受欢迎的。例如,在 ConT_EXt wiki 中,对于每个 ConT_EXt 命令,有时会有两种语法(通常完全相同)。我推测一种属于 Mark II,另一种属于 Mark IV 和 / 或 LMTX;基于这个假设,我还推测第一个版本来自 Mark II。但事实是 wiki 对此只字未提。



事实上,在语言层面上差异相对较少,这意味着在许多情况下,我们谈论的不是两个版本,而是 ConT_EXt 的两种「风味」。但无论你怎么称呼它们,事实是,为一种 ConT_EXt 风味准备的文档通常不能用另一种风味编译;如果文档混合了两种或三种版本,它很可能无法与其中任何一种很好地编译。这意味着源文件的作者必须首先决定是为 Mark II、Mark IV 还是 LMTX 编写。(自从 LMTX 出现以来,很少有或根本没有理由为 Mark II 或 Mark IV 编写新文档;但是,如果想要编辑为这两种风味中的任何一种编写的现有文档,那么使用原始文档风味的命令可能比重写文档以使用适合 LMTX 的命令更容易。)

如果我们使用不同版本的 ConT_EXt,一个区分 Mark II 和 Mark IV 文件的好技巧是使用不同的文件扩展名。因此,例如,对于我为 Mark II 编写的任何文件,我使用「.mkii」作为扩展名;对于 Mark IV,我使用「.mkiv」,而对于为 LMTX 编写的文件,我使用「.tex」。诚然,ConT_EXt 期望所有源文件都有扩展名「.tex」,但只要我们在将 ConT_EXt 应用于文件时明确指定文件扩展名,就可以更改文件扩展名。

截至 2026 年,「ConT_EXt Standalone」发行版包括 Mark IV 和 LMTX,两者都可以使用命令「context」访问。Mark II 仍然可用(主要供需要处理旧文档的人使用),但它不包含在「ConT_EXt Standalone」发行版中。然而,著名的 T_EXlive 发行版包含了 Mark II,可以使用命令「texexec」访问。

如今,Mark II 和 Mark IV 已冻结,而 LMTX 仍在开发中,这意味着只有发现需要更正的错误或缺陷时才会发布旧版本的新版本,而 LMTX 的新版本则继续定期发布;有时一个月两到三次,尽管在大多数情况下,这些「新版本」并没有引入语言上可感知的变化,而是仅限于在低级别上改进命令的实现方式,或更新发行版中包含的众多手册中的一些。即便如此,如果我们安装了开发版本——这是我推荐的,也是「ConT_EXt Standalone」默认安装的版本——定期更新我们的版本是有意义的(参见附录 A 了解更新已安装「ConT_EXt Standalone」版本的方法)。

5.2 ConT_EXt 与 L^AT_EX 的比较

鉴于从 T_EX 衍生出的最流行的格式是 L^AT_EX,这两者之间的比较是不可避免的。显然,我们谈论的是不同的语言,尽管它们在某种程度上相互关联,因为它们都源自 T_EX。这种关系类似于例如西班牙语和法语之间的关系:有共同起源(拉丁语)的语言,这意味着它们的语法相似,并且每种语言中的许多单词在另一种语言中都有对应的单词。但除了这种家族相似性之外,L^AT_EX 和 ConT_EXt 在理念和实现

上有所不同,因为两者的初始目标在某种程度上是相反的。 $\text{\LaTeX}$ 声称要简化 $\text{\TeX}$ 的使用,将作者与具体的排版细节隔离开来,以帮助专注于内容,将排版细节留给 $\text{\LaTeX}$ 。这意味着简化 $\text{\TeX}$ 的使用是以牺牲 $\text{\TeX}$ 的巨大灵活性为代价的,通过预定义基本格式并限制作者必须决定的排版问题的数量。与这种理念相反,Con $\text{\TeX}$ t 诞生于一家致力于排版文档的公司内部。因此,它远非想要将作者与排版细节隔离开来,而是旨在赋予作者对它们的绝对和完全控制。为了实现这一点,Con $\text{\TeX}$ t 提供了一个统一且一致的接口,比 $\text{\LaTeX}$ 更接近 $\text{\TeX}$ 的原始精神。

这种理念和创始目标的差异进而转化为实现上的差异。 $\text{\LaTeX}$ 倾向于尽可能简化,不需要使用 $\text{\TeX}$ 的所有资源。在某种程度上,其核心相当简单。因此,当需要扩大其可能性时,有必要明确编写一个包来实现。这种与 $\text{\LaTeX}$ 相关的打包既是优点也是缺点:优点是因为 $\text{\LaTeX}$ 的巨大知名度,加上其用户的慷慨,意味着我们可能遇到的几乎任何需求都已被某人满足过,并且有一个包可以实现它;但这也是一个缺点,因为这些包之间常常不兼容,而且它们的语法并不总是统一的。这意味着使用 $\text{\LaTeX}$ 需要不断搜索成千上万个已有的包来满足需求,并确保它们能一起工作。

与通过包扩展来补充的 $\text{\LaTeX}$ 核心的简单性相反,Con $\text{\TeX}$ t 被设计为在其内部包含所有——或几乎所有—— $\text{\TeX}$ 的排版可能性,因此它的概念更加一体化,但同时也更加模块化。Con $\text{\TeX}$ t 核心允许我们做几乎所有事情,并且我们保证其不同实用程序之间不会有不兼容性,无需为我们需要的功能研究扩展,语言的语法也不会仅仅因为我们需要一个特定的实用程序而改变。

诚然,Con $\text{\TeX}$ t 有所谓的扩展模块,有些人可能认为它们发挥着类似于 $\text{\LaTeX}$ 包的功能,但实际上它们的工作方式不同:Con $\text{\TeX}$ t 模块专门设计用于包含额外的实用程序,这些实用程序要么因为仍处于实验阶段而尚未并入核心,要么允许访问由 Con $\text{\TeX}$ t 开发团队之外的某人所编写的扩展。

我不认为这两种理念中哪一种更可取。问题更多地取决于用户的概况和需求。如果用户不想处理排版问题,只想制作非常高质量的标准化文档,那么选择像 $\text{\LaTeX}$ 这样的系统可能更可取;另一方面,喜欢尝试、或需要控制文档每一个细节的用户,或者必须为文档设计特殊布局的用户,使用像 Con $\text{\TeX}$ t 这样的系统可能更好,作者可以掌握所有控制权;当然,也有不知道如何正确使用这种控制的风险。

5.3 理解使用 Con $\text{\TeX}$ t 的工作流程

当我们使用 Con $\text{\TeX}$ t 时,我们总是从编写一个文本文件(我们称之为源文件)开始,在其中,除了最终文档的实际内容外,我们还将包含指示我们希望如何精确格式化文档的指令(用 Con $\text{\TeX}$ t 的话说):我们希望页面和段落具有的总体外观、我们希望应用于某些段落的边距、我们想要显示的字体、我们希望以不同字体显示的片段等。一旦我们编写了源文件,我们从终端(或者可能是我们的文本编辑器)应用「context」程序,它将处理该文件,并从中生成一个不同的文件,其中文档的内容将根据源文件中为此目的包含的指令进行格式化。这个新文件可以发送给(商业)打印机、在屏幕上显示、放在互联网上或在联系人、朋友、客户、老师、学生... 之间分发,或者换句话说,分发给任何我们为其编写文档的人。

这意味着在使用 ConT_EXt 时,作者处理的是一个视觉外观与最终文档毫无关系的文件:作者直接处理的文件是一个没有进行排版格式化的文本文件。因此,ConT_EXt 的工作方式与所谓的文字处理器程序不同,后者在我们编写文档的同时就显示编辑后文档的最终外观。对于那些习惯于文字处理器的人来说,使用 ConT_EXt 的工作方式起初会感到奇怪,甚至可能需要一段时间来适应。然而,一旦习惯了,人们就会明白,这种区分工作文件和最终结果的工作方式实际上是一种优势,原因有很多,在此我无特定顺序地强调以下几点:

1. 因为文本文件比文字处理器的二进制文件『更轻』,处理它们需要更少的计算机内存,它们更不容易损坏,并且当我们更改创建它们的程序版本时,它们不会变得不可读。它们也与任何操作系统兼容,并且可以用许多文本编辑器编辑,因此为了处理它们,计算机系统不必安装创建该文件的程序:任何其他编辑程序都可以;并且在每个计算机系统中,总会有某种文本编辑程序。
2. 因为区分工作文档和最终文档有助于区分文档的实际内容与其外观,允许作者在创作阶段专注于内容,在排版阶段专注于外观。
3. 因为它允许快速准确地更改文档的外观,因为这由可以轻松识别的 ConT_EXt 命令决定。
4. 因为这种更改外观的便利性,另一方面,允许我们轻松地由单一内容生成两个(或更多)不同的版本:可能一个版本针对纸上打印进行了优化,另一个版本设计为在屏幕上显示,适应后者的尺寸,并可能包含在纸质文档中没有意义的超链接。
5. 因为文字处理器中常见的排版错误(例如将斜体扩展到单词最后一个字符之后)也很容易避免。
6. 因为当工作文件不被分发并且「仅供我们自己查看」时,可以为我们自己后续的修订或版本加入注释和观察、评论和警告,安心地知道这些不会出现在要分发的格式化文件中。
7. 因为同时处理整个文档所能获得的质量远高于使用必须在编写文档时做出排版决策的程序所能达到的质量。
8. 等等。

以上所有意味着,一方面,在使用 ConT_EXt 时,一旦我们掌握了窍门,我们就会更高效、更有生产力,另一方面,我们能获得的排版质量远优于使用所谓的文字处理器所能获得的质量。尽管后者确实更容易使用,但实际上它们并没有那么容易使用。因为虽然 ConT_EXt 如前所述包含 3500 条指令,但 ConT_EXt 的普通用户并不需要知道所有这些指令。要做通常用文字处理器做的事情,我们只需要知道允许我们指示文档结构的指令、一些关于常见排版资源的指令(如粗体或斜体),以及可能如何生成列表或脚注的指令。总共不超过 15 或 20 条指令就能让我们完成几乎所有用文字处理器能做的事情。其余的指令允许我们做不同的、通常无法用文字处理器完成

或很难实现的事情。我们可以说,虽然学习使用 ConT_EXt 比学习使用文字处理器更困难,这是因为用 ConT_EXt 可以做得更多。

5.4 获取 ConT_EXt 的帮助

在我们还不熟悉的时候,获取 ConT_EXt 帮助的最佳地点无疑是 [wiki](#),那里有很多示例,并且有很好的搜索引擎,尤其是如果你英语好的话。当然,我们也可以在互联网上找到帮助,但在这里,ConT_EXt 名称中的文字游戏会给我们带来麻烦,因为搜索单词「context」会返回数百万个结果,其中大多数与我们寻找的内容无关。要查找有关 ConT_EXt 的信息,你需要在「context」一词上添加一些内容;例如,「tex」、或「Mark IV」、或「Hans Hagen」(ConT_EXt 的创建者之一)、或「Pragma ADE」,或类似的东西。使用 wiki 名称「contextgarden」搜索信息也可能有用。

当我们对 ConT_EXt 有了更多了解后,我们可以查阅「ConT_EXt Standalone」中包含的众多文档中的一些,甚至在 [TeX – LaTeX Stack Exchange](#) 或 ConT_EXt 邮件列表([NTG-context](#))上寻求帮助。后者涉及最了解 ConT_EXt 的人,但良好的网络礼仪规则要求,在提问之前,我们应该事先努力自己寻找答案。

第2章 我们的第一个源文件

Table of Contents: 第1节 准备实验:必备工具; 第2节 实验本身; 第3节 我们示例文件的结构; 第4节 关于如何运行「context」的一些额外细节; 第5节 错误管理;

本章致力于我们的第一次实验,并将解释 ConT_EXt 文档的基本结构,以及处理潜在错误的最佳策略。

第1节 准备实验:必备工具

要编写并编译第一个源文件,我们需要在系统上安装以下工具。

1. 一个**文本编辑器**,用于编写我们的测试文件。市面上有很多文本编辑器,很难想象哪个操作系统没有预装一个。我们可以使用其中任何一个:有简单的、复杂的、强大的、付费的、免费的(免费啤酒意义上的)、自由的(自由言论意义上的)、专门用于 T_EX 系统的、通用型的等等。如果我们习惯使用某个特定的编辑器,最好继续使用它;如果到目前为止还不习惯使用任何编辑器,我最初的建议是找一个简单的编辑器,以免在学习 ConT_EXt 的难度上再增加学习使用文本编辑器的任务。尽管确实,最难学的程序往往是最强大的程序。

本文是用 GNU Emacs 编写的,它是有史以来最强大、最通用的编辑器之一;它确实有自己的特点,也有批评者,但总的来说,“Emacs 爱好者”比“Emacs 反对者”更多。有一个名为 AucTeX 的 GNU Emacs 扩展,用于处理 T_EX 文件或其衍生文件,它为编辑器提供了一些非常有趣的额外实用程序,尽管 AucTeX 通常为处理 L^AT_EX 文件做的准备比处理 ConT_EXt 文件更充分。如果我们不知道选择哪个编辑器,GNU Emacs 结合 AucTeX 可能是一个不错的选择;两者都是自由软件,因此它们有适用于所有操作系统的版本。事实上,说 GNU Emacs 是“自由软件”是一种保守的说法,因为这个程序比其他任何程序都更能体现“自由软件”是什么以及意味着什么的精神。毕竟,它的主要开发者是 RICHARD STALLMAN,他是 GNU 项目和自由软件基金会的创始人和理论家。

除了 GNU Emacs + AucTeX,如果你不知道选择哪个,其他不错的选择有 Scite 和 TexWorks。前者,尽管是一个通用编辑器,并非专门为处理 ConT_EXt 文件而设计,但易于定制,并且由于它是 ConT_EXt 开发者通常使用的编辑器,「ConT_EXt Standalone」包含了这个编辑器的配置文件,由 HANS HAGEN 本人编写。而 TexWorks 是一个快速的文本编辑器,专门处理 T_EX 文件及其衍生语言的文件。将其配置用于 ConT_EXt 非常简单,「ConT_EXt Standalone」也预见了其配置。

无论使用什么编辑器,我们绝对不能用作文本编辑器的是文字处理器,例如 OpenOffice Writer 或 Microsoft Word。这些程序在我看来也过于缓慢和笨重,如果明确指定,它们可以将文件保存为“纯文本(txt)”,但它们并非为此而设计,我们最终很可能将文件保存为与 ConT_EXt 不兼容的某种二进制格式。

2. 一个 ConT_EXt 发行版,用于处理我们的测试文件。如果我们的系统上已经安装了 T_EX (或 L_T_EX),那么可能已经安装了一个版本的 ConT_EXt。要测试这一点,只需打开一个终端并输入

```
$> context --version
```

注意:对于那些不熟悉终端操作的人,我写的前两个字符(「\$>」)不需要在终端中输入。我只是表示了所谓的终端提示符;那个闪烁的小标志,表示终端正在等待指令。

如果已经安装了某个版本的 ConT_EXt,将会出现类似下面的内容:

```
mtx-context | ConTeXt Process Management 1.06
mtx-context |
mtx-context | main context file: .../context/tex/texmf-context/
               | tex/context/base/mkiv/context.tex
mtx-context | current version: 2026.02.19 11:49
mtx-context | main context file: .../context/tex/texmf-context/
               | tex/context/base/mkx1/context.mkx1
mtx-context | current version: 2026.02.19 11:49
```

最后一行告诉我们安装版本发布的日期。如果这太旧了,我们应该更新它或安装一个新版本。我推荐安装名为「ConT_EXt Standalone」的发行版,其安装说明可以在 [ConT_EXt wiki](#) 上找到。所有这些的摘要可以在[附录 A](#) 中找到。

3. 一个 PDF 文件阅读器,以便我们在屏幕上看到实验的结果。在 Windows 和 Mac OS 中,通常有 Adobe Acrobat Reader。它不是默认安装的(或者在我 15 多年前停止使用 Microsoft Windows 时不是),但当你第一次尝试打开 PDF 文件时它会安装,所以很可能它已经安装了。Linux/Unix 系统没有当前版本的 Acrobat Reader,但你也不需要它,因为 literally 有几十个免费且非常好的 PDF 阅读器可用。此外,这些系统上几乎总是默认安装了其中一个。我最喜欢的是 MuPDF,因为它速度快且易于使用;不过如果你使用带重音字符的英语以外的语言,它有一些缺点,并且不允许你选择文本或将文档发送到打印机;它只是一个阅读器;但它非常快速且使用舒适。当我需要一些 MuPDF 中不可用的功能时,我通常使用 Okular 或 qPdfView。但同样,这是一个品味问题:你可以选择任何你喜欢的。

我们可以选择我们的编辑器、PDF 阅读器、ConT_EXt 发行版.....欢迎来到自由自由软件的世界!

第2节 实验本身

编写源文件

如果上述工具已经可用,我们需要打开文本编辑器并用它创建一个文件,我们将它命名为「rain.tex」。我们将以下内容写为此文件的内容:

```
% 文档的第一行

\mainlanguage[en] % 语言 = 英语

\setuppapersize[S5] % 纸张大小

\setupbodyfont
[modern,12pt] % 字体 = Latin Modern,12 点

\setuphead    % 章节的格式
[chapter]
[style=\bfc]

\starttext % 开始文档内容

\startchapter
[title=西班牙的雨.....]

你真好,让我来。
再说一遍,哪里下雨?
在平原上,在平原上。
那个该死的平原在哪里?
在西班牙,在西班牙。
西班牙的雨主要落在平原上。
西班牙的雨主要落在平原上。

\stopchapter

\stoptext % 文档结束
```

在编写时,更改(几乎)任何内容都无关紧要,尤其是添加或删除空格或换行。重要的是,「\」后面的单词以及花括号内的内容必须完全按照所示编写。其余部分可以有变化。

文件的字符编码

一旦我们写完了上面的内容,我们将文件保存到磁盘上,确保字符编码是 UTF-8(这种字符编码是当今的标准)。无论如何,如果我们不确定,我们可以从文本编辑器本身查看编码,并在需要时更改它。如何做显然取决于我们使用的文本编辑器。例如,在 GNU Emacs 中,按 Ctrl-X,然后按 Return,接着按『f』,会在窗口底部

行(GNU Emacs 称之为小缓冲区)出现一条消息,询问我们新的编码并告诉我们当前的编码;例如,你可能会看到一行如

Coding system for saving file (default utf-8-unix):

这告诉你在保存文件时将使用 UTF-8 字符编码。在其他编辑器中,我们通常可以在「另存为」菜单中找到编码设置。

一旦我们确认编码正确,并将文件保存到磁盘上,我们关闭编辑器,专注于分析我们写的内容。

查看我们的第一个为 ConT_EXt 编写的源文件的内容

第一行以「%」字符开头。这是一个保留字符,告诉 ConT_EXt 不要处理从该字符到行尾之间的文本。这在我们想在源文件中写入只有作者才能阅读的注释时很有用,因为它不会成为最终文档的一部分。在这个例子中,我使用这个字符来引起对某些行的注意,解释它们的作用。

接下来的行以「\」字符开头,这是 ConT_EXt 的另一个保留字符,表示其后的内容是命令的名称。这个示例展示了 ConT_EXt 源文件中出现的许多命令:文档所用的语言、纸张大小、文档中将使用的字体以及章节的格式化方式。在后面的其他章节中,我们将看到这些命令的细节,但此刻我只想让读者看到它们的样子:它们总是以「\」开头,然后是命令名称,然后根据情况在花括号(也称为大括号,但在本文档中我们使用“花括号”以明确区别)或方括号内,放置命令产生其效果所需的数据。在命令名称与其后的方括号或花括号之间,可以有空格或一个换行。

在我们示例的第九行(我只计算有文本的行)是重要的 `\starttext` 命令:它告诉 ConT_EXt 文档内容从此处开始;而在我们示例的最后一行,我们看到命令 `\stoptext` 表示文档在此结束。这是两个非常重要的命令,我很快会进一步说明。在它们之间是文档的实际内容,在我们的示例中,包括来自「My Fair Lady」「西班牙的雨...」的著名对话。我将其写成散文形式,以便我们可以看到 ConT_EXt 如何格式化段落。

处理源文件

下一步,在确认 ConT_EXt 已正确安装在我们的系统上之后,我们需要在与源文件「rain.tex」保存的相同目录中打开一个终端。

许多文本编辑器允许我们在不打开终端的情况下编译正在处理的文档。然而,用 ConT_EXt 处理文档的规范过程意味着通过直接执行程序从终端进行。在本文档中,我将以这种方式进行(或假定以这种方式进行),原因有几个:首先,我无法知道读者正在使用什么文本编辑器。但最重要的是,通过使用终端,我们将能够访问「context」的屏幕输出,并看到来自程序的消息。

要从终端运行 context,我们必须告诉命令解释器(「shell」)context 在我们系统中的安装位置。这取决于我们选择将 context 安装在哪里,但假设我们将其安装在我们的主目录中,那么我们的主目录现在包含一个名为「context」的目录。

在类 Unix 系统(如 Linux)中,命令解释器有一个名为 `PATH` 的变量,它给出一个目录列表,当尝试执行程序时会搜索这些目录。我们可以将包含 `context` 程序的目录放在此列表的前面¹,通过输入

```
$> PATH=$HOME/context/tex/texmf-linux-64/bin:$PATH
```

如果我们把「ConTeXt Standalone」安装在其他地方,我们需要相应地调整上面的行。

为了避免每次打开终端时都输入这一行,可以将上面的行放在 shell 的配置文件中。根据我们选择的特定 shell,这个配置文件可能是 `$HOME/.bashrc`(如果我们的 shell 是 `bash`)、`$HOME/.zshrc`(如果我们的 shell 是 `zsh`),或 `$HOME/.profile`(如果我们的 shell 是 `sh`)。

如果我们使用的是某种非类 Unix 操作系统,我们需要做类似上述的操作。如果我们还不知道怎么做,可以请教隔壁那个无所不知的 10 岁孩子。

一旦我们设置好 shell 程序知道如何找到 `context`,我们就可以运行它。我们通过在终端中输入以下命令来实现:

```
$> context rain
```

注意,尽管源文件名为「`rain.tex`」,但在调用「`context`」时我们省略了文件扩展名,因为 `context` 会假定它应该寻找一个扩展名为 `.tex` 的文件。如果我们把源文件命名为例如「`rain.mkiv`」(我经常这样做,以便知道这个文件是为 Mark IV 编写的),我们就必须通过输入「`context rain.mkiv`」来明确指定文件扩展名。

在终端中运行「`context`」后,屏幕上会出现几十行文字,告诉我们 ConTeXt 正在做什么。这些信息出现的速度人类无法跟上,但我们不必担心,因为除了显示在屏幕上,相同的信息也存储在一个辅助文件中,其扩展名是(在这种情况下,因为我们的 ConTeXt 文件名为 `rain.tex`)「`rain.log`」。它是在处理时生成的,如果需要,我们可以稍后从容地查阅它。

几秒钟后,如果我们在源文件中没有犯任何严重错误,终端消息将会结束。最后一条消息会告诉我们编译文件花费了多长时间。第一次编译文档时需要稍多时间,因为 ConTeXt 必须将告知它正在使用哪些字体的文件加载到内存中,而后续处理时这些已经加载。当显示耗时信息的最终消息出现时,处理完成。如果一切顺利,我们运行「`context`」的目录现在将包含三个额外的文件:

- `rain.pdf`
- `rain.log`
- `rain.tuc`

第一个是我们处理的结果,或者换句话说,是格式化后的 PDF。第二个是「`.log`」文件,存储了处理文件时屏幕上显示的所有信息;第三个是 ConTeXt 在编译

¹ 我们可能也想把它放在列表末尾,或者中间的某个位置。我们不会在这里争论这个!

时生成的辅助文件,用于构建索引和交叉引用。目前,如果一切如预期,我们可以删除这两个文件(`rain.log` 和 `rain.tuc`)。如果存在任何问题,这些文件中的信息将帮助我们找出问题所在,并帮助我们找到解决方案。

如果我们没有得到这些结果,可能是由于以下可能性之一:

- ConT_EXt 发行版未正确安装。在这种情况下,在终端输入「`context`」命令后,我们会看到一条包含「`command not found: context`」等文本的消息。
- 我们的文件不是 UTF-8 编码,导致处理错误。
- 我们在源文件中编写命令名称或其相关数据时出错,或者输入了其他无效的 ConT_EXt 输入。

如果在运行「`context`」后,终端开始发出消息,然后在没有重新出现提示符的情况下停止,我们需要在继续之前按 `Ctrl-C` 来中止因错误而中断的 ConT_EXt 运行。(这在 ConT_EXt LMTX 中不太可能发生。)

然后我们需要检查发生了什么,并解决它,直到我们得到正确的编译。

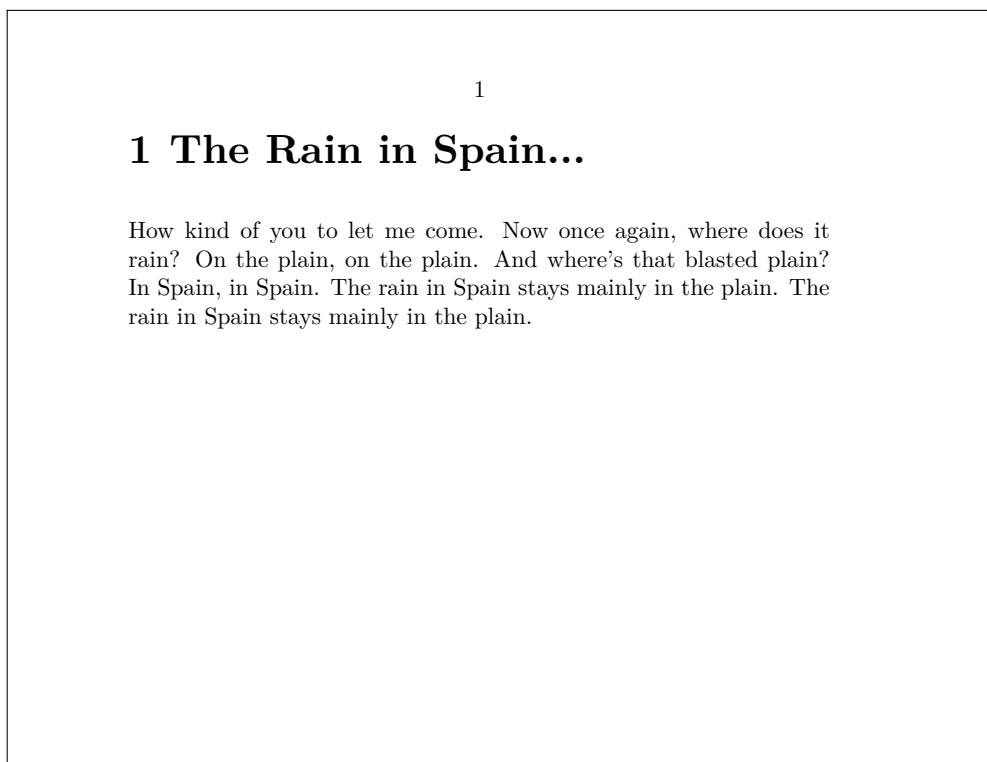


图 2.1 西班牙的雨

在图 2.1 中,我们看到了「`rain.pdf`」的内容。我们还看到 ConT_EXt 为页面和章节编号,并用我们指示的字体书写文本。这里碰巧没有任何单词断字,但默认情况下,ConT_EXt 会根据所选语言的断字规则在行尾断字,在我们的例子中,源文件的第一行通过命令 (`\mainlanguage[en]`) 指定了英语。

总结:ConT_EXt 转换了源文件并生成了一个文件,其中我们有一个根据源文件中的指令格式化的文档。所有的注释都消失了,就命令而言,我们现在看到的不是它们的名称,而是它们执行的结果。

第3节 我们示例文件的结构

在仅由一个源文件开发的项目中,结构非常简单,由命令 `\starttext ... \stoptext` 标记。从文件的第一行到命令 `\starttext` 之间的所有内容称为导言区。实际文档的内容插入在命令 `\starttext` 和 `\stoptext` 之间。在我们的示例中,导言区包括三个全局配置命令:一个指示文档语言的命令 (`\mainlanguage`),另一个指示页面尺寸的命令 (`\setuppapersize`) (在我们的例子中是「S5」,代表计算机屏幕的尺寸),以及第三个命令 (`\setuphead`),它允许我们配置章节标题的外观。

文档主体由命令 `\starttext` 和 `\stoptext` 界定。这些命令分别表示可处理文本的开始和结束点:在它们之间,我们需要包含所有我们希望 ConTeXt 处理的文本,以及不应影响整个文档而只影响部分文档的命令。目前,我们假设命令 `\starttext` 和 `\stoptext` 在每个 ConTeXt 文档中都是必需的,尽管在后续讨论多文件项目(节 6)时,我们会看到一些例外情况。

第4节 关于如何运行「context」的一些额外细节

我们可以用各种选项调用「context」。选项紧跟在命令名称之后引入,前面加两个连字符(即减号,而不是任何其他看起来相似的字符)。如果要引入多个选项,我们用空格分隔它们。在你获得一些专业知识或有某些不寻常的需求之前,你可能不需要使用任何这些选项。但如果你好奇,「help」选项会列出所有选项,并附带简要说明:

```
$> context --help
```

一些比较有趣的选项如下:

interface: 正如我在介绍章节中已经说过的,ConTeXt 界面已被翻译成多种语言。默认情况下界面是英文的;但这个选项允许我们告诉它使用捷克语 (cs)、荷兰语 (nl)、法语 (fr)、意大利语 (it)、德语 (de) 或罗马尼亚语 (ro)。

purge, purgeall: 删除处理过程中生成的辅助文件。

result=Name: 指定生成的 PDF 文件的名称。默认情况下,它将与正在处理的源文件相同,但扩展名为「.pdf」。

usemodule=list: 在运行 ConTeXt 之前加载指定的模块(模块是 ConTeXt 的扩展,不是其核心的一部分,提供一些额外的实用功能)。

useenvironment=list: 在运行 ConTeXt 之前加载指定的环境文件(环境文件是包含配置指令的文件)。

version: 显示 ConT_EXt 版本。

help: 打印程序选项的帮助信息。注意还有一个「`--expert`」选项,它列出更多的选项。

noconsole: 抑制编译期间通常发送到屏幕的一些消息。不过,这些消息仍保存在「`.log`」文件中。

在学习 ConT_EXt 的早期阶段,我认为使用 `noconsole` 选项不是一个好主意,因为当产生错误时,我们将不知道错误在哪里或是什么导致的(除非查看 `.log` 文件)。亲爱的读者,请相信我,你迟早会在处理过程中遇到错误。

第5节 错误管理

在使用 ConT_EXt 时,迟早会在处理过程中出现一些错误,这是不可避免的。我们基本上可以将错误分为以下四类:

1. **书写错误。**当我们弄错命令名称时会产生这些错误。在这种情况下,我们向编译器发送了一个它无法理解的指令。例如,当我们没有写命令 `\TeX` 而是写了 `\Tex`(最后一个「x」是小写)时,因为 ConT_EXt 区分大小写,因此认为「TeX」和「Tex」是不同的单词。或者,如果命令的功能选项被放在方括号内而不是花括号内,或者我们试图将保留字符当作普通字符使用,等等。
2. **遗漏错误。**在 ConT_EXt 中,有些指令开始一个任务,需要我们明确指示其何时结束;这就像启用数学模式的保留字符「`$`」,它一直持续到被禁用,如果我们忘记禁用它,当遇到在数学模式下没有意义的文本或指令时就会产生错误。如果我们以保留字符「`{`」或 `\startSomething` 命令开始一个文本块,而在后面没有找到明确的结束字符「`}`」或 `\stopSomething` 命令,也会发生同样的情况。
3. **概念错误。**这是我称之为当调用一个需要某些参数但没有提供这些参数的命令时产生的错误,或者当调用命令的语法不正确时产生的错误。
4. **情境错误。**有些命令被设计为仅在特定上下文或环境中工作,在这些环境之外不被识别。这在数学模式下尤其常见:一些 ConT_EXt 命令仅在编写数学公式时有效,如果在其他环境中调用它们会产生错误。

当「`context`」在处理过程中警告我们产生了错误时,我们该怎么办?首先,显然是要确定错误是什么。为此,我们可能需要分析处理过程中生成的「`.log`」文件,尽管通常这并非必要,因为错误消息将在我们运行「`context`」的同一个终端中可见。

```

3  \setuppapersize[S5] % 纸张大小
4
5  \setupbodyfont
6  [modern,12pt] % 字体 = Latin Modern 12 点
7
8  \setuphead % 章节的格式
9  [chapter]
10 [style=\bfc]
11
12 >> \starttext % 开始文档内容
13
14 \startchapter
15 [title=西班牙的雨.....]
16
17 你真好让我来。
18 再说一遍哪里下雨?
19 在平原上在平原上。
20 那个该死的平原在哪里?
21 在西班牙在西班牙。
22 西班牙的雨主要落在平原上。
错误消息顶部行末尾的控制序列从未被 \def'ed。你可以继续我会忘记任何未定义的东西。
mtx-context | fatal error: return code: 1

```

图 2.2 编译错误情况下的屏幕输出

例如,如果在我们的测试文件「rain.tex」中,错误地将 `\starttext` 写成了 `\startxt`(只有一个『t』),这是一个非常常见的错误,当运行「context rain」时,处理将停止,在终端中我们可以看到图 2.2 中显示的信息¹(注意该图只显示了终端输出的末尾,前面会有许多其他“样板”输出行)。在那里我们可以看到源文件的行被编号,在其中一行(这里是第 12 行),编译器在行号和文本之间添加了「>>」,以指示这是它发现错误的行。文件「rain.log」将给我们更多线索。在这个例子中,「rain.log」不是一个大文件,因为被编译的源文件很小;在其他情况下,它可能包含几乎难以应付的大量信息。但我们必须深入研究它。如果我们用文本编辑器打开「rain.log」²,我们会看到它存储了 ConTeXt 正在做的所有事情。我们需要在那里找到以错误消息开头的行,为此我们可以使用文本编辑器的搜索功能。我们将寻找「tex error」,这会带我们找到以下行:

```

tex error    > tex error on line 12 in file ./rain-error.tex: |
               Undefined control sequence

<line 3.12>
  \starttext
  % Begin document contents

```

注意:「rain.log」文件中告诉我们错误的行很长。为了使这一行在此页面上看起来美观,考虑到本页的宽度,我将其分成了两行。字符『|』表示我分割的位置。

如果我们注意错误消息的三行,我们看到第一行告诉我们是哪一行产生了错误(第 12 行)以及错误的类型:「Undefined control sequence」,也就是:未知的控制序列,换句话说,未知的命令。日志文件接下来的两行向我们显示了第 12 行,并在产生

¹ 注意:错误报告的确切格式可能会随时间变化,但你应该会看到类似的信息。

² 如果你的文本编辑器能够自动搜索并显示此错误消息,如果你已经“升级”到从编辑器内部运行 ConTeXt,那就可以。例如,如果你使用带有「AUCTeX」包的 Emacs,并使用「Ctrl-C Ctrl-C」让 Emacs 为你编译文档,那么输入「Ctrl-C `」会将你的编辑窗口分成两部分:一部分会显示错误消息,而另一部分会显示有问题的 ConTeXt 代码行。

错误的位置进行了分割。因此,毫无疑问错误在于 `\startext`。我们仔细阅读,凭借运气和经验,我们会意识到我们写了「startext」而不是「startttext」(有两个『t』)。

想想看,计算机非常擅长且非常快速地执行指令,但非常不擅长读取我们的思想,单词「startext」与「startttext」不同。程序知道如何执行后者,而不是前者。它不知道如何处理那个。

其他时候,找到错误不会那么容易。特别是当错误在于某事物已经开始但未指示其必须结束的位置时。有时,我们不应该在「.log」文件中搜索「tex error」,而应该寻找一个星号。文件中行首的这个字符与其说是致命错误,不如说是一个警告。然而,警告有助于找到错误。

如果「.log」文件中的信息不够,我们需要逐位检查我们的主文件,寻找错误。一个很好的策略是更改 `\stoptext` 命令的位置:记住 ConT_EXt 在遇到此命令时会停止处理文本。因此,如果我将一个 `\stoptext` 大致放在文件中间并编译,只有前半部分会被编译;如果错误再次出现,那么我知道它在源文件的前半部分,如果没有,则意味着它在后半部分... 如此类推,通过逐位更改 `\stoptext` 命令的位置,我们将能够找到错误所在。一旦找到它,我们就可以尝试理解并纠正它,或者,如果我们无法理解为什么会产生错误,至少通过找到它所在的位置,我们可以尝试以另一种方式编写以避免重现它。

在实践中,当用 ConT_EXt 制作相对较长的文档时,通常会在起草文档的过程中不时编译,这样如果它“抛出”错误,我们就会大致清楚自上次处理文件以来我们添加或更改了什么,以及为什么 ConT_EXt 会抛出错误。换句话说,通常建议在创建和/或编辑 ConT_EXt 文档时,每隔几分钟重新编译一次。

第3章 ConTeXt 的命令与其他基本概念

Table of Contents: 第1节 ConTeXt 的保留字符; 第2节 命令本身; 第3节 命令的作用范围; 3.1 需要或不需要指明作用范围的命令; 3.2 需要明确指示开始和结束位置的命令(环境); 第4节 命令的操作选项; 4.1 可以以多种不同方式工作的命令; 4.2 配置其他命令工作方式的命令(\setupSomething); 4.3 设置可配置命令的定制版本(\defineSomething); 第5节 命令语法和选项概要,以及调用时方括号和花括号的使用; 第6节 ConTeXt 命令的官方列表; 第7节 定义新命令; 7.1 定义新命令的通用机制; 7.2 创建新环境; 第8节 其他基本概念; 8.1 组; 8.2 尺寸; 第9节 ConTeXt 的自学方法;

我们已经看到,在源文件中需要向 ConTeXt 解释我们希望如何转换稿件的指令。这些指令可以称为「命令」、「宏」或「控制序列」。

从 ConTeXt 内部运作的角度来看(实际上是 T_EX 的运作),原语和宏之间存在区别。原语是一个无法分解为其他更简单指令的简单指令。宏是一个可以分解为其他更简单指令的指令,而这些更简单的指令又可能进一步分解为更简单的指令,依此类推。ConTeXt 的大多数命令实际上都是复杂的宏。从程序员的角度来看,宏和原语之间的区别很重要。但从用户的角度来看,这个问题不那么重要:在这两种情况下,我们拥有的是指令,无需担心它们在底层如何运作。因此,ConTeXt 文档通常从用户角度谈论命令,从程序员角度谈论宏。由于本入门书仅从用户角度出发,我将交替使用这两个术语,视它们为同义词。

命令 是给 ConTeXt 去做某事的指令;我们通过它们控制 程序的执行。KNUTH, T_EX 之父,使用术语控制序列 来指代原语和宏,我认为这是其中最准确的术语。当我认为区分控制符号和控制词 很重要时,我会使用它。

ConTeXt 的指令基本上有两种:保留字符和真正意义上的命令。

第1节 ConTeXt 的保留字符

当 ConTeXt 读取仅由文本字符组成的源文件时,由于它是一个文本文件,ConTeXt 需要某种方式来区分哪些是要格式化的实际文本,哪些是它必须执行的指令。ConTeXt 的保留字符正是使其能够进行这种区分的关键。原则上,ConTeXt 会假定源文件中的每个字符都是要处理的文本,除非该字符是 11 个应被视为指令 的保留字符之一。

只有 11 个指令? 不,只有 11 个保留字符;但由于其中一个字符,即「\」字符,其功能是表示紧随其后的一个或多个字符是指令,那么实际上潜在的命令数量是无限的。ConTeXt 有超过 5700 个命令;幸运的是,大多数人只需要了解其中非常小的一部分就能制作出漂亮的文档!

保留字符如下：

$\backslash \% \{ \} \# \sim | \$ _ \wedge \&$

ConTeXt 对它们的解释如下：

- $\backslash$ 这个字符对我们来说是最重要的：它表示紧随其后的内容不能被解释为文本，而应被解释为指令。它被称为「转义字符」或「转义序列」（尽管它与大多数键盘上的「Esc」键无关）。¹
- $\%$ 百分号告诉 ConTeXt，从该字符到行尾的内容是注释，不得处理或包含在最终的格式化文件中。在源文件中引入注释非常有用。注释可以帮助解释为什么以某种方式做了某事，这对于完成后的源文件非常有用，以便在日后修订时，有时我们记不起为什么做了某事；或者它也可以作为提醒自己的备忘录，提醒可能需要修改的内容。它甚至可以用来帮助定位源文件中某个错误的原因，因为通过在行首放置注释标记，我们可以将该行排除在编译之外，从而判断是否是那一行导致了错误；它也可以用来让我们在同一个宏的两个不同版本之间进行选择，从而在编译后得到不同的结果；或者阻止编译某个我们不确定的片段，但又不用从源文件中删除它，以防日后想恢复它... 等等。一旦我们意识到源文件可以包含只有我们自己才能看到的文本，我们对这个字符的用途就只受限于我们自己的想象力了。我承认，当唯一的文本写作工具是文字处理器时，这是我最为怀念的功能之一。
- $\{$ 此字符开启一个「组」。组是受某些特性影响的文本块。我们将在 [节 8.1](#) 中讨论它们。
- $\}$ 此字符关闭先前用 $\{$ 开启的组。
- $\#$ 此字符用于定义宏。它指代宏的参数。参见本章的 [节 7.1](#)。
- $\sim$ 此字符在文档中引入一个空白，以防止换行，这意味着由 $\sim$ 字符分隔的两个单词将始终保留在同一行。你可能已经知道这个概念，称为「不间断空格」。我们将在 [节 3.1](#) 中讨论这个指令及其使用位置。
- $|$ 此字符用于表示由分隔元素连接的两个单词构成一个复合词，该复合词可以按音节分割第一个组成部分，但不能分割第二个组成部分。参见 [节 4](#)。
- $\$$ 此字符是数学模式的开关²。如果数学模式未启用，它会启用该模式；如果已启用，则会禁用它。在数学模式下，ConTeXt 应用一些与普通模式不同的字

¹ 在计算机术语中，影响后续字符解释的字符称为转义字符。相比之下，键盘上的转义键之所以这么叫，是因为它生成 ASCII 码中的第 27 号字符，该字符在此 ASCII 编码中用作转义字符。如今，转义键的用途更多地与取消正在进行的操作的概念相关联。

² 『\$』是 Knuth 在 T_EX 中选择的用于切换进入或退出数学模式的字符；然而，虽然在 ConTeXt 中可以使用 $\$$ 切换进入或退出数学模式，但命令 $\backslash im$ （用于「行内」数学）和 $\backslash startformula \dots \backslash stopformula$ （用于「显示」数学）是（其中）推荐的用 ConTeXt LMTX 排版数学的方式。在你的 ConTeXt 发行版中，你应该能找到文件 `mathincontext-paper.pdf`；该文档的第 1 章提供了排版数学的基础知识，文档的其余部分提供了更多的细节。

体和规则,旨在优化数学公式的书写。尽管书写数学是 ConT_EXt 一个非常重要的用途,但我不会在本入门书中进一步展开。作为一个文科生,我对此力不从心!

- 此字符(下划线)在数学模式中用于表示后面的内容是下标。例如,要得到 x_1 , 我们需要写 `$x_1$`。
- ^ 此字符在数学模式中用于表示后面的内容是上标。例如,要得到 $(x + i)^n$, 我们需要写 `$(x+i)^{n^3}$`。
- & 『&』字符在 Plain T_EX 的许多地方使用。然而,在 ConT_EXt LMTX 中,它似乎是特殊字符的唯一地方是当使用 `\starttable... \stoptable` 制作表格时,用于分隔列条目。此外,这种制作表格的方式已不再推荐,因此你可能永远不会在 ConT_EXt LMTX 中看到 & 不是它自身的情况。可以说,& 在 LMTX 中不是一个保留字符,因为(除了在 `\starttable... \stoptable` 环境中)你不需要在其前面加 『\』 来使用它。

可以推测,在选择哪些字符作为保留字符时,会选择大多数键盘上都有但在书面文本中不常用的字符。然而,尽管不常见,但总有可能其中一些字符会出现在我们的文档中;例如,当我们要写某物花费 100 美元 (\$100),或者在西班牙,2018 年 65 岁以上驾驶者的百分比为 16% 时。在这些情况下,我们不能直接写保留字符,而必须使用一个命令,该命令将在最终文档中正确地输出保留字符。每个保留字符对应的命令见表 3.1。

保留字符	生成它的命令
\	<code>\backslash</code>
%	<code>\%</code>
{	<code>\{</code>
}	<code>\}</code>
#	<code>\#</code>
~	<code>\lettertilde</code>
	<code>\ </code>
\$	<code>\\$</code>
_	<code>_</code>
^	<code>\letterhat</code>
&	<code>\&</code>

表 3.1 书写保留字符

获取保留字符的另一种方法是使用 `\type` 命令。此命令将其作为参数的内容发送到最终文档,而不以任何方式处理它,因此也不对其进行解释。在最终文档中,从 `\type` 接收的文本将以计算机终端和打字机特有的等宽字体显示。

通常,我们会将要显示的文本放在花括号内作为 `\type` 的参数。然而,当此文本本身包含左花括号或右花括号时,我们必须更加小心;`\type` 会配对花括号,一旦它看到的右花括号数

量与左花括号相等,它就结束了。例如,`\type{a{b}c}` 输出 `a{b}c`,但 `\type{a}{b{c}}` 输出 `abc`;注意『a』后面的右花括号结束了 `\type` 命令,而 `b{c}` 作为普通文本输出。(为什么这里的括号会消失将在后面解释。)当我们想逐字输出花括号时,使用其他字符来分隔逐字文本更安全。例如:`\type*{*,`或 `\type+}{+}`。

如果我们错误地直接使用了保留字符(而不是用于其预期目的),因为我们忘记了它是保留字符而不能像普通字符一样使用,那么可能发生三种情况:

1. 最常见的是,编译时产生错误。
2. 我们得到意想不到的结果。这尤其在「~」和「%」时发生;在前一种情况下,最终文档中不会出现我们期望的「~」,而是会插入一个空白;在后一种情况下,从「%」开始,同一行的所有内容将停止被处理。不当使用「\」也可能产生意想不到的结果,如果它或紧随其后的字符组成了一个 ConT_EXt 知道的命令。然而,更常见的是,不当使用「\」会产生编译错误。
3. 没有问题发生:这发生在两个主要用于数学的保留字符(`_` `^`)以及 `&` 上:如果在这些环境之外使用,它们会被视为普通字符。

第3点是我的结论。事实上,我在 ConT_EXt 文档中任何地方都没有找到 关于可以直接使用这些保留字符的说明;然而,在我的测试中,当这样做时,我没有看到任何错误;这与例如在 L^AT_EX 中不同。



第2节 命令本身

命令本身总是以「\」字符开头。根据转义序列之后的内容,可以分为:

- a. **控制符号**。控制符号以转义序列(「\」)开头,并且仅由一个非字母字符组成,例如「\,」、「\1」、「\'」或「\%」。任何严格意义上不是字母的字符或符号都可以是控制符号,包括数字、标点符号、符号甚至空格。在本文档中,为了表示空格(空白)当需要强调其存在时,我使用的符号是 `_`。实际上,「_」(反斜杠后跟一个空格)是一个常用的控制符号,我们很快就能看到。

空白或空格是一个「不可见」字符,这在像本文这样的文档中是一个问题,因为有时我们需要明确指定源文件中要写什么。Knuth 已经意识到了这个问题,并在他的「《T_EX 书》」中开创了使用「_」符号表示重要空格的惯例。因此,例如,如果我们想表明源文件中的两个单词需要用两个空格分隔,那么我们会写「word₁__word₂」。

- b. **控制词**。如果反斜杠后面的字符是一个(严格意义上的)字母,则该命令将是一个控制词。这类命令数量最多,其特点是命令名称只能由大写和/或小写字母组成;不允许使用数字、标点符号或任何其他类型的符号。另一方面,请记住,ConT_EXt 区分大小写,这意味着 `\mycommand` 和 `\MyCommand` 命令是不同的。但是 `\MyCommand1` 和 `\MyCommand2` 将被视为同一命令的两种用法,因为『1』和『2』不是字母,因此不是命令名称的一部分。

ConT_EXt 参考手册中没有关于命令名称的规则,「ConT_EXt Standalone」中包含的其他「手册」中也没有。我在上一段中陈述的是基于 T_EX 中发生的情况得出的结论(此外,在 T_EX 中,像带重音元音这样不出现在英语字母表中的字符不被视为「字母」)。这条规则可以很好地解释命令名称后空白的吸收问题。

当 ConT_EXt 读取源文件并找到一个转义字符(「\」)时,它知道后面会跟着一个命令。然后它读取转义序列后面的第一个字符。如果该字符不是字母,则意味着该命令是一个控制符号,并且仅由这第一个符号组成。但另一方面,如果转义序列后的第一个字符是字母,那么 ConT_EXt 将继续读取每个字符,直到找到第一个非字母字符,然后它就知道命令名称已经结束。这就是为什么作为控制词的命令名称不能包含非字母字符。

当命令名称末尾的「非字母」是空格时,假定该空格不是要处理的文本的一部分,而是专门插入以指示命令名称结束的位置,因此 ConT_EXt 会丢弃这个空格。这会产生一个令 ConT_EXt 初学者感到惊讶的效果,因为当相关命令的效果是在最终文档中写入某些内容时,命令的输出会与下一个单词连接在一起。例如,源文件中的以下两个句子

```
Knowing \TeX helps with learning \ConTeXt.
Knowing \TeX, although not essential, helps with learning \ConTeXt
```

分别产生以下结果:

```
Knowing TEXhelps with learning ConTEXt.
Knowing TEX, although not essential, helps with learning ConTEXt.1
```

注意,在第一种情况下,单词「T_EX」与后面的单词连接在一起,而在第二种情况下则没有。这是因为,在源文件的第一种情况下,命令名 `\TeX` 之后的第一个「非字母」是一个空格,该空格被抑制,因为 ConT_EXt 假定它只是为了指示命令名称的结束;而在第二种情况下,后面是一个逗号,由于逗号不是空格,因此没有被抑制。

另一方面,仅仅添加一个额外的空格并不能解决这个问题,例如,写

```
Knowing \TeX helps with learning \ConTeXt2
```

也解决不了问题,因为 ConT_EXt 的一条规则(我们将在节 2.1 中看到)是,一个空格会「吸收」它后面的所有空格和制表符。因此,当我们遇到这个问题时(幸运的是不常发生),我们必须确保命令名称后的第一个「非字母」不是空格。有两个候选:

- 保留字符「{ }」。如前所述,保留字符「{」开启一个组,「}」关闭一个组,因此序列「{ }」引入了一个空组。空组对最终文档没有影响,但它帮助 ConT_EXt 知道它前面的命令名称已经结束。或者,我们也可以在相关命令周围创建一个组,例如写「{\TeX}」。无论哪种方式,结果都是 `\TeX` 之后的第一个「非字母」不是空格。

¹ 注意:在本入门中,为了说明某个问题,有时会同时写出源代码片段及其编译结果,此时遵循两个惯例:有时代码和编译结果并排放置在两栏段落中;其他时候,代码用深紫红色书写(本文档通常用此颜色表示 ConT_EXt 命令),编译结果用红色表示。

² 关于「」符号,请回想 page 26 的注释。

- 控制符号「_」(反斜杠后跟一个空格,见 [page 26](#) 的注释)。此控制符号的效果是在最终文档中插入一个空格。为了正确理解 ConTeXt 的逻辑,花点时间看看当 ConTeXt 遇到一个控制词(例如 `\TeX`)后跟一个控制符号(例如「_」)时会发生什么,可能是值得的:
 - ConTeXt 遇到 \ 字符后跟一个『T』,并且知道这个反斜杠出现在控制词之前,它会继续读取字符,直到遇到一个「非字母」,当遇到引入下一个控制符号的 \ 字符时,就会发生这种情况。
 - 一旦它知道命令名称是 `\TeX`,它就运行该命令并在最终文档中打印 `\TeX`。然后它返回到停止读取的位置,检查第二个反斜杠后面的字符。
 - 它检查发现是一个空格,即一个「非字母」,这意味着控制序列恰好就是那个,因此它可以运行它。它运行了,并插入了一个空格。
 - 最后,它再次返回到停止读取的位置(作为控制符号的空格),并从那里继续处理源文件。

我已经比较详细地解释了这个机制,因为空格的消除常常令新手感到惊讶。然而,应该注意的是,这个问题相对较小,因为控制词通常不直接打印到最终文档,而是影响格式和外观。相比之下,控制符号很常见地向最终文档打印内容。

还有第三种避免空格问题的方法,即(以 `\TeX` 风格)定义一个类似的命令,并在命令名称的末尾包含一个「非字母」。例如,以下序列:

```
\def\txt-{\TeX}
```

将创建一个名为 `\txt` 的命令,该命令的作用与命令 `\TeX` 完全相同,并且只有在后面跟一个连字符时才能正常工作:`\txt-`。这个连字符在技术上不是命令名称的一部分,但除非名称后面有连字符,否则此命令将无法工作。为什么会这样与定义 `\TeX` 宏的机制有关,这里解释起来太复杂了。但它确实有效:一旦定义了这个命令,每次我们使用 `\txt-` 时,ConTeXt 会将其替换为 `\TeX`,同时删除连字符,但在内部使用它来知道命令名称已经结束,因此紧随其后的空格不会被删除。

这个「技巧」对于 `\define` 命令(这是 ConTeXt 专门用于定义宏的命令)将无法正常工作。

第3节 命令的作用范围

3.1 需要或不需要指明作用范围的命令

许多 ConTeXt 命令,尤其是那些影响字体格式特性(粗体、斜体、小型大写字母等)的命令,不带任何参数,因为它们并非设计为仅应用于特定文本。它们仅限于开启某项功能(粗体、斜体、无衬线体、某种字体大小等)。

此类命令的效果可以通过两种方式终止。一种方式是遇到另一个禁用它或启用与其不兼容的另一个功能的命令。例如,命令 `\bf` 启用粗体,文本的粗体可以通过遇到一个不兼容的命令(例如 `\tf` 或 `\it`)来终止。

限制此类命令范围的另一种方法是在组内执行它们(参见节 8.1);这些命令的效果在它们所在的组关闭时终止。因此,为了使这些命令只影响一部分文本,通常的做法是生成一个包含该命令和我们希望它影响的文本的组。通过将文本放在花括号之间来创建一个组。因此,以下文本

```
In {\it The \TeX Book}, {\sc Knuth}
explained everything you need to know
about \TeX.
```

```
In The TEX Book, KNUTH explained everything
you need to know about TEX.
```

创建了两个组,一个用于指定 `\it`(斜体)命令的范围,另一个用于指定 `\sc`(小型大写字母)命令的范围。

与这类命令相反,还有其他命令,由于它们产生的效果或其他原因,需要明确指示它们将应用于哪些文本。在这些情况下,要受命令影响的文本被放在紧跟在命令之后的花括号内。作为例子,我们可以提到 `\framed`: 此命令在其所取参数的文本周围绘制一个框架,因此

```
\framed{Tweedledum and Tweedledee}
```

将产生

```
Tweedledum and Tweedledee
```

注意,虽然在第一组命令(那些不需要参数的命令)中,有时也使用花括号来指定作用域;但括号对于命令的工作不是必需的。该命令被设计为从其出现的位置开始应用。因此,当使用括号确定其应用范围时,命令被放在这些括号之内,这与第二组命令不同,在第二组命令中,框定命令要应用的文本的括号出现在命令之后。

对于 `\framed` 命令,很明显它产生的效果需要一个参数——要应用该效果的文本。在其他情况下,命令属于哪种类型取决于程序员。因此,例如, `\it` 和 `\color` 命令的作用非常相似: 它们将某种特性(格式或颜色)应用于文本。但决定将前者编程为不带参数,而将后者编程为带参数的命令。

3.2 需要明确指示开始和结束位置的命令(环境)

有些命令确定其作用范围的方式不是使用花括号,而是明确指示它们开始应用的点以及效果停止的点。因此,这些命令成对出现: 一个指示命令何时启用,另一个指示此操作何时必须停止。使用「start」后跟命令名称表示操作的开始,使用「stop」后跟命令名称表示结束。例如,命令「itemize」变为 `\startitemize` 表示列表项的开始, `\stopitemize` 表示其结束。

在 ConT_EXt 官方文档中,这些命令对没有特殊的名称。参考手册和入门书只是称它们为「start ... stop」。有时它们被称为环境,这是 L^AT_EX 对类似结构的称呼,尽管这样做的缺点是,在 ConT_EXt 中,术语「环境」用于其他东西(一种特殊类型的文

件,我们将在节 6 中讨论多文件项目时看到)。即便如此,由于术语“环境”很清晰,并且上下文可以很容易地区分我们是在谈论环境命令 还是环境文件,我将使用这个术语。

因此,环境由一个开启或开始它的命令和另一个关闭或结束它的命令组成。如果源文件包含一个开启环境的命令,但随后没有关闭,通常会产生一个错误。¹ 另一方面,这类错误更难发现,因为错误可能发生在距离开启命令很远的地方。有时「.log」文件会向我们显示未正确关闭的环境开始的行;但其他时候,环境未关闭意味着 ConT_EXt 误解了某个段落,而不是在那个有问题的环境中,这意味着「.log」文件对我们查找问题所在没有太大帮助。

环境可以嵌套,这意味着可以在现有环境内开启另一个环境,但在嵌套环境的情况下,环境需要在其被开启的环境内部关闭。换句话说,环境关闭的顺序必须与它们开启的顺序一致。我相信下面的例子应该能清楚地说明这一点:

```
\startSomething
...
\startSomethingElse
...
\startAnotherSomethingElse
...
\stopAnotherSomethingElse
\stopSomethingElse
\stopSomething
```

在示例中,您可以看到「AnotherSomethingElse」环境是在「SomethingElse」环境内部开启的,因此也需要在其内部关闭。否则,在编译文件时会产生错误。我们可以说环境需要「正确嵌套」。

通常,被设计为环境 的命令是实现某些更改的命令,这些更改旨在应用于不小于段落的文本单元。例如,更改边距的「narrower」环境仅在段落级别应用时才有意义;类似地,「framedtext」环境为一个或多个段落添加框架。后一个环境可以帮助我们理解为什么有些命令被设计为环境,而另一些被设计为单个命令:如果我们想给一个或多个单词(都在同一行)加框,我们会使用命令 `\framed`,但如果我们想给整个段落(或多个段落)加框,那么我们会使用「framedtext」环境。

另一方面,位于特定环境内的文本通常构成一个组(参见节 8.1),这意味着如果在环境内发现一个激活命令(属于那些应用于之后所有文本的命令,例如 `\it`),此命令将仅应用直到找到它的环境结束;事实上,ConT_EXt 有一个未命名的环境,以 `\start` 命令开始(后面没有其他文本;只有 `start`。这就是为什么我称之为未命名环境),并以 `\stop` 命令结束。我怀疑这唯一的目的是创建一个组。

我没有在 ConT_EXt 文档中的任何地方读到环境的效果之一是对其内容进行分组,但这是我对许多预定义环境进行测试的结果,尽管我必须承认我的测试不够详尽。我只是随机选择了一些环境进行了检查。然而,我的测试表明,这样的陈述(如果正确的话)仅适用于某些预定义环



¹ 但并不总是;这取决于所讨论的环境以及文档其余部分的情况。ConT_EXt 在这方面与 L^AT_EX 不同,后者严格得多。

境:那些用 `\definestartstop` 命令创建的环境(在节 7.2 中解释)不会创建任何组,除非在定义新环境时我们包含了创建组所需的命令(参见节 8.1)。

我还假设我称之为未命名 的环境(`\start`)只是为了创建一个组:它确实创建了一个组,但它是否有其他用途我不知道。这是参考手册中未记录的命令之一。

第4节 命令的操作选项

4.1 可以以多种不同方式工作的命令

许多命令可以以不止一种方式工作。在这种情况下,总有一种预定的工作方式,可以通过在命令名称后的括号中指示所需操作的参数来改变。

我们在上一节提到的 `\framed` 命令中找到了一个很好的例子。此命令在其所取参数的文本周围绘制一个框架。默认情况下,框架的高度和宽度与其应用的文本相同;但我们可以指定不同的高度和宽度。因此,我们可以看到默认 `\framed` 的工作方式:

```
\framed{Tweedledum}
```



与定制版本的工作方式之间的区别:

```
\framed  
[width=3cm, height=1cm]  
{Tweedledum}
```



在第二个示例中,我们在方括号内为包围其参数文本的框架指定了特定的宽度和高度。在这些括号内,不同的配置选项用逗号分隔;两个或多个选项之间的空格甚至换行(只要不是双换行)都不予考虑,因此,例如,同一命令的以下四个版本产生完全相同的结果:

```
\framed[width=3cm,height=1cm]{Tweedledum}  
  
\framed[width=3cm, height=1cm]{Tweedledum}  
  
\framed  
[width=3cm, height=1cm]  
{Tweedledum}  
  
\framed  
[width=3cm,  
height=1cm]  
{Tweedledum}
```


许多人可能会发现最后一个版本最容易阅读:我们一眼就能看出有多少选项以及它们是如何使用的。在这样一个只有两个选项的示例中,这可能看起来不那么重要;但在选项列表很长的情况下,如果每个选项在源文件中都占一行,那么更容易理解源文件要求 ConT_EXt 做什么,并且在必要时也更容易发现潜在的错误。因此,最后一种(或类似的)编写命令的格式是用户『首选』的。

关于配置选项的语法,请参见后面的(节 5)。

4.2 配置其他命令工作方式命令(\setupSomething)

我们已经看到,支持多种工作方式的命令总是有一种默认的工作方式。如果这些命令之一在我们的源文件中被多次调用,而我们想为所有这些调用更改默认值,那么与其每次调用命令时更改这些选项,不如更改默认值更方便、更高效。为此,几乎总是有一个可用的命令,其名称以 \setup 开头,后跟我们想要更改其默认选项的命令名称。

我们在本节中一直用作示例的 \framed 命令仍然是一个很好的例子。因此,如果我们在文档中使用了大量框架,但它们都需要精确的尺寸,那么最好重新配置 \framed 的工作方式,使用 \setupframed 来完成。因此

```
\setupframed
[
  width=3cm,
  height=1cm
]
```

将确保从那时起,每次我们调用 \framed 时,默认情况下它将生成一个宽 3 厘米、高 1 厘米的框架,而无需每次都明确指示。

ConT_EXt 中大约有 300 个命令允许我们配置其他命令的工作方式。因此,我们可以配置框架(\framed)、列表(\itemize)、章节标题(\chapter)、节标题(\section)等的默认工作方式。

4.3 设置可配置命令的定制版本(\defineSomething)

继续以 \framed 为例,很明显,如果我们的文档使用几种不同类型的框架,每种都有不同的尺寸,那么理想情况下,我们可以预定义 \framed 的不同配置,并将它们与特定名称关联起来,以便根据需要进行选择使用其中之一。我们可以在 ConT_EXt 中使用 \defineframed 命令来完成,其语法为:

```
\defineframed[名称][配置]
```

其中名称 是分配给要配置的特定类型框架的名称;而配置 是与该名称关联的特定配置。

这一切的效果是,指定的配置与我们已经建立的名称相关联,该名称在所有意图和目的上都将作为一个新命令工作,并且我们可以在任何原本可以使用原始命令(`\framed`)的上下文中使用它。

这种可能性不仅存在于 `\framed` 命令的具体情况,也存在于许多具有 `\setup` 可能性的命令。`\defineSomething + \setupSomething` 的组合是一种赋予 ConTeXt 极致灵活性和强大功能的机制。如果我们详细检查 `\defineSomething` 命令的作用,我们会看到:

- 首先,它克隆一个支持多种配置的特定命令。
- 然后,它将该克隆与一个新命令的名称关联起来。
- 最后,它为克隆设置一个预定的配置,与原始命令的配置方式不同。

在我们给出的示例中,我们在创建特殊框架的同时对其进行了配置。但我们也可以先创建它,稍后再配置它,因为如前所述,一旦克隆被创建,它就可以在原始命令可以使用的任何地方使用。因此,例如,如果我们创建了一个名为「MySpecialFrame」的框架,我们可以使用 `\setupframed` 来配置它,指示要配置的实际框架。在这种情况下,`\setup` 命令将接受一个新参数,即要配置的框架的名称:

```
\defineframed[MySpecialFrame]

\setupframed
[MySpecialFrame]
[...]
```

第5节 命令语法和选项概要,以及调用时方括号和花括号的使用

总结我们到目前为止所看到的内容,在 ConTeXt 中

- 真正意义上的命令总是以「\」字符开头。
- 有些命令可以带一个或多个参数。
- 告诉命令如何工作或以某种方式影响其工作的参数,放在方括号内。
- 告诉命令必须作用于哪部分文本的参数,放在花括号内。¹

当命令仅作用于一个字母时,例如 `\buildtextcedilla` 命令(仅举例,如加泰罗尼亚语中常用的『ç』),参数周围的花括号可以省略:该命令将应用于第一个非空格的字符。

- 有些参数可以是可选的,在这种情况下我们可以省略它们。但我们绝不能改变命令所期望的参数顺序。

方括号之间的参数可以由一个或多个选项组成。选项可以是多种类型:

¹ 可以定义以其他方式界定其参数的命令,但标准的 ConTeXt LMTX 命令不常这样做。

- 由单个单词组成的选项。例如,前面我们看到如何使用 `\setuppapersize[S5]` 指定文档的纸张大小。在这种情况下,「S5」是一个单一的「单词」选项。
- 由单个「单词」组成,但带有可选重复次数的选项。例如,ConT_EXt 有一个 `\blank` 命令(用于在输出文档中放置垂直空间),它可以接受像 `\blank[2*line]` 这样的选项,以插入类似于两行所用空间量的空间。这种类型的选项实际上只是第一种类型的特例。
- 为「变量」或「键」设置值的选项。例如,前面我们看到 `\framed[width=3cm,height=1cm]{Tweedledum}`,其中选项将 `width` 设置为 3cm,将 `height` 设置为 1cm。

有些命令可以同时接受「键 = 值」类型和「单词」(包括重复)类型的参数。重要的是要记住,这两种类型的参数不能放在同一组方括号内。如果某个命令同时提供这两种类型的参数,可以使用两组方括号。前面,我们看到

```
\setuphead % 章节的格式
[chapter]
[style=\bfc]
```

这显示了 `\setuphead` 命令使用一个单词选项 `chapter` 和一个键值选项 `style=\bfc`。此示例表明值选项可以是命令(`\bfc` 命令切换到非常大的粗体字体,正如您在章节标题中看到的那样)。

当给定类型(即单词或键值)有多个选项时,相同类型的选项用逗号分隔。这允许键值选项写得比较长;例如,本节标题是使用以下选项设置的: `title={命令语法和选项概要,以及调用时方括号和花括号的使用}`。在这种情况下,因为标题中包含一个『,』,整个标题必须用花括号括起来,以表示『,』是值的一部分,而不是分隔两个键值对的逗号。

分隔选项的逗号后面可以跟任意数量的空格和/或单个换行。然而,对于键值选项,任何出现在后面『,』或终止符『』之前的空白都会成为该键值的一部分。在许多情况下,这是不希望的,并且在编译文档时可能导致意外结果。总之,除非空格确实是最后一个值的一部分,否则绝不要紧挨着逗号前面放置空格。

第 6 节 ConT_EXt 命令的官方列表

在 ConT_EXt 文档中,有一份特别重要的文档,其中列出了所有命令,指明了每个命令期望的参数数量、类型,以及所设想的各种选项及其允许的值。这份文档名为「`setup-en.pdf`」,并为每个新版本的 ConT_EXt 自动生成。它可以在名为「`tex/texmf-context/doc/context/documents/general/qrcs`」的目录中找到。

实际上,「`qrc`」有七个版本的这份文档,对应于拥有 ConT_EXt 界面的每种语言:德语、捷克语、法语、荷兰语、英语、意大利语和罗马尼亚语。对于每种语言,该目录中有两份文档:一份名为「`setup-LangCode`」(其中 `LangCode` 是两个字母的国际语言标识代码),第二份文档名为「`setup-mapping-LangCode`」。第二份文档包含按字母顺序排列的命令列表,并指示命令原型,但没有关于每个参数可能值的进一步信息。

这份文档对于学习使用 ConT_EXt 至关重要,因为我们可以从中了解某个命令是否存在;考虑到命令(或环境)+ setup 命令 + define 命令的组合,这一点尤其有用。例如,如果我知道使用 `\blank` 命令引入空行,我可以找出是否存在一个名为 `\setupblank` 的命令允许我配置它,以及另一个允许我为空行设置定制配置的命令(`\defineblank`)。

因此,「`setup-en.pdf`」有助于学习 ConT_EXt。但我更希望它能不仅告诉我们每个命令允许的参数类型和数量,还能告诉我们这些参数的用途,那就更好了。这将大大减少 ConT_EXt 文档的缺陷。有些命令允许一些可选参数,我甚至在本入门书中都没有提到,因为我不知道它们的用途,而且由于它们是可选的,也没有必要提及。这令人非常沮丧。

第7节 定义新命令

7.1 定义新命令的通用机制

我们刚刚看到,使用 `\defineSomething`,我们可以克隆一个已存在的命令并开发其新版本。从那时起,新版本将在所有意图和目的上作为一个新命令工作。

除了这种可能性(仅适用于某些特定命令,当然不少,但并非全部),ConT_EXt 还有一个定义新命令的通用机制,该机制非常强大,但在某些用途上也相当复杂。在像这样面向初学者的文本中,我认为最好从其最简单的用途开始介绍。最简单的用途是将文本片段与一个单词关联起来,以便每次该单词出现在源文件中时,都被替换为链接的文本。这将使我们一方面节省大量打字时间,另一方面,作为一个额外优势,它减少了打字时出错的可能性,同时确保相关文本始终以相同的方式书写。

让我们想象一下,例如,我们正在写一篇关于拉丁语文本头韵的论文,其中经常引用拉丁语句子「*O Tite tute Tati tibi tanta tyranne tulisti*」(哦,提图斯·塔提乌斯,你这个暴君,你给自己带来了这么多!)。这是一个相当长的句子,其中有两个单词是专有名词,以大写字母开头,而且,让我们承认,无论我们多么热爱拉丁语诗歌,我们很容易在写它时「犯错」。在这种情况下,我们可以在源文件的导言区中放入:

```
\define\Tite{\quotation{O Tite tute Tati tibi tanta tyranne tulisti}}
```

基于这样的定义,每次命令 `\Tite` 出现在我们的源文件中,它都会被指示的句子替换,并且也会像原始定义中那样放在引号内,这使我们能够确保该句子出现的方式始终保持一致。我们也可以将其写成斜体、更大的字体大小... 任何我们想要的。重要的是我们只需要写一次,并且在整个文本中,它将按照所写的方式被复制,次数不限。我们也可以创建两个版本的命令,称为 `\Tite` 和 `\tite`,具体取决于句子是否需要使用大写字母。替换文本可以是纯文本,也可以包含命令,或者形成数学表达式(至少对我来说,数学表达式中更容易打错)。例如,如果表达式 $(x_1, \dots, x_n)$ 需要经常出现在我们的文本中,我们可以创建一个命令来表示它。例如

```
\define\xvec{$(x_1,\ldots,x_n)$}
```

这样,每当 `\xvec` 出现在文本中时,它就会被替换为与之关联的表达式。

`\define` 命令的一般语法如下:

`\define[参数个数]\命令名称{替换文本}`

其中

- **参数个数**指的是新命令将带有的参数数量。如果不需要任何参数,如到目前为止的示例所示,则可以省略。
- **命令名称**指的是新命令将使用的名称。这里,关于命令名称的一般规则适用。名称可以是一个非字母的单个字符,或者一个或多个不包含任何「非字母」字符的字母。
- **替换文本**包含每次在源文件中找到新命令名称时将替换它的文本。

在定义中为新命令提供参数的可能性赋予了这种机制极大的灵活性,因为它允许根据所取参数定义可变的替换文本。

例如:让我们想象一下,我们想写一个命令来生成商业信件的开头。一个非常简单的版本可能是:

```
\define\LetterHeading{
  \rightaligned{Peter Smith}\par
  \rightaligned{Consultant}\par
  Maryborough, \date\par
  Dear Sir,\par
}
```

但最好有一个命令版本,可以在信头中写入收件人的姓名。这需要使用一个参数将收件人的姓名传递给新命令。这将需要重新定义命令如下:

```
\define[1]\LetterHeading{
  \rightaligned{Peter Smith}\par
  \rightaligned{Consultant}\par
  Maryborough, \date\par
  Dear Mr #1,\par
}
```

注意我们在定义中引入了两个更改。首先,在关键字 `\define` 和新命令名称之间,我们在方括号内包含了一个『1』(l1)。这告诉 ConT_EXt 我们正在定义的命令将带有一个参数。其次,在命令定义的最后一行,我们写了「Dear Mr #1,」,使用了保留字符「#」。这表示在替换文本中「#1」出现的位置,将插入第一个(在这种情况下也是唯一的)参数的内容。如果它有两个参数,「#1」将指代第一个参数,「#2」指代第二个参数。为了在源文件中调用该命令,在命令名称之后,参数必须包含在花括号内,每个参数有自己的花括号。因此,我们刚刚定义的命令应该以如下方式在源文件文本中调用:

`\LetterHeading{收件人姓名}`

例如:`\LetterHeading{Anthony Moore}`。

我们还可以进一步改进上述函数,因为它假设信件将寄给男性(它写的是「Dear Sir」),所以也许我们可以包含另一个参数来区分男性和女性收件人。例如:

```
\define[2]\LetterHeading{
  \rightaligned{Peter Smith}\par
  \rightaligned{Consultant}\par
  Maryborough, \date\par
  #1\ #2,\par
}
```

这样,该函数可以例如通过以下方式调用

```
\LetterHeading{Dear Ms}{Eloise Merriweather}
```

尽管这不是很优雅(从编程的角度来看)。最好为第一个参数定义符号值(男/女;0/1;m/f),以便宏本身根据该值选择适当的文本。但解释如何实现这一点需要深入更多细节,我认为初学者读者在现阶段无法理解。

7.2 创建新环境

为了创建新环境,ConTeXt 提供了 `\definestartstop` 命令,其语法如下:

`\definestartstop[名称][选项]`

在 `\definestartstop` 的官方定义中(参见节 6),还有一个我上面没有列出的额外参数,因为它是可选的,而且我未能找出它的用途。ConTeXt 入门「一瞥」和不完整的参考手册都没有解释它。我曾假设这个参数(必须在名称和配置之间输入)可能是某个现有环境的名称,作为新环境的初始模型,但我的测试表明这个假设是错误的。我查阅了 ConTeXt 邮件列表,没有看到这个可能参数的任何用法。



其中

- **名称**是新环境将使用的名称。
- **选项**允许我们配置新环境的行为。我们可以使用以下值来配置它:
 - ◊ **before** ——进入环境之前必须运行的命令。
 - ◊ **after** ——离开环境之后必须运行的命令。
 - ◊ **style** ——新环境文本将使用的样式。
 - ◊ **setups** ——使用 `\startsetups ... \stopsetups` 创建的一组命令。此命令及其用法不在本入门书中解释。
 - ◊ **color, inbetween, left, right** ——未记录的选项,我未能使它们工作。我们可以根据名称假设某些选项的作用,例如 **color**,但从我做的更多测试来看,为该选项指定某些值,我没有看到环境内部有任何变化。



环境定义的一个示例如下:


```

\definestartstop
[TextWithBar]
[before=\startmarginrule\noindeatation,
after=\stopmarginrule,
style=\ss\sl,
]

\starttext

The first two fundamental laws of human stupidity state unambiguously
that:

\startTextWithBar

\startitemize[n,broad]

\item Always and inevitably we underestimate the number of stupid
individuals in the world.

\item The probability that a given person is stupid is independent
of any other characteristic of the same person.

\stopitemize

\stopTextWithBar

\stoptext

```

结果将是：

人类愚蠢的第一条基本定律明确指出：

1. 我们总是且不可避免地低估世界上愚蠢个体的数量。
2. 一个人愚蠢的概率独立于该人的任何其他特征。

如果我们希望新环境成为一个组(节 8.1), 以便在其内部发生的任何对 ConT_EXt 正常工作的改变在离开环境时消失, 我们必须在「before」选项中包含 `\bgroup` 命令, 并在「after」选项中包含 `\egroup` 命令。

第 8 节 其他基本概念

除了命令之外, 还有其他概念对于理解 ConT_EXt 工作方式的逻辑至关重要。其中一些, 由于其复杂性, 不适合在入门书中介绍, 因此本文档将不予讨论; 但现在有两个概念应该加以研究: 组和尺寸。

8.1 组

组 是源文件中一个明确定义的片段, ConT_EXt 将其用作工作单元(稍后将解释其含义)。每个组都有一个开始和一个结束, 需要明确指示。组通过以下方式之一开始:

- 使用保留字符「`{`」或命令 `\bgroup`。
- 使用命令 `\begingroup`。
- 使用命令 `\start`。
- 通过开启某些环境(`\startSomething` 命令)。
- 通过使用保留字符「`$`」开始数学环境。

并通过以下方式之一关闭:

- 使用保留字符「`}`」或命令 `\egroup`。
- 使用命令 `\endgroup`。
- 使用命令 `\stop`。
- 通过关闭环境(`\stopSomething` 命令)。
- 通过离开数学环境(使用保留字符「`$`」)。

某些命令也会自动生成一个组,例如 `\hbox`、`\vbox`,以及一般来说,与创建盒子相关的命令¹。除了后一种情况(由某些命令自动生成的组)之外,关闭组的方式必须与开启组的方式一致。这意味着用「`{`」开始的组必须用「`}`」关闭,用 `\begingroup` 开始的组必须用 `\endgroup` 关闭。此规则只有一个例外,即用「`{`」开始的组可以用 `\egroup` 关闭,用 `\bgroup` 开始的组可以用「`}`」关闭;实际上,这意味着「`{`」和 `\bgroup` 是完全同义且可互换的,「`}`」和 `\egroup` 也是如此。

命令 `\bgroup` 和 `\egroup` 的设计目的是让用户能够定义一对命令,其中一个命令开启一个组,另一个命令关闭该组。因此,由于 TeX 语法的内部原因,这些组不能使用花括号开启和关闭,因为这会在源文件中产生不平衡的花括号,并且在编译时总是会抛出错误。

相比之下,命令 `\begingroup` 和 `\endgroup` 不能与花括号或 `\bgroup ... \egroup` 命令互换,因为用 `\begingroup` 开始的组必须用 `\endgroup` 关闭。后两个命令旨在允许更深入的错误检查。通常,普通用户不必使用它们。

我们可以有嵌套的组(一个组内的另一个组),在这种情况下,组关闭的顺序必须与它们开启的顺序一致:任何子组必须在其开始的组内关闭。也可以有空组,用「`{}`」生成。空组原则上对最终文档没有影响,但它可能有用,例如,用于指示命令名称的结束。

组的主要效果是封装其内容:通常,在组内进行的定义、格式和值分配,一旦离开该组就会被「遗忘」。这样,如果我们希望 ConTeXt 暂时改变其正常工作方式,最有效的方法是创建一个组,并在该组内改变其工作方式。因此,当我们离开该组时,之前的所有值和格式都将恢复。我们在提到像 `\it`、`\bf`、`\sc` 等命令时已经看到了一些这样的例子。但这不仅发生在格式命令上:组在某种程度上隔离了其内容,因此,在 ConTeXt 不断管理的众多内部变量中的任何一个发生的任何更改,将仅在我们处于发生该更改的组内时保持有效。同样,在组内定义的命令在组外将不可知。

因此,如果我们处理以下示例

```
\define\A{B}
```

¹ 盒子 概念也是 ConTeXt 的核心概念,但其解释未包含在本入门书中。

```

\A
{
  \define\A{C}
  \A
}
\A

```

我们将看到,第一次运行命令 `\A` 时,结果与其初始定义(『B』)相对应。然后我们创建了一个组,并在其中重新定义了命令 `\A`。如果我们现在在组内运行它,该命令将给出新的定义(在我们的示例中是『C』),但是当我们离开重新定义了命令 `\A` 的组时,如果再次运行它,它将再次输出『B』。在组内进行的定义一旦我们离开该组就被「遗忘」了。

组的另一个可能用途涉及那些专门设计应用于其后所写字符的命令或指令。在这种情况下,如果我们希望命令应用于多个字符,我们必须将希望命令或指令应用的字符括在一个组中。因此,例如,保留字符「^」,正如我们已经知道的,当在数学环境中使用时,它会将后面的字符转换为上标;因此,例如,如果我们写「`$4^2x$`」,我们将得到「 4^2x 」。但是如果我们写「`$4^{\{2x\}}$`」,我们将得到「 4^{2x} 」。

最后,分组的第三个用途是告诉 ConT_EXt,组内包含的内容必须被视为一个整体。这就是为什么之前(节 5)提到,在某些情况下,最好将某些命令选项的内容放在花括号内。

8.2 尺寸

尽管我们可以完美地使用 ConT_EXt 而不必担心尺寸,但如果不加以考虑,我们将无法充分利用所有配置可能性。因为(在很大程度上)T_EX 及其衍生品所达到的排版完美性在于系统内部对尺寸的高度关注。字符有尺寸;单词之间、行之间或段落之间的间距有尺寸;行有尺寸,边距、页眉和页脚也有尺寸。对于我们能想到的页面上的几乎每个元素,都会有一个或多个相关的尺寸。

在 ConT_EXt 中,尺寸由一个十进制数后跟测量单位表示。可以使用的单位¹见表 3.2。

表 3.2 中的前三个单位是标准长度单位;第一个在英语世界的某些地区使用,第二个和第三个在英语世界内外都使用。其余单位来自排版领域。最后两个,我没有给出等价关系,是基于当前字体的相对测量单位。一「em」等于所谓的 em-dash(「—」)的宽度,在此处使用的字体中,它比「M」大约宽 1pt。一「ex」等于一个「x」的高度。使用与字体大小相关的度量允许创建 ConT_EXt 源材料(包括用户定义的宏),其中尺寸可以根据当前字体改变大小,因此即使更改字体或大小,也能保持良好的外观。这就是为什么通常推荐使用这些单位。

除了极少数例外,我们可以使用任何我们偏好的测量单位,因为 ConT_EXt 会在内部进行转换。但是,每当指示一个尺寸时,必须指示测量单位,即使我们想指示

¹ 事实上,ConT_EXt LMTX 中还定义了其他单位,但为了避免在本入门书中涉及过多细节,这里不列出它们。

名称	ConT _E Xt 中的名称	等价关系
英寸	in	1 in = 2.54 cm
厘米	cm	2.54 cm = 1 英寸
毫米	mm	10 mm = 1 cm
点	pt	72.27 pt = 1 英寸
大点	bp	72 bp = 1 英寸
缩放点	sp	65536 sp = 1 点
十二点活字	pc	1 pc = 12 点
迪多点	dd	1157 dd = 1238 点
西塞罗	cc	1 cc = 12 迪多点
	em	
	ex	

表 3.2 ConT_EXt 中的测量单位

「0」的测量值,也必须提供一个单位,例如『0pt』或『0cm』。在数字和单位名称之间,可以留或不留空格。如果单位有小数部分,我们可以使用小数点分隔符,可以是句点『.』或逗号『,』。

测量值通常用作某些命令的选项。但只要我们知道内部变量的名称,我们也可以直接为 ConT_EXt 的某些内部测量值赋值。例如,普通段落第一行的缩进在内部由 ConT_EXt 使用一个名为 `\parindent` 的变量控制。通过明确地给这个变量赋值,我们将改变 ConT_EXt 从那时起使用的测量值。因此,例如,如果我们想消除第一行的缩进,我们只需要在源文件中写:

```
\parindent=0pt
```

我们也可以写 `\parindent 0pt`(不带等号)或 `\parindent0pt`(测量名称与其值之间没有空格),但使用等号可能使意图更明显。

然而,直接给内部测量值赋值被认为是「不优雅的」。通常,建议使用控制该变量的命令,并在源文件的导言区中这样做。相反的技术会导致源文件非常难以调试,因为并非所有配置命令都在同一个地方,并且很难在排版特性上获得一定的一致性。

ConT_EXt 使用的某些尺寸是「弹性的」:也就是说,根据上下文,它们可以采用指定范围内的值。这些测量值使用以下语法赋值:

```
\测量名称 plus 最大增量 minus 最大减量
```

例如

```
\parskip 3pt plus 2pt minus 1pt
```

通过这条指令,我们告诉 ConT_EXt 给 `\parskip`(表示段落之间的垂直距离)分配一个正常测量值 3 点,但如果页面排版需要,该测量值可以大到 5 点(3 加 2)或

小到 2 点(3 减 1)。在这些情况下,将由 ConT_EXt 在给定页面上选择分隔段落的距离,介于最小 2 点和最大 5 点之间。

使用具有拉伸和收缩能力的尺寸有利有弊。如上所述,当 ConT_EXt 将单词排列成行,或者因为当前页已满而必须开始新页时,这种灵活性可以帮助 ConT_EXt 做出选择,避免难看或不方便的断行或断页。但是(例如)赋予 `\parskip` 一些拉伸和/或收缩能力意味着段落之间的间隙可能会改变,有些人可能不喜欢这样。最终,设计文档「外观」的人将不得不根据自己的偏好做出决定。

第 9 节 ConT_EXt 的自学方法

ConT_EXt 庞大的命令和选项数量确实令人望而生畏,并可能给我们一种永远无法学好用好它的感觉。这种印象是错误的,因为 ConT_EXt 的优点之一是其处理所有结构的方式统一:学好少数几个结构,并或多或少知道其余结构的用途,当我们需要某些额外功能时,学会使用它将相对容易。因此,我认为这本入门书是一种训练,它将使我们做好准备,进行自己的研究。

要用 ConT_EXt 创建一个文档,可能只需要了解以下五件事(我们可以称之为 ConT_EXt 五大要点):

1. 知道如何创建源文件或项目;这在本书的[章 4](#)中有所解释。
2. 设置文档的主字体,并了解更改字体和颜色的基本命令([章 6](#))。
3. 了解用于结构化文档内容的基本命令,例如章、节、小节等。这些都在[章 7](#)中解释。
4. 也许要知道如何处理 *itemize* 环境,这在[节 3](#)中有一些详细的解释。
5. ... 仅此而已。

对于其余部分,我们只需要知道它是可能的。当然,如果人们不知道某个工具的存在,就不会使用它。其中许多工具在本入门书中有所解释;但最重要的是,我们可以反复观察 ConT_EXt 在面对某种类型的结构时是如何行动的:

- 首先,会有一个命令允许它这样做。
- 其次,几乎总是有一个命令允许我们配置和预设任务将如何执行;一个名称以 `\setup` 开头的命令,通常与基本命令名称相同。
- 最后,通常可以创建一个新命令来执行类似的任务,但使用不同的配置。

要查看这些命令是否存在,请查阅命令的官方列表(参见[节 6](#)),该列表还会告诉我们这些命令支持的配置选项。尽管乍一看这些选项的名称可能看起来神秘,但我们很快就会看到,有许多选项在许多命令中重复出现,并且在所有命令中以相同或非

常相似的方式工作。如果我们对某个选项的作用或工作方式有疑问,生成一个文档并测试它就足够了。我们也可以查阅丰富的 ConTeXt 文档。正如自由软件世界中常见的那样,「ConTeXt Standalone」在其发行版中包含了几乎所有文档的源代码。像「grep」(用于 GNU Linux 和类似系统)这样的实用程序可以帮助我们搜索我们有疑问的命令或选项是否在任何这些源文件中使用,以便我们手头有一个示例。

这就是学习 ConTeXt 的设想方式:本入门书详细解释了我强调的五个(实际上是四个)方面,以及更多内容:当我们阅读时,脑海中会形成对序列的清晰画面:执行任务的命令 — 配置前一个命令的命令 — 允许我们创建类似命令的命令。我们还将学习 ConTeXt 的一些主要结构,并发现它们的用途。

第4章 源文件与项目

Table of Contents: 第1节 源文件的编码; 第2节 源文件中 ConT_EXt 以特殊方式处理的字符; 2.1 空格和制表符; 2.2 换行; 2.3 连字符/短划线; 第3节 简单项目与多文件项目; 第4节 简单项目中源文件的结构; 第5节 T_EX 风格的多文件管理; 5.1 `\input` 命令; 5.2 `\ReadFile` 和 `\readfile`; 第6节 ConT_EXt 项目本身; 6.1 环境文件; 6.2 组件与产品; 6.3 项目本身; 6.4 环境、组件、产品和项目的共同方面;

正如我们已经知道的,在使用 ConT_EXt 时,我们总是从一个文本文件开始,其中除了最终文档的内容外,还包括许多指令,告诉 ConT_EXt 它必须应用哪些转换,以便从源文件生成我们最终正确格式化的 PDF 文档。

考虑到那些迄今为止只知道如何使用文字处理器的读者,我认为花一些时间研究源文件本身是值得的。或者更确切地说,是源文件们,因为有时只有一个源文件,而其他时候我们会使用多个源文件来生成最终文档。在后一种情况下,我们可以谈论「多文件项目」。(本书本身就是一个多文件项目,并且(在你学习 ConT_EXt 的某个阶段)检查其源文件的结构可能会有所启发。)

第1节 源文件的编码

源文件必须是文本文件。在计算机术语中,这是指仅包含人类可读文本且不含二进制代码的文件。这些文件也称为简单文本或纯文本文件。

由于计算机系统内部只处理二进制数,文本文件实际上由表示字符的数字组成。使用一个表将数字与字符联系起来。对于文本文件,有几种可能的表。术语文本文件编码指的是特定文本文件所使用的特定字符匹配表。

文本文件存在不同编码表是计算机科学自身历史的结果。在早期发展阶段,当计算机设备的内存和存储容量稀缺时,人们决定使用一个名为 ASCII(代表「美国信息交换标准代码」)的表,该表仅允许 128 个字符,并于 1963 年由美国标准委员会制定。显然,128 个字符不足以表示世界上所有语言中使用的所有字符和符号;但对于英语来说绰绰有余,在所有西方语言中,英语是字符最少的语言,因为它不使用任何变音符号(重音和其他在字母上方、下方或穿过字母的标记)。使用 ASCII 的优点是文本文件占用的空间非常小,因为 127(表中的最大数字)可以用 7 位二进制数表示,并且许多早期计算机使用 8 位字节作为内存测量单位。表中的任何字符都适合放入单个字节。由于字节有 8 个二进制位(bits),而 ASCII 只使用 7 位,甚至还有空间可以添加 128 个其他字符,以表示(一些)其他语言的字符。

但是当计算机的使用扩大时,ASCII 的不足变得明显,有必要开发替代表,以包含英语字母表中不熟悉的字符,如西班牙语的『ñ』、带重音的元音、加泰罗尼亚语或法语的『ç』等。另一方面,对于这些 ASCII 的替代表应该是什么,最初没有达成一致,因此不同的专业计算机公司逐渐

自行解决这个问题。因此,不仅为不同的语言或语言组创建了特定的表,而且根据创建它们的公司(微软、苹果、IBM 等)的不同,也有不同的表。

随着计算机内存的增加和存储设备成本的降低,以及相应容量的增加,创建一种可用于所有语言的单一表的想法才得以出现。但是,再次强调,实际创建的不是一个包含所有字符的单一表,而是一个标准编码(称为 Unicode)以及不同的表示方式(UTF-8、UTF-16、UTF-32 等)。在所有这些系统中,最终成为事实标准的是 UTF-8,它能够表示几乎任何现存语言,以及许多已经消亡的语言,还有众多附加符号,全部使用可变长度(1 到 4 字节)的数字,这反过来又有助于优化文本文件的大小。与使用纯 ASCII 的文件相比,这个大小并没有太大的增加。

在 X_YTeX 出现之前,基于 T_EX 的系统——它也是在美国诞生,因此以英语为母语——假定编码是纯 ASCII;因此,要使用不同的编码,您必须以某种方式在源文件中指明这一点。

ConT_EXt LMTX 假定编码为 UTF-8。然而,在较旧的计算机系统中,默认可能仍使用不同的编码。我不太确定 Windows 使用的默认编码,因为微软面向大众的策略在于隐藏复杂性(但即使隐藏,并不意味着它消失了!)。关于其默认使用的编码系统,没有太多可用信息(或者我没能找到)。

在任何情况下,无论默认编码是什么,任何文本编辑器都允许您以所需的编码保存文件。打算由 ConT_EXt LMTX 处理的源文件必须以 UTF-8 保存¹,除非有非常好的理由使用不同的编码(尽管我想不出这个理由可能是什么)。

如果我们想编写一个以其他编码编写的文件(也许是一个旧文件),我们可以:

- a. 将文件转换为 UTF-8,这是推荐的选项;有多种工具可以做到这一点;例如,在 Linux 中,命令 `iconv` 或 `recode`。
- b. 在源文件中告诉 ConT_EXt 编码不是 UTF-8。为此,我们需要使用命令 `\enableregime`,其语法为:

`\enableregime[编码]`

其中编码指的是 ConT_EXt 知道的相关文件实际编码的名称。在表 4.1 中,您将找到各种编码以及 ConT_EXt 认识它们的名称。

ConT_EXt LMTX 强烈推荐使用 UTF-8;我同意这个建议。从现在开始,在本入门书中,我们假定编码始终为 UTF-8。

除了 `\enableregime`,ConT_EXt 还包含命令 `\useregime`,它允许我们使用一种或另一种编码的代码作为参数。我没有找到关于此命令的信息,也没有找到它与 `\enableregime` 的区别,只有一些关于其使用的示例。我怀疑 `\useregime` 是为使用许多源文件的复杂项目设计的,期望并非所有源文件都具有相同的编码。但这只是一个猜测。



第 2 节 源文件中 ConT_EXt 以特殊方式处理的字符

特殊字符是我给一组不同于保留字符的字符起的名字。如节 1 所示,它们对 ConT_EXt 具有特殊含义,因此不能直接用作源文件中的字符。除此之外,还有另一组

¹ 注意 ASCII 是 UTF-8 的子集,因此如果您不需要 ASCII 中可用的字符以外的字符,您的编辑器可能指示您的文件是 ASCII 而不是 UTF-8;这完全没问题

编码	ConT _E Xt 中的名称	注释
Windows CP 1250	cp1250, windows-1250	西欧
Windows CP 1251	cp1251, windows-1251	西里尔字母
Windows CP 1252	cp1252, win, windows-1252	西欧
Windows CP 1253	cp1253, windows-1253	希腊语
Windows CP 1254	cp1254, windows-1254	土耳其语
Windows CP 1257	cp1257, windows-1257	波罗的海语
ISO-8859-1, ISO Latin 1	iso-8859-1, latin1, il1	西欧
ISO-8859-2, ISO Latin 2	iso-8859-2, latin2, il2	西欧
ISO-8859-15, ISO Latin 9	iso-8859-15, latin9, il9	西欧
ISO-8859-7	iso-8859-7, grk	希腊语
Mac Roman	mac	西欧
IBM PC DOS	ibm	西欧
UTF-8	utf	Unicode
VISCII	vis, viscii	越南语
DOS CP 866	cp866, cp866nav	西里尔字母
KOI8	koi8-r, koi8-u, koi8-ru	西里尔字母
Mac Cyrillic	maccyr, macukr	西里尔字母
其他	cp855, cp866av, cp866mav, cp866tat, ctt, dbk, iso88595, iso111, mik, mls, mnk, mos, ncc	各种

表 4.1 ConT_EXt 中的主要编码

字符, 尽管 ConT_EXt 在源文件中遇到它们时会按原样处理, 但确实以特殊规则处理它们。这组字符包括空格、制表符、换行符和连字符。

2.1 空格和制表符

在源文件中, 制表符和空格在所有意图和目的上都被同等对待。制表符(取决于您的编辑器和其设置, 您可能通过按键盘上的 Tab 键将其输入文件)将被 ConT_EXt 转换为空格。空格会被其后紧随的任何其他空格(或制表符)吸收。因此, 在源文件中写

Tweedledum and Tweedledee.

或

Tweedledum and Tweedledee.

绝对没有区别。ConT_EXt 认为这两个句子完全相同。因此, 如果我们想在单词之间引入一个额外的空格, 我们需要使用一些 ConT_EXt 命令来做到这一点。通常, 使用「_」会起作用, 意思是 \ 字符后跟一个空格。但是还有其他方法, 将在关于水平空格的章 3 中讨论。

连续空格的吸收使我们能够通过简单地增加或减少所使用的缩进来视觉上突出我们想要突出的部分来编写源文件, 并且放心地知道这不会以任何方式影响最终文档。因此, 在以下示例中

马德里七十年代末的音乐团体 `\em La Romántica Banda Local` 创作了风格 `eclectic` 且难以归类的歌曲。例如,在他们的歌曲 `El Egipcio` 中,他们说:

`\quotation{\em Esto es una farsa más que una comedia, página muy seria de la histeria musical; sueños de princesa, vicios de gitano pueden en su mano acariciar la verdad}}`, 仅仅因为词语具有内在韵律(`comedia-histeria-seria, gitano-mano`)而混合词语和短语。

您可以看到有些行稍微向右缩进。这些行是属于将以斜体显示的部分。缩进这些行有助于(作者)看到斜体结束的位置。

有些人可能会想,真是一团糟!我必须费心缩进行吗?事实上,这种特殊的缩进是由我的文本编辑器(GNU Emacs)在编辑 `ConTEXt` 源文件时自动完成的。正是这种小小的帮助让您选择使用某个文本编辑器而不是另一个。

空格被吸收的规则仅适用于源文件中连续的空格。因此,如果在源文件中的两个空格之间放置一个空组(`{ }`),尽管空组不会在最终文件中产生任何东西,但它的存在将确保这两个空格不是连续的。例如,如果我们写 `「Tweedledum {} and Tweedledee」`,我们将得到 `「Tweedledum and Tweedledee」`,如果您仔细观察,会看到前两个单词之间的空格更大。

同样的情况也发生在保留字符`「~」`上,尽管它的效果是生成一个空格,即使它实际上不是一个空格:一个空格后跟一个`~`不会吸收后者,而`~`后的空格也不会被吸收。(回想一下,`~`产生一个「不间断空格」,用于那些我们希望确保两个单词在输出的 PDF 文件中保持在同一行的情况。)

2.2 换行

在大多数文本编辑器中,当一行超过最大宽度时,会自动插入换行。我们也可以通按`「Enter」`或`「Return」`键明确地插入换行。

`ConTEXt` 对换行应用以下规则:

- a. 单个换行在所有意图和目的上等同于一个空格。因此,如果在换行之前或之后有任何空格或制表符,它们将被换行或第一个空格吸收,并且在最终文档中将插入一个简单的空格。
- b. 两个或更多连续的换行创建一个段落分隔符。为此,如果第一个和第二个换行之间只有空格或制表符(因为这些被第一个换行吸收),则认为两个换行是连续的;这简而言之意味着源文件中一个或多个完全空白的连续行(其中没有任何字符,或者只有空格或制表符)成为一个段落分隔符。

注意我说的是「两个或更多连续的换行」,然后是「一个或多个连续的空白行」,这意味着如果我们想增加段落之间的间距,不能仅仅通过插入另一个换行来实现。为此,我们需要使用增加垂直空间的命令。如果我们只想要一个额外的行距,可以使用 `\blank` 命令。但是还有其他增加垂直空间的方法。我建议您参考[节 2](#)。

在某些情况下,当一个换行变成空格时,我们最终可能会得到一些不期望和意想不到的空格。尤其是在编写宏时,很容易在我们没有意识到的情况下“溜进”一个空格。为了避免这种情

况,我们可以使用保留字符「%」,正如我们所知,它会导致其所在行的处理终止,这意味着行尾的换行也不会被处理。因此,例如,命令

```
\define[3]\Test{
  {\em #1}
  {\bf #2}
  {\sc #3}
}
```

将其第一个参数写成斜体,第二个写成粗体,第三个写成小型大写字母,会在这些参数中的每一个之间插入一个空格,而

```
\define[3]\Test{%
  {\em #1}%
  {\bf #2}%
  {\sc #3}%
}
```

不会在它们之间插入任何空格,因为保留字符 % 阻止了换行被处理并变成空格。

您可能在想「但是行首的空格呢?为什么它们不会在输出中产生空格?」。原因是 ConT_EXt 遵循与 T_EX 相同的规则:行首的空格和制表符被忽略。这使得您可以方便地缩进行,当这样做允许您以一种增强查看 ConT_EXt 源文件结构的能力的方式编写文本(或您的宏!)时。

现在您可能在想「但是本入门书中怎么有以空格开头的行?」。好问题!答案是您可以告诉 ConT_EXt 输出逐字文本;也就是说,输出与 ConT_EXt 源文件中的输入完全一致。这将在节 2.3 中更详细地讨论。但通常情况下,空格、制表符和换行的规则如上所述。

2.3 连字符/短划线

连字符是计算机键盘与印刷文本之间差异的一个很好的例子。在普通键盘上,通常只有一个字符用于短划线(或用排版术语来说,称为“规则”),我们称之为连字符(「-」);但印刷文本使用多达四种不同长度的规则:

- 短规则(连字符)(-),例如用于行尾断字时分隔音节,或用于复合词(至少在英语中),如「thought-provoking」。
- 中等长度的规则(「en 短划线」或「en 规则」)(-),比连字符稍长。它们有多种用途,包括:对于某些欧洲语言(英语较少),用于对话行的开头,或用于分隔日期或页码范围中大小数字的范围,例如「pp. 12-33」。
- 长规则(「em 短划线」或「em 规则」)(-),用作括号的替代,将一个句子包含在另一个句子中,包围句子中间的列表,以及用于许多其他情况。
- 减号(-),表示减法或负数。在 ConT_EXt 中(排版上)正确的负数排版方式是,例如,输入「`$-40$`」、「`\im{-40}`」或「`\minus40`」,而不是「`-40`」;比较 -40(前三种方式中的任何一种)与 -40 的外观。

如今,上述所有规则以及其他规则都可以作为 UTF-8 编码中的单个字符使用。但由于只有连字符可以通过键盘上的单个键生成,其他的在源文件中就不那么容易产生了。幸运的是, T_EX 看到了在我们的最终文档中包含比键盘所能产生的更多规则/短划线的必要性,并设计了一个简单的程序来实现这一点。ConT_EXt 通过添加生

成这些不同种类规则的命令来补充这一程序。我们可以使用两种方法来生成四种规则:要么使用命令的普通 ConT_EXt 方式,要么直接从键盘输入。这些程序显示在表 4.2 中:

规则类型	外观	直接输入	命令
连字符	-	-	<code>\hyphen</code>
En 短划线	—	--	<code>\endash</code>
Em 短划线	—	---	<code>\emdash</code>
减号	—	\$-\$	<code>\minus</code>

表 4.2 ConT_EXt 中的规则/短划线

命令名称 `\hyphen` 和 `\minus` 是英语中通常使用的名称。尽管印刷业的许多人称它们为『规则』,但 T_EX 的术语,即 `\endash` 和 `\emdash`,在排版术语中也很常见。「en」和「em」是排版中使用的测量单位名称¹。「en」大致(但不一定精确!)等于所用字体中『n』的宽度,而「em」大致(但不一定精确)等于所用字体中『m』的宽度。

第 3 节 简单项目与多文件项目

在 ConT_EXt 中,我们可以只使用一个源文件,其中包含我们最终文档的所有内容以及与它相关的所有细节,在这种情况下,我们谈论的是「简单项目」,或者相反,我们可以使用多个共享我们最终文档内容的源文件,在这种情况下,我们谈论的是「多文件项目」。

通常使用多个源文件进行工作的场景如下:

- 如果我们正在编写一个由多位作者合作的文档,每位作者负责自己与其他部分不同的部分;例如,如果我们正在编写一本包含不同作者贡献的纪念性出版物,或者一本期刊的某一期等。
- 如果我们正在编写一个长篇文档,其中每个部分(章节)具有相对自主性,因此这些部分的最终安排允许有多种可能性,并将在最后决定。这对于许多学术文本(手册、入门书等)来说相对频繁地发生,其中章节的顺序可能会变化。
- 如果我们正在编写多个共享某些样式特征的相关文档。
- 简单来说,如果我们正在处理的文档很大,以至于计算机在编辑或编译它时变慢;在这种情况下,将材料拆分到多个源文件中将大大加快编译速度(如果我们能单独编译每个文件的话)。
- 此外,如果我们编写了许多宏,想要应用于某些(或所有)文档,或者如果我们生成了一个控制或样式化我们文档的模板,并希望将它们应用于这些文档,等等。

¹ 我们在表 3.2 中看到 ConT_EXt 使用「em」作为测量单位,但 LMTX 不支持「en」作为单位。

第4节 简单项目中源文件的结构

在单个源文件中开发的简单项目中,结构非常简单,并围绕着必须出现在同一文件中的「text」环境。我们区分该文件的以下部分:

- **文档导言区:**从文件的第一行到「text」环境(`\starttext`)开始之前的所有内容。
- **文档主体:**这是「text」环境的内容;或者换句话说,`\starttext`和`\stoptext`之间的所有内容。

在图 4.1 中,我们看到了一个非常简单的源文件。在命令 `\starttext`(在图中,仅计算有文本的行,它在第 5 行)之前的所有内容构成导言区;`\starttext` 和 `\stoptext` 之间的所有内容构成文档主体。`\stoptext` 之后的任何内容都将被忽略。

```
% 文档的第一行

% 导言区:
% 包含文档的全局配置命令

\starttext % 文档主体从这里开始

...
... % 文档内容
...

\stoptext % 文档结束
```

图 4.1 包含简单项目的文件

导言区用于包含要影响整个文档的命令,即决定其整体配置的命令。在导言区中编写任何命令都不是必需的。如果没有,ConTeXt 将采用一个默认配置,这个配置不是很完善,但可能适用于许多文档。在一个规划良好的文档中,导言区将包含所有影响整个文档的命令,比如要在源文件中使用的宏和定制命令。在一个典型的导言区中,这可能包括以下内容:

- 文档主语言的指示(参见[节 5](#))。
- 纸张大小([节 1](#))和页面布局([节 3](#))的指示。
- 文档主字体的特性([节 3](#))。
- 要使用的章节命令的自定义([节 4](#)),以及如果需要,定义新的章节命令([节 5](#))。
- 页眉和页脚的布局([节 6](#))。
- 我们自己的宏的设置([节 7](#))。
- 等等。

导言区用于文档的整体配置;因此,任何与文档内容或可处理文本有关的东西都不应该在那里。理论上,包含在导言区中的任何可处理文本都将被忽略,尽管有时,如果它在那里,会导致编译错误。

文档主体,位于 `\starttext` 和 `\stoptext` 命令之间,包括实际内容,即可处理文本,以及不应影响整个文档的 ConTeXt 命令。

第5节 T_EX 风格的多文件管理

为了能够处理多个源文件,T_EX 包含了一个称为 `\input` 的原语,它在 ConTeXt 中也可以工作,尽管后者包含两个特定命令,在某种程度上完善了 `\input` 的功能。

5.1 `\input` 命令

`\input` 命令插入它指示的文件的内容。其格式为:

`\input 文件名`

其中文件名 是要插入的文件的名称。注意,文件名不必放在花括号之间,即使这样做也不会抛出错误。但是,它绝不应该放在方括号之间。如果文件扩展名是「`.tex`」,则可以省略。

当 ConTeXt 编译文档并找到 `\input` 命令时,它会查找指示的文件,并继续编译,就好像该文件的内容是属于调用它的文件的一部分,位于找到 `\input` 的位置。当 ConTeXt 处理完 `\input` 文件后,它返回到原始文件,并从它停止的地方继续。因此,实际结果是,通过 `\input` 调用的文件的内容被插入到调用该文件的位置。使用 `\input` 调用的文件必须在我们的操作系统中有一个有效的名称,并且名称中没有空格。ConTeXt 将在工作目录中查找它,如果在那里找不到,它将在 TEXINPUTS 环境变量(如果您已设置)中指定的目录中查找。最后,如果您的主目录中有一个名为 `texmf` 的目录,ConTeXt 也会在那里查找 `\input` 文件。如果最终找不到该文件,则应该会导致编译错误¹。

`\input` 命令最常见的用法如下:编写一个文件,我们称之为「`principal.tex`」,它将用作一个容器,通过 `\input` 命令调用构成我们项目的各个文件。这显示在以下示例中:

¹ 截至 2026 年 3 月,并不总是出现明确的错误消息;context 运行可能会因没有明显原因而突然结束。如果您检查 .log 文件或终端输出,在非常接近末尾的地方,您可能会看到一行如 `open source > level 1, order 2, name './FileName.tex'`,而没有相应的 `close source ...` 行。有 `open` 而没有 `close` 告诉您 ConTeXt 无法找到文件 `FileName.tex` 并停止了执行。


```
% 全局配置命令:

\input MyConfiguration

\starttext

\input TitlePage
\input Preface
\input Chap1
\input Chap2
\input Chap3

...

\stoptext
```

注意,对于文档的全局配置,我们调用了文件「MyConfiguration.tex」,我们假设它包含我们想要应用的全局命令。然后,在命令 `\starttext` 和 `\stoptext` 之间,我们调用包含我们文档各个部分内容的几个文件。如果在某个时刻,为了加快编译过程,我们想跳过编译一些文件,我们只需在调用该文件或那些文件的行的开头放置一个注释标记。例如,如果我们正在编写第三章,并且我们想编译它只是为了检查其中是否有错误,我们不需要编译其余部分,因此可以写:

```
% 全局配置命令:

\input MyConfiguration

\starttext

% \input TitlePage
% \input Preface
% \input Chap1
% \input Chap2

\input Chap3

...

\stoptext
```

并且只有第 3 章将被编译。注意,另一方面,更改章节的顺序就像更改调用它们的行的顺序一样简单。

当我们在一个多文件项目中排除某个文件不被编译时,我们获得了处理速度的提升,但结果是,正在编译的部分对尚未编译的其他部分的所有引用将不再起作用。参见 [节 2](#)。这些缺失的引用不会导致编译错误,但是您会在您编译的部分中看到诸如「section ??」之类的内容,对于每个对您未编译部分的引用。不要太担心这些,当您最终编译整个文档时,它们会出现。

重要的是要清楚,当我们使用 `\input` 工作时,只有主文件(调用所有其他文件的文件)必须包含 `\starttext` 和 `\stoptext` 命令,因为如果其他文件包含它们,将会出现错误。另一方面,这意味着我们不能直接编译构成项目的不同文件,而必须从主文件编译它们,主文件是存放文档基本结构的文件。

5.2 `\ReadFile` 和 `\readfile`

正如我们刚刚看到的,如果 ConT_EXt 找不到用 `\input` 调用的文件,它将产生一个错误。对于我们希望仅当文件存在时才插入文件,但允许其可能不存在的情况,ConT_EXt 提供了 `\input` 命令的一个变体。即

`\ReadFile{文件名}`

此命令在所有方面都与 `\input` 相似,唯一的例外是,如果要插入的文件未找到,它将继续编译而不产生任何类型的错误。它在语法上也与 `\input` 不同,因为我们知道使用 `\input` 时,不必将要插入的文件名放在花括号之间。但是对于 `\ReadFile`,这是必需的。如果我们不使用花括号,ConT_EXt 会认为要查找的文件名与 `\ReadFile` 命令之后的第一个字符相同,后跟扩展名 `.tex`。因此,例如,如果我们写

`\ReadFile MyFile`

ConT_EXt 将理解是要读取的文件名为「`M.tex`」,因为紧接在命令 `\ReadFile` 之后的字符(不包括我们知道在命令名称末尾被忽略的空格)是一个『`M`』。由于 ConT_EXt 通常不会找到名为「`M.tex`」的文件,并且 `\ReadFile` 在找不到文件时不会产生错误,因此 ConT_EXt 将在「`MyFile`」中的『`M`』之后继续编译,并将插入文本「`yFile`」。

`\ReadFile` 的一个更精细的版本是 `\readfile`,其格式为

`\readfile{文件名}{存在时文本}{不存在时文本}`

第一个参数类似于 `\ReadFile` 的参数:它是一个用花括号括起来的文件名。第二个参数包含如果文件存在,在插入文件内容之前要写入的文本。第三个参数包含如果找不到相关文件时要写入的文本。这意味着,根据作为第一个参数输入的文件是否找到,将执行第二个参数(如果文件存在)或第三个参数(如果文件不存在)。

`\readfile` 的用途之一是(在众多用途中)如果输入文件未找到,则在文档中放入一些信息性消息。例如,您可以使用

`\readfile{Chap23}{}{Chap23 应该在这里! 它在哪里? }`
在最终文档中给读者留下一条相当明显的消息。

第 6 节 ConT_EXt 项目本身

ConT_EXt 为多文件项目提供的第三种机制更加复杂和完整:它首先区分「项目」文件、「产品」文件、「组件」文件和「环境」文件。为了理解每种文件类型之间的关系和功能,我认为最好逐一解释它们。

6.1 环境文件

环境文件是一种存储特定样式的宏和配置的文件,该样式旨在应用于多个文档,无论它们是完全独立的文档还是复杂文档的一部分。因此,环境文件可以包含我们通常在 `\starttext` 之前写入的所有内容;即:文档的全局配置。

我保留了「环境文件」这个术语,以便不偏离 ConT_EXt 官方术语,尽管我认为更好的术语可能是「格式文件」或「全局配置文件」。

与所有 ConT_EXt 源文件一样,环境文件是文本文件,并假定扩展名为「`.tex`」,尽管如果我们愿意,可以更改它,也许改为「`.env`」。然而,通常在 ConT_EXt 中不这样做。最常见的是,环境文件通过以『`env`』开头或结尾来标识。例如:「`MyMacros_env.tex`」或「`env_MyMacros.tex`」。这样的环境文件内部看起来类似于以下内容:

```
\startenvironment MyEnvironment

\mainlanguage[en]

\setupbodyfont
[modern]

\setupwhitespace
[big]

...

\stopenvironment
```

或者换句话说,定义和配置命令包含在 `\startenvironment` 和 `\stopenvironment` 之间。在 `\startenvironment` 之后,我们立即写入我们想要标识相关环境的名称,然后包含我们希望环境组成的所有命令。

关于环境的名称,根据我的测试,我们在 `\startenvironment` 之后立即添加的名称仅仅是指示性的,如果我们不提供名称,那么什么(坏)事也不会发生。

环境文件旨在与组件和产品(将在下一节中解释)一起工作。这就是为什么可以从组件或产品中使用 `\environment` 命令调用一个或多个环境的原因。但是,此命令如果用于任何 ConT_EXt 源文件的配置区域(导言区),即使它不是打算分部分编译的源文件,也能工作。

`\environment` 命令可以使用以下两种格式之一调用:

`\environment 文件`

`\environment[文件]`

在任何一种情况下,此命令的效果都是加载作为参数的文件的内容。如果找不到该文件,ConT_EXt 将继续正常编译而不产生任何错误。如果文件扩展名是「`.tex`」,则可以省略。

6.2 组件与产品

如果我们把一本书的每个章节放在不同的源文件中,那么我们可以说章节是组件,而书是产品。这意味着组件是一个产品的自主部分,能够拥有自己的样式并且可以独立编译。每个组件都在自己的文件中,此外,还会有一个产品文件将所有组件组合在一起。

一个典型的组件文件如下:

```
\environment MyEnvironment
\environment MyMacros

\startcomponent Chapter1

\startchapter[title={第 1 章}]

...

\stopchapter

\stopcomponent
```

一个产品文件如下所示:

```
\environment MyEnvironment
\environment MyMacros

\startproduct MyBook

\component Chapter1
\component Chapter2
\component Chapter3

...

\stopproduct
```

注意,我们文档的实际内容将分布在各个「组件」文件中,而「产品」文件仅限于确定组件的顺序。另一方面,(单独的)组件和产品都可以直接编译。编译一个产品将生成一个包含该产品所有组件的 PDF 文件。相反,如果单独编译其中一个组件,它将生成一个仅包含已编译组件的 PDF 文件。

在组件文件中,在 `\startcomponent` 命令之前,我们可以使用 `\environment` 环境名称调用一个或多个环境文件。我们可以在产品文件中的 `\startproduct` 之前做同样的事情。可以同时加载多个环境文件。例如,我们可以将我们最喜欢的宏集合以及应用于我们文档的不同样式放在不同的文件中。但是请注意,当我们使用两个或多个环境时,它们按照被调用的顺序加载,因此,如果同一个配置命令包含在多个环境中,并且该命令在不同的环境中具有不同的值,则最后加载的环境的值将生效。另一方面,环境文件只加载一次,因此,在前面的示例中,从产品文件和特定组件文件中都调用了环境,如果我们编译产品,那就是加载环境的时候,

并且按照那里指示的顺序;当从任何组件调用环境时,ConTeXt 将检查该环境是否已加载,如果已加载,则环境文件将不会被再次处理。

从产品中调用的组件的名称必须是包含该组件的文件的名称,尽管如果此文件的扩展名是「.tex»,则可以省略。

6.3 项目本身

在大多数情况下,产品和组件之间的区别就足够了。尽管如此,ConTeXt 还有一个更高的级别,我们可以在其中对多个产品进行分组:这就是项目。

一个典型的项目文件大致如下:

```
\startproject MyCollection

\environment MyEnvironment
\environment MyMacros

\product Book01
\product Book02
\product Book03

...

\stopproject
```

我们需要使用项目的情况是,例如,我们需要编辑一套具有相同格式规范的书籍;或者如果我们正在编辑一本期刊:书籍的合集或期刊本身将是项目;每本书或每期期刊将是一个产品;而一本书的每个章节或一期期刊的每篇文章将是一个组件。

另一方面,项目不打算直接编译。考虑到根据定义,属于该项目的每个产品(合集里的每本书,或每期期刊)应该单独编译并生成自己的 PDF。因此,\product 命令实际上什么也不做:它只是一个给作者的提醒,提醒哪些产品属于该项目。

显然,有人可能会问,既然项目不能编译,为什么还要有项目呢?答案是项目文件将某些环境与项目关联起来。这就是为什么,如果我们在组件或产品文件中包含 \project 项目名称命令,ConTeXt 将读取项目文件并自动加载与其关联的环境。这就是为什么在项目中,\environment 命令必须放在 \startproject 之后;然而,在产品和组件中,\environment 必须放在之前 \startproduct 或 \startcomponent。

就像 \environment 和 \component 命令一样,\project 命令允许我们将项目名称放在方括号内或根本不使用方括号。这意味着 \project 文件名和 \project[文件名] 是等效的命令。

加载环境的不同方式总结

从以上内容可知,可以通过以下任何过程加载环境:

- a. 在 `\starttext` 或 `\startcomponent` 之前插入 `\environment` 环境名称命令。这将仅为了编译相关文件而加载环境。
- b. 在产品文件中的 `\startproduct` 之前插入 `\environment` 环境名称命令。这将在编译产品时加载环境,但如果其组件单独编译则不会。
- c. 在产品或环境中插入 `\project` 命令:这将加载与项目关联的所有环境(在项目文件中)。

6.4 环境、组件、产品和项目的共同方面

环境、组件、产品和项目的名称 我们已经看到,对于所有这些元素,在启动特定环境、

组件、产品或项目的 `\start` 命令之后,直接输入其名称。通常,此名称必须与包含该环境、组件或产品的文件的名称一致,因为,例如,当 ConT_EXt 编译一个产品并且根据产品文件必须加载一个环境或组件时,我们无法知道该环境或组件在哪个文件中,除非该文件与要加载的元素具有相同的名称。

否则,根据我的测试,在产品文件和环境文件中的 `\startproduct` 或 `\startenvironment` 之后写入的名称仅仅是指示性的。如果省略它,或者与文件名不匹配,什么坏事也不会发生。但是,对于组件,重要的是组件的名称与包含它的文件的名称匹配。

项目目录的结束 我们知道,默认情况下,ConT_EXt 在工作目录和 TEXINPUTS 变量

指示的路径中查找文件。然而,当我们使用 `\project`、`\product`、`\component` 或 `\environment` 命令时,假定项目有一个目录结构,其中公共元素位于父目录中,特定元素位于某个子目录中。因此,如果在工作目录中找不到指示的文件,将在其父目录中搜索,如果在那里也找不到,则在该目录的父目录中搜索,依此类推。

Global aspects of the document

第5章 页面与文档分页

Table of Contents: 第1节 页面尺寸; 1.1 设置页面尺寸; 1.2 使用非标准页面尺寸; 1.3 在文档中的任意点更改页面尺寸; 1.4 调整页面尺寸以适应其内容; 第2节 页面上的元素; 第3节 页面布局(\setuplayout); 3.1 为不同的页面组件分配大小; 3.2 调整页面布局; 3.3 使用多个页面布局; 3.4 与页面布局相关的其他事项; 3.4.1 区分奇偶页; 3.4.2 具有多栏的页面; 第4节 页码编号; 第5节 强制或建议的分页; 5.1 \page 命令; 5.2 连接某些行或段落以防止在它们之间插入分页; 第6节 页眉和页脚; 6.1 建立页眉和页脚内容的命令; 6.2 格式化页眉和页脚; 6.3 定义特定的页眉和页脚并将其链接到节命令; 第7节 在页面边缘和边距中插入文本元素;

ConTeXt 将源文档转换为正确格式化的页面。这些页面的外观、文本和空白的分布方式以及它们包含的元素,都是良好排版的基础。本章致力于讨论所有这些问题,以及一些与分页相关的其他事项。

第1节 页面尺寸

1.1 设置页面尺寸

默认情况下,ConTeXt 假定文档为 A4 尺寸,这是欧洲标准。我们可以使用 \setuppapersize 设置不同的尺寸,这是一个典型的命令,通常出现在文档导言区中。该命令的常规语法是:

`\setuppapersize[逻辑页面][物理页面]`

其中两个参数都采用符号名称。¹ 第一个参数(我称之为逻辑页面)表示排版时要考虑的页面尺寸;第二个参数(物理页面)表示要打印的页面尺寸。通常两种尺寸相同,此时第二个参数可以省略;然而,有时两种尺寸可能不同,例如,当以 8 页或 16 页为一印张打印书籍时(一种常见的印刷技术,大约在 1960 年代之前尤其常用于学术书籍)。在这些情况下,ConTeXt 允许我们区分这两种尺寸;如果我们希望在一张纸上打印多个页面,我们还可以使用 \setuparranging 命令(本入门不解释)指示要遵循的折页方案。

对于排版尺寸,我们可以指定造纸工业(或我们自己)使用的任何预定义尺寸。这包括:

¹ 回想一下,在节 5 中,我指出 ConTeXt 命令所接受的选项基本上有两种:符号名称(其含义已被 ConTeXt 知晓)或我们必须显式赋予某个值的变量。

- ISO-216 定义的 A、B 和 C 系列中的任何尺寸(从 A0 到 A10, 从 B0 到 B10, 从 C0 到 C10), 通常在欧洲使用。
- 任何美国标准尺寸: letter、ledger、tabloid、legal、folio、executive。
- ISO-217 标准定义的任何 RA 和 RSA 尺寸。
- 瑞士 SIS-014711 标准定义的 G5 和 E5 尺寸(用于博士论文)。
- 信封: 北美标准定义的任何尺寸(9 号信封到 14 号信封)或 ISO-269 标准定义的任何尺寸(C6/C5、DL、E4)。
- CD, 用于 CD 封面。
- S3-S6、S8、SM 和 SW, 用于不打算打印而是屏幕显示的文档的屏幕尺寸。

与纸张尺寸一起, 使用 `\setuppapersize` 我们可以指示页面方向:「纵向」(垂直)或「横向」(水平)。

根据 ConTeXt wiki, `\setuppapersize` 允许的其他选项有「rotated」、「90」、「180」、「270」、「mirrored」和「negative」。在我自己的测试中, 我只注意到使用「rotated」时有一些明显的变化, 它会翻转页面, 尽管不完全是翻转。数值选项本应产生相应的旋转度数, 可以单独使用或与「rotated」组合使用, 但我无法让它们工作。我也没确切发现「mirrored」和「negative」的用途。



`\setuppapersize` 的第二个参数(我已经说过, 当打印尺寸与排版尺寸相同时可以省略)可以采用与第一个参数相同的值, 指示纸张尺寸和方向。它也可以取「oversized」作为值——根据 ConTeXt wiki——这会在纸张的每个角上增加一厘米半。

根据 wiki, 第二个参数还有其他可能的值:「undersized」、「doublesized」和「doubleoversized」。但在我的测试中, 使用这些值后我没有看到任何变化; 命令的官方定义(参见节 6)也没有提到这些选项。

1.2 使用非标准页面尺寸

如果我们想使用非标准的页面尺寸, 可以做两件事:

1. 使用 `\setuppapersize` 的另一种语法, 允许我们以尺寸值的形式引入纸张的高度和宽度。
2. 定义我们自己的页面尺寸, 为其分配一个名称, 并像使用标准纸张尺寸一样使用它。

`\setuppapersize` 的替代语法

除了我们已经看到的语法, `\setuppapersize` 还允许我们使用另一种语法:

`\setuppapersize[名称][选项]`

其中名称 是一个可选参数,表示使用 `\definepapersize`(我们将在下一节中介绍)分配给纸张尺寸的名称,而选项 是那种我们赋予显式值的选项。在允许的选项中,我们可以强调以下几点:

- **width** 和 **height**, 分别表示页面的宽度和高度。
- **page** 和 **paper**, 其中第一个指的是要排版的页面尺寸,第二个指的是实际打印的纸张尺寸。这意味着「page」相当于 `\setuppapersize` 正常语法(上面解释过)的第一个参数,而「paper」相当于第二个参数。这些选项可以采用前面提到的相同值(A4、S3 等)。
- **scale**, 表示页面的缩放因子。
- **topspace**、**backspace** 和 **offset**: 附加距离。

定义自定义页面尺寸

要定义自定义页面尺寸,我们使用 `\definepapersize` 命令,其语法为:

`\definepapersize[名称][选项]`

其中名称 指赋予新尺寸的名称,而选项 可以是:

- `\setuppapersize` 正常语法中允许的任何值(A4、A3、B5、CD 等)。
- 宽度(纸张宽度)、高度(纸张高度)和偏移(位移)的尺寸值,或缩放值(「scale」)。

不能混合使用 `\setuppapersize` 允许的值与尺寸值或缩放因子。这是因为在第一种情况下,选项是符号词,而在第二种情况下,是赋予显式值的变量;在 ConT_EXt 中,正如我已经说过的,不能在同一个方括号对中混合两种类型的选项。

当我们使用 `\definepapersize` 指示与某些标准测量值一致的纸张尺寸时,实际上我们并不是在定义一个新的纸张尺寸,而是在为已经存在的纸张尺寸定义一个新名称。如果我们想将纸张尺寸与方向结合起来,这可能会很有用。因此,例如,我们可以写

```
\definepapersize[vertical][A4, portrait]
\definepapersize[horizontal][A4, landscape]
```

1.3 在文档中的任意点更改页面尺寸

在大多数情况下,文档只有一种页面尺寸,这就是为什么 `\setuppapersize` 是我们包含在导言区中且在每个文档中只使用一次的典型命令。然而,有时可能需要

在文档中的某个点更改尺寸;例如,如果从某个点开始包含一个附录,其中纸张是横向的。

在这种情况下,我们可以在希望更改发生的确切位置使用 `\setuppapersize`。但由于尺寸会立即更改,为避免意外结果,我们通常会在 `\setuppapersize` 之前插入一个强制分页(使用 `\page`)。

如果我们只需要为单个页面更改页面尺寸,而不是使用两次 `\setuppapersize`(一次更改为新尺寸,第二次恢复到原始尺寸),我们可以使用 `\adaptpapersize`,它会更改页面尺寸,并在下一页之后自动重置为调用 `\adaptpapersize` 之前的值。与 `\setuppapersize` 一样,在使用 `\adaptpapersize` 之前,我们应该插入一个强制分页。

1.4 调整页面尺寸以适应其内容

ConTeXt 中有三个环境可以生成精确尺寸的页面来容纳其内容。它们是 `\startMPpage`、`\startpagefigure` 和 `\startTEXpage`。第一个生成一个页面,其中包含用 MetaPost 生成的图形,MetaPost 是一种与 ConTeXt 集成的图形设计语言,但本入门不会涉及。第二个对图像执行相同的操作,并可选择在图像下方添加一些文本。它有两个参数:第一个标识包含图像的文件。我将在专门讨论外部图像的章节(节 2)中讨论这一点。第三个(`\startTEXpage`)创建一个完全适合其内容的页面。其语法为:

```
\startTEXpage[选项] ... \stopTEXpage
```

其中选项可以是以下任何一项:

- **strut**: 我不确定此选项的用途。在 ConTeXt 术语中,*strut* 是一个没有宽度但具有最大高度和深度的字符,但我不太明白这与该命令的整体用途有什么关系。根据 wiki,此选项允许的值有「yes」、「no」、「global」和「local」,默认值为「no」。
- **align**: 指示文本对齐方式。可以是「normal」、「flushleft」、「flushright」、「middle」、「high」、「low」或「lohi」。
- **offset**: 指示文本周围的空白量。可以是「none」、「overlay」(如果需要叠加效果)或实际尺寸。
- **width、height**: 我们可以为页面指定宽度和高度,或者指定值「fit」,使得宽度和高度由环境中包含的文本所需的大小决定。(「fit」是每个参数的默认值。)
- **frame**: 默认是「off」,但如果我们希望页面上的文本周围有边框,可以取「on」值。



第2节 页面上的元素

ConTeXt 将页面划分为许多不同的区域(「元素」), 这为文档设计者在决定页面布局方式时提供了相当大的灵活性; 许多用户, 尤其是 ConTeXt 新手, 不会使用所有这些页面元素。但对于那些需要复杂布局的用户, 现在将描述许多页面布局参数。(如果你提前查看图 5.1, 你会看到许多页面元素和相应距离的图示; 但不要让那个图的复杂性吓到你!)

这些元素的大小可以使用 `\setuplayout` 进行配置。我们很快就会看到这个命令, 但在此之前, 最好先描述每个页面元素, 并指出 `\setuplayout` 识别它们各自的名称:

- **边缘:** 文本区域周围的空白区域。尽管大多数文字处理器称之为「边距」, 但使用 ConTeXt 的术语更可取, 因为它使我们能够区分没有文本的边缘(尽管在电子文档中可能有导航按钮等)和有时可以放置某些文本元素(例如边注)的边距。
 - ◊ 上边缘的高度由两个测量值控制: 上边缘本身(「top」)以及上边缘与页眉之间的距离(「topdistance」)。这两个测量值的总和称为「topspace」。
 - ◊ 左右边缘的大小分别取决于「leftedge」和「rightedge」测量值。如果我们希望两者长度相同, 可以使用「edge」选项同时配置它们。

在用于双面打印的文档中, 我们不谈论左边缘和右边缘, 而是谈论内边缘和外边缘; 前者是离纸张装订或缝线最近的一边, 即奇数页上的左边缘和偶数页上的右边缘。外边缘位于页面与内边缘相反的一侧。

- ◊ 下边缘的高度称为「bottom」。
- **边距**(严格意义上的)。ConTeXt 仅将「边距」一词用于左右边距。边距位于边缘和主文本区域之间, 旨在承载某些文本元素, 如边注, 并且在某些文档设计中, 节标题或其编号位于边距中。

控制边距大小的尺寸有:

- ◊ **margin:** 用于将左右边距同时设置为相同大小。
- ◊ **leftmargin、rightmargin:** 分别用于设置左边距和右边距的大小。
- ◊ **edgedistance:** 用于设置边缘与边距之间的距离。(如果需要左右不同的距离, 可以使用选项 **leftedgedistance** 和 **rightedgedistance**。)
- ◊ **leftedgedistance、rightedgedistance:** 分别用于设置边缘与左右边距之间的距离。
- ◊ **margindistance:** 用于设置边距与主文本区域之间的距离。

- ◇ **leftmargindistance**、**rightmargindistance**: 如果需要不同的距离, 分别用于设置主文本区域与左右边距之间的距离。
- ◇ **backspace**: 此测量值表示纸张左边缘与主文本区域左边缘之间的空间。因此, 它由「leftedge」+「leftedgedistance」+「leftmargin」+「leftmargindistance」之和组成。在不使用边距的文档中, 通常直接设置「backspace」。
- **页眉和页脚**: 页面的页眉和页脚是两个区域, 分别位于页面书写区域的顶部或底部。它们通常包含有助于提供文本上下文的信息, 例如页码、作者姓名、作品标题、章节标题等。在 ConT_EXt 中, 页面上的这些区域受以下尺寸的影响:
 - ◇ **header**: 页眉的高度。
 - ◇ **footer**: 页脚的高度。
 - ◇ **headerdistance**: 页眉与页面主文本区域顶部之间的距离。
 - ◇ **footerdistance**: 页脚与页面主文本区域底部之间的距离。
 - ◇ **topdistance**、**bottomdistance**: 前面都提到过。它们分别是上边缘与页眉顶部之间, 以及下边缘与页脚底部之间的距离。
- **主文本区域**: 这是页面上最宽的区域, 容纳文档的文本。其大小取决于「width」和「textheight」变量。而「height」变量测量的是「header」、「headerdistance」、「textheight」、「footerdistance」和「footer」的总和。

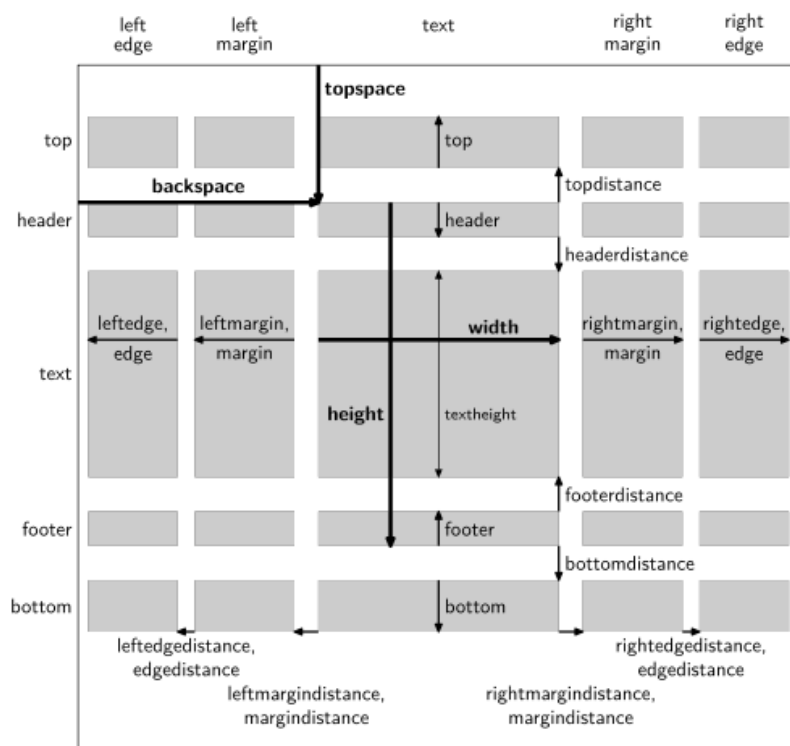


图 5.1 页面上的区域和测量值

我们可以在图 5.1 中看到所有这些区域, 以及相应测量值的名称和指示其范围的箭头。箭头的粗细以及箭头名称的大小旨在反映这些距离对页面布局的重要性。如果仔细观察, 我们会看到此图显示一个页面可以表示为一个九行九列的表格, 或者, 如果不考虑不同区域之间的间距值, 那么将有五行五列, 其中只有三行三列可以有文本。中间行与中间列的交点构成主文本区域, 通常占据页面的大部分。

在文档的布局阶段, 我们可以使用 `\showsetups` 查看所有页面测量值。要查看页面上视觉指示的文本分布主要轮廓, 可以使用 `\showframe`; 而使用 `\showlayout` 可以结合前两个命令的效果。

第3节 页面布局 (`\setuplayout`)

3.1 为不同的页面组件分配大小

页面设计涉及为页面的各个区域分配特定的大小。这通过 `\setuplayout` 完成。此命令允许我们更改上一节中提到的任何尺寸。其语法如下:

`\setuplayout[名称][选项]`

其中名称 是一个可选参数, 仅在我们设计了多个布局时使用(参见节 3.3), 而选项, 除了我们稍后将看到的其他选项外, 可以是前面提到的任何测量值。但请记住, 这些测量值是相互关联的, 因为影响宽度的组件总和以及影响高度的组件总和必须分别与页面的宽度和高度一致。原则上, 这意味着当更改水平长度时, 我们必须调整其余的水平长度; 调整垂直长度时也是如此。

默认情况下, ConTeXt 仅在某些情况下自动调整尺寸, 但另一方面, 其文档中并未以任何完整或系统的方式指明这些情况。通过进行多次测试, 我能够验证, 例如, 手动增加或减少页眉或页脚的高度会导致「`textheight`」的调整; 然而, 根据我的测试, 手动更改某些边距并不会自动调整文本宽度(「`width`」)。这就是为什么在页面大小(用 `\setuppapersize` 设置)与其各个组件的大小之间不产生任何不一致的最有效方法是:

- 关于水平测量值:
 - 通过调整「`backspace`」(包括「`leftedge`」和「`leftmargin`」)。
 - 通过调整「`width`」(文本宽度), 不使用尺寸值, 而使用「`fit`」或「`middle`」值:
 - * `fit` 根据页面其余水平组件的宽度计算文本宽度。
 - * `middle` 执行相同的操作, 但首先使左右边距相等。
- 关于垂直测量值:

- ◊ 通过调整「`topspace`」。
 - ◊ 通过将「`fit`」或「`middle`」值应用于「`height`」。它们的工作方式与宽度情况相同。第一个根据其余组件计算高度,第二个使上下边距相等,然后计算文本高度。
 - ◊ 一旦「`height`」被调整,如果需要,再调整页眉或页脚的高度,因为知道在这种情况下「`textheight`」将被自动重新调整。
- 间接确定主文本区域高度的另一种可能性是指示要容纳在其中的行数(考虑到当前的 `interline` 间距和字体大小)。这就是为什么 `\setuplayout` 包含「`lines`」选项的原因。

将逻辑页面放置在物理页面上

在逻辑页面大小与物理页面大小不同的情况下(参见节 1.1),`\setuplayout` 允许我们配置一些影响逻辑页面在物理页面上放置的附加选项:

- **location**: 此选项确定页面将放置在物理页面上的位置。其可能值为 `left`、`middle`、`right`、`top`、`bottom`、`singlesided`、`doublesided` 或 `duplex`。
- **scale**: 指示在将页面放置到物理页面上之前应用于页面的缩放因子。
- **marking**: 将在页面上打印视觉标记,以指示纸张的裁剪位置。
- **horoffset**、**veroffset**、**clipoffset**、**cropoffset**、**trimoffset**、**bleedoffset**、**artoffset**: 一系列指示物理页面上不同位移的测量值。其中大部分在 2013 年的参考手册中有所解释。

这些 `\setuplayout` 选项必须与 `\setuparranging` 的指示结合使用,后者指示逻辑页面在物理纸张上的排列顺序。我不会在本入门中解释这些命令,因为我还没有对它们进行任何测试。

获取文本区域的宽度和高度

命令 `\textwidth` 和 `\textheight` 分别返回文本区域的宽度和高度。这些命令提供的值不能直接显示在最终文档中,但它们可以用于其他命令来设置其宽度或高度测量值。因此,例如,要指示我们希望图像的宽度为行宽的 60

3.2 调整页面布局

可能我们某个特定页面上的页面布局会产生不理想的结果;例如,章节的最后一页只有一两行,这在排版或美学上都是不可取的。为了解决这些情况,ConT_EXt 提供

了 `\adaptlayout` 命令,允许我们更改一个或多个页面上文本区域的大小。此命令旨在仅在我们已经完成文档编写并进行一些小的最终调整时使用。因此,它的自然位置是在文档的导言区。该命令的语法为:

`\adaptlayout[页面][选项]`

其中 **页面** 指的是我们想要更改其布局的页码。这是一个可选参数,仅当 `\adaptlayout` 放在导言区中时才使用。我们可以指定一个页面,也可以指定多个页面,用逗号分隔数字。如果省略这第一个参数,`\adaptlayout` 将只影响它找到该命令的页面。

至于选项,它们可以是:

- **height**: 允许我们以尺寸值的形式指示相关页面应具有的高度。我们可以指示绝对高度(例如「19cm」)或相对高度(例如「+1cm」、「-0.7cm」)。例如,「height=+5mm」。
- **lines**: 我们可以包含要增加或减少的行数。要增加行数,值前面加『+』,要减少行数,前面加『-』。例如,「lines=-3」。

考虑到当我们更改页面上的行数时,这可能会影响文档其余部分的分页。这就是为什么建议仅在文档最终确定不再更改时,在最后使用 `\adaptlayout`,并在导言区中进行。然后我们转到我们希望调整的第一个页面,进行调整并检查它如何影响后续页面;如果它以需要调整另一页的方式影响它,我们添加其编号并再次编译,依此类推。这可能是一个有点漫长、乏味的过程,这就是为什么建议仅在文档其他部分最终确定后才执行此操作。

3.3 使用多个页面布局

如果我们需要在文档的不同部分使用不同的布局,最好的方法是首先定义通用布局,然后定义各种替代布局,这些替代布局只更改需要不同的尺寸。这些替代布局将继承整体布局的所有特性,这些特性在其定义中不会更改。要指定一个替代布局并为其命名以便以后调用,我们使用 `\definelayout` 命令,其一般语法为:

`\definelayout[名称/编号][配置]`

其中 **名称/编号** 是与新设计关联的名称,或者是新布局将自动激活的页码,而 **配置** 将包含我们希望通过与整体布局比较而更改的布局方面。

当新布局与名称关联时,要在文档中的特定点调用它,我们使用:

`\setuplayout[布局名称]`

并返回到通用布局:

`\setuplayout[reset]`

另一方面,如果新布局与特定的页码关联,则当到达该页时,它将自动激活。但是,一旦激活,要返回到通用设计,我们必须显式地指示这一点,尽管我们可以半自动化这一点。例如,如果我们想将布局仅应用于第 1 页和第 2 页,我们可以在文档的导言区中写:

```
\definelayout[1][...]  
\definelayout[3][reset]
```

这些命令的效果是,第一行定义的布局在第 1 页上激活,在第 3 页上激活另一个布局,其功能仅仅是返回到通用布局。

使用 `\definelayout[even]` 我们创建一个在所有偶数页上激活的布局;使用 `\definelayout[odd]`,该布局将应用于所有奇数页。

3.4 与页面布局相关的其他事项

3.4.1 区分奇偶页

在双面打印的文档中,奇数页和偶数页的页眉、页码和侧边距通常不同。偶数页也称为左手页(verso),奇数页称为右手页(recto)。在这些情况下,关于边距的术语也通常会改变,我们谈论内边距和外边距。前者位于离页面装订或缝合最近的位置,后者位于相反的一侧。在奇数页上,内边距对应于左边距,而在偶数页上,外边距对应于右边距。

`\setuplayout` 没有任何选项允许我们明确告诉它我们想要区分奇偶页的布局。这是因为对于 ConT_EXt,两种页面之间的区别是通过一个不同的选项设置的,即 `\setuppagenumbering`,我们将在节 4 中看到。一旦设置了双面,ConT_EXt 就会假定用 `\setuplayout` 描述的页面是奇数页,并通过将奇数页的值反转来构建偶数页。也就是说,适用于奇数页左侧的规范将适用于偶数页的右侧,反之亦然。

3.4.2 具有多栏的页面

使用 `\setuplayout`,我们还可以使文档的文本分布在两栏或多栏中,例如报纸和一些杂志那样。这由「columns」选项控制,其值必须是一个整数。当有多栏时,栏之间的距离由「columndistance」选项指示。

此选项适用于所有文本(或大部分文本)分布在多栏中的文档。如果在一个主要是单栏的文档中,我们希望特定部分为两栏或三栏,我们不需要更改页面布局,而只需使用「columns」环境(参见节 2)。

第4节 页码编号

默认情况下, ConT_EXt 使用阿拉伯数字进行页码编号, 并且数字显示在页眉中央。要更改这些特性, ConT_EXt 有不同的方法, 在我看来, 它们在这方面引入了不必要的复杂性。

首先, 编号的基本特性由两个不同的命令控制: `\setuppagenumbering` 和 `\setupuserpagenumber`。

`\setuppagenumbering` 允许以下选项:

- **alternative**: 此选项控制文档的设计是使页眉和页脚在所有页面上相同(「`singlesided`」), 还是区分奇偶页(「`doublesided`」)。当此选项取后者值时, 「`setuplayout`」引入的页面布局值会自动受到影响, 因此假定「`setuplayout`」中指示的仅适用于奇数页, 因此为左边距安排的实际上指的是内边距(在偶数页上位于右侧), 而为右侧安排的实际上指的是外边距, 在偶数页上位于左侧。
- **state**: 指示是否显示页码。它允许两个值: `start`(将显示页码)和 `stop`(将隐藏页码)。这些值的名称(`start` 和 `stop`)可能让我们认为当「`state=stop`」时页面停止编号, 而当「`state=start`」时编号重新开始。但事实并非如此: 这些值只影响页码是否显示。
- **location**: 指示页码将显示在何处。通常我们需要在此选项中指定两个值, 用逗号分隔。首先, 我们需要指定我们想要页码在页眉(「`header`」)还是页脚(「`footer`」)中, 然后, 在页眉或页脚中的位置: 可以是「`left`」、「`middle`」、「`right`」、「`inleft`」、「`inright`」、「`margin`」、「`inmargin`」、「`atmargin`」或「`marginedge`」。例如: 要在页脚中显示右对齐的编号, 我们应该指示「`location={footer,right}`」。注意我们如何用花括号将此选项括起来, 以便 ConT_EXt 能够正确解释分隔逗号。
- **style**: 指示用于页码的字体大小和样式。
- **color**: 指示应用于页码的颜色。
- **left**: 获取要放置在页码左侧的命令或文本。
- **right**: 获取要放置在页码右侧的命令或文本。
- **command**: 获取一个命令, 页码将作为参数传递给该命令。
- **width**: 指示页码占用的宽度。
- **strut**: 我不太确定这个。在我的测试中, 当「`strut=no`」时, 数字正好打印在页眉的上边缘或页脚的底部, 而当「`strut=yes`」(默认值)时, 数字和边缘之间会有一个空间。

`\setupuserpagenumber` 允许这些额外选项:

- **numberconversion**: 控制编号的类型; 可以是阿拉伯数字(「n」、「numbers」)、小写字母(「a」、「characters」)、大写字母(「A」、「Characters」)、小型大写阿拉伯数字(「KA」)、小写罗马数字(「i」、「r」、「romannumerals」)、大写罗马数字(「I」、「R」、「Romannumerals」)或小型大写罗马数字(「KR」)。
- **number**: 指示要分配给第一页的编号, 在此基础上计算其余页面。
- **numberorder**: 如果我们为此分配「reverse」作为值, 页码将按递减顺序排列; 这意味着最后一页将是 1, 倒数第二页是 2, 依此类推。
- **way**: 允许我们指示编号将如何进行。可以是:「byblock」、「bychapter」、「bysection」、「bysubsection」等。
- **prefix**: 允许我们指示页码的前缀。
- **numberconversionset**: 在下面解释。

除了这两个命令, 还需要考虑涉及文档宏观结构(参见节 6)的数字控制。从这个角度来看, `\defineconversionset` 允许我们为每个宏观结构块指示不同类型的编号。例如:

```
\defineconversionset
[frontpart:pagenumber][][romannumerals]
```

```
\defineconversionset
[bodypart:pagenumber][][numbers]
```

```
\defineconversionset
[appendixpart:pagenumber][][Characters]
```

将看到我们文档中的第一个块(frontmatter)使用小写罗马数字编号, 中央块(bodymatter)使用阿拉伯数字, 附录使用大写字母。

我们可以使用以下命令获取页码:

- `\userpagenumber`: 返回页码, 与使用 `\setuppagenumbering` 和 `\setupuser-pagenumber` 配置的完全一样。
- `\pagenumber`: 返回与上一个命令相同的数字, 但仍为阿拉伯数字。
- `\realpagenumber`: 返回页面的实际阿拉伯数字编号, 不考虑任何这些规范。

要获取文档最后一页的编号, 有三个与上述命令并行的命令。它们是: `\lastuserpagenumber`、`\lastpagenumber` 和 `\lastrealpagenumber`。

第5节 强制或建议的分页

5.1 `\page` 命令

ConTeXt 中文本分布的算法相当复杂, 基于大量的计算和内部变量, 这些变量告诉程序从排版正确性的角度来看, 哪里是进行实际分页的最佳可能点。`\page` 命令允许我们影响这个算法:

a. 通过建议某些点作为包含分页的最佳或不适当位置:

- ◇ **no**: 指示命令所在的位置不是插入分页的好候选位置, 因此, 应尽可能在文档的其他点进行分页。
- ◇ **preference**: 告诉 ConTeXt 遇到命令的点是尝试分页的好位置, 尽管它不会在那里强制分页。
- ◇ **bigpreference**: 指示遇到命令的点是尝试分页的非常好的位置, 但同样不会强制分页。

请注意, 这三个选项既不强制也不阻止分页, 只是告诉 ConTeXt 在寻找分页的最佳位置时, 应该考虑此命令中指示的内容。然而, 最终, 分页发生的位置仍将由 ConTeXt 决定。

b. 通过在某个点强制分页; 在这种情况下, 我们还可以指示应进行多少次分页以及要插入的页面的某些特性。

- ◇ **yes**: 在此点强制分页。
- ◇ **makeup**: 类似于「yes」, 但强制分页是立即的, 不会先放置任何待处理的浮动对象(参见节 1)。
- ◇ **empty**: 在文档中插入一个完全空白的页面(即, 甚至没有任何页眉或页脚文本)。
- ◇ **even**: 根据需要插入尽可能多的页面, 使下一页为偶数页。
- ◇ **odd**: 根据需要插入尽可能多的页面, 使下一页为奇数页。
- ◇ **left, right**: 类似于前两个选项, 但仅适用于双面打印的文档, 根据页面是奇数还是偶数, 具有不同的页眉、页脚或边距。
- ◇ **quadruple**: 插入所需数量的页面, 使下一页的页码为 4 的倍数。

除了这些专门控制分页的选项外, `\page` 还包括其他影响此命令工作方式的选项。特别是「disable」选项, 它导致 ConTeXt 忽略从那时起找到的 `\page` 命令, 而「reset」选项产生相反的效果, 恢复后续 `\page` 命令的有效性。

5.2 连接某些行或段落以防止在它们之间插入分页

有时, 如果我们想防止在几个段落之间分页, 使用 `\page` 命令可能会很繁琐, 因为它必须在每个可能插入分页的位置写入。一个更简单的做法是将我们想要保持在同一页上的材料放入 TeX 所称的垂直盒子中。

在本文档的开头([page 5](#)),我指出在内部,对 TeX 来说一切都是盒子。盒子概念对于任何类型的高级操作在 TeX 中都是基础;但管理它过于复杂,无法包含在本入门中。这就是为什么我只偶尔提及盒子。

TeX 盒子一旦创建,就是不可分割的¹,这意味着我们不能插入一个会将盒子一分为二的分页。这就是为什么,如果我们把想要保持在一起的材料放在一个不可见的盒子中,我们就可以避免插入会分割这些材料的分页。执行此操作的命令是 `\vbox`,其语法为

```
\vbox{材料}
```

其中材料 是我们想要保持在一起的文本。

ConTeXt 的某些环境确实将其内容放在一个盒子中,例如「`framedtext`」。因此,如果我们把想要保持在一起的材料放在这个环境中,并设置边框不可见(通过使用 `frame=off` 选项),我们就达到了保持材料在一起的目标。

但是... 请注意,我们最终可能会在页面上出现多余的空白,例如,如果带框的文本几乎适合页面,但没有完全适合。如果发生这种情况,您将不得不决定是要重写一些文本、撤销保持文本在一起的决定,还是接受一个带有大量空白的页面。

第 6 节 页眉和页脚

6.1 建立页眉和页脚内容的命令

如果我们在页面布局中为页眉和页脚分配了特定的大小,我们可以使用 `\setupheadertexts` 和 `\setupfootertexts` 命令在它们中包含文本。这两个命令相似,唯一的区别是前者激活页眉内容,后者激活页脚内容。两者都有一到五个参数。

1. 使用单个参数时,这些命令会将页眉或页脚的文本放置在页面的(水平)中央。例如:`\setupfootertexts[pagenum]` 将在页脚中央写入页码。
2. 使用两个参数时,第一个参数的内容将放置在页眉或页脚的左侧,第二个参数的内容将放置在右侧。例如,`\setupheadertexts[Preface][pagenum]` 将排版一个页眉,其中单词「Preface」写在左侧,页码打印在右侧。
3. 如果我们使用三个参数,第一个将指示区域,其中将打印其他两个参数。我所说的区域指的是[节 2](#)中提到的页面区域,即:「`edge`」、「`margin`」、「`header`」、... 其他两个参数分别包含要放置在左侧和右侧的文本。

使用四个或五个参数调用 `\setupheadertexts` 或 `\setupfootertexts` 类似于分别使用两个或三个参数,但适用于区分奇偶页的情况。我们知道,当使用

¹ 这不正确... 它们可以被分割,但那是另一个对于本入门来说过于高级的话题。

- 对于 `part`、`partnumber`、`chapter`、`chapternumber` 等, 使用 `\getmarking[标记]` 命令, 该命令返回所请求的标记的内容。因此, 例如, `\getmarking[chapter]` 将返回当前章节的标题, 而 `\getmarking[chapternumber]` 将返回当前章节的编号。

要禁用特定页面上的页眉和页脚, 请使用 `\noheaderandfooterlines` 命令, 该命令仅作用于它所在的页面。如果我们只想删除特定页面上的页码, 我们必须使用 `\page[blank]` 命令。

6.2 格式化页眉和页脚

页眉或页脚文本显示的具体格式可以在 `\setupheadertexts` 或 `\setupfootertexts` 的参数中使用相应的格式命令来指示。但是, 我们也可以使用 `\setupheader` 和 `\setupfooter` 全局配置, 它们允许以下选项:

- **state**: 允许以下值: `start`、`stop`、`empty`、`high`、`none`、`normal` 或 `nomarking`。
- **style**、**leftstyle**、**rightstyle**: 页眉和页脚文本样式的配置。`style` 影响所有页面, `leftstyle` 影响偶数页, `rightstyle` 影响奇数页。
- **color**、**leftcolor**、**rightcolor**: 页眉或页脚颜色。它可以影响所有页面(`color` 选项)或仅影响偶数页(`leftcolor`)或奇数页(`rightcolor`)。
- **width**、**leftwidth**、**rightwidth**: 所有页眉和页脚的宽度(`width`)或偶数页上的页眉 / 页脚(`leftwidth`)或奇数页上的页眉 / 页脚(`rightwidth`)。
- **before**: 在写入页眉或页脚之前要执行的命令。
- **after**: 在写入页眉或页脚之后要执行的命令。
- **strut**: 如果是「yes」, 则在页眉与边缘之间建立垂直分隔空间。当是「no」时, 页眉或页脚紧贴上下边缘区域的边界。

6.3 定义特定的页眉和页脚并将其链接到节命令

ConTeXt 的页眉和页脚系统允许我们在更改章节或节时自动更改页眉或页脚中的文本; 或者当我们更改页面时, 如果我们为奇偶页设置了不同的页眉或页脚。但它不允许区分(文档、章节或节的)第一页和其余页面。要实现后者, 我们必须:

1. 定义特定的页眉或页脚。
2. 将其链接到适用的节。

特定页眉或页脚的定义使用 `\definertext` 命令完成, 其语法为:

```
\definetext
[名称] [类型]
[内容 1] [内容 2] [内容 3]
[内容 4] [内容 5]
```

其中名称是分配给正在处理的页眉或页脚的名称;类型可以是 header 或 footer,取决于我们定义的是哪一个,其余五个参数包含我们想要的新页眉或页脚的内容,类似于我们看到的 `\setupheadertexts` 和 `\setupfootertexts` 的工作方式。完成后,我们需要使用 `\setuphead` 的 header 和 footer 选项(这些在章 7 中没有解释)将新页眉或页脚链接到某个特定的节。

因此,以下示例将在每个章节的第一页上隐藏页眉,并显示一个居中的页码作为页脚:

```
\definetext[ChapterFirstPage] [footer] [pagenumber]
\setuphead
[chapter]
[header=high, footer=ChapterFirstPage]
```

第 7 节 在页面边缘和边距中插入文本元素

上下边缘以及左右边距通常不包含任何文本。然而,ConTeXt 允许在那里放置一些文本元素。特别地,以下命令可用于此目的:

- `\setuptoptexts`: 允许我们在页面上边缘(页眉区域上方)放置文本。
- `\setupbottomtexts`: 允许我们在页面下边缘(页脚区域下方)放置文本。
- `\margintext`、`\atleftmargin`、`\atrightmargin`、`\ininner`、`\ininneredge`、`\ininnermargin`、`\inleft`、`\inleftedge`、`\inleftmargin`、`\inmargin`、`\inother`、`\inouter`、`\inouteredge`、`\inoutermargin`、`\inright`、`\inrightedge`、`\inrightmargin`: 允许我们在文档的侧边边缘和边距中放置文本。

前两个命令的功能与 `\setupheadertexts` 和 `\setupfootertexts` 完全相同,甚至可以使用 `\setuptop` 和 `\setupbottom` 预先配置这些文本的格式,类似于 `\setupheader` 允许我们配置 `\setupheadertexts` 的文本。关于所有这些,我请读者参考我已经在节 6 中说过的内容。唯一需要补充的小细节是,如果在页面布局中没有为上下边缘(top 或 bottom)预留空间,则为 `\setuptoptexts` 或 `\setupbottomtexts` 设置的文本将不可见。为此,请参见节 3.1。

至于旨在将文本放置在文档边距中的命令,它们都具有类似的语法:

```
\命令名称[引用][配置]{文本}
```

其中引用和配置是可选参数;第一个用于可能的交叉引用,第二个允许我们设置边距文本。最后一个参数,用花括号括起来,包含要放置在边距中的文本。

在这些命令中,`\margintext`更通用,因为它允许将文本放置在页面的任何边距或侧边边缘中。其余命令,如其名称所示,将文本放置在边距本身(右或左,内或外)或边缘(右或左,内或外)。这些命令与页面布局密切相关,因为例如,如果我们使用`\inrightrightedge`但未在页面布局中为右边缘预留任何空间,则什么也看不到。

`\margintext`的配置选项如下:

- **location**: 指示文本将放置在哪个边距中。可以是 `left`、`right`, 或者在双面文档中, 可以是 `outer` 或 `inner`。默认情况下, 在单面文档中是 `left`, 在双面文档中是 `outer`。
- **width**: 用于打印文本的可用宽度。默认情况下, 将使用边距的全部宽度。
- **margin**: 指示文本将放置在 `margin` 本身还是 `edge` 中。
- **align**: 文本对齐方式: 这里使用的值与 `\setupalign 6.1` 中的相同。
- **line**: 允许我们指示边距中文本的位移行数。因此, `line=1` 将文本向下移动一行, `line=-1` 将文本向上移动一行。
- **style**: 用于指示要放置在边距中的文本样式的命令。
- **color**: 边距文本的颜色。
- **command**: 一个命令的名称, 要放置在边距中的文本将作为参数传递给该命令。此命令将在写入文本之前执行。例如, 如果我们想在文本周围绘制一个框架, 可以使用「`[command=\framed]{Text}`」。

下面是一个演示 `\inmargin` 的示例: 在本句末尾, 使用了文本 **!!!**「`\inmargin[color=red]{!!!}`」。

其余的命令允许相同的选项, 除了 `location` 和 `margin`。特别是, `\atrightmargin` 和 `\atleftmargin` 命令将文本完全紧贴页面主体放置。我们可以使用 `distance` 选项设置一个分隔空间, 我在谈论 `\margintext` 时没有提到这个选项, 因为在我的测试中, 我没有看到该选项对该命令有任何效果。

除了上述选项之外, 这些命令还支持其他选项(`strut`、`anchor`、`method`、`category`、`scope`、`option`、`hoffset`、`voffset`、`dy`、`bottomspace`、`threshold` 和 `stack`), 我没有提到它们, 因为它们没有文档, 而且坦率地说, 我不太确定它们的用途。像 `distance` 这样的名称我们可以猜到, 但其余的? wiki 只提到了 `stack` 选项, 说它用于模拟 L^AT_EX 中的 `\marginpars` 命令, 但这对我来说似乎不是很清楚。



`\setupmargindata` 命令允许我们全局配置每个边距中的文本。因此, 例如,

`\setupmargindata[right][style=slanted]`

将确保右边距中的所有文本都以倾斜样式书写。

我们还可以使用以下命令创建自己的定制命令：

```
\definemargindata[名称][配置]
```

之后像这样使用它：

```
\名称{我边距中的定制文本}
```

第6章 ConT_EXt 中的字体与颜色

Table of Contents: 第1节「ConT_EXt Standalone」中包含的排版字体; 第2节 字体特性; 2.1 字体、样式和样式变体; 2.2 字体大小; 第3节 设置文档的主字体; 第4节 更改字体或某些字体特性; 4.1 `\setupbodyfont` 和 `\switchtobodyfont` 命令; 4.2 快速更改样式、替代形式和大小的后缀; 4.3 为字体大小、样式和替代形式定义命令和关键字; 第5节 与使用某些替代形式相关的其他事项; 5.1 斜体、倾斜体和强调; 5.2 小型大写字母和伪小型大写字母; 第6节 颜色的使用与配置; 6.1 排版彩色文本片段的方法; 6.2 更改文档的背景和前景颜色; 6.3 为特定文本片段着色的命令; 6.4 预定义颜色; 6.5 查看可用颜色; 6.6 定义我们自己的颜色;

第1节 「ConT_EXt Standalone」中包含的排版字体

ConT_EXt 的字体系统提供了许多可能性,但也相当复杂。我不会在本手册中分析所有高级字体可能性,而是仅限于假设我们使用的是随 ConT_EXt Standalone 安装提供的 21 种字体中的一些,即表 6.1 中显示的字体。

官方名称	ConT _E Xt 中的名称	示例
Latin Modern	modern, modern-base	Emily Brontë book
Antykwa Poltawskiego	antypol	Emily Brontë's book
Antykwa Toruńska	antykwa	Emily Brontë's book
Cambria	cambria	Emily Brontë book
DejaVu	dejavu	Emily Brontë book
DejaVu Condensed	dejavu-condensed	Emily Brontë book
Gentium	gentium	Emily Brontë book
Iwona	iwona	Emily Brontë book
Latin Modern Variable	modernvariable, modern-variable	Emily Brontë book
PostScript	postscript	Emily Brontë book
TeX Gyre Adventor	adventor, avantgarde	Emily Brontë's book
TeX Gyre Bonum	bonum, bookman	Emily Brontë's book
TeX Gyre Cursor	cursor, courier	Emily Brontë's book
TeX Gyre Heros	heros, helvetica	Emily Brontë book
TeX Gyre Schola	schola, schoolbook	Emily Brontë book
TeX Gyre Chorus	chorus, chancery	Emily Brontë book
TeX Gyre Pagella	pagella, palatino	Emily Brontë book
TeX Gyre Termes	termes, times	Emily Brontë book
Euler	eulernova	Emily Brontë book
Stix2	stix	Emily Brontë book
Xits	xits	Emily Brontë book

表 6.1 ConT_EXt 发行版中包含的字体

表 6.1 的中间列指示了 ConT_EXt 识别相关字体的一个或多个名称。当有两个名称时,它们是同义的。最后一列是使用该字体的示例。至于字体的显示顺序,第一个是 ConT_EXt 默认使用的字体,最后三个是专门为数学设计的字体,其余字体按字母顺序排列。请注意,Euler 字体不能直接表示带重音的字母,所以我们得到的是 Bront's,而不是 Brontë's。

对于来自 Windows 世界及其默认字体的读者,我将指出 *heros* 类似于 Windows 中的 Arial,而 *termes* 类似于 Times New Roman。它们不完全相同,但足够相似,以至于需要非常细心才能看出区别。

Windows 使用的字体不是自由软件(事实上 Windows 中几乎没有什么自由软件),因此它们不能包含在 ConT_EXt 发行版中。然而,如果 ConT_EXt 安装在 Windows 中,那么这些字体已经安装好了,并且可以像运行 ConT_EXt 的系统上安装的任何其他字体一样使用。不过,在本入门中,我不会讨论如何使用系统上已安装的字体。这方面的帮助可以在 [ConT_EXt wiki](#) 上找到。

第2节 字体特性

2.1 字体、样式和样式变体

关于字体的术语有些混乱,因为有时所谓的字体实际上是一个字体系列,包括共享基本设计的不同样式和变体。我不会讨论哪种术语更正确;我只想澄清 ConT_EXt 中使用的术语。在那里,它区分了字体、样式和每种样式的变体(或替代形式)。ConT_EXt 发行版中包含的字体(实际上它们是字体系列)就是我们上一节中看到的。我们现在来看一下样式和替代形式。

字体样式

DONALD E. KNUTH 为 T_EX 设计了 *Computer Modern* 字体,赋予它三种不同的样式,称为 *Roman*、*sans serif* 和 *teletype*。*Roman* 样式是一种字符带有装饰性衬线的设计,在排版术语中称为 *serif*,这也是为什么这种字体样式也被称为 *serif*。这种样式被认为是正常或默认样式。*sans serif* 样式,顾名思义,没有这些衬线,因此是一种更简单、更风格化的字体,有时也以其他名称出现;例如,在西班牙语中称为 *paloseco*。这种字体可以作为文档的主要字体,但也适合在以 *roman* 样式为主要字体的文本的某些片段中使用,例如标题或页眉。最后,*teletype* 样式被包含在 *Computer Roman* 中,因为它被设计用于编写与计算机编程相关的书籍,其中涉及大量计算机代码,在印刷材料中,这些代码通常以模仿计算机终端和旧式打字机的等宽样式呈现。

除了这三种字体样式之外,还可以添加第四种用于数学片段的样式。但由于 T_EX 在进入数学模式时会自动使用这种样式,并且不包含明确启用或禁用它命令,也没有其他样式的变体或替代形式,因此通常不将其视为真正意义上的样式。

ConT_EXt 包含两种可能的附加样式的命令:手写体和书法体。我不太确定它们之间的区别,因为一方面,ConT_EXt 发行版中包含的字体都没有在其设计中包含这些样式,另一方面,在我

看来,书法体也是手写的。ConT_EXt 包含的这些用于启用此类样式的命令,如果用于未实现它们的字体,编译时不会产生任何错误:只是什么都不会发生。

字体替代形式

每种样式 都允许许多替代形式,这就是 ConT_EXt 对它们的称呼(*alternative*):

- 常规或正常(「**tf**」,来自 *typeface*)。
- 粗体(「**bf**」,来自 *boldface*)。
- 斜体(「**it**」,来自 *italic*)
- 粗斜体(「**bi**」,来自 *bold italic*)
- 倾斜体(「**sl**」,来自 *slanted*)
- 粗倾斜体(「**bs**」,来自 *bold slanted*)
- 小型大写字母(「**sc**」,来自 *small caps*)
- 旧式数字(「**os**」,来自 *old style*)

这些替代形式,顾名思义,是互斥的:当一个被启用时,其他的被禁用。这就是为什么 ConT_EXt 提供了启用它们的命令,而没有提供禁用的命令;因为当我们启用一个替代形式时,我们就禁用了之前使用的那个;因此,例如,如果我们正在用斜体书写并启用了粗体,那么斜体将被禁用。如果我们想同时使用粗体和斜体,我们不必先启用一个再启用另一个,而是启用同时包含两者的替代形式(「**bi**」)。

另一方面,必须记住,尽管 ConT_EXt 假定每种字体都有这些替代形式,并因此提供了启用它们的命令,但为了在最终文档中起作用并产生一些可察觉的效果,这些命令需要字体在其设计中为每种样式和替代形式提供特定的形式。

特别是,许多字体在其设计中不区分倾斜体和斜体字母,或者不包含小型大写字母的特殊形式。

斜体与倾斜体的区别

斜体和倾斜体在排版功能上的相似性导致许多人混淆这两种替代形式。倾斜体是通过稍微倾斜常规字形得到的。但斜体意味着——至少在某些字体中——一种不同的设计,其中字母看起来是倾斜的,因为它们是按照那种样子绘制的;但实际上并没有真正的倾斜。这可以在下面的示例中看到,其中我们以相同的大小将同一个词写了三次,大小足够大,以便容易看出差异。第一个版本使用常规形式,第二个使用倾斜体,第三个使用斜体:

italics – *italics* – italics

注意前两个示例中字符的设计是相同的,但在第三个示例中,某些字母的笔画存在微妙(或者可能并不微妙!)的差异,这非常明显,尤其是在『a』的绘制方式上,尽管差异实际上几乎发生在所有字符上。

斜体和倾斜体的通常用法是相似的,每个人决定使用哪一种。这里有自由,但我们应该指出,如果斜体和倾斜体的使用是一致的,文档的排版会更好,看起来也会更好。此外,在许多字体中,斜体和倾斜体之间的设计差异可以忽略不计,因此使用哪一种都没有区别。

另一方面,斜体和倾斜体都是字体的替代形式,这主要意味着两件事:

1. 我们只能在字体中定义了它们时才能使用它们。
2. 当启用其中一个时,我们正在禁用之前一直使用的替代形式。

除了斜体和倾斜体的命令之外,ConT_EXt 还提供了一个用于强调文本特定部分的额外命令。与斜体或倾斜体相比,它的使用意味着微妙的差异。参见[节 5.1](#)。

2.2 字体大小

ConT_EXt 处理的所有字体都基于矢量图形,因此理论上它们可以以任何字体大小显示,尽管正如我们将看到的,这取决于我们用来确定字体大小的实际指令。除非另有说明,否则假定字体大小为 12 点。

ConT_EXt 使用的所有字体都基于矢量图形,因此是 Opentype 或 Type 1 字体,这意味着起源早于这项技术的字体已经被重新实现。特别是,由 Knuth 设计的 T_EX 默认字体 *Computer Modern* 最初仅以某些离散大小存在,每个字符由「位图」表示,而不是矢量图形。后来 *Computer Modern* 被重新实现为 Type 1 字体。再后来,*Computer Modern* 被用作开发 *Latin Modern* 的基础,其中添加了许多字形,改进了各种字体度量以提供更好的字母间距,并整合了各种其他改进。*Latin Modern* 是 ConT_EXt 使用的默认字体,尽管在许多文档中它仍被称为 *Computer Modern*。这可能是因为该字体对于 T_EX 系统仍然具有强烈的象征意义,因为这些系统最初是由 Knuth 与另一个名为 MetaFont 的程序一起开发和发展的,该程序旨在设计可与 T_EX 一起使用的字体。

第 3 节 设置文档的主字体

默认情况下,除非指定了其他字体,否则 ConT_EXt 将使用 12 点的 *Latin Modern Roman* 作为主字体。如前所述,这种字体是从 *Computer Modern* 发展而来的,后者最初由 KNUTH 设计用于 T_EX。它是一种优雅的罗马风格字体,在某些笔画中具有丰富的比例和装饰性「花饰」——称为衬线——非常适合打印文本和屏幕显示,尽管——这是个人观点——它不太适合像智能手机这样的小屏幕,因为衬线或花饰往往会「堆积」,使阅读变得困难。

要设置不同的字体,我们使用 `\setupbodyfont`,它不仅允许我们更改实际字体,还允许更改其大小和样式。当我们希望这适用于整个文档时,需要将其包含在源文件的导言区中。但如果我们只是想在某些点更改字体,这就是我们需要包含后续内容的地方。

`\setupbodyfont` 的格式为:

`\setupbodyfont[选项]`

其中命令的各种选项允许我们指示：

- **字体名称**, 可以是表 6.1 中的任何符号字体名称。
- **大小**, 可以通过其尺寸(使用点作为测量单位)或某些符号名称来指示。但请注意, 尽管我之前说过 ConT_EXt 使用的字体几乎可以缩放到任何大小, 但在 `\setupbodyfont` 中, ConT_EXt 仅支持 4 到 12 之间的整数大小, 以及 14.4 和 17.3 这些值。默认情况下, 它假定大小为 12 点。

`\setupbodyfont` 建立了我们可以称之为文档基本大小的东西; 换句话说, 就是正常字符大小, 基于此计算其他大小, 例如标题和脚注。当我们使用 `\setupbodyfont` 更改主大小时, 所有基于主字体计算的其他大小也会更改。

除了直接指示字符大小(10pt、11pt、12pt 等)之外, 我们还可以使用一些基于当前大小计算要应用的字符大小的符号名称。相关的符号名称, 从大到小依次为: 「big」、「small」、「script」、「x」、「scriptscript」和「xx」。因此, 例如, 如果我们想使用 `\setupbodyfont` 设置大于 12 点的正文文本, 我们可以使用「big」来实现。

- **字体样式**, 正如我们已经指出的, 可以是 roman(带衬线)、无衬线(sans serif)、打字机体, 以及对于某些字体, 手写体和书法体样式。`\setupbodyfont` 允许不同的符号名称来指示不同的样式。这些可以在表 6.2 中找到:

样式	允许的符号名称
Roman	rm, roman, serif, regular
Sans Serif	ss, sans, support, sansserif
等宽	tt, modo, type, teletype
手写体	hw, handwritten
书法体	cg, calligraphic

表 6.2 `setupbodyfont` 中的样式

据我所知, 每种样式支持的不同名称是完全同义的。

查看字体的外观

在决定在文档中使用特定字体之前, 我们通常想看看它是什么样子。这几乎总是可以从操作系统完成, 因为通常有一些实用程序可以检查系统上安装字体的外观; 但为了方便起见, ConT_EXt 本身提供了一个实用程序, 允许我们查看 ConT_EXt 中启用的任何字体的外观。这就是 `\showbodyfont`, 它会生成一个包含我们所指示字体示例的表格。

`\showbodyfont` 的格式如下:

`\showbodyfont`[选项]

其中我们可以像在 `\setupbodyfont` 中一样精确地指示相同的符号名称作为选项。因此,例如, `\showbodyfont[schola, 8pt]` 将向我们展示下面的表格,其中包含 schola 字体在基本大小为 8 点时的不同示例:

[schola] [schola,8pt]													
	<code>\tf</code>	<code>\tf</code>	<code>\bf</code>	<code>\sl</code>	<code>\it</code>	<code>\bs</code>	<code>\bi</code>	<code>\tfx</code>	<code>\tfxx</code>	<code>\tfa</code>	<code>\tfb</code>	<code>\tfc</code>	<code>\tfd</code>
<code>\rm</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag
<code>\ss</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag
<code>\tt</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag
<code>\mr</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag

请注意表格的第一行和第一列中有某些命令。稍后,当这些命令的含义被解释后,我们将再次查看 `\showbodyfont` 生成的表格。

如果我们想查看特定字体的完整字符范围,可以使用 `\showfont[字体名称]` 命令。此命令将显示每个字符的主要设计,而不应用样式和替代形式的命令。

第 4 节 更改字体或某些字体特性

4.1 `\setupbodyfont` 和 `\switchtobodyfont` 命令

要更改字体、样式或大小,我们可以使用与在文档开头不想使用 ConT_EXt 默认字体时设置字体的相同命令: `\setupbodyfont`。我们只需要将此命令放在我们希望更改字体的文档位置。它将产生一个永久性的字体更改,这意味着它将直接影响主字体,并间接影响所有与之相关的字体。

与 `\setupbodyfont` 非常相似的是 `\switchtobodyfont`。这两个命令都允许我们更改字体的相同方面(字体本身、样式和大小),但它们在内部的工作方式不同,并且用于不同的用途。第一个命令(`\setupbodyfont`)旨在建立文档的主字体(通常也是唯一一个);一个文档拥有多个主字体既不常见,也不符合排版规范(这就是为什么它被称为主字体)。相比之下, `\switchtobodyfont` 旨在以不同的字体书写文本的某些部分,或者为我们在文档中定义的特定类型的段落分配特定的字体。

除了上述内容——实际上影响了这两个命令各自的内部运作——从用户的角度来看,使用一个命令或另一个命令之间存在一些差异。特别是:

1. 正如我们已经知道的, `\setupbodyfont` 仅限于特定的大小范围,而 `\switchtobodyfont` 允许我们指示几乎任何大小,因此如果字体没有该大小,字体将缩放到该大小。

2. `\switchtobodyfont` 除了在其使用的位置之外,不会以任何方式影响文本元素。这与 `\setupbodyfont` 形成对比,如上所述,后者会建立主字体,并通过更改主字体,也会更改所有其字体是基于主字体计算的文本元素的字体。

另一方面,这两个命令不仅更改字体、样式和大小,还更改与字体相关的其他方面,例如,行间距。

有趣的是,`\setupbodyfont` 如果请求不允许的字体大小会产生编译错误,但如果请求不存在的字体则不会产生错误,在这种情况下,将启用默认字体(*Latin Modern Roman*)。`\switchtobodyfont` 在字体方面的行为相同,在大小方面,正如我已经说过的,它试图通过缩放字体来实现。然而,有些字体无法缩放到某些大小,在这种情况下,默认字体将再次被启用。

4.2 快速更改样式、替代形式和大小的后缀

更改样式和替代形式

除了 `\switchtobodyfont`, ConT_EXt 还提供了一组命令,允许我们快速更改样式、替代形式或大小。关于这些命令, ConT_EXt wiki 警告说,有时当它们出现在段落的开头时,可能会产生一些不希望的副作用,因此它建议在这种情况下,在相关命令之前加上 `\dontleavehmode` 命令。

样式	启用它的命令
Roman	<code>\rm, \roman, \serif, \regular</code>
Sans Serif	<code>\ss, \sans, \support, \sansserif</code>
等宽	<code>\tt, \mono, \teletype,</code>
手写体	<code>\hw, \handwritten,</code>
书法体	<code>\cf, \calligraphic</code>

表 6.3 在不同样式之间切换的命令

表 6.3 包含了允许我们更改样式的命令,而不改变任何其他方面;而表 6.4 包含了允许我们专门更改替代形式的命令。

替代形式	启用它的命令
正常	<code>\tf, \normal</code>
斜体	<code>\it, \italic</code>
粗体	<code>\bf, \bold</code>
粗斜体	<code>\bi, \bolditalic, \italicbold</code>
倾斜体	<code>\sl, \slanted</code>
粗倾斜体	<code>\bs, \boldslanted, \slantedbold</code>
小型大写字母	<code>\sc, \smallcaps</code>
旧式数字	<code>\os, \mediaeval</code>

表 6.4 启用特定替代形式的命令

所有这些命令都会保持其有效性,直到显式启用另一种样式或替代形式,或者声明该命令的组结束。因此,当我们希望命令只影响文本的一部分时,我们所必须做

的是将该部分括在一个组内,如下例所示,其中每当单词 *thought* 作为名词而不是动词出现时,它就以斜体出现,为其创建一个组。

I thought a {\it thought} but
the {\it thought} I thought wasn't
the {\it thought} I thought I thought.
If the {\it thought} I thought I thought
had been the {\it thought} I thought
I wouldn't have thought so much!

I thought a *thought*, but the *thought* I thought,
wasn't the *thought* I thought I thought. If the *thought*
I thought I thought had been the *thought* I thought
I wouldn't have thought so much!

用于同时更改替代形式和大小的后缀

更改样式或替代形式的命令在其双字母版本(`\tf`、`\it`、`\bf` 等)中允许使用一系列影响字体大小的后缀。后缀『a』、『b』、『c』和『d』分别将字体大小乘以 1.2、 $1.2^2 (= 1.44)$ 、 $1.2^3 (= 1.728)$ 或 $1.2^4 (= 2.42)$ 。例如:

`\tf test, \tfa test, \tfb test, \tfc test, \tfd test`

test, test, test, test, test

而后缀 x 和 xx 会减小字体大小,分别乘以 0.8 和 0.6:

`\tf test, \tfx test, \tfxx test`

test, test, test

应用于 `\tf` 的后缀『x』和『xx』允许我们缩短命令,因此 `\tfx` 可以写成 `\tx`, `\tfxx` 可以写成 `\txx`。

这些不同后缀的可用性取决于字体的实际实现。根据 ConT_EXt 2013 年参考手册(主要针对 Mark II),唯一保证始终有效的后缀是『x』,其他的可能实现也可能不实现;或者它们可能仅适用于某些替代形式。

无论如何,为了消除疑问,我们可以使用我之前提到过的 `\showbodyfont` (在节 3 中)。此命令显示一个图表,不仅让我们能够欣赏字体的外观,还能看到字体在其每种样式和替代形式中的样子,以及哪些调整大小的后缀可用。

让我们再次查看 `\showbodyfont` 显示的表格：

[modern]													
	<code>\tf</code>	<code>\tf</code>	<code>\bf</code>	<code>\sl</code>	<code>\it</code>	<code>\bs</code>	<code>\bi</code>	<code>\tfx</code>	<code>\tfxx</code>	<code>\tfa</code>	<code>\tfb</code>	<code>\tfc</code>	<code>\tfd</code>
<code>\rm</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag
<code>\ss</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag
<code>\tt</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag
<code>\mr</code>	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag	Ag

如果我们仔细观察表格,可以看到第一列包含字体样式(`\rm`、`\ss` 和 `\tt`)。第一行左侧包含替代形式(`\tf`、`\sc`、`\sl`、`\it`、`\bf`、`\bs` 和 `\bi`),而第一行的右侧包含其他可用的后缀,尽管仅适用于常规替代形式。

需要注意的是,通过这些后缀中的任何一个更改字体大小都只会严格地更改字体大小,而保持通常与字体大小相关的其他值不变,例如行距。

自定义后缀的缩放因子

要自定义缩放因子,我们可以使用 `\definebodyfontenvironment`,其格式可以是：

```
\definebodyfontenvironment[特定大小][缩放]
\definebodyfontenvironment[default][缩放]
```

在第一个版本中,我们将为 `\setupbodyfont` 或 `\switchtobodyfont` 设置的主字体的特定大小重新定义缩放。例如：

```
\definebodyfontenvironment[10pt][a=12pt, b=14pt, c=2, d=3]
```

将确保当主字体为 10 点时,后缀『a』将其更改为 12 点,后缀『b』更改为 14 点,后缀『c』将原始字体乘以 2,后缀『d』乘以 3。请注意,对于『a』和『b』指示了固定尺寸,但对于『c』和『d』指示了原始大小的乘数因子。

然而,当 `\definebodyfontenvironment` 的第一个参数等于「default」时,我们将为所有可能的字体大小重新定义缩放值,并且作为缩放值,我们只能输入一个乘数数字。因此,例如,如果我们写：

```
\definebodyfontenvironment[default][a=1.3, b=1.6, c=2.5, d=4]
```

我们将指示,无论主字体大小如何,『a』后缀指示的字体大小是默认大小的 1.3 倍,『b』是 1.6 倍,『c』是 2 倍,『d』是 4 倍。

除了后缀『xx』、『x』、『a』、『b』、『c』和『d』之外,使用 `\definebodyfontenvironment` 我们还可以为『big』、『small』、『script』和『scriptscript』关键字分配缩放值。这些值分配给 `\setupbodyfont` 和 `\switchtobodyfont` 中与这些关键字关联的所有大小。它们也应用于以下命令中,我认为从它们的名称可以推断出它们的用途:

- `\smallbold`
- `\smallslanted`
- `\smallboldslanted`
- `\smallslantedbold`
- `\smallbolditalic`
- `\smallitalicbold`
- `\smallbodyfont`
- `\bigbodyfont`

如果我们想查看特定字体的默认大小,可以使用 `\showbodyfontenvironment[字体]`。例如,将此命令应用于 modern 字体,会得到以下结果:

[modern]						
text	script	scriptscript	x	xx	small	big
10pt	7pt	5pt	8pt	6pt	8pt	12pt
11pt	8pt	6pt	9pt	7pt	9pt	12pt
12pt	9pt	7pt	10pt	8pt	10pt	14.4pt
14.4pt	11pt	9pt	12pt	10pt	12pt	17.3pt
17.3pt	12pt	10pt	14.4pt	12pt	14.4pt	20.7pt
20.7pt	14.4pt	12pt	17.3pt	14.4pt	17.3pt	20.7pt
4pt	4pt	4pt	4pt	4pt	4pt	6pt
5pt	5pt	5pt	5pt	5pt	5pt	7pt
6pt	5pt	5pt	5pt	5pt	5pt	8pt
7pt	6pt	5pt	6pt	5pt	5pt	9pt
8pt	6pt	5pt	6pt	5pt	6pt	10pt
9pt	7pt	5pt	7pt	5pt	7pt	11pt

4.3 为字体大小、样式和替代形式定义命令和关键字

用于更改字体大小、样式和变体的预定义命令非常丰富。此外,ConTeXt 允许我们:

1. 添加我们自己的命令来更改字体样式、大小或变体。
2. 为 `\switchtobodyfont` 识别的样式或变体名称添加同义词。

它提供以下命令来实现这一点：

- `\definebodyfontswitch`: 允许我们定义一个更改字体大小的命令。例如, 如果我们想定义 `\eight` 命令(或 `\viii` 命令)来设置 8 pt 字体, 我们需要写:

```
\definebodyfontswitch[eight][8pt] 或 \definebodyfontswitch[viii][8pt]
```

- `\definefontstyle`: 允许我们定义一个或多个可以在 `\setupbodyfont` 或 `\switchtobodyfont` 中使用的词来设置特定的字体样式; 因此, 例如, 如果我们想用其他名称来称呼 *sans serif* (例如, 在西班牙语中称为「paloseco」), 我们可以写

```
\definefontstyle[paloseco][ss]
```

`\definefontstyle` 的一个特点是它允许将多个词同时关联到同一样式, 因此, 继续西班牙语的例子:

```
\definefontstyle[paloseco, sosa, sinrebordes][ss]
```

- `\definealternativestyle`: 允许我们将一个名称与字体变体关联起来。这个名称可以作为命令使用, 或者被允许配置要应用样式的命令的 `style` 选项识别。因此, 例如, 以下片段

```
\definealternativestyle[strong][\bf][]
```

将启用 `\strong` 命令和关键字「strong」, 该关键字将被允许此选项的命令的 `style` 选项识别。我们本可以说「bold」, 但这个词已经在 ConTeXt 中使用, 所以我选择了一个 HTML 中使用的术语, 即「strong」作为替代。

我不知道 `\definealternativestyle` 的第三个参数是做什么的。它不是可选的, 因此不能省略; 但我找到的关于它的唯一信息是在 ConTeXt 参考手册中, 其中说第三个参数仅与章节标题相关:「除了 `\cap` 之外, 我们必须遵守此处使用的字体」(??)



第5节 与使用某些替代形式相关的其他事项

在字体的不同替代形式中, 有两种的使用需要一定的澄清:

5.1 斜体、倾斜体和强调

斜体和倾斜体都主要用于在排版上突出文本片段以吸引注意力。换句话说, 就是强调它。

¹ 请记住, 除了控制符号的情况外, ConTeXt 命令名称只能由字母组成。

当然,我们可以通过显式启用斜体或倾斜体来强调文本。但 ConT_EXt 提供了一个更有用和有趣的替代命令,专门用于强调文本片段。这就是源自单词 *emphasis* 的 `\em` 命令。与纯粹排版命令的 `\it` 和 `\sl` 相比,`\em` 是一个概念性命令;它的工作方式不同,因此更通用,以至于 ConT_EXt 文档建议优先使用 `\em` 而不是 `\it` 或 `\sl`。当我们使用后两个命令时,我们是在告诉 ConT_EXt 我们想要使用哪种字体替代形式;但当我们使用 `\em` 时,我们是在告诉它我们想要产生什么效果,让 ConT_EXt 决定如何做到这一点。通常,为了达到强调或突出某物的效果,我们会启用斜体或倾斜体,但这取决于上下文。因此,如果我们在一个已经是斜体——或倾斜体——的文本中使用 `\em`,该命令将以相反的方式突出显示:在这种情况下是直立文本。

因此,下面的示例:

```
{\em One of the most beautiful
orchids in the world is the
{\em Thelymitra variegata}
or Southern Queen of Sheba.}
```

*One of the most beautiful orchids in the world is
the Thelymitra variegata or Southern Queen of Sheba.*

注意,第一个 `\em` 启用了斜体(实际上是倾斜体,见下文),而第二个 `\em` 禁用了斜体,而是将单词「Thelymitra variegata」设置为正常的直立样式。

`\em` 的另一个优点是它不是一种替代形式,因此不会禁用我们之前拥有的替代形式,因此,例如,在粗体文本中,使用 `\em` 我们将得到粗倾斜体,而无需显式调用 `\bs`。类似地,如果 `\bf` 命令出现在已经强调的文本中,这种强调不会停止。

默认情况下,`\em` 启用的是倾斜体而不是斜体,但我们可以使用 `\setupbodyfontenvironment[default][em=italic]` 来更改这一点。

5.2 小型大写字母和伪小型大写字母

小型大写字母是一种排版资源,通常比使用大写字母好得多。小型大写字母给我们大写字母的形状,但保持与行中小写字母相同的高度。这就是为什么小型大写字母是小写字母的样式变体。小型大写字母在某些上下文中替换大写字母,对于书写罗马数字或章节标题特别有用。在学术文本中,通常也使用小型大写字母来书写被引用作者的姓名。

问题在于并非所有字体都实现了小型大写字母,而那些实现了的字体,也并非总是为其某些字体样式实现了小型大写字母。此外,由于小型大写字母是斜体、粗体或倾斜体的替代形式,根据我们在本章中阐述的一般规则,所有这些排版特性不能同时使用。

这些问题可以通过使用伪小型大写字母来解决,ConT_EXt 允许我们使用 `\cap` 和 `\Cap` 命令来创建伪小型大写字母;有关此主题的更多信息,请参见[节 2.1](#)。

第6节 颜色的使用与配置

ConTeXt 提供了更改整个文档、其某些元素或文本某些部分的颜色命令。它还提供了将数百种预定义颜色加载到内存中以及查看其成分的命令。

6.1 排版彩色文本片段的方法

大多数 ConTeXt 的可配置命令都允许一个名为「color」的选项,该选项允许我们指示受该命令影响的文本应以何种颜色书写。因此,例如,要指示章节标题用蓝色书写,我们只需要写:

```
\setuphead
[chapter]
[color=blue]
```

使用此方法,我们可以为标题、题头、脚注、边注、条和线、表格、表格或图像标题等着色。使用此方法的优点是最终结果将保持一致(所有履行相同功能的文本将使用相同的颜色书写),并且更容易全局更改。

我们也可以直接为文本的一部分或片段着色,但是,为了避免颜色过于斑驳(从排版角度来看不愉快)或不一致的使用,通常建议避免直接着色,而使用我们可以称之为语义着色的方法。也就是说,例如,我们不写

```
\color[red]{Very important text}
```

而是为非常重要的文本定义一个命令,并赋予其颜色。例如

```
\definehighlight[important][color=red]
\important{Very important text}
```

6.2 更改文档的背景和前景颜色

如果我们想更改整个文档的颜色,无论是想更改背景颜色还是前景(文本)颜色,我们将使用 `\setupbackgrounds` 或 `\setupcolors`。因此,例如:

```
\setupbackgrounds
[page]
[background=color, backgroundcolor=blue]
```

将设置页面的背景颜色为蓝色。作为「backgroundcolor」的值,我们可以使用任何预定义颜色的名称。

要全局更改整个文档的前景颜色(从插入命令的位置开始),请使用 `\setupcolors`,其中「textcolor」选项控制文本颜色。例如:

```
\setupcolors[textcolor=red]
```

将确保文本颜色为红色。

6.3 为特定文本片段着色的命令

为小段文本着色的通用命令是

`\color[颜色名称]{要着色的文本}`

对于较大的文本片段,最好使用

`\startcolor[颜色名称] ... \stopcolor`

这两个命令都使用某种预定义颜色。如果我们想即时定义颜色,可以使用 `\colored` 命令。例如:

Three `\colored[r=0.1, g=0.8, b=0.8]`
`{coloured}` cats

Three coloured cats.

生成一种颜色,由红色(最大可能量的 10%)、绿色(80%)和蓝色(80%)组成。
(在这种方案中,白色将是 `r=1, g=1, b=1`,这在白色页面背景上是看不见的。

6.4 预定义颜色

ConT_EXt LMTX 自动加载表 6.5 中列出的颜色定义。

名称	浅色调	中色调	深色调
black			
white			
gray	lightgray	middlegray	darkgray
red	lightred	middlered	darkred
green	lightgreen	middlegreen	darkgreen
blue	lightblue	middleblue	darkblue
cyan	lightcyan	middlecyan	darkcyan
magenta	lightmagenta	middlemagenta	darkmagenta
yellow	lightyellow	middleyellow	darkyellow
orange	lightorange	middleorange	darkorange
更多灰色调			
palegray	gray-1	gray-2	gray-3
gray-4	gray-5	gray-6	gray-7
gray-8	gray-9		

表 6.5 ConT_EXt 的预定义颜色

还有其他颜色集默认未加载,但可以使用以下命令加载:

`\usecolors[集合名称]`

其中集合名称可以是：

- 「crayola」, 235 种模仿马克笔色调的颜色。
- 「dem」, 91 种颜色。
- 「ema」, 540 种基于 Emacs 使用的颜色的颜色定义。
- 「rainbow」, 91 种用于数学公式的颜色。
- 「ral」, 213 种来自 *Deutsches Institut für Gütesicherung und Kennzeichnung* (德国质量保证与认证协会) 的颜色定义。
- 「ryb」, 36 种颜色。
- 「solarized」, 16 种基于「solarized」方案的颜色。
- 「svg」, 147 种颜色。
- 「x11」, 550 种 X11 标准颜色。
- 「xwi」, 125 种颜色。

颜色定义文件包含在发行版的「context/base/mkiv」目录中, 其名称遵循「colo-imp-NAME.mkiv」模式。(默认颜色位于 `../colo-imp-rgb.mkiv` 文件中, 即使对于 ConT_EXt LMTX 也是如此。)我刚刚提供的关于不同预定义颜色集的信息是基于我的特定发行版。具体的集合或其中定义的颜色数量在将来的版本中可能会改变。还要注意, 这些集合之间的颜色名称有很多重叠, 并且与特定名称关联的实际颜色在不同的集合中并不总是完全相同。

要查看这些集合中的每一个包含哪些颜色, 我们可以使用下面描述的 `\showcolor[集合名称]` 命令。要使用这些颜色中的一些, 我们首先需要使用 `(\usecolors[集合名称])` 命令将它们加载到内存中, 然后我们必须告诉 `\color` 或 `\startcolor` 命令颜色的名称。例如, 以下序列:

```
\usecolors[xwi]
\color[darkgoldenrod]{Tweedledum and Tweedledee}
```

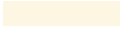
将写出

Tweedledum and Tweedledee

6.5 查看可用颜色

`\showcolor` 命令显示一个颜色列表, 您可以在其中看到颜色的外观、颜色在灰度中使用时的外观、颜色的红、绿、蓝分量, 以及 ConT_EXt 识别它的名称。不带任何参数使用时, `\showcolor` 将显示当前文档中使用的颜色。但作为参数, 我们可以指示节 6.4 中讨论的任何预定义颜色集, 因此, 例如, `\showcolor[solarized]` 将向我们显示该集合中的 16 种 solarized 颜色:

		0.561	0.514	0.580	0.588	base0
		0.460	0.396	0.482	0.514	base00
		0.409	0.345	0.431	0.459	base01
		0.162	0.027	0.212	0.259	base02
		0.123	0.000	0.169	0.212	base03

		0.615	0.576	0.631	0.631	base1
		0.909	0.933	0.910	0.835	base2
		0.965	0.992	0.965	0.890	base3
		0.457	0.149	0.545	0.824	blue
		0.487	0.165	0.631	0.596	cyan
		0.510	0.522	0.600	0.000	green
		0.429	0.827	0.212	0.510	magenta
		0.422	0.796	0.294	0.086	orange
		0.395	0.863	0.196	0.184	red
		0.473	0.424	0.443	0.769	violet
		0.530	0.710	0.537	0.000	yellow

如果我们想查看特定颜色的 rgb 分量,可以使用 `\showcolorcomponents[颜色名称]`。如果我们试图定义一种特定的颜色,查看与其接近的某种颜色的组成,这会很有用。例如,`\showcolorcomponents[darkgoldenrod]` (使用「xwi」颜色)将向我们显示:

color	name	transparency	specification
	 darkgoldenrod		r=0.7200,g=0.5300,b=0.0400

6.6 定义我们自己的颜色

`\definecolor` 允许我们克隆现有颜色或定义新颜色。克隆现有颜色就像为其创建一个替代名称一样简单。为此,您需要写:

`\definecolor[新颜色][旧颜色]`

这将确保「新颜色」与「旧颜色」的颜色完全相同。

但 `\definecolor` 的主要用途是创建新颜色。为此,必须按以下方式使用该命令:

`\definecolor[颜色名称][定义]`

其中定义 可以通过应用最多六种不同的颜色生成方案来完成:

- **RGB 颜色:** RGB 颜色的定义是最广泛使用的方法之一;它基于通过加法混合三种原色来表示颜色的想法:红色(『r』代表 *red*)、绿色(『g』代表 *green*)和蓝色(『b』代表 *blue*)。这些分量中的每一个都表示为一个介于 0 和 1 之间(含)的十进制数。

`\definecolor[lime 1][r=0.75, g=1, b=0]: “lime 1”颜色的文本。`

- **Hex 颜色:** 这种表示颜色的方式也基于 RGB 方案, 但红、绿、蓝分量表示为一个三字节的十六进制数, 其中第一个字节(两个字符)代表红色的值, 第二个字节代表绿色的值, 第三个字节代表蓝色的值。例如:

`\definecolor[lime 2][x=BFFF00]: “lime 2”颜色的文本。`

- **CMYK 颜色:** 这种颜色生成模型称为「减色模型」, 基于以下颜色颜料的混合: 青色(『c』)、品红色(『m』)、黄色(『y』)和黑色(『k』, 来自 *key*)。这些分量中的每一个都表示为一个介于 0 和 1 之间的十进制数:

`\definecolor[lime 3][c=0.25, m=0, y=1, k=0]: “lime 3”颜色的文本。`

- **HSL/HSV:** 这种颜色模型基于测量色相(『h』)、饱和度(『s』)和亮度(『l』)(有时是明度(『v』))。色相对应于 0 到 360 之间的数字; 饱和度和亮度/明度必须是 0 到 1 之间的十进制数。例如

`\definecolor[lime 4][h=75, s=1, v=1]: “lime 4”颜色的文本。`

- **HWB 颜色:** HWB 模型是 CSS4 的一个建议标准, 它测量色相(『h』)、白度(『w』)和黑度(『b』)。色相对应于 0 到 360 之间的数字, 而白度和黑度由 0 到 1 之间的十进制数表示。(注意: 2026 年 3 月的实验显示, 使用 HWB 产生的颜色与假定的 RGB 等效色之间的一致性不是很好。不清楚是转换错误还是 ConTeXt 的 HWB 处理有问题。)

`\definecolor[Tomato 1][h=9, w=0.28, b=0.] “Tomato 1”颜色的文本。`

- **灰度:** 基于一个称为(『s』, 来自 *scale*)的分量, 测量灰色的程度。它需要是一个介于 0 和 1 之间的数字。例如:

`\definecolor[my dark grey][s=0.35]: “my dark grey”颜色的文本。`

也可以从另一种颜色定义新颜色。例如, 本入门中标题所用的颜色定义为:

`\definecolor[maincolour][0.6(orange)]`

第7章 文档结构

Table of Contents: 第1节 文档中的结构划分; 第2节 节的类型及其层级; 第3节 节命令的通用语法; 第4节 节及其标题的格式与配置; 4.1 `\setuphead` 和 `\setupheads` 命令; 4.2 节标题的组成部分; 4.3 控制编号(在编号节中); 4.4 标题颜色和样式; 4.5 编号和标题文本的位置; 4.6 在打印标题之前或之后执行的命令或操作; 4.7 其他可配置特性; 4.8 其他 `\setuphead` 选项; 第5节 定义新的节命令; 第6节 文档的宏观结构;

第1节 文档中的结构划分

除了非常简短的文本(例如信件)之外,文档通常被组织成块或文本组,这些块或文本组通常遵循层级顺序。这些块的命名没有标准方式:例如在小说中,结构划分通常称为“章”,尽管有些较长的小说有更大的块,通常称为“部”,将若干章组合在一起。戏剧作品区分“幕”和“场”。学术手册有时分为“部分”和“课”、“专题”或“章”,而这些内部又常有进一步的划分;其他学术或技术文档(如解释计算机程序或系统的本文)中也常存在类似复杂的层级划分。甚至法律条文也结构化为“卷”(最长最复杂的,如法典)、“编”、“章”、“节”、“条”。科技文档的嵌套深度有时可达六、七层,甚至偶尔八层。

本章重点分析 ConTeXt 为支持这些结构划分所提供的机制。我将用“节”(section)这个总称来指代它们。

没有一个明确的术语可以让我们概括性地指代所有这些结构划分。我选择的“节”这个词侧重于结构划分而非其他,但缺点是 ConTeXt 预定义的结构划分之一就叫做“节”(section)。我希望这不会造成混淆,相信从上下文中很容易判断我们是在泛泛指称结构划分,还是特指 ConTeXt 称为 section 的那种具体划分。

每个(泛指)的“节”都意味着:

- 文档中一个相当大块的结构划分,它本身可能包含其他更低层级的划分。从这个角度看,“节”意味着具有层级关系的文本块。从节的角度看,整个文档可以看作一棵树。文档本身是树干,每个章节是一个树枝,树枝上又可以有细枝(节),细枝还可以再分(小节),依此类推。

拥有清晰的结构对于文档的阅读和理解非常重要。然而,这个任务取决于作者,而不是排字工。尽管让 ConTeXt 把我们变成比实际更好的作者并非其职责,但它所包含的全范围的节命令(其中层级关系相当清晰)可以帮助我们写出结构更良好的文档。

- 一个可以称为“标题”或“标签”的标题。该标题会被打印:

- ◇ 总是(或几乎总是)在文档中该结构划分开始的位置。
- ◇ 有时也会出现在目录中,以及可能出现在该节所在页面的页眉或页脚中。

ConTeXt 允许我们自动完成所有这些任务,使得打印结构单元标题的格式特性只需指定一次,并且是否也应包含在目录、页眉或页脚中也只需指定一次。为此,ConTeXt 只需要知道每个结构单元从哪里开始和结束、它的名称以及它的层级级别。

第2节 节的类型及其层级

ConTeXt 区分编号节和无编号节。顾名思义,前者会自动编号并被发送到目录,有时也会发送到页眉和/或页脚。

ConTeXt 具有预定义的、按层级顺序排列的节命令,见 Table 7.1。

层级	编号节	无编号节
1	<code>\part</code>	—
2	<code>\chapter</code>	<code>\title</code>
3	<code>\section</code>	<code>\subject</code>
4	<code>\subsection</code>	<code>\subsubject</code>
5	<code>\subsubsection</code>	<code>\subsubsubject</code>
6	<code>\subsubsubsection</code>	<code>\subsubsubsubject</code>
...	...	...

表 7.1 ConTeXt 中的节命令

关于预定义的节,需要澄清以下几点:

- 在 Table 7.1 中,节命令以传统形式展示。但我们很快就会看到,它们也可以作为环境使用(例如 `\startchapter ... \stopchapter`),而且这实际上是推荐的方式。
- 表中只列出了前 6 个节层级。然而,在我的测试中,我发现了多达 12 个层级:在 `\subsubsubsection` 之后是 `\subsubsubsubsection`,以此类推,直到 `\subsubsubsubsubsubsubsubsubsubsubsection` 或 `\subsubsubsubsubsubsubsubsubsubsubject`。

但我们应该记住,上述那种(过深的)较低层级几乎无助于提高文本的可理解性!首先,我们很可能会遇到大的节不可避免地涉及多个事项,这会让读者难以把握其内容。层级过深也可能导致读者失去对文本的整体感,产生的效果是所涉及材料的过度碎片化。我的理解是,一般来说四个层级就足够了;极偶尔可能需要用到六到七个层级,但更深的层次很少是个好主意。

从编写源文件的角度来看,为了创建更深子层级而需要在先前层级上再加一个“sub”,可能会使源文件几乎不可读:试图弄清楚名为“subsubsubsubsubsection”的命令的层级可不是开

玩笑的,因为我得数清有多少个“sub”! 所以我的建议是,如果我们真的需要这么多深度层级,从第五级(subsubsection)开始,我们最好定义自己的节命令(参见 [Section 5](#)),给它们起比预定义命令更清晰的名称。

- 最高的节层级(`\part`)只存在于编号标题,并且有一个特殊性:`part` 的标题不会被打印。然而,即使标题不被打印,也会引入一个空白页(可以假设用户重新配置命令后会在该页上打印标题),并且 `part` 的编号会被考虑用于计算 `chapter` 和其他节的编号。

为什么 `\part` 的默认版本不打印任何内容? 根据 ConTeXt wiki 的说法,因为在这个层级上的标题几乎总是需要特定的布局;虽然这是事实,但在我看来这并不足以成为好理由,因为实际上 `chapter` 和 `section` 也经常被重新定义,而 `part` 不打印任何内容的事实会迫使新手用户深入研究文档以查看哪里出了问题。

- 尽管第一个节层级是“`part`”,但这只是理论上的和抽象的。在具体文档中,第一个节层级将是文档中第一个节命令对应的层级。也就是说,在包含 `part` 但不包含 `chapter` 的文档中,`chapter` 将是第一级。但如果文档也不包含 `chapter`,而只有 `section`,那么该文档的层级将从 `section` 开始。

第3节 节命令的通用语法

所有节命令,包括用户创建的任何层级(参见 [Section 5](#)),都允许以下替代语法形式(例如,如果我们使用「`section`」层级):

```
\section [标签] {标题}
\section [选项]
\startsection [选项] [变量] ... \stopsection
```

在上述三种形式中,方括号内的参数是可选的,可以省略。我们将分别讨论它们,但首先需要明确的是,在 LMTX 中,推荐使用第三种方式。

- 在第一种语法形式中,我们可以称之为“经典”形式,该命令接受两个参数:一个可选的方括号参数,一个必选的花括号参数。可选参数用于为命令关联一个标签,该标签将用于内部引用(参见 [Section 2](#))。花括号内的必选参数是节标题。
- 另外两种语法形式更符合 ConTeXt 的风格:命令所需了解的一切都通过方括号内引入的值和选项来传达。

回忆一下,在 [Section 3.1](#) 和 [Section 4](#) 中,我曾说过在 ConTeXt 中,命令的作用域用花括号表示,其选项用方括号表示。但仔细想想,某个节命令的标题并不是它的应用范围,因此为了与通用语法一致,它不应该被放在花括号之间,而应该作为一个选项。ConTeXt 允许这种例外,因为这是 TeX 中的经典做法,但 ConTeXt 提供了更符合其整体设计的替代语法形式。

这些选项属于赋值(键值)类型(`选项名=值`),具体如下:

- ◇ **reference**: 用于交叉引用的标签。
- ◇ **title**: 将在文档正文中打印的节标题。
- ◇ **list**: 将在目录中打印的节标题。
- ◇ **marking**: 将在页眉或页脚中打印的节标题。
- ◇ **bookmark**: 将转换为 PDF 文件中“书签”的节标题。
- ◇ **ownnumber**: 此选项用于未自动编号的节; 在这种情况下, 此选项指示分配给该节的编号。

当然,「list」、「marking」和「bookmark」选项只应在希望使用不同的标题来替换通过「title」选项设置的主标题时使用。这非常有用,例如当标题太长而不适合页眉时;尽管我们也可以使用 `\nomarking` 和 `\nolist` 命令(非常类似)来实现这一点。另一方面,我们需要记住,如果标题文本(“title”选项)包含任何逗号,则整个标题需要用花括号括起来,以确保 ConTeXt 知道逗号是标题的一部分。这同样适用于选项:“list”、“marking”和“bookmark”。因此,为了避免时刻留意标题中是否有逗号,我认为养成习惯,始终将这些选项的值用花括号括起来是个好主意。

因此,例如,以下行将创建一个标题为“A Test Chapter”的章,并关联用于交叉引用的标签“test”,而页眉将使用“Chapter test”而不是“A Test Chapter”。

```
\chapter
[
  title={A Test Chapter},
  reference={test},
  marking={Chapter test}
]
```

`\startSectionType` 语法将节变成了一个环境。这更符合这样的事实:正如我在开头所说,在后台每个节都是一个差异化的文本块,尽管 ConTeXt 默认不将节命令生成的环境视为组。尽管如此,这个方式是 LMTX 所推荐的;很可能是因为这种建立节的方式要求我们明确说明每个节的开始和结束,这使得结构更容易保持一致,并且很可能对 XML 和 EPUB 输出有更好的支持。事实上,对于 XML 输出,这是必不可少的。

当我们使用 `\startSectionName` 时,允许在方括号之间使用一个或多个变量作为参数。然后可以在文档的其他位置使用 `\structureuservariable` 命令来使用这些变量的值。

拥有用户变量使得 ConTeXt 可以有非常高级的用法,因为可以根据特定变量的值来决定是否编译某个片段、如何编译或使用哪个模板。然而,这些 ConTeXt 工具超出了我在本介绍中打算处理的范围。

第4节 节及其标题的格式与配置

4.1 `\setuphead` 和 `\setupheads` 命令

默认情况下, ConT_EXt 为每个节层级分配了某些特性, 这些特性主要(但不限于)影响标题在文档正文中显示的格式, 但不影响标题在目录或页眉页脚中的显示方式。我们可以使用 `\setuphead` 命令更改这些特性, 其语法为:

`\setuphead[节列表][选项]`

其中:

- **节列表**指一个或多个节(用逗号分隔)的名称, 这些节将受该命令影响。可以是:
 - ◇ 任何预定义的节(part, chapter, title 等), 此时可以通过名称或层级来引用它们。要通过层级引用, 我们使用单词“section-层级编号”, 其中层级编号是所涉节层级的数字。因此, 「section-1」等同于「part」, 「section-2」等同于「chapter」, 依此类推。
 - ◇ 任何我们自己定义的类型。关于这一点, 请参见 [Section 5](#)。
- **选项**是配置选项。这些选项属于显式赋值类型(**选项名=值**)。可用的选项数量非常多(超过六十个!), 因此我将按功能分组解释它们。但我必须指出, 我未能确定其中一些选项的用途或使用方法。我不会讨论那些选项。

之前我说过 `\setuphead` 会影响明确指定的节。但这并不意味着修改某个特定节不会以任何方式影响其他节, 除非它们在命令中被明确提及。事实上, 恰恰相反: 修改一个节会影响与之关联的其他节, 即使没有在命令中明确说明。不同节之间的关联有两种:

- 无编号命令与同层级的相应编号命令相关联, 因此更改编号命令的外观会影响同层级的无编号命令; 但反过来则不行: 更改无编号命令不会影响编号命令。这意味着, 例如, 如果我们更改「chapter」(第2级)的某个方面, 我们也会更改「title」的同一方面; 但更改「title」不会影响「chapter」。
- 命令按层级关联, 因此如果我们更改特定层级的某些特性, 该更改将影响其后的所有层级。这仅发生在某些特性上。例如颜色: 如果我们设定 subsection 显示为红色, 那么 subsubsection、subsubsubsection 等也会变成红色。但其他特性(如字体样式)则不会如此。

除了 `\setuphead`, ConT_EXt 还提供了 `\setupheads` 命令, 该命令全局影响所有节命令。ConT_EXt wiki 提到, 有些人说这个命令不起作用。根据我的测试, 该命令对某些选项有效, 对另一些则无效。特别地, 它对「style」选项无效, 这很令人惊讶, 因为标题样式很可能正是我们想要全局更改以影响所有标题的东

西。但根据我的测试,它对其他选项如「number」或「color」是有效的。因此,例如,`\setupheads[color=blue]` 将确保我们文档中的所有标题都打印为蓝色。

由于我懒得去测试每个选项是否适用于 `\setupheads` (记住有六十多个), 下面我只引用 `\setuphead`。

最后: 在检查具体选项之前, 我们应注意 ConT_EXt wiki 中提到的一点, 尽管可能没在正确的地方提到: 某些选项只有在 `\startSectionName` 语法时才有效。

此信息是与 `\setupheads` 相关的内容, 而不是 `\setuphead`, 而后者才是解释大部分选项的地方, 并且如果只在一个地方说明, 那似乎是最合理的地方。另一方面, 该信息只提到了「insidesection」选项, 没有明确说明其他选项是否也会发生同样的情况。

4.2 节标题的组成部分

在深入讨论允许我们配置标题外观的具体选项之前, 最好先指出一个节标题最多可以有三个不同的部分, ConT_EXt 允许我们单独或一起格式化这些部分。这些标题元素如下:

- **标题本身**, 即标题的文本。原则上标题总是显示的, 除了「part」类型的节, 默认情况下标题不显示。控制标题是否显示的选项是「placehead», 其值可以是「yes」、「no」、「hidden」、「empty」或「section」。前两个的含义很清楚。但我



因此, 如果我们希望标题在第一级节中显示, 我们的设置应该是:

```
\setuphead
[part]
[placehead=yes]
```

某些节的标题, 正如我们已经知道的, 可以自动发送到页眉和目录。使用节命令的 `list` 和 `marking` 选项, 我们可以指定要发送的替代标题。也可以在编写标题时使用 `\nolist` 或 `\nomarking` 命令, 让标题的某些部分在目录或页眉中被省略号替换。例如:

```
\chapter{Influences of \nomarking{19th century} impressionism
\nomarking{in the 21st century}}
```

将在页眉中写入“Influences of ... impressionism ...”。

- **编号**: 仅适用于编号节 (part, chapter, section, subsection...), 不适用于无编号节 (title, subject, subsubject)。事实上, 某个节是否编号取决于「number」和「incrementnumber」选项, 其可能值为「yes」和「no」。在编号节中, 这两个选项都设为 yes, 在无编号节中设为 no。

为什么有两个选项来控制同一件事? 因为实际上这两个选项控制两件事: 一个指定节是否被编号 (incrementnumber), 另一个指定编号是否显示 (number)。如果为某个节设置

了 `incrementnumber=yes` 和 `number=no`,我们将得到一个外部(视觉上)无编号但内部仍被计数的节。这对于将这样的节包含在目录中很有用,因为通常目录只包含编号节。有关更多信息,请参见 [Subsection 1.7.1](#) 在 [Section 1.7](#) 中。

- **标题的标签。**原则上标题中的这个元素是空的。但我们可以为其关联一个值,在这种情况下,在编号和实际标题之前,将打印我们为此层级分配的标签。例如,在章标题中,我们可能希望打印“Chapter”这个词,或者在部分标题中打印“Part”一词。我们不是使用 `\setuphead` 来做这件事,而是使用 `\setuplabeltext` 命令。该命令允许我们为不同节层级的标签分配文本值。因此,例如,如果我们想在章标题之前写入“Chapter”,我们应该设置:

```
\setuplabeltext
[chapter=Chapter~]
```

在示例中,在分配的名称之后,我包含了保留字符“~”,它会在单词后插入一个不间断空格。如果我们不介意标签和编号之间发生换行,可以只加一个(普通)空格。但无论哪种空格都很重要;没有它,编号就会与标签连在一起,我们会看到例如“Chapter1”而不是“Chapter 1”。

4.3 控制编号(在编号节中)

我们已经知道预定义的编号节(`part`, `chapter`, `section...`),以及某个节是否编号取决于用 `\setuphead` 设置的「`number`」和「`incrementnumber`」选项。

默认情况下,各个层级的编号是自动的,除非我们将「`ownnumber`」选项的值设为「`yes`」。当「`ownnumber=yes`」时,必须为每个命令指定编号。这可以通过以下方式完成:

- 如果使用“经典”语法调用命令,则在标题文本之前添加一个包含编号的参数来定义「`ownnumber`」。例如:

```
\chapter{13}{Chapter title} 将生成一个手动分配了编号 13 的章。
```

- 如果使用 ConT_EXt 特有语法(即 `\SectionType [Options]` 或 `\startSectionType [Options]`)调用命令,则通过「`ownnumber`」选项定义。例如:

```
\chapter[title={Chapter title}, ownnumber=13], 将生成一个手动分配了编号 13 的章。
```

当 ConT_EXt 自动进行编号时,它使用内部计数器来存储不同层级的编号;因此有一个用于 `part` 的计数器,另一个用于 `chapter`,另一个用于 `section`,等等。每次 ConT_EXt 遇到一个节命令时,它会执行以下操作:

- 将与命令对应层级的计数器加 1。
- 将该命令以下所有层级的关联计数器重置为 0。

这意味着,例如,每次遇到新的 `chapter` 时,`chapter` 计数器加 1,所有 `section`、`subsection`、`subsubsection` 等命令的计数器重置为 0;然而,`part` 的计数器不受影响。

要更改起始计数的数字, 请使用 `\setupheadnumber` 命令, 如下所示:

`\setupheadnumber[节类型][起始计数数字]`

其中起始计数数字 是从此开始计算任何类型节的数字。因此, 如果起始计数数字 等于 0, 则第一个节将为 1; 如果等于 10, 则第一个节将为 11。

除了使用无符号数字, 您还可以使用类似 `\setupheadnumber[section][-5]` 的命令, 这将使节编号减少 5, 这意味着下一个节编号将比当前编号少 4 (因为每次调用 `\section` 时节编号都会增加 1)。当然, 这在任何“正常”情况下都不是您可能想要做的。此外, 尽管 wiki (2026 年 3 月 20 日) 说可以使用像“+10”这样的数字来将节编号增加 10, 但在该日期的 LMTX 中这并未生效。

默认情况下, 节编号显示阿拉伯数字, 并包含所有先前层级的编号。也就是说: 在包含 part、chapter、section 和 subsection 的文档中, 特定的 subsection 将指示它属于哪个 part、chapter 和 section。因此, 第一部分第三章第二节的第四个子节将编号为“1.3.2.4”。

控制编号显示方式的两个基本选项是:

- **conversion:** 控制将使用的编号类型。它允许许多值, 具体取决于我们想要的编号类型:
 - ◊ **阿拉伯数字编号:** 经典编号: 1, 2, 3, ... 通过值 `n`, `N` 或 `numbers` 获得。
 - ◊ **罗马数字编号:** 有三种方式:
 - * 大写罗马数字: `I`, `R`, `Romannumerals`。
 - * 小写罗马数字: `i`, `r`, `romannumerals`。
 - * 小型大写罗马数字: `KR`, `RK`。
 - ◊ **字母编号:** 有三种方式:
 - * 大写字母: `A`, `Character`
 - * 小写字母: `a`, `character`
 - * 小型大写字母: `AK`, `KA`
 - ◊ **单词编号:** 写出表示数字的单词。例如, 『3』变成『Three』。有两种方式:
 - * 首字母大写的单词: `Words`。
 - * 全部小写的单词: `words`。
 - ◊ **符号编号:** 基于符号的编号使用不同的符号集, 每个符号被赋予一个数值。由于 ConTeXt 使用的每个符号集的符号数量有限, 只有当要达到的最大“数字”不太高时才适合使用这种编号类型。ConTeXt 提供了四种不同的符号集: 分别称为 `set 0`、`set 1`、`set 2` 和 `set 3`。下面是每个集合用于编号的符号。请注意, `set 0` 和 `set 1` 能达到的最大数字都是 9, 而 `set 2` 和 `set 3` 都是 12:

Set 0: • - * ▷ ◦ ○ □ √
Set 1: * ** *** ‡ ‡‡ ‡‡‡ * ** ***
Set 2: * † ‡ ** †† ‡‡ *** ††† ‡‡‡ **** ‡‡‡‡ ‡‡‡‡‡
Set 3: * ** *** ‡ ‡‡ ‡‡‡ ◐ ◑ ◒ ◓ § §§ \$\$\$

- **sectionsegments**: 此选项允许我们控制是否显示先前层级的编号。我们可以指示将显示哪些先前的层级。这是通过标识要显示的起始层级和结束层级来完成的。层级的标识可以通过其编号(part=1, chapter=2, section=3 等)或名称(part、chapter、section 等)进行。因此,例如,「sectionsegments=2:3」表示应显示 chapter 和 section 的编号。这与说「sectionsegments=chapter:section」完全相同。如果我们想显示某个层级以上的所有编号,我们可以使用「optionsegments」的值为起始层级:all 或起始层级:*。例如,「sectionsegments=3:*」表示从第 3 级(section)开始显示编号。

因此,例如,假设我们希望 part 用大写罗马数字编号;chapter 用阿拉伯数字编号,但不包含所属 part 的编号;section 和 subsection 用包含 chapter 和 section 编号的阿拉伯数字编号;subsubsection 用大写字母编号(但不包含 part、chapter 或 section 编号)。在这种情况下,我们应该编写以下内容:

```
\setuphead[part][conversion=I]
\setuphead[chapter] [conversion=n, sectionsegments=2]
\setuphead[section] [conversion=n, sectionsegments=2:3]
\setuphead[subsection] [conversion=n, sectionsegments=2:4]
\setuphead[subsubsection][conversion=A, sectionsegments=5]
```

4.4 标题颜色和样式

我们有以下选项来控制样式和颜色:

- **样式**由「style」、「numberstyle」和「textstyle」选项控制,具体取决于我们希望影响整个标题、仅编号还是仅文本。通过这些选项中的任何一个,我们可以包含影响字体的命令:即特定字体、样式(罗马、无衬线或打字机)、变体(斜体、粗体、倾斜等)和大小。如果我们只想指定一种样式特性,我们可以通过使用样式名称(例如,“bold”表示粗体)、或其缩写(“bf”)或生成它的命令(\bf,对于粗体)来实现。如果我们想同时指定多个特性,则必须通过生成它们的命令来实现,将它们一个接一个地写下来。另一方面,请记住,如果我们只指定一个特性,其余的样式特性将自动设置为文档的默认值,这就是为什么很少建议只设置一个样式特性。
- **颜色**通过「color」、「numbercolor」和「textcolor」选项设置,具体取决于我们希望设置整个标题的颜色,还是仅设置编号或文本的颜色。此处指定的颜色可以是 ConTeXt 的预定义颜色之一,或者是我们自己定义并事先分配了名称的其他颜色。但是,我们不能直接在此处使用颜色定义命令。

除了这六个选项之外,还有五个选项可用于建立一些更复杂的功能,通过这些功能我们几乎可以做任何想做的事情。它们是:「command」、「numbercommand」、

「`textcommand`」、「`deepnumbercommand`」和「`deeptextcommand`」。我们先解释前三个：

- **command** 指示一个将接受两个参数(编号和节标题)的命令。它可以是普通的 ConTeXt 命令,也可以是我们自己定义的命令。
- **numbercommand** 类似于「`command`」,但这个命令只接受一个带有节编号的参数。
- **textcommand** 也类似于「`command`」,但它只接受一个带有标题文本的参数。

这三个选项使我们几乎可以做任何想做的事情。例如,如果我想要 section 右对齐、放在一个框架内,并且在编号和文本之间有一个换行,我可以简单地创建一个执行此操作的命令,然后将该命令指定为「`command`」选项的值。这可以通过以下行实现：

```
\define[2]\AlignSection
{\framed[frame=on, width=broad, align=flushright]{#1\\#2}}

\setuphead
[section]
[command=\AlignSection]
```

当我们同时设置「`command`」和「`style`」选项时,命令会应用于带有其样式的标题。这意味着,例如,如果我们设置了「`textstyle=\em`」和「`textcommand=\WORD`」,则命令 `\WORD` (将其参数文本大写)将应用于带有其样式的标题,即 `\WORD{\em Title text}`。如果我们希望反过来,即样式在命令应用之后应用于标题的内容,我们应该使用「`deeptextcommand`」和「`deepnumbercommand`」选项而不是「`textcommand`」和「`numbercommand`」。在上面给出的示例中,这将生成「`{\em\WORD{Title text}}`」。

在大多数情况下,以一种方式或另一种方式执行不会有区别;但在某些情况下可能会有区别。

4.5 编号和标题文本的位置

`\setuphead` 的「`alternative`」选项同时控制两件事:编号相对于标题文本的位置,以及标题(包括编号和文本)相对于显示它的页面和节内容的位置。这是两件不同的事情,但由于它们都由同一个选项控制,因此它们被同时控制。

标题相对于页面和节内容第一段的位置由「`alternative`」的以下可能值控制：

- **text**: 节标题与其内容的第一段整合在一起。效果类似于 L^AT_EX 中使用 `\paragraph` 和 `\subparagraph` 产生的效果。
- **paragraph**: 节标题将是一个独立的“段落”。

- **normal**: 节标题将放置在 ConT_EXt 为特定类型节提供的默认位置。通常是「paragraph」。
- **middle**: 标题作为一个独立的居中段落书写。如果是编号节, 则编号和文本分到不同的行, 都居中。

使用「**alternative=middle**」获得的效果类似于使用「**align**」选项控制标题对齐获得的效果。「**align**」可以取「**left**」、「**middle**」或「**flushright**」值。但是如果我们使用此选项将标题居中, 编号和文本将出现在同一行。

- **margin_{text}**: 此选项导致整个标题(编号和文本)打印在留作边距的空间中。

编号相对于标题文本的位置由「**alternative**」的以下可能值指示:

- **margin/inmargin**: 标题是一个独立的“段落”。编号写在留作边距的空间中。我还没弄清楚使用「**margin**」和使用「**inmargin**」之间的区别。
- **reverse**: 标题构成一个独立的段落, 但正常顺序颠倒, 先打印文本, 然后打印编号。
- **top/bottom**: 在标题文本占用多行的情况下, 这两个选项控制编号是与标题的第一行对齐还是与最后一行对齐。

4.6 在打印标题之前或之后执行的命令或操作

可以指定一个或多个在打印标题之前(「**before**」选项)或之后(「**after**」选项)执行的命令。这些选项广泛用于视觉上标记标题。例如: 如果我们在标题和其前面的文本之间添加更多垂直空间, 「**before=\blank**」将添加一个空行。要添加更多空间, 我们可以写成「**before={\blank[3*big]}**」。在这种情况下, 我们用花括号将选项的值括起来以避免错误。

与「**before**」和「**after**」选项非常相似的是「**commandbefore**」和「**commandafter**」选项。根据我的测试, 我推测区别在于前两者在开始排版标题本身之前和之后执行操作, 而后两者指的是在排版标题文本之前和之后执行的命令。 

如果我们在标题之前插入分页, 我们必须使用「**page**」选项, 该选项允许(除其他值外)使用「**yes**」插入分页, 使用「**left**」插入尽可能多的分页以确保标题从偶数页开始, 使用「**right**」确保标题从奇数页开始, 或者使用「**no**」禁用强制分页。另一方面, 对于低于「**chapter**」的层级, 此选项仅在同时使用「**continue=no**」选项时才有效; 否则, 如果相关的 section、subsection 或命令位于某个 chapter 的第一页, 它将不起作用。

默认情况下, ConT_EXt 中的 chapter 从新的一页开始。如果设定 section 也另起一页, 就会出现: 对于某个 chapter 的第一个 section(可能位于 chapter 的开头), 如果该 section 也触发分页, 我们最终会得到一个只包含 chapter 标题的页面, 这不太美观。这就是为什么我们可以设置

「continue」选项(我得说这个名称对我来说不太清楚):如果「continue=yes」,分页将不适用于位于 chapter 第一页上的 section。如果「continue=no」,分页仍然会被应用。

如果我们使用节环境(\start ... \stop)而不是节命令,我们还有「insidesection」选项,通过它我们可以指定一个或多个命令,这些命令将在标题排版完毕并且我们已经进入节内部之后执行。例如,此选项允许我们确保在开始一个 chapter 后立即自动排版一个目录(通过「insidesection=\placecontent」)。

4.7 其他可配置特性

除了我们已经看到的那些,我们还可以使用 \setuphead 配置以下附加特性:

- **行距**:由「interlinespace」控制,其值是一个先前用 \defineinterlinespace 创建并用 \setupinterlinespace 配置的行距命令的名称。
- **对齐**:「align」选项影响包含标题的段落的对齐方式。它可以具有以下值(包括其他值):「flushleft」(左对齐)、「flushright」(右对齐)、「middle」(居中)、「inner」(内边距)和「outer」(外边距)。
- **边距**:使用「margin」选项,我们可以手动设置标题的边距。
- **首段缩进**:「indentnext」选项的值(可以是「yes」、「no」或「auto」)控制节的第一段的第一行是否缩进。是否应该缩进(在通常段落首行缩进的文档中)是一个品味问题。
- **宽度**:默认情况下,标题占据其需要的宽度,除非此宽度大于行宽,在这种情况下标题将占据多行。但是使用「width」选项,我们可以为标题指定特定宽度。「numberwidth」和「textwidth」选项分别指定编号的宽度或标题文本的宽度。
- **分隔编号和文本**:「distance」和「textdistance」选项允许我们控制分隔编号与其文本的距离。
- **节页眉和页脚的样式**:为此我们使用「header」和「footer」选项。

4.8 其他 \setuphead 选项

通过我们已经看到的选项,我们可以看到节标题的配置可能性几乎是无限的。然而,\setuphead 还有大约三十个我没有提到的选项。大多数是因为我还没有发现它们的用途或使用方法,但有一些是因为解释它们将迫使我涉及我不打算在本介绍中处理的方面。



第5节 定义新的节命令

我们可以使用 \definehead 定义自己的节命令,其语法为:

`\definehead[命令名][模型][配置]`

其中:

- **命令名**表示新节命令将具有的名称。
- **模型**是现有节命令的名称,新命令将最初从该模型继承其所有特性。

事实上,新命令从模型继承的不仅仅是初始特性:它成为模型的一种定制实例,但与之共享例如控制编号的内部计数器。

- **配置**是我们新命令的定制配置。这里我们可以使用与 `\setuphead` 中完全相同的选项。

不一定要在创建新命令时配置它。这可以稍后使用 `\setuphead` 完成,事实上,在 ConTeXt 手册及其 wiki 给出的示例中,这似乎是正常的方式。

第6节 文档的宏观结构

章节、节、小节、标题.....组织文档;它们构建文档。但除了这些命令产生的结构之外,在某些印刷书籍中,特别是来自学术界的书籍,还存在一种书籍材料的“宏观排序”,考虑的不仅是其内容,而是这些大部分在书中所执行的功能。这就是我们区分以下部分的方式:

- 文档的开头部分,包含标题页、致谢页、题献页、目录,或许还有前言、介绍页等。
- 文档的主体部分,包含文档的基本文本,分为部、章、节、小节等。这部分通常是最广泛和最重要的。
- 由附录或附件组成的附加材料,这些材料展开或举例说明主体中处理的某些问题,或提供非主体作者撰写的附加文档等。
- 文档的结尾部分,我们可以在其中找到参考文献、索引、词汇表等。

在源文件中,我们可以通过 Table 7.2 中看到的环境来划定每个部分。

文档部分	命令
开头部分	<code>\startfrontmatter [Options] ... \stopfrontmatter</code>
主体部分	<code>\startbodymatter [Options] ... \stopbodymatter</code>
附录部分	<code>\startappendices [Options] ... \stopappendices</code>
结尾部分	<code>\startbackmatter [Options] ... \stopbackmatter</code>

表 7.2 反映文档宏观结构的环境

这四个环境允许相同的四个选项:「page」、「before」、「after」和「number」,它们的值和用途与 `\setuphead` 中的相同(参见 Section 4),不过我们应注意,此处的「number=no」选项将消除该环境内所有节命令的编号。

在我们的文档中包含这些大部分中的任何一个,仅当要在它们之间建立某种区分时才有意义。例如开头部分中的页眉或页码编号。每个块的配置通过 `\setupsectionblock` 实现,其语法为:

`\setupsectionblock[块名称][选项]`

其中块名称可以是 `frontpart`、`bodypart`、`appendix` 或 `backpart`,选项可以是刚刚提到的那些:「`page`」、「`number`」、「`before`」和「`after`」。因此,例如,为了确保在 *frontmatter* 中页面用罗马数字编号,在我们文档的导言中我们可以写:

```
\setupsectionblock
[frontpart]
[
  before={\setuppagenumbering[conversion=romannumerals]}
]
```

尽管还有其他方法可以实现这一点。

ConT_EXt 对这些块的默认配置意味着:

- 这四个块都从新的一页开始。
- 每个块中的节编号会发生变化:
 - ◊ 在 `frontmatter` 和 `backmatter` 中,默认情况下所有编号节都是无编号的。
 - ◊ 在 `bodymatter` 中,章使用阿拉伯数字编号。
 - ◊ 在 `appendices` 中,章使用大写字母编号。

也可以使用 `\definesectionblock` 创建新的节块。

第8章 目录与索引

Table of Contents: 第1节 目录; 1.1 目录概观; 1.2 带标题的完全自动目录; 1.3 无标题的自动目录; 1.4 要纳入目录的元素:`criterion`选项; 1.5 目录的布局:`alternative`选项; 1.6 目录条目的格式; 1.7 目录的手动调整; 1.7.1 在目录中包含未编号的章节; 1.7.2 手动向目录添加条目; 1.7.3 从目录中排除属于某个包含在目录中的章节类型的特定章节; 1.7.4 章节标题文本在目录中与文档正文中的标题不同; 第2节 列表、组合列表以及基于列表的目录; 2.1 ConT_EXt 中的列表; 2.2 图像、表格和其他项目的列表或索引; 2.3 组合列表; 第3节 索引; 3.1 生成索引; 3.1.1 定义索引条目并标记源文件中它们所指的点; 3.1.2 生成最终索引; 3.2 格式化主题索引; 3.3 创建其他索引;

目录和索引是文档的全局性方面。几乎所有文档都会有目录,而只有部分文档会有索引。对于许多语言(但不是英语)来说,目录和索引都归入“索引”(index)这个通用术语。对于英语读者来说,目录通常出现在开头(或者在某些情况下也出现在各章开头,如本文档所做的那样),而索引出现在末尾。

这两者都涉及内部引用机制的具体应用,其解释包含在 [Section 2](#) 中。

第1节 目录

1.1 目录概观

在上一章中,我们研究了允许在编写文档时建立文档结构的命令。本节重点介绍目录和索引,它们在某种程度上“镜像”了文档的结构。目录对于了解整个文档(有助于将信息置于上下文中)以及查找特定段落的准确位置非常有用。结构非常复杂的书籍,有许多不同深度的章节和小节,似乎需要不同类型的目录,因为过于简略的目录(可能只包含前两三个层级)虽然有助于了解文档内容的整体概况,但对于定位特定段落却不太有用。另一方面,在非常详细的目录中,很容易只见树木不见森林,失去文档的整体观。这就是为什么有时结构特别复杂的书籍会包含多个目录:开头有一个不太详细的目录显示主要部分,每章开头有一个更详细的目录,最后可能还有一个索引。

所有这些都可以由 ConT_EXt 相对容易地自动生成。我们可以:

- 在文档的任何位置生成完整或部分的目录。
- 决定这两者的内容。

- 极其细致地配置它们的外观。
- 在目录中包含超链接,允许我们直接跳转到相应的章节。

事实上,只要在文档中启用了交互功能,所有目录都默认包含最后这项实用性。这在 [Section 3](#) 中讨论。

在我看来,ConTeXt 参考手册中对目录的解释有些令人困惑,我认为这是因为一次性引入了太多信息。ConTeXt 构建目录的机制有很多部分,因此试图一次性解释所有部分的文本很难清晰,尤其是对于新手读者。相比之下,wiki 上的解释基本上限于示例:对于学习“技巧”非常有用,但在我看来,对于理解机制及其工作原理是不够的。这就是为什么我决定在本介绍中使用的解释策略,首先假设一个并非严格正确(或不完全正确)的事情:即 ConTeXt 中存在一个叫做“目录”的东西。以此为基础,解释生成目录的“正常”命令,当这些命令及其配置被熟知后,我认为是时候介绍那些迄今为止被省略的机制部分的信息了——尽管是在理论层面而非更实践层面。了解这些额外的“部分”使我们能够创建比我们称之为“正常”的目录(由之前解释的命令创建)更加定制化的目录;然而,在大多数情况下,我们不需要这样做。

1.2 带标题的完全自动目录

根据文档中编号章节(part、chapter、section 等)生成自动目录(TOC)的基本命令是 `\completecontent` 和 `\placecontent`。这两个命令的主要区别在于,第一个命令会为目录添加一个“标题”;为此,它会在目录之前立即插入一个“未编号的章节”,其默认标题为“Table of Contents”(目录)。

因此,`\completecontent`:

- 在它出现的位置插入一个名为“Table of Contents”(目录)的新未编号章节。

我们回想一下,在 ConTeXt 中,用于生成与章节同级别的未编号章节的命令是 `\title`(见 [Section 2](#))。因此,实际上 `\completecontent` 插入的不是“章节”(`\chapter`),而是一个“标题”(`\title`)。我之所以没有这样说,是因为我认为读者在此处使用未编号章节命令的名称可能会感到困惑,因为术语“标题”(Title)也有更广泛的含义,读者很容易不将其与我们所指的具体章节层级联系起来。

- 这个“章节”(实际上是 `\title`)的格式与文档中其余未编号章节完全相同;默认情况下,这包括一个分页。
- 目录紧接在标题之后打印。

最初生成的目录是“完整的”,正如生成它的命令名称(`\completecontent`)所暗示的那样。但一方面,我们可以如 [Section 1.3](#) 所述限制目录的深度;另一方面,由于此命令对其在源文件中的位置“敏感”(参见下文关于 `\placecontent` 的说明),如果 `\completecontent` 不在文档开头,则生成的目录可能不完整。在源文件的某些位置,该命令甚至可能被明显忽略。如果发生这种情况,解决方案是使用「`criterium=all`」选项调用该命令。关于此选项,另请参见 [Section 1.3](#)。

要更改目录的默认标题, 我们使用 `\setupheadtext` 命令, 其语法为:

```
\setupheadtext[语言][元素=名称]
```

其中语言是可选的, 指的是 ConTeXt 使用的语言标识符(见 [Section 5](#)), 元素指的是我们要更改名称的元素(目录的情况下为「content」), 名称是我们想要赋予目录的标题。例如:

```
\setupheadtext[en][content=Contents]
```

将确保由 `\completecontent` 生成的目录标题为“Contents”而不是“Table of Contents”。

此外, `\completecontent` 允许与 `\placecontent` 相同的配置选项, 关于其解释, 我请读者参考(下一节)。

1.3 无标题的自动目录

用于插入无标题、根据文档章节命令自动生成的目录的通用命令是 `\placecontent`, 其语法为:

```
\placecontent[选项]
```

原则上, 目录将包含所有编号的章节, 尽管我们可以使用 `\setupcombinedlist` 命令(我们将在后面讨论)来限制其深度。因此, 例如:

```
\setupcombinedlist[content][list={chapter, section}]
```

会将目录的内容限制为章(chapter)和节(section)。

此命令的一个特点是它对其在源文件中的位置敏感。这通过几个例子很容易解释, 但如果我们想精确指定命令如何工作以及在每种情况下哪些标题包含在目录中, 则要困难得多。所以让我们从例子开始:

- 放在文档开头、第一个章节命令(根据情况是 part、chapter 或 section)之前的 `\placecontent` 将生成完整的目录。

我不太确定默认生成的目录是否“完整”, 我相信它确实包含了足够多的章节层级, 在大多数情况下是完整的; 但我怀疑它不会超过第八级章节。无论如何, 如上所述, 我们可以通过以下方式调整目录达到的章节级别:

```
\setupcombinedlist[content][list={chapter, section, subsection, ...}]
```

- 相反, 位于 part、chapter 或 section 内部的同一个命令将专门生成该元素内容的目录, 换句话说, 即特定 part 的 chapters、sections 和其他更低层级, 或特定 chapter 的 sections(和其他层级), 或特定 section 的 subsections。

至于技术性和详细的解释,为了正确理解 `\placecontent` 的默认操作,必须记住,各个 section 实际上是 ConTeXt LMTX 的“环境”,以 `\startSectionType` 开始,以 `\stopSectionType` 结束,并且可以包含在其他更低级别的 section 命令中。因此,考虑到这一点,我们可以说 `\placecontent` 默认生成的目录将仅包含:

- 属于放置命令的“环境”(章节级别)的元素。这意味着放在 chapter 中的命令不会包含来自其他 chapters 的 sections 或 subsections。
- 章节级别低于命令所在位置对应级别的元素。也就是说,如果命令位于 chapter 中,则仅包含 sections、subsection 和其他更低级别;但如果命令位于 section 中,则索引将被拆分以生成 subsection 级别的目录。

此外,要生成目录,要求 `\placecontent` 位于其所在 chapter 的第一个 section 之前,或位于其所在 section 的第一个 subsection 之前,等等。

我不确定上面的解释是否清楚。也许用一个比之前的例子更详细的例子,我们可以更好地理解我的意思:让我们想象以下文档结构:

- 第 1 章
 - ◊ 第 1.1 节
 - ◊ 第 1.2 节
 - * 第 1.2.1 小节
 - * 第 1.2.2 小节
 - * 第 1.2.3 小节
 - ◊ 第 1.3 节
 - ◊ 第 1.4 节
- 第 2 章

因此:放在第 1 章之前的 `\placecontent` 将生成完整的目录,类似于 `\completecontent` 生成的目录但没有标题。但如果命令放在第 1 章内部且在第 1.1 节之前,目录将只是该章的目录;如果放在第 1.2 节的开头,目录将只是该节的内容。但是,如果命令放在例如第 1.1 节和第 1.2 节之间,它将被忽略。如果放在节的末尾或文档末尾,它也将被忽略。

当然,所有这些仅适用于命令不包含选项的情况。特别是, `criterion` 选项将改变这种默认行为。

在 `\placecontent` 允许的选项中,我将只解释其中两个,它们是设置目录最重要的选项,而且也是 ConTeXt 参考手册中(部分)记录的选项。 `criterion` 选项,影响目录内容与命令在源文件中位置的关系;以及 `alternative` 选项,影响要生成的目录的总体布局。

1.4 要纳入目录的元素: `criterion` 选项

上面已经解释了 `\placecontent` 关于命令在源文件中位置的默认操作。 `criterion` 选项会改变此操作。除其他外,它可以采用以下值:

- `all`: 无论命令在源文件中的位置如何, 目录都是完整的。
- `previous`: 目录仅包含 `\placecontent` 之前的(我们所在层级的)章节命令。此选项适用于写在文档或相关章节末尾的目录。
- `part, chapter, section, subsection...`: 表示目录应限于所指示的章节级别。
- `component`: 在多文件项目中(见 [Section 6](#)), 它将仅生成找到 `\placecontent` 或 `\completecontent` 命令的“组件”(component)对应的目录。

1.5 目录的布局: `alternative` 选项

`alternative` 选项控制目录的整体布局。其主要值可见于 [Table 8.1](#)。

alternative	目录条目的内容	注释
a	编号 – 标题 – 页码	每个条目一行
b	编号 – 标题 – 空格 – 页码	每个条目一行
c	编号 – 标题 – 引导点 – 页码	每个条目一行
d	编号 – 标题 – 页码	连续目录
e	标题	带框
f	标题	左对齐, 右对齐或居中
g	标题	居中

表 8.1 目录的格式化方式

前四个替代值提供每个章节的所有信息(其编号、标题和起始页码), 因此适用于纸质和电子文档。最后三个替代值仅提供标题信息, 因此仅适用于电子文档, 只要目录包含指向该章节的超链接(这在 ConT_EXt 中默认发生), 就不需要知道章节开始的页码。

此外, 我相信要真正欣赏各种替代方案之间的差异, 最好的办法是让读者生成一个测试文档, 在其中详细分析它们。

1.6 目录条目的格式

我们已经看到, `\placecontent` 或 `\completecontent` 的 `alternative` 选项允许我们控制目录的总体“布局”, 即每个标题将显示哪些信息, 以及不同标题之间是否有换行分隔。对每个目录条目的最终调整是使用 `\setuplist` 命令完成的, 其语法如下:

`\setuplist[元素][配置]`

其中元素指的是特定类型的章节。这可以是 `part`、`chapter`、`section` 等。我们也可以同时配置多个元素, 用逗号分隔。配置有多达 54 种可能性, 其中许多, 像往

常一样,没有明确记录;但这并不妨碍那些有记录的,或者那些不够清晰的,能够完全调整目录。

我现在将解释最重要的选项,根据它们的用途进行分组,但在深入之前,让我们记住,根据 `alternative` 的值,一个目录条目最多可以有三个不同的组成部分:章节编号、章节标题和页码。配置选项允许我们全局或单独配置各个组成部分:

- 包含(或不包含)不同组成部分:如果我们选择的替代方案除了标题之外还包括章节编号和页码(替代方案『a』、『b』、『c』或『d』),则选项 `headnumber=no` 或 `pagenumber=no` 意味着对于我们正在配置的特定级别,不显示章节编号(`headnumber`)或页码(`pagenumber`)。
- 颜色和样式:我们已经知道,目录中特定章节生成的条目(取决于替代方案)最多可以有三个不同的组成部分:章节编号、标题和页码。我们可以使用 `style` 和 `color` 选项共同指示这三个组成部分的样式和颜色,或者通过 `numberstyle`、`textstyle` 或 `pagestyle`(用于样式)以及 `numbercolor`、`textcolor` 或 `pagecolor`(用于颜色)为每个组成部分单独设置。

要控制每个条目的外观,除了样式本身之外,我们还可以对整个条目或其不同元素应用某些命令。为此,有 `command`、`numbercommand`、`pagecommand` 和 `textcommand` 选项。这里指示的命令可以是标准的 ConTeXt 命令或我们自己创建的命令。章节编号、标题文本和页码将作为参数传递给 `command` 选项,而章节标题将作为参数传递给 `textcommand`,页码传递给 `pagecommand`。因此,例如,以下指令将确保章节标题以(伪)小型大写字母书写:

```
\setuplist[section][textcommand=\Cap]
```

- 与其他目录条目的分隔:`before` 和 `after` 选项允许我们指示在排版目录条目之前(`before`)和之后(`after`)执行的命令。通常这些命令用于设置间距或前后条目之间的某些分隔元素。
- 元素的缩进:使用 `margin` 选项设置,它允许我们设置正在配置的级别的条目的左缩进量。
- 嵌入目录的超链接:默认情况下,索引条目包含指向相关章节开始的文档页面的超链接。使用 `interaction` 选项,我们可以禁用此功能(`interaction=no`)或将超链接限制在索引条目的部分,可以是章节编号(`interaction=number` 或 `interaction=sectionnumber`)、章节标题(`interaction=text` 或 `interaction=title`)或页码(`interaction=page` 或 `interaction=pagenumber`)。
- 其他方面:
 - ◊ `width`:指定章节编号和标题之间的分隔距离。可以是一个尺寸,或者是关键字 `fit`,它设置章节编号的精确宽度。

- ◊ `symbol`: 允许将章节编号替换为“符号”。支持三个可能的值: `one`、`two` 和 `three`。此选项的值为 `none` 会从目录中移除章节编号。
- ◊ `numberalign`: 指示编号元素的对齐方式; 可以是 `left`、`right`、`middle`、`flushright`、`flushleft`。

在目录的众多配置选项中, 没有一个允许直接控制行距。默认情况下, 行距将适用于整个文档。然而, 通常目录中的行最好比文档的其余部分稍微“紧凑”一些。要实现这一点, 我们应该将生成目录的命令(`\placecontent` 或 `\completecontent`) 包含在一个设置了不同行距的组内。例如:

```
\start
\setupinterlinespace[small]
\placecontent
\stop
```

1.7 目录的手动调整

我们已经解释了生成目录的两个基本命令(`\placecontent` 和 `\completecontent`) 及其选项。使用这两个命令, 目录是自动生成的, 由文档中(或目录所指的文档块或段中)现有的编号章节构建而成。我现在将解释某些“设置”, 使目录的内容不那么“自动”。这意味着:

- 也可能在目录中包含一些未编号的章节标题。
- 可以手动向目录发送一个不对应于编号章节存在的特定条目。
- 可以从目录中排除特定的编号章节。
- 目录中反映的特定章节的标题可以不与文档正文中包含的标题完全相同。

1.7.1 在目录中包含未编号的章节

ConTeXt 构建目录的机制意味着所有编号章节都会自动包含在内, 正如我已经说过的(见 [Section 4.2](#)), 这取决于我们可以为每种章节类型使用 `\setuphead` 更改的两个选项(`number` 和 `incrementnumber`)。那里还解释了, 一个 `incrementnumber=yes` 且 `number=no` 的章节类型将是一个内部编号但外部不编号的章节。

因此, 如果我们希望某个特定的未编号章节类型——例如 `title`——包含在目录中, 我们必须更改该章节类型的 `incrementnumber` 选项的值, 将其设置为 `yes`, 然后将该章节类型包含在目录要显示的章节类型中, 这可以通过 `\setupcombinedlist` 完成, 如上所述:

```
\setuphead
```



```
[title]
[incrementnumber=yes]

\setupcombinedlist
[content]
[list={chapter, title, section, subsection, subsubsection}]
```

然后,如果我们愿意,可以使用 `\setuplist` 与任何其他条目完全相同的方式格式化此条目;例如:

```
\setuplist[title][style=bold]
```

注意:刚刚解释的过程将包含我们文档中相关未编号章节类型的所有实例(在我们的示例中是 `title` 类型的章节)。如果我们只想在目录中包含该章节类型的特定出现,最好使用下面解释的过程。

1.7.2 手动向目录添加条目

从源文件中的任何位置,我们可以向目录提交一个条目(模拟一个实际上不存在的章节)或一个命令。

要发送一个模拟实际上不存在的章节的条目,请使用 `\writetolist` 命令,其语法为:

```
\writetolist[章节类型][选项]{编号}{文本}
```

其中:

- 第一个参数指示此章节条目在目录中必须具有的级别:`chapter`、`section`、`subsection` 等。
- 第二个参数是可选的,允许以特定方式配置此条目。如果省略手动提交的条目,它将像第一个参数指示的级别的所有条目一样被格式化;尽管我必须指出,在我的测试中,我未能使其工作。

无论是在 ConTeXt 命令的官方列表中(见 [Section 6](#))还是在维基中,我们都被告知此参数允许与 `\setuplist` 相同的值,`\setuplist` 是允许我们格式化不同目录条目的命令。但是,我坚持认为,在我的测试中,我未能以任何方式更改手动发送的目录条目的外观。

- 第三个参数应该反映发送到目录的元素的编号,但在我的测试中我也无法让它工作。
- 最后一个参数包含要发送到目录的文本。

例如,如果我们想将特定的未编号章节添加到目录中,但仅限于那一章节,这很有用。在 [Section 1.7.1](#) 中,它解释了如何将整个类别的未编号章节添加到目录中;但如果我们只想将某个章节类型的特定出现添加到目录中,使用 `\writetolist` 命令

更方便。因此,例如,如果我们希望文档中包含参考书目的章节是一个未编号的章节,但仍然包含在目录中,我们可以这样写:

```
\subject{Bibliography}
\writetolist[section]{}{Bibliography}
```

请注意,我们对章节使用了 `section` 的未编号版本,即 `subject`,但我们手动将其添加到索引中,就好像它是一个编号章节(`section`)一样。

另一个旨在手动影响目录的命令是 `\writebetweenlist`,它用于从文档中的特定点向目录添加的不是条目本身,而是一个“命令”。例如,如果我们想在目录中的两个项目之间包含一条线,我们可以在文档中位于两个相关章节之间的任何位置写入以下内容:

```
\writebetweenlist[section]{\hrule}
```

1.7.3 从目录中排除属于某个包含在目录中的章节类型的特定章节

目录是根据 `\setupcombinedlist` 的 `list` 选项建立的“章节类型”构建的;因此,如果某个“章节类型”出现在目录中,则无法排除我们出于任何原因不希望在目录中出现的特定章节(属于该类型)。

通常,如果我们不希望某个章节出现在那里,我们会使用它的“未编号等价物”,例如,用 `title` 代替 `chapter`,用 `subject` 代替 `section` 等。这些章节不会被发送到目录,也不会被编号。

然而,如果出于任何原因,我们希望某个章节被编号但不出现在目录中,即使其他这种类型的章节出现了,我们可以使用一个“技巧”,即创建一个新的章节类型,它是相关章节的克隆。例如:

```
\definehead[MySubsection][subsection]
\section{第一节}
\subsection{第一小节}
\MySubsection{第二小节}
\subsection{第三小节}
```

这将确保在插入 `MySubsection` 类型的章节时,subsection 计数器会增加,因为此章节是 subsections 的“克隆”,但目录不会包含此类章节的标题或编号,因为默认情况下它不包含 `MySubsection` 类型。

1.7.4 章节标题文本在目录中与文档正文中的标题不同

如果我们不希望特定章节在目录中包含的标题与文档正文中显示的标题相同,可以使用以下方法。不要使用传统语法(章节类型{标题})创建章节,而是使用 `\章节类型[选项]` 或 `\start 章节类型[选项]`,并将我们希望写入目录的文本分配给 `list` 选项(见 [Section 3](#))。

例如,如果一个章节标题很长,并且希望在目录中使用截断版本,可以如下实现:

```
\startsection
[
  reference=sec:investigations,
  title={发现为什么海水沸腾,猪如何获得飞行能力,以及蛇鲨是否真的能用斩首刀击败},
  list={发现为什么海水沸腾等},
]
```

第2节 列表、组合列表以及基于列表的目录

在内部,对于 ConT_EXt 来说,目录只不过是一个“组合列表”(combined list),而组合列表,顾名思义,由简单列表组合而成。因此,ConT_EXt 构建目录的基本概念是列表。多个列表组合起来形成一个目录。默认情况下,ConT_EXt 包含一个名为「content」的预定义组合列表,到目前为止我们检查的命令:`\placecontent` 和 `\completecontent`,正是使用这个列表。

2.1 ConT_EXt 中的列表

在 ConT_EXt 中,“列表”是一系列编号元素,我们需要记住三件事:

1. 编号。
2. 名称或标题。
3. 找到它的页码。

这不仅发生在编号章节上,也发生在文档的其他元素上,如图像、表格等。通常,对于那些有以 `\place` 开头的命令的元素,例如 `\placetable`、`\placefigure` 等,都是如此。

在所有这些情况下,ConT_EXt 会自动生成一个列表,收集相关元素类型的各种出现、其编号、标题和页码。因此,例如,有一个章节列表,称为 `chapter`,另一个节的列表,称为 `section`;但也有一个表格列表(称为 `table`)和另一个图形列表(称为 `figure`)。ConT_EXt 自动生成的列表总是以它们存储的元素类型命名。

如果我们创建一种新的编号章节类型,也会自动生成一个列表:创建新的编号章节将隐含地创建存储它们的列表。如果对于默认未编号的章节,我们设置了 `incrementnumber=yes`(使其成为编号章节),我们也将隐含地创建一个以该名称命名的列表。

除了隐式列表(由 ConT_EXt 自动定义)之外,我们还可以使用 `\definelist` 创建自己的列表,其语法为:

```
\definelist[列表名称][配置]
```

列表中的项目通过以下方式添加：

- 在 ConT_EXt 预定义的列表中, 或者由于创建新的浮动对象而由它创建的列表中(见 Section 5), 每当向文档添加项目时, 无论是通过使用章节命令还是通过使用其他类型列表的 `\placeWhatever` 命令, 相应的条目都会自动插入到其列表中。例如: `\placefigure` 会将图像插入文档, 但也会将相应的条目插入列表。
- 在任何类型的列表中, 都可以使用 `\writetolist[列表名称]` 手动添加, 如 Section 1.7 的 Subsection 1.7.2 中所述。同样在该节中解释的 `\writebetweenlist` 命令也可用。

一旦创建了列表并包含了所有项目, 与之相关的三个基本命令是 `\setuplist`、`\placelist` 和 `\completelist`。第一个允许我们配置列表的外观; 后两个在文档中找到它们的位置插入相关列表。`\placelist` 和 `\completelist` 之间的区别类似于 `\placecontent` 和 `\completecontent` 之间的区别(见 1.2 和 1.3)。

因此, 例如:

```
\placelist[section]
```

将插入一个节的列表, 如果文档的交互性已启用, 并且如果在 `\setuplist` 中我们没有设置 `interaction=no`, 则包含指向它们的超链接。节的列表并不完全等同于基于节的目录: 目录的概念通常还包括更低级别(subsection、subsubsection 等)。但节的列表将仅包含节本身。

这些命令的语法是:

```
\placelist[列表名称][选项]
```

```
\setuplist[列表名称][配置]
```

`\setuplist` 的选项已在 Section 1.6 中解释, 而 `\placelist` 的选项与 `\placecontent` 的选项相同(见 Section 1.3)。

2.2 图像、表格和其他项目的列表或索引

从以上所述可以看出, 由于 ConT_EXt 会自动创建使用 `\placefigure` 命令放置在文档中的图像的列表, 因此在文档的特定位置生成图形列表或索引就像使用 `\placelist[figure]` 命令一样简单。如果我们想要生成一个带标题的列表(类似于使用 `\completecontent` 得到的结果), 可以使用 `\completelist[figure]` 来完成。类似地, 我们可以将 `\placelist` 和 `\completelist` 用于 ConT_EXt 中其他四种预定义的浮动对象类型: 表格(「table」)、图形(「graphic」)、“间奏”(「intermezzo」)和化学公式(「chemical」)。

对于每种预定义的列表类型, ConT_EXt 已经包含了一个生成无标题列表的命令(`\placelistoffigures`、`\placelistoftables`、`\placelistoffgraphics`、`\placelistofintermezzi` 和 `\placelistofchemicals`), 以及另一

个生成带标题列表的命令 (`\completelistoffigures`、`\completelistoftables`、`\completelistofgraphics`、`\completelistofintermezzi` 和 `\completelistofchemicals`)，方式类似于 `\completecontent`。注意：虽然英语中“intermezzo”的常见复数可能是“intermezzos”，但 ConTeXt 的 `\placelistof...` 和 `\completelistof...` 命令使用 *intermezzi*。

同样地，对于我们自己创建的浮动对象（见 [Section 5](#)），`\placelistof<浮动对象名称>` 和 `\completelistof<浮动对象名称>` 命令将被自动创建。

对于我们使用 `\definelist` 自己创建的列表，我们可以使用 `\placelist[列表名称]` 或 `\completelist[列表名称]` 来创建索引。

2.3 组合列表

组合列表，顾名思义，是组合来自不同先前定义列表的项目的列表。默认情况下，ConTeXt 定义了一个名为「content」的默认内容索引组合列表，但我们可以使用 `\definecombinedlist` 创建其他组合列表，其语法为：

```
\definecombinedlist[名称][列表][选项]
```

其中：

- 名称是新的组合列表将拥有的名称。
- 列表指的是要组合的列表名称，用逗号分隔。
- 选项是列表的配置选项选择。这些选项可以在定义列表时指定，或者可能更可取的是在调用列表时指定。主要选项（已经解释过）是 `criterium`（[Section 1.3](#) 中的 [Subsection 1.4](#)）和 `alternative`（同一节中的 [Subsection 1.5](#)）。

使用 `\definecombinedlist` 创建组合列表的一个副作用是，还会创建一个名为 `\place 列表名称` 的命令，用于调用列表，即将其包含在输出文件中。例如：

```
\definecombinedlist[TOC]
```

将创建命令 `\placeTOC`；并且：

```
\definecombinedlist[content]
```

将创建命令 `\placecontent`。

但是等等，“`\placecontent`”？这不就是用于创建“普通”目录的命令吗？确实！这意味着标准目录实际上是由 ConTeXt 通过以下命令创建的：

```
\definecombinedlist
[content]
[part, chapter, section, subsection,
```

`subsubsection, subsubsubsection,
subsubsubsubsection]`

一旦定义了组合列表,我们就可以使用 `\setupcombinedlist` 来配置(或重新配置)它,该命令允许使用已经解释过的选项 `criterium`(见 [Subsection 1.4](#))和 `alternative`(见 [Subsection 1.5](#)),以及用于“更改”组合列表中包含的列表的 `list` 选项。

ConTeXt 命令的官方列表(见 [Section 6](#))没有将 `list` 选项列为 `\setupcombinedlist` 允许的选项之一,但在维基中该命令的几个示例中使用了它(而且,维基在专门介绍该命令的页面上也没有提到它)。我也检查过,该选项确实有效。

第3节 索引

3.1 生成索引

“主题索引”(subject index)通常位于文档末尾,由一系列重要术语组成,指示每个术语在文档中出现的页码。

在手排书籍的时代,生成主题索引是一项复杂且繁琐的任务。页面顺序的任何更改都可能影响索引中的许多(如果不是全部)条目。因此,索引并不常见。如今,计算机排版程序的存在意味着,虽然这项任务可能仍然繁琐,但不再那么复杂,因为计算机程序很容易保持与每个索引条目关联的数据是最新的。

要生成主题索引,我们需要:

1. 确定哪些单词、术语或概念将成为索引的一部分。这实际上是由作者(或其他同样熟悉文档主题的人)决定的任务。
2. 找到未来索引中每个条目在文档中出现的位置。然而,准确地说,当我们使用 ConTeXt 工作时,不仅仅是“找到”源文件中讨论概念或问题的位置,我们“标记”这些位置,通过插入一个命令,该命令随后将用于自动生成索引。这是繁琐的部分。
3. 通过在文档中我们选择的位置放置一个命令来生成和格式化索引。后者使用 ConTeXt 非常简单,只需要一个命令:`\placeindex`。

3.1.1 定义索引条目并标记源文件中它们所指的点

基础工作在于第二步。诚然,计算机系统也简化了这一过程,因为我们可以进行全局文本搜索来定位源文件中讨论特定主题的位置。但我们也不应盲目依赖这种文本搜索:一个好的主题索引必须能够检测到讨论特定主题的每个位置,即使没有使用“标准”术语来指代它。此外,通常会有一些地方使用了特定术语,但并非以值得在索引中提及的说明性方式。

为了“标记”源文件中的实际点,并将其与将出现在索引中的单词、术语或想法相关联,我们使用 `\index` 命令,其语法如下:

`\index[排序用文本]{索引条目}`

其中排序用文本是一个可选参数,用于指示与索引条目本身不同的文本,以便按字母顺序排序,而索引条目是将出现在索引中的文本,与此标记相关联。我们还可以应用我们希望使用的格式化功能,如果文本中出现保留字符,必须按照 ConT_EXt 中的常规方式书写。

以不同于实际书写方式对索引条目进行字母排序的可能性非常有用。例如,考虑本文档,为所有引用 `\TeX` 命令的地方生成一个索引条目可能很有用。您可能尝试

```
\index{\TeX}
```

但这将创建一个(在“t”下的)索引条目,内容是“T_EX”,而不是“\TeX”。如果您实际上希望在索引中看到 `\TeX`,那么您可以尝试

```
\index{\backslash TeX}
```

但这会将索引条目放在“b”下。(ConT_EXt LMTX 在选择索引条目所列的字母时,会忽略任何前导的“\”。)要让 `\TeX` 出现在“t”下,您可以使用

```
\index[tex]{\backslash TeX}
```

“索引条目”将是我们选择的任何内容。要使主题索引真正有用,我们在考虑文档读者最可能查找哪些概念时需要更加努力;因此,例如,将条目定义为“疾病,霍奇金氏”可能比定义为“霍奇金氏病”更好,因为更具包容性的术语是“疾病”。

如果索引有多个深度级别(最多支持三级),要将特定的索引条目关联到特定级别,使用“+”字符,如下所示:

```
\index{条目 1+条目 2}
```

```
\index{条目 1+条目 2+条目 3}
```

在第一种情况下,我们定义了一个名为「条目 2」的第二级条目,它将是「条目 1」的子条目。在第二种情况下,我们定义了一个名为「条目 3」的第三级条目,它将是「条目 2」的子条目,而「条目 2」又是「条目 1」的子条目。例如:

```
My \index{dog}{dog is a \index{dog+greyhound}{greyhound called Rocket}.  
He does not like \index{cat+stray}{stray cats}.
```

值得注意上述内容的一些细节:

- `\index` 命令通常放在与其关联的单词“之前”,并且通常不与其用空格分隔。这是为了确保命令与它链接的单词在同一页上:
 - ◊ 如果有一个空格分隔它们,则 ConT_EXt 可能会选择该空格作为换行位置,这也可能最终成为分页位置,在这种情况下,`\index` 命令将在一页上,而与之关联的单词在下一页上。

- 如果命令放在单词“之后”,则该单词可能会被连字过程打断音节,并且其两个音节之间的换行也可能是分页,在这种情况下,`\index` 条目将引用包含单词结尾的页面,而不是包含单词开头的页面。
- 第二级术语在 `\index` 命令的第二次和第三次使用中引入。
- 还要注意,在 `\index` 命令的第三次使用中,虽然出现在文本中的词是“stray”,但将插入到索引中的术语是“cat”。
- 最后:看看在两行中为主题索引编写了三个条目。我之前说过,标记源文件中的精确位置是繁琐的。我现在补充说,标记太多位置会适得其反。过于广泛的索引绝不一定比所有信息都相关、更简洁的索引更可取。这就是为什么我之前说过,决定哪些词将出现在索引中应该是作者有意识决定的结果。

如果我们希望索引真正有用,用作同义词的术语必须在索引中归入一个主术语之下。但由于读者可能会按任何其他主术语搜索索引中的信息,索引中通常包含指向其他条目的条目。例如,在民法手册的主题索引中,我们很容易找到类似这样的内容:

合同无效性
 参见无效。

我们不是使用 `\index` 命令来实现这一点,而是使用 `\seeindex`,其格式为:

`\seeindex[排序用文本] {条目 1} {条目 2}`

其中条目 1 是指向另一个条目的索引条目,而条目 2 是引用目标。对于我们之前的例子,我们会写:

`\seeindex{合同无效性}{无效}`

在 `\seeindex` 中,我们也可以使用“+”符号来指示其花括号中两个参数中任何一个的子级别。

3.1.2 生成最终索引

一旦我们在源文件中标记了索引的所有条目,实际生成索引就使用 `\placeindex` 或 `\compleindex` 命令来执行。这两个命令(概念上)扫描源文件中的 `\index` 命令,并生成索引应具有的所有条目的列表,将术语与找到 `\index` 命令的位置对应的页码相关联。然后,它们按字母顺序对出现在索引中的术语列表进行排序,并合并相同术语出现多次的情况,最后,将正确格式化的结果插入到最终文档中。

`\placeindex` 和 `\completeindex` 之间的区别类似于 `\content` 和 `\completecontent` 之间的区别(见 [Section 1.2](#)):`\placeindex` 仅限于生成索引并将其插

入,而 `\completeindex` 在最终文档中插入一个名为“Index”(索引)的新章节,索引将在该章节内排版。

3.2 格式化主题索引

主题索引是 ConTeXt 称为“寄存器”(register)的更通用结构的特定应用;因此索引使用以下命令格式化:

`\setupregister[index][配置]`

使用此命令,我们可以:

- 确定索引及其不同元素的外观。即:
 - ◊ 索引标题,通常是字母表中的字母,可以使用样式、颜色和命令进行修改。默认情况下,这些字母是小写的。使用 `alternative=A` 可以将它们设置为大写。
 - ◊ 条目本身及其页码。外观取决于实际条目的 `textstyle`, `textcolor`, `textcommand` 和 `deeptextcommand` 选项,以及页码的 `pagestyle`, `pagecolor` 和 `pagecommand` 选项。使用 `pagenumber=no` 我们也可以生成没有页码的主题索引(尽管我不知道这是否有用)。
 - ◊ `distance` 选项测量条目名称和页码之间的分隔宽度;然而,它也测量条目的缩进量。

`style`、`textstyle`、`pagestyle`、`color`、`textcolor` 和 `pagecolor` 选项的名称足够清晰,我认为可以告诉我们每个选项的作用。关于 `command`、`pagecommand`、`textcommand` 和 `deeptextcommand`,我请读者参考 [Section 4.4](#) 中关于章节命令配置的类似命名选项的解释。

- 设置索引的总体外观,其中包括在索引之前(before)或之后(after)执行的命令、需要的列数(n)、列是否应该相等(balance)、条目的对齐方式(align)等。

3.3 创建其他索引

我解释主题索引时,好像文档中只能有一个这样的索引;事实上,文档可以有任意多个索引。可以有一个个人姓名索引(一个“专名索引”),它收集文档中提到的人名并指示引用它们的位置;这是一种特定类型的索引。在法律文本中,我们也可以为《民法典》的提及创建一个特殊索引;或者,在像本文档这样的文档中,创建一个所解释宏的索引等。

要在文档中创建一个额外的索引,我们使用 `\defineregister` 命令,其语法为:

`\defineregister[索引名称][配置]`

其中索引名称是新索引将拥有的名称,而配置控制其工作方式。也可以稍后通过以下方式配置索引:

`\setupregister[索引名称][配置]`

一旦创建了名为索引名称的新索引,我们就可以使用 `\索引名称` 命令来标记此索引将拥有的条目,方式类似于使用 `\index` 标记条目的方式。`\see 索引名称` 命令也允许我们创建指向其他条目的条目。

例如:我们可以在此文档中创建一个 ConTeXt 命令的索引,使用命令:

`\defineregister[macro]`

这将创建 `\macro` 命令。这让我们可以将所有对 ConTeXt 命令的引用标记为索引条目,然后使用 `\placemacro` 或 `\completemacro` 生成索引。

创建一个新索引会启用 `\索引名称` 命令来标记其条目,并创建相应的用于生成索引的 `\place 索引名称` 和 `\complete 索引名称` 命令。但后两个命令实际上是应用于相关索引的两个更通用命令的缩写。因此,`\place 索引名称` 等同于 `\placeregister[索引名称]`,而 `\complete 索引名称` 等同于 `\completeregister[索引名称]`。

第9章 引用与超链接

Table of Contents: 第1节 引用类型; 第2节 内部引用; 2.1 引用目标中的标签; 2.2 在引用源点用于检索目标点数据的命令; 2.2.1 从标签检索信息的基本命令; 2.2.2 使用 `\ref` 命令检索与标签关联的信息; 2.2.3 检测链接指向的位置; 第3节 交互式电子文档; 3.1 启用文档中的交互性; 3.2 交互性的基本配置; 第4节 指向外部文档的超链接; 4.1 帮助排版超链接但不创建它们的命令; 4.2 建立链接的命令; 第5节 在最终 PDF 中创建书签;

第1节 引用类型

科技文献中充满了各种引用:

- 有时这些引用指向其他文档,这些文档是所述内容的基础,或者与所述内容相矛盾,或者对所述观点进行展开或进一步细化。在这些情况下,该引用被称为外部引用,如果文档要求学术严谨,则该引用采用文献引文的形式。
- 但文档中也常见到,在某个节中引用文档的另一节,此时该引用被称为内部引用。当文档中的某一点评论特定图像、表格、注释或类似元素的某个方面,并通过其编号或所在页码来指代时,也属于内部引用。

为了精确起见,内部引用需要指向文档中精确且易于识别的位置。因此,这类引用总是引用编号元素(例如,当我们说「见表 3.2」或「第 7 章」)或页码。像「如前所述」或「后文将见」这样的模糊引用不是真正的引用,排版它们没有特殊要求,也没有专门的工具来处理。此外,我个人不鼓励我的博士生或硕士生习惯性地使用这种做法。

内部引用通常也被称为「交叉引用」,但在本文档中,我通常只使用「引用」一词,需要明确时则使用「内部引用」。

为了澄清我在引用中使用的术语,我将引入引用的点称为源点,将其指向的位置称为目标点。这样来看,当源点和目标点位于同一文档中时,该引用称为内部引用;当它们位于不同文档中时,称为外部引用。

从文档排版的角度来看:

- 外部引用不构成特殊问题,因此原则上不需要任何工具来引入它们:我需要的来自目标文档的所有数据都是可用的,可以直接在引用中使用。然而,如果源文档是电子文档,并且目标文档也可在网络上访问,则可以在引用中包含超链接,以便直接跳转到目标。在这种情况下,源文档可以说是交互式的。

- 相比之下,内部引用确实对文档排版构成了挑战,因为任何有准备中等长度科技文档经验的人都知道,页面、节、图像、表格、定理等的编号几乎不可避免地会在文档准备过程中发生变化,这使得手动保持更新非常困难。

在计算机出现之前,作者会避免使用内部引用;而那些不可避免的引用,例如目录(如果附带各节的页码,则它是内部引用的一个例子),会在最后才编写。

即使是最受限的排版系统,如文字处理器,也允许包含某种内部交叉引用,例如目录。但这与 ConT_EXt 中包含的全面引用管理机制相比微不足道,该机制还可以将旨在保持引用更新的内部引用管理机制与超链接的使用结合起来,而超链接显然并非外部引用所独有。

第2节 内部引用

建立内部引用需要两样东西:

1. 在目标点处的一个标签或标识符。在编译时,ConT_EXt 会将特定的数据与该标签关联起来。关联什么数据取决于标签的类型;它可以是节编号、注释编号、图像编号、与编号列表中特定项关联的编号、节标题等。
2. 在源点处的一个命令。该命令访问与目标点标签相关联的数据,并将其插入到源点处。命令的选择取决于我们希望从标签中提取哪种数据插入到源点。

当我们考虑引用时,我们是从「源点 → 目标点」的角度思考的,因此似乎应该先解释与源点相关的事项,然后再解释与目标点相关的。然而,我认为如果反过来解释,会更容易理解引用的逻辑。

2.1 引用目标中的标签

在本章中,我所说的标签是指一个文本字符串,它将与引用的目标点相关联,并在内部用于检索有关引用目标点的某些信息,例如页码、节编号等。实际上,与每个标签关联的信息取决于创建它的过程。ConT_EXt 称这些标签为引用,但我认为“引用”这个术语含义更广,不够清晰。

与引用目标关联的标签:

- 要求文档中每个潜在的目标点具有唯一性,以便能够无歧义地识别。如果对不同的目标点使用相同的标签,ConT_EXt 不会抛出编译错误,但会导致所有引用都指向它找到的第一个标签(在源文件中),这会产生副作用:我们的一些引用可能是错误的,更糟糕的是,我们可能注意不到。因此,在创建标签时,确保我们分配的新标签之前没有被分配过是很重要的。
- 可以包含字母、数字、标点符号、空格等。当存在空格时,ConT_EXt 关于此类字符的一般规则仍然适用(参见 [Section 2.1](#)),因此,例如「My nice label」和「My nice label」被视为相同,即使它们使用了不同数量的空格。

由于标签可以包含哪些字符或多少个字符没有限制,我的建议是使用清晰且有助于我们理解源文件的标签名称,尤其是在我们可能很久之后重新阅读源文件时。这就是为什么我之前给出的例子(「My nice label」)不是一个好例子,因为它没有告诉我们标签指向的目标的任何信息。例如,对于本节,使用标签「sec:Target labels」会好得多。

要将特定目标与标签关联起来,主要有两种方法:

1. 通过用于创建标签所指向的元素的参数或命令选项。从这个角度来看,所有创建某种结构或文本元素的命令(这些元素可以成为引用目标)都包含一个称为「reference」的选项,用于包含标签。有时,标签本身就是整个参数的内容,而不是选项。

我在节命令中找到了一个很好的例子来说明这一点,如(Section 3)所知,节命令允许使用多种语法。在经典语法中,命令写作:

```
\section[Label]{Title}
```

而在 ConTeXt 特有语法中,命令写作

```
\startsection  
[title=Title, reference=Label, ...]
```

在这两种情况下,该命令都提供了引入一个标签的途径,该标签将与相应的节(或章、小节等)关联。

我说过,这种可能性存在于所有允许我们创建可以作为引用目标的文本元素的命令中。这些是所有可以编号的文本元素,包括节、各种浮动对象(表格、图像等)、脚注或尾注、引用、编号列表、描述、定义等。

当标签直接作为参数输入,而不是作为赋值的选项时,ConTeXt 允许将多个标签与同一个目标关联。例如:

```
\chapter[label1, label2, label3]{My chapter}
```

我不清楚让一个目标拥有多个不同标签有什么优势,并且猜测这样做不是因为它能提供优势,而是因为 ConTeXt 的某些内部要求适用于某些类型的参数。

2. 通过 `\pagereference`、`\reference` 或 `\textreference` 命令,其语法如下:

```
\pagereference[Label]  
\reference[Label]{Text}  
\textreference[Label]{Text}
```

- ◇ 使用 `\pagereference` 创建的标签允许我们检索页码。
- ◇ 使用 `\reference` 和 `\textreference` 创建的标签允许我们检索页码以及作为参数包含的任何关联文本。

在 `\reference` 和 `\textreference` 中,与标签关联的文本不会出现在命令所在位置(引用目标点)的最终文档中,但可以在引用的源点处被检索并打印出来。

我之前说过,每个标签都关联着关于目标点的某些信息。这些信息是什么取决于标签的类型:

- 所有标签都记住(即它们使得检索成为可能)创建它们的命令所在的页码。对于可能跨多页的节,该页码将是该节开始的页码。
- 使用创建编号文本元素(节、注释、表格、图像等)的命令插入的标签,记住与该元素关联的编号(节编号、注释编号等)。
- 如果该元素有一个标题(如节的情况,以及使用 `\placetable` 命令插入的表格等),它们将记住这个标题。
- 使用 `\pagereference` 创建的标签仅记住页码。
- 使用 `\reference` 或 `\textreference` 创建的标签还会记住这些命令作为参数所带的关联文本。

实际上我不确定 `\reference` 和 `\textreference` 命令之间的真正区别。我想猜测,这三个允许创建标签的命令的设计试图与三个允许从标签检索信息的命令(我们稍后会看到)保持平行;但事实上,根据我的测试,`\reference` 和 `\textreference` 似乎是冗余的命令。

2.2 在引用源点用于检索目标点数据的命令

我接下来要解释的命令从标签中检索信息,并且,如果我们的文档是交互式的,还会生成到引用目标的连接。但这些命令的重点是从标签中检索到的信息。如果我们只想生成连接,而不从标签中检索任何信息,则必须使用 [Section 4.2](#) 中解释的 `\goto` 命令。

2.2.1 从标签检索信息的基本命令

考虑到与目标点关联的每个标签可以存储不同的信息项,ConTeXt 包含三个不同的命令来检索这些信息是合乎逻辑的;根据我们想要从引用目标点检索哪种信息,我们选择以下命令之一:

- `\at` 命令允许我们检索标签的页码。
- 对于除了页码之外还记住元素编号(节编号、注释编号、项编号、表格编号等)的标签,`\in` 命令允许我们检索这个编号。
- 最后,对于记住与标签关联的文本(节标题、使用 `\placefigure` 插入的图像标题等)的标签,`\about` 命令允许我们检索这个文本。

这三个 `\at`、`\in` 和 `\about` 命令具有相同的语法:

```
\at{Text}[Label]
\in{Text}[Label]
\about{Text}[Label]
```

- 「Label」是我们希望从中检索信息的标签。
- 「Text」是写在我们要用命令检索的信息之前的文本。在文本和命令检索到的标签数据之间,将插入一个不间断空格,并且如果交互功能被启用,使得该命令除了检索信息外还生成一个允许跳转到目标点的链接,那么作为参数包含的文本将成为链接的一部分(它将是可点击的文本)。

因此,在下面的例子中,我们看到 `\in` 检索节编号, `\at` 检索页码。

```
In \in{Section}[sec:target labels],
that begins on \at{page}
[sec:target labels], the
characteristics of labels used for
internal references are explained.
```

在 [Section 2.1](#) 中,该节开始于 [page 127](#),解释了用于内部引用的标签的特性。

注意 ConT_EXt 自动创建了超链接(参见 [Section 3](#)),并且 `\in` 和 `\at` 作为参数接受的文本是链接的一部分。但如果我们以另一种方式书写,结果将是:

```
In section \in{}[sec:target
labels], that begins on page \at{}
[sec:target labels], the
characteristics of labels used for
internal references are explained.
```

在 [section 2.1](#) 中,该节开始于 [page 127](#),解释了用于内部引用的标签的特性。

文本保持不变,但前面的“section”和“page”这两个词不再包含在链接中,因为它们不再是命令的一部分。

如果 ConT_EXt 无法找到 `\at`、`\in` 或 `\about` 命令指向的标签,不会导致编译错误,但在最终文档中本应显示这些命令检索到的信息的位置,我们会看到「??」。

ConT_EXt 找不到标签有两个原因:

1. 我们写标签时犯了错误(无论是在源点还是目标点)。
2. 我们只编译了文档的一部分,而标签指向尚未编译的部分(参见 [Sections 5.1](#) 和 [6](#))。

在第一种情况下,需要修复错误。因此,当我们完成整个文档的编译(且第二种情况不再存在)后,最好在 PDF 中搜索所有出现的「??」,以检查文档中是否存在损坏的引用。

2.2.2 使用 `\ref` 命令检索与标签关联的信息

`\at`、`\in` 和 `\about` 各自检索标签的特定元素。另一个命令允许我们检索给定标签的任何元素。这就是 `\ref` 命令,其语法为:

```
\ref[Element to retrieve][Label]
```

其中第一个参数可以是:

- `text`: 返回与标签关联的文本。
- `title`: 返回与标签关联的标题。
- `number`: 返回与标签关联的编号。例如,在节中,返回节编号。
- `page`: 返回页码。
- `realpage`: 返回实际页码。
- `default`: 返回 ConTeXt 认为是标签的自然元素。通常这与 `number` 返回的内容一致。

实际上,`\ref` 比 `\at`、`\in` 或 `\about` 精确得多;例如,它区分页码和实际页码。如果文档的页码从 1500 开始(因为本文档是前一个文档的续篇),或者前言部分使用罗马数字编号并在前言结束后重新开始编号,那么页码可能与实际页码不一致。同样,`\ref` 区分与引用关联的文本和标题,而 `\about` 则不做区分。

如果使用 `\ref` 请求一个不具备特定信息的标签(例如,与脚注关联的标签的标题),该命令将返回一个空字符串。

2.2.3 检测链接指向的位置

ConTeXt 还有两个对链接方向敏感的命令。我所说的「链接方向」,是指确定源文件中的链接目标是在源点之前还是之后。例如:我们正在编写文档,想要引用一节,该节在最终的目录结构中可能出现在当前节之前或之后——我们还没有决定。在这种情况下,拥有一个命令,能够根据目标在最终文档中最终出现在源点之前还是之后来写入两种不同文本之一,将非常有用。针对此类需求,ConTeXt 提供了 `\somewhere` 命令,其语法为:

```
\somewhere{Text if before}{Text if after}[Label].
```

例如,在下面的文本中,我在源文件中使用了本章中的两个实际标签。一个出现在以下文本之前,一个出现在之后。(以下文本中的“before”和“after”这两个词不是红色的,不是因为 `\somewhere` 命令,而是因为它们是链接,已被配置为具有示例中看到的特定颜色。)

The hyperlink's address can also be detected by the `\type{\somewhere}` command. This way we can also find chapters or other text elements `\somewhere {before}{after} [sec:references]` and discuss their descriptions in some other place `\somewhere{before}{after} [sec:interactivity]`.

超链接的地址也可以通过 `\somewhere` 命令检测。这样我们就可以找到章节或其他文本元素 `before`, 并在其他地方讨论它们的描述 `after`。

另一个能够检测其指向的标签是在前还是在后的命令是 `\atpage`, 其语法为:

`\atpage[label]`

这个命令与前一个非常相似, 但它不让我们自己根据标签在前还是在后来编写文本, 而是为两种情况分别插入默认文本, 并且如果文档是交互式的, 还会插入一个超链接。

`\atpage` 插入的文本是与其作为参数的 `label` 相关的内容: 如果标签在命令之前, 则插入与「`precedingpage`」标签关联的文本; 在相反的情况下, 则插入与「`hereafter`」关联的文本。

当我写到这一点时, 我被之前的一个决定所困扰: 在本章中, 我决定将 `ConTEXt` 称为「reference」的东西称为「label」。这在我看来更清晰。但是, 由 `ConTEXt` 命令(例如 `\atpage`)生成的某些文本片段也被称为「labels」(这次是另一种含义)。(参见 [Section 5.3](#))。我希望读者不会感到困惑。我认为上下文可以让我们正确区分我在每种情况下所指的 `label` 的不同含义。

因此, 我们可以像更改任何其他标签的文本一样更改 `\atpage` 插入的文本:

```
\setuplabeltext[en][precedingpage=New text ]
\setuplabeltext[en][hereafter=New text ]
```

在这一点上, 我认为「`ConTEXt Standalone`」(我正在使用的发行版)中有一个小错误。检查 `ConTEXt` 中可以用 `\setuplabeltext` 更改的预定义标签名称, 有两对标签可能是 `\atpage` 使用的候选:

- 「`precedingpage`」和「`followingpage`」。
- 「`hencefore`」和「`hereafter`」。

我们可以假设 `\atpage` 会使用第一对或第二对。但实际上, 对于前面的项, 它使用「`precedingpage`」, 对于后面的项, 它使用「`hereafter`」, 我认为这是不一致的。

第3节 交互式电子文档

只有电子文档才能是交互式的; 但并非所有电子文档都是。电子文档是指存储在计算机文件中, 可以直接在屏幕上打开和阅读的文档。另一方面, 交互式电子文档是指配备了允许用户与之交互的实用程序的文档; 也就是说: 我们可以做的不仅仅是阅读。例如, 当文档包含执行某些操作的按钮、可以通过其跳转到文档另一点(或外部文档)的链接, 或者文档中有用户可以书写的区域、可以播放的视频或音频片段等时, 就存在交互性。

所有由 ConT_EXt 生成的文档都是电子文档(因为 ConT_EXt 生成 PDF, 而 PDF 本质上就是电子文档), 但它们并不总是交互式的。要使它们具有交互性, 需要如下一节所示明确指示。

但请记住, 尽管 ConT_EXt 可以生成交互式 PDF, 但要欣赏这种交互性, 我们需要一个能够支持它的 PDF 阅读器, 因为并非所有 PDF 阅读器都允许我们使用超链接、按钮以及交互式文档特有的类似功能。

3.1 启用文档中的交互性

ConT_EXt 除非明确指示, 否则不会创建具有交互功能的 PDF; 这通常在文档的前言中完成。启用此功能的命令是:

```
\setupinteraction[state=start]
```

通常, 当我们要生成交互式文档时, 此命令只会在文档前言中使用一次。但实际上, 我们可以根据需要多次使用它来改变文档的交互状态。「state=start」命令启用交互性, 而「state=stop」禁用交互性, 因此我们可以在文档中想要禁用的某些章或部分中禁用交互性。

我想不出任何理由希望交互式文档中包含非交互部分。但 ConT_EXt 理念的一个关键部分是, 如果某件事在技术上是可行的, 即使我们不太可能使用它, 也会提供相应的操作步骤。正是这种理念赋予了 ConT_EXt 如此多的可能性, 也使得像这样的简单介绍无法做到简短。

一旦建立了交互:

- 某些 ConT_EXt 命令默认包含超链接。例如:
 - ◊ 用于创建目录的命令, 原则上(除非另有明确指示)将是交互式的; 即, 单击目录中的条目将跳转到相应节开始的页面。
 - ◊ 我们在本章第一部分看到的用于内部引用的命令提供了交互性: 单击它们会使支持此功能的 PDF 阅读器跳转到引用目标。
 - ◊ 对于脚注和尾注, 单击正文中的注释锚点将带我们到注释所在的页面; 此外, 单击注释文本中的注释标记将带我们到正文中引用该脚注或尾注的位置。
 - ◊ 等等。
- 还有其他专门为交互式文档(例如演示文稿)设计的命令可用。演示文稿使用了许多与交互性相关的工具, 例如按钮、菜单、图像叠加、嵌入的声音或视频等。解释所有这些内容会太冗长, 而且演示文稿是一种相当特殊的文档类型。因此, 在下面的内容中, 我将只描述与交互性相关的一个功能: 超链接。

3.2 交互性的基本配置

`\setupinteraction` 除了启用或禁用交互之外,还允许我们配置与之相关的一些事项;主要(但不限于)是链接的颜色和样式。这通过以下命令选项完成:

- `color`: 控制链接的正常颜色。
- `contrastcolor`: 确定目标与源点在同一页面上的链接的颜色。我建议此选项始终设置为与前一个选项相同的内容。
- `style`: 控制链接样式;例如,「`style=\it`」会使链接以斜体排版。默认情况下,链接以粗体设置。
- `title, subtitle, author, date, keyword`: 分配给这些选项的值将转换为 ConTeXt 生成的 PDF 的元数据。
- `click`: 此选项控制链接在被点击时是否应高亮显示。

例如,本文档的 `\setupinteraction` 命令某一天如下所示:

```
\setupinteraction
[
  state=start,
  color=darkblue,
  contrastcolor=darkblue,
  style=,
  title={A Not So Short Introduction to ConTeXt LMTX (Community Edition)},
  author={Joaquín Ataz-López et al},
]
```

第 4 节 指向外部文档的超链接

我将区分不创建链接但有助于排版链接 URL 的命令和创建超链接的命令。让我们分别来看。

4.1 帮助排版超链接但不创建它们的命令

URL 通常很长,并且包含各种字符,甚至是 ConTeXt 中的保留字符,因此不能直接使用。此外,虽然可能希望有一个由简短的「人类可读」单词组成的可点击链接,但在其他情况下,可能希望写出完整的 URL。(如果 PDF 阅读器实现了交互式文档的功能,将鼠标光标悬停在链接上应该会使阅读器显示链接;但是,如果有人打印 PDF,则不会看到链接的实际目标。)

此外,当 URL 本身显示在文档中时,可能很难优雅地排版它,因为 URL 可能不包含 ConTeXt 可以插入换行的可接受位置。较长的 URL 可能超过一行的长度,但

即使较短的 URL 也可能碰巧出现在段落中一行的末尾,可能导致换行问题。此外,对 URL 进行断字以插入换行是不合理的,因为读者很难知道连字符是否实际上是 URL 的一部分。

ConT_EXt 提供了两个用于排版 URL 的实用程序,将依次讨论。第一个在必要时能较好地处理 URL 的换行。第二个可能功能更强,但它排版的 URL 不可点击(即,它们不是交互式的)。

`\useURL`

此命令允许我们编写一个 URL 并为其关联一个名称,以便当我们在文档中想要使用该 URL 时,可以通过关联的名称来调用它。我们可以在文档正文中的方便位置(但在我们想要使用 URL 之前)或在文档前言中使用 `\useURL`。对于将在文档中多次使用的 URL,在前言中定义它们特别有用。

该命令有两种用法:

1. `\useURL[Associated name] [URL]`
 2. `\useURL[Associated name] [URL] [] [Link text]`
- 在第一个版本中,URL 简单地与一个名称关联,我们将在文档中通过该名称调用它。也就是说,当调用 URL 时,完整的 URL 会显示在文档中,并且整个 URL 都是可点击的文本。
 - 在第二个版本中,最后一个参数提供了要在文档中排版的可点击文本。第三个参数的存在是为了我们想要将 URL 分成两部分的情况,以便第一部分包含访问地址,第二部分包含要打开的特定文档或页面的名称。例如,考虑解释 ConT_EXt 是什么的文档的地址:<http://www.pragma-ade.com/general/manuals/what-is-context.pdf>。这个地址可以完整地写在第二个参数中,留空第三个参数:

```
\useURL [WhatIsCTX]
[http://www.pragma-ade.com/general/manuals/what-is-context.pdf]
[]
[What is \ConTeXt?]
```

但我们也可以将其拆分为两个参数:

```
\useURL [WhatIsCTX]
[http://www.pragma-ade.com/general/manuals]
[what-is-context.pdf]
[What is \ConTeXt?]
```

在这两种情况下,我们都将该地址与词「WhatIsCTX」关联起来;要在此地址中包含一个链接,当我们使用链接创建命令(如 Section 4.2 中所述)时,我们只需使用「WhatIsCTX」作为该命令的参数,而不是 URL 本身,从而得到链接「What is ConT_EXt?»。您可以看到此文本是本文档中通常的链接颜

色,如果您正在使用支持交互的 PDF 阅读器阅读本文,当您将鼠标悬停在文本上时,它应该会显示链接的目标。

如果在文本中的任何时候,我们想要重现我们已经使用 `\useURL` 与名称关联的 URL,我们可以使用 `\url[Associated name]`,它将与该名称关联的 URL 插入到文档中。但此命令虽然将 URL 排版到文档中,却不会创建任何链接,如下所示:`\url[WhatIsCTX]` 产生「<http://www.pragma-ade.com/general/manuals/what-is-context.pdf>」。

使用 `\url` 写入的 URL 的显示格式不是由 `\setupinteraction` 建立的格式,而是由 `\setupurl` 专门为此命令建立的格式,它允许我们设置样式(选项 `style`)和颜色(选项 `colour`)。在本文档的前言中,两种情况下指定了相同的颜色,这就是为什么上面的 URL 与可点击链接的颜色相同。

`\hyphenatedurl`

此命令用于将在文档文本中完整写出的 URL。此命令指示 ConTeXt 在必要时在 URL 内部包含换行,以正确排版段落。其格式为:

`\hyphenatedurl{URLaddress}`

尽管 `\hyphenatedurl` 命令的名称如此,但它并不会对 URL 名称进行断字。它所做的是将 URL 中常见的某些字符视为可以插入换行的好位置(在这些字符之前或之后)。ConTeXt 允许我们将额外的字符添加到允许换行的字符列表中。我们三个命令可以做到这一点:

```
\sethyphenatedurlnormal{Characters}
\sethyphenatedurlbefore{Characters}
\sethyphenatedurlafter{Characters}
```

这些命令将它们作为参数的字符分别添加到允许之前和之后换行的字符列表、仅允许之前换行的字符列表以及仅允许之后换行的字符列表。

只要希望 URL 的完整文本出现在最终文档中(大致原样),就可以使用 `\hyphenatedurl`。它甚至可以用作 `\useURL` 的最后一个参数,在该命令的版本中,最后一个参数指定要在最终文档中显示的可点击文本。例如:

```
\useURL [WhatIsCTX]
[http://www.pragma-ade.com/general/manuals/what-is-context.pdf]
[]
[\hyphenatedurl{http://www.pragma-ade.com/general/manuals/what-is-context.pdf}]
```

在 `\hyphenatedurl` 的参数中,可以使用 ConTeXt 通常所有的保留字符,除了三个必须替换为命令的字符:

- % 必须替换为 `\letterpercent`
- # 必须替换为 `\letterhash`
- \ 必须替换为 `\letterescape` 或 `\letterbackslash`。

每次 `\hyphenatedurl` 插入换行时,它都会执行 `\hyphenatedurlseparator` 命令,默认情况下该命令什么都不做。但我们可以重新定义它,以便在换行之前插入一个代表性字符,类似于普通单词的处理方式,在单词中插入连字符以表示该单词在下一行继续。例如,使用:

```
\def\hyphenatedurlseparator{\curvearrowright}
```

下面这个特别长的网址在换行前会有『 $\curvearrowright$ 』,如下所示:

[https://wiki.contextgarden.net/Input_and_compilation/Project_and \$\curvearrowright\$ _file_management#Naming_conventions](https://wiki.contextgarden.net/Input_and_compilation/Project_and_file_management#Naming_conventions)

在此示例中,为了在 URL 中得到『 $\#$ 』,必须使用 `\letterhash`。

4.2 建立链接的命令

要创建指向已使用 `\useURL` 定义的 URL 的链接,我们可以使用命令 `\from`。该命令将可点击文本插入文档;该文本将是:如果使用双参数形式的 `\useURL`,则为完整的 URL 本身;如果使用四参数形式的 `\useURL`,则为第四个参数。其语法为:

```
\from[Name]
```

其中 `Name` 是先前使用 `\useURL` 与 URL 关联的名称。

要创建链接并将其与尚未预先定义的可点击文本关联,我们有 `\goto` 命令,它适用于内部和外部链接。其语法为:

```
\goto{Clickable text}[Target]
```

其中 `Clickable text` 是要在最终文档中显示的文本(鼠标点击该文本将产生跳转),而 `Target` 可以是以下之一:

- 我们文档中的一个标签。在这种情况下,`\goto` 将生成跳转,类似于我们已经讨论过的 `\in` 或 `\at` 命令。但与这些命令不同的是,在 `\goto` 使用的位置不会(自动)显示与标签关联的任何信息。例如,这里有一个指向“Internal References”节的链接,使用 `\goto{xyzzzy}[sec:references]` 生成:xyzzzy。(链接名称“xyzzzy”非常没有意义,但通过使用该名称,您可以相当确定链接文本不是从最初创建引用的地方检索的。)
- 一个 URL 本身。在这种情况下,必须通过如下写法明确指明它是一个 URL:

```
\goto{Clickable text}[url(URL)]
```

其中 URL 又可以是先前通过 `\useURL` 与 URL 关联的名称,或者是 URL 本身,在后一种情况下,在编写 URL 时,我们必须确保 ConT_EXt 的保留字符在 ConT_EXt 中正确书写。按照 ConT_EXt 规则书写 URL 不会影响链接的功能。

例如, 已经将 URL 与 WhatIsCTX 关联后, 我们可以写 `\goto{What is ConTeXt, anyway?}[url(WhatIsCTX)]`, 这会产生此链接: [What is ConTeXt, anyway?](#)。再举一个例子, `\goto` 命令可以创建一个「mailto:」链接; 假设您的 PDF 阅读器支持邮件链接并且您的系统已配置好邮件链接, 那么命令 `\goto{Email}[url(mailto:spam@example.com)]` 将在您的文档中插入一个文本为「Email」的链接, 如果点击该链接, PDF 阅读器将请求您的邮件程序开始向收件人 `spam@example.com` 撰写电子邮件。

第5节 在最终 PDF 中创建书签

PDF 文件可以有一个内部书签列表, 允许读者在 PDF 查看器程序的特殊窗口中看到这些书签, 并且只需单击其中一个书签即可在文档中移动。

默认情况下, ConTeXt 不会为输出的 PDF 提供书签列表。然而, 要获得与文档中各节相对应的书签列表, 只需包含 `\placebookmarks` 命令即可, 其语法为:

```
\placebookmarks[List of sections]
```

其中 `List of sections` 是一个逗号分隔的列表, 列出了应该出现在书签列表中的节级别。例如, 命令

```
\placebookmarks[chapter,section,subsection]
```

将为每一章、每一节和每一小节在书签列表中放置一个书签。

关于此命令, 请记住以下几点:

- `\placebookmarks` 必须放在 `\setupinteraction` 之后。当使用一个或多个环境文件时, 最好将 `\placebookmarks` 放在 `\setupinteraction` 命令之后不久(或紧接其后)。
- 由用户定义的节类型(使用 `\definehead`)并不总是位于书签列表中的正确位置。最好将它们排除在外。
- 如果任何节的节标题包含尾注或脚注, 脚注的文本将被视为书签的一部分。
- 作为参数, 除了节列表, 我们可以简单地指示象征性的词「all」, 顾名思义, 它将包含所有节; 然而, 根据我的测试, 这个词除了确实是一些节之外, 还包含了一些使用非节命令放置的文本, 因此生成的列表有些不可预测。

并非所有 PDF 查看器程序都允许我们查看书签; 而且许多允许的程序默认并未启用此功能。因此, 要检查此功能的结果, 我们必须确保我们的 PDF 阅读器程序支持此功能并已启用。例如, Acrobat 默认不显示书签, 尽管其工具栏上有一个按钮可以打开书签显示。

最后:默认情况下,根据 PDF 阅读器的不同,书签列表可能只显示顶级书签;在这种情况下,要查看次级书签,读者必须单击某个图形元素以显示下一级别的书签(依此类推)。如果您希望默认显示更多级别,可以给 `\placebookmarks` 添加第二个参数,指示默认情况下哪些级别的书签将是可见的。例如,以下命令告诉 ConTeXt 在书签列表中包含四个级别的节,并默认显示前三个级别:

```
\placebookmarks[chapter,section,subsection,subsubsection]  
[chapter,section,subsection]
```

Particular issues

第10章 字符、单词、文本与水平间距

Table of Contents: 第1节 获取键盘上通常无法直接输入的字符; 1.1 变音符号和特殊字母; 1.2 传统连字; 1.3 希腊字母; 1.4 各种符号; 1.5 定义字符; 1.6 使用预定义符号集; 第2节 特殊字符格式; 2.1 大写、小写与伪小型大写字母; 2.2 上标或下标文本; 2.3 照排文本; 第3节 字符间距与词间距; 3.1 自动设置水平间距; 3.2 改变单词内字符之间的间距; 3.3 用于在单词之间添加水平间距的命令; 第4节 复合词; 第5节 文本的语言; 5.1 设置和更改语言; 5.2 配置语言; 5.3 与特定语言关联的标签; 5.4 一些与语言相关的命令; 5.4.1 与日期相关的命令; 5.4.2 `\translate` 命令; 5.4.3 `\quote` 和 `\quotation` 命令;

所有文本文档的基本核心元素是字符:字符组成单词,单词又构成行,行形成段落,段落最终组成页面。

本章从“字符”开始,解释 ConT_EXt 中与字符、单词和文本相关的一些实用工具。

第1节 获取键盘上通常无法直接输入的字符

在编码为 UTF-8 的文本文件中(参见 [Section 1](#)),我们可以使用任何字符或符号,无论是现存语言的还是许多已消亡语言的。但是,由于键盘的能力有限,UTF-8 允许使用的大多数字符和符号通常无法直接从键盘获得。这一点尤其适用于许多变音符号,即放置在特定字母上方(或下方)以赋予其特殊含义的符号,同时也适用于许多其他字符,如数学符号、传统连字等。我们可以通过 ConT_EXt 使用命令来获得其中许多字符。

1.1 变音符号和特殊字母

几乎所有西方语言都有变音符号(英语在很大程度上是个重要例外),通常键盘可以生成相应区域语言的变音符号。因此,西班牙语键盘可以生成西班牙语所需的所有变音符号(基本上是重音和分音符号),以及加泰罗尼亚语(钝重音和软音符)或法语(软音符、钝重音和扬抑符)等其他语言使用的一些变音符号;但无法生成葡萄牙语中使用的一些变音符号,例如在「*navegação*」等单词中元音上的波浪号。

T_EX 是在美国设计的,那里的键盘通常无法让我们获得变音符号,因此 Donald Knuth 为其提供了一组命令,使我们能够获得几乎所有已知的变音符号(至少在使用拉丁字母的语言中)。如果我们使用西班牙语键盘,使用这些命令来获得可以直接从键盘获得的变音符号并没有太大意义。然而,了解这些命令的存在及其内容仍然很重要,因为西班牙语(或意大利语、法语.....)键盘无法让我们生成所有可能的变音符号。

名称	字符	缩写	命令
锐音	ú	\'u	\uacute
钝音	ù	\`u	\ugrave
扬抑符	û	\^u	\ucircumflex
分音符或元音变音	ü	\'u	\udiaeresis, \uumlaut
波浪号	ũ	\~u	\utilde
长音符号	ū	\=u	\umacron
短音符号	ǔ	\u u	\ubreve

表 10.1 重音及其他变音符号

在 Table 10.1 中,我们找到了获取这些变音符号的命令和缩写。在所有情况下,使用命令还是缩写并不重要。在表中,我使用了字母『u』作为示例,但这些命令适用于任何元音以及一些辅音和半元音。

- 由于大多数缩写命令是控制符号(参见 Section 2),带有变音符号的字母可以紧跟在命令后面书写,也可以与命令分开书写。例如,要获得葡萄牙语的『ã』,我们可以写入 `\~a` 或 `\~_a` 字符。¹ 但对于短音符号 (`\u`),由于它是一个控制词,空白字符是必需的。
- 对于命令的长格式,带有变音符号的字母将是命令名称的第一个字母。例如, `\emacron` 将在小写『e』上方放置长音符号 (ē), `\Emacron` 将在大写『E』上方执行相同操作 (Ē),而 `\Amacron` 将在大写『A』上方执行相同操作 (Ā)。

虽然 Table 10.1 中的命令适用于元音和一些辅音,但还有其他命令可以生成一些仅适用于一个或几个字母的变音符号和特殊字母。它们显示在 Table 10.2 中。

名称	字符	缩写	命令
斯堪的纳维亚 O	ø, Ø	\o, \O	
斯堪的纳维亚 A	å, Å	\aa, \AA, {\r a}, {\r A}	\aring, \Aring
波兰语 L	ł, Ł	\l, \L	
德语 Eszett	ß	\ss, \SS	
不带点的『i』和『j』	ı, Ĳ	\i, \j	
匈牙利语 umlaut	ű, Ű	\H u, \H U	
软音符	ç, Ç	\c c, \c C	\ccedilla, \Ccedilla

表 10.2 更多变音符号和特殊字母

我想指出,上表中的一些命令是从其他字符生成字符的,而其他命令仅在我们使用的字体明确提供了该字符时才有效。因此,对于德语 Eszett (ß),表中显示了两个命令但只有一个字符,因为我用于此文本的字体仅提供了大写版本的德语 Eszett(这很常见)。

这大概也是我无法让 `\AA` 生成大写斯堪的纳维亚 A 的原因,尽管「`{\r A}`」和 `\Aring` 能正确工作。

¹ 请记住,在本文档中,当空白字符很重要时,我们用『_』来表示它。

匈牙利语 umlaut 也适用于字母『o』, 软音符适用于字母『k』、『l』、『n』、『r』、『s』和『t』, 无论大小写。要使用的命令是 `\kcedilla`、`\lcedilla`、`\ncedilla` 等。

1.2 传统连字

连字是由两个或多个通常分开书写的字素结合而成。这种字符间的“融合”通常始于手写文本中的一种速记方式, 直到最终获得了一定的排版独立性。其中一些甚至被纳入通常排印字体中定义的字符之列, 例如 ampersand 符号『&』, 它最初是拉丁语系词(连词)「et」的缩写, 还有德语 Eszett (ß), 正如其名称所示, 它最初是『s』和『z』的组合。在某些字体设计中, 即使在今天, 我们也能追溯这两个字符的起源; 或者我之所以能看到, 是因为我知道它们在那里。具体来说, 使用 Pagella 字体的『&』和使用 Bookman 字体的『ß』可以明显看出其起源。

作为练习, 我建议(在阅读 [Chapter 6](#) 之后, 其中解释了如何操作)尝试用这些字体以足够大的尺寸(例如 30 pt)打印这些字符, 以便能直觉感受它们原有的组成部分。

其他不那么流行但至今仍偶尔使用的传统连字是拉丁语词尾「oe」和「ae」, 它们有时被写成『œ』或『æ』以表示它们在拉丁语中构成双元音。这些连字可以通过 [Table 10.3](#) 中的命令在 ConT_EXt 中实现。

连字	缩写	命令
æ, Æ	<code>\ae</code> , <code>\AE</code>	<code>\aeligature</code> , <code>\AEligature</code>
œ, Œ	<code>\oe</code> , <code>\OE</code>	<code>\oeligature</code> , <code>\OEligature</code>

表 10.3 传统连字

西班牙语(卡斯蒂利亚语)中曾经有一个传统连字, 如今通常不在字体中出现了, 那就是『D』: 它是『D』和『E』的缩写。据我所知, ConT_EXt 中没有命令让我们使用这个符号,¹ 但我们可以自己创建一个, 如 [Section 1.5](#) 中所述。

除了前面我称之为“传统”的连字(因为它们源于手写体)之外, 印刷术发明后, 还发展出了一些印刷文本连字, 我称之为「排印连字」。在 ConT_EXt 中, 这些被视为字体「实用工具」, 并由程序自动管理, 尽管我们可以通过 `\definefontfeature` 来影响这些字体实用工具的处理方式(包括连字)(本入门指南中不对此进行解释)。

1.3 希腊字母

在数学和物理公式中经常使用希腊字符。这就是为什么 ConT_EXt 包含了生成全部大小写希腊字母的可能性。这里的命令基于所讨论的希腊字母的英文名称构建。如果第一个字符用小写书写, 我们将获得小写希腊字母; 如果使用大写书写, 我们将获得大写希腊字母。例如, 命令 `\mu` 将生成该字母的小写版本(μ), 而 `\Mu` 将生

¹ 相比之下, 在 L^AT_EX 中, 我们可以使用「fontenc」包实现的 `\DH` 命令。

成大写版本 (M)。在 Table 10.4 中,我们可以看到哪个命令生成希腊字母表中的每个字母,包括小写和大写。

英文名称	字符(小写 / 大写)	命令(小写 / 大写)
Alpha	α , A	<code>\alpha</code> , <code>\Alpha</code>
Beta	β , B	<code>\beta</code> , <code>\Beta</code>
Gamma	γ , Γ	<code>\gamma</code> , <code>\Gamma</code>
Delta	δ , Δ	<code>\delta</code> , <code>\Delta</code>
Epsilon	ϵ , ε , E	<code>\epsilon</code> , <code>\varepsilon</code> , <code>\Epsilon</code>
Zeta	ζ , Z	<code>\zeta</code> , <code>\Zeta</code>
Eta	η , H	<code>\eta</code> , <code>\Eta</code>
Theta	θ , ϑ , Θ	<code>\theta</code> , <code>\vartheta</code> , <code>\Theta</code>
Iota	ι , I	<code>\iota</code> , <code>\Iota</code>
Kappa	κ , $\varkappa$, K	<code>\kappa</code> , <code>\varkappa</code> , <code>\Kappa</code>
Lambda	λ , Λ	<code>\lambda</code> , <code>\Lambda</code>
Mu	μ , M	<code>\mu</code> , <code>\Mu</code>
Nu	ν , N	<code>\nu</code> , <code>\Nu</code>
Xi	ξ , Ξ	<code>\xi</code> , <code>\Xi</code>
Omicron	o , O	<code>\omicron</code> , <code>\Omicron</code>
Pi	π , ϖ , Π	<code>\pi</code> , <code>\varpi</code> , <code>\Pi</code>
Rho	ρ , ϱ , P	<code>\rho</code> , <code>\varrho</code> , <code>\Rho</code>
Sigma	σ , ς , Σ	<code>\sigma</code> , <code>\varsigma</code> , <code>\Sigma</code>
Tau	τ , T	<code>\tau</code> , <code>\Tau</code>
Ypsilon	υ , Y	<code>\upsilon</code> , <code>\Upsilon</code>
Phi	ϕ , φ , Φ	<code>\phi</code> , <code>\varphi</code> , <code>\Phi</code>
Chi	χ , X	<code>\chi</code> , <code>\Chi</code>
Psi	ψ , Ψ	<code>\psi</code> , <code>\Psi</code>
Omega	ω , Ω	<code>\omega</code> , <code>\Omega</code>

表 10.4 希腊字母表

请注意,对于某些字符 (epsilon、kappa、theta、pi、rho、sigma 和 phi) 的小写版本,存在两种可能的变体。

1.4 各种符号

与我们刚刚看到的字符一起,TeX (以及 ConTeXt) 提供了生成无数符号的命令。这样的命令有很多;我在 Appendix B 中提供了一个扩展的(尽管不完整)列表。

1.5 定义字符

如果我们需要使用任何无法从键盘直接输入的字符,我们总可以找到一个包含这些字符的网页,然后将它们复制到我们的源文件中。如果我们使用 UTF-8 编码(推荐),这几乎总是可行的。此外,在 ConTeXt wiki 中有一个页面,包含各种可以简单地复制粘贴到我们文档中的符号。要获取它们,请单击[此链接](#)。

然而,如果我们需要多次使用某个字符,那么复制粘贴并不是最有效的方式。更好的做法是定义该字符,使其与一个命令相关联,每次运行该命令就会生成该字符。为此,我们使用 `\definecharacter`,其语法为:

`\definecharacter Name Character`

其中:

- `Name` 是与新字符关联的名称。它不应是现有命令的名称,否则会覆盖该命令。
- `Character` 是每次运行 `\Name` 时生成的字符。我们可以通过三种方式指明该字符:
 - ◊ 直接将其写入或粘贴到我们的源文件中(如果我们从其他电子文档或网页复制了它)。
 - ◊ 通过指明该字符在当前使用的字体中关联的编号。要查看字体中包含的字符及其关联的编号,我们可以使用 `\showfont[Font name]` 命令。
 - ◊ 使用我们接下来将立即看到的组合字符构建命令之一来构建新字符。

与本入门指南中看到的其他 ConT_EXt 命令相比,`\definecharacter` 的语法非常不寻常。在 Knuth 的原始 T_EX 中,通常编写其参数由空白字符分隔的命令。然而,ConT_EXt 的设计倾向于将参数括在方括号或花括号中,因此除非你学习如何编写自己的命令并决定使用这样的语法,否则你可能不会看到很多具有这种语法的命令。

作为第一种用法的示例,让我们暂时回到关于连字的章节(Section 1.2)。在那里我提到了一个西班牙语中的传统连字,如今通常无法在字体中找到:『D』。我们可以将这个字符关联到 `\decontract` 命令,这样每当我们写入 `\decontract` 时就会生成该字符。我们可以这样做:

`\definecharacter decontract Đ`

要构建一个不在我们字体中、也无法从键盘获得的新字符,就像我刚才给出的例子那样,首先我们必须找到包含该字符的文本,复制它,并能够将其粘贴到我们的定义中。在我刚刚给出的实际示例中,我最初是从维基百科复制了『D』。

ConT_EXt 还包含一些允许我们创建组合字符的命令,这些命令可以与 `\definecharacter` 结合使用。我所说的组合字符是指那些带有变音符号的字符。命令如下:

```
\buildtextaccent Accent Character
\buildtextbottomcomma Character
\buildtextbottomdot Character
\buildtextcedilla Character
\buildtextgrave Character
\buildtextmacron Character
\buildtextogonek Character
```

例如:正如我们已经知道的,默认情况下 ConT_EXt 只有用于书写带有软音符的特定字母(c、k、l、n、r、s、t)的命令;这些字母是通常被纳入字体的带有软音符的版本。如果我们想使用带有软音符的『b』,我们可以使用 `\buildtextcedilla` 命令,如下所示:

```
\definecharacter bcedilla {\buildtextcedilla b}
```

此命令将创建新的 `\bcedilla` 命令, 该命令将生成带有软音符的『b』:『b̃』。这些命令真正地「构建」了新字符, 即使我们的字体没有它, 也会生成它。这些命令所做的是将一个字符叠加在另一个字符上, 然后为该叠加命名。

`\buildtextaccent` 接受两个字符作为参数, 并将一个字符叠加在另一个字符上, 同时将第一个字符稍微抬高。尽管它被称为「`buildtextaccent`」, 但作为参数的字符不一定非得是重音符号。然而, 如果一个字符是重音符号, 叠加效果会更好, 因为在这种情况下, 将重音符号叠加在字符上, 重音符号不太可能覆盖字符。另一方面, 在正常情况下具有相同基线的两个字符的叠加会受到以下事实的影响: 该命令将一个字符略微抬高到另一个字符之上。这就是为什么我们不能使用此命令来获得上面提到的连字『D』, 因为如果我们写入

```
\definecharacter decontract {\buildtextaccent D E}
```

在我们的源文件中, 此命令产生的相对于『D』基线的轻微抬高意味着它产生的效果(『D̂』)不是很好。但如果字符的高度允许, 我们可以创建一个组合。例如,

```
\definecharacter unusual {\buildtextaccent \_ "}
```

将定义字符『_』, 它与 `\unusual` 命令相关联。

其余的 `build` 命令接受单个参数: 将为其添加由每个命令生成的变音符号的字符。下面我将展示每个命令的一个示例, 都基于字母『z』:

- `\buildtextbottomcomma` 在其所取字符下方添加一个逗号(『z̸』)。
- `\buildtextbottomdot` 在其所取字符下方添加一个点(『ẓ』)。
- `\buildtextcedilla` 在其所取字符下方添加一个软音符(『z̃』)。
- `\buildtextgrave` 在其所取字符上方添加一个钝音重音(『ẑ』)。
- `\buildtextmacron` 在其所取字符下方添加一个小横线(『z̄』)。
- `\buildtextogonek` 在其所取字符下方添加一个小钩(『z̧』)。

乍一看, `\buildtextgrave` 似乎是多余的, 因为我们已经有了 `\buildtextaccent`。但是, 如果你检查由这两个命令中的第一个生成的钝音重音, 它看起来会好一些。下面的示例显示了两个命令的结果, 字体大小足够大以分辨差异:

ẑ — ẑ

1.6 使用预定义符号集

ConTeXt 本身包含了许多预定义的符号集, 我们可以在文档中使用。这些集合被称为「cc」、「cow」、「fontawesome」、「jmn」、「mvs」和「nav」。(更多信息请参见 https://wiki.contextgarden.net/Characters_words_and_fonts/Emoji_and_symbols。*) 这些集合中的每一个还包含一些子集:

- **cc**(Creative Commons)包含集合「cc」。
- **cow**(Cow-TeX 徽标)包含「cownormal」和「cowcontour」。
- **fontawesome** 包含「fontawesome」。
- **jmn**(Janusz M. Nowacki 的导航符号)包含集合「navigation 1」、「navigation 2」、「navigation 3」和「navigation 4」。
- **mis**(排版项目符号)。
- **mvs**(Martin Vogel 的符号)包含「astronomic」、「zodiac」、「europe」、「martinvogel 1」和「martinvogel 2」。
- **nav**(Hans Hagen 的导航符号)包含「navigation 1」、「navigation 2」和「navigation 3」。
- **was**(Roland Waldi 的符号)包含「wasy general」、「wasy music」、「wasy astronomy」、「wasy astrology」和「wasy geometry」。

要查看每个集合中包含的特定符号, 使用以下语法:

```
\usesymbols[Set]
\showsymbolset[Subset]
```

例如: 如果我们想查看「mvs/zodiac」中包含的符号, 那么在源文件中写入:

```
\usesymbols[mvs]
\showsymbolset[zodiac]
```

我们将得到以下结果:

Aquarius	♈	♈
Aries	♈	♈
Cancer	♋	♋
Capricorn	♏	♏
Gemini	♊	♊
Leo	♌	♌
Libra	♎	♎
Pisces	♐	♐
Sagittarius	♐	♐
Scorpio	♏	♏
Taurus	♉	♉
Virgo	♍	♍

请注意, 每个符号的名称以及符号本身都被指示出来。`\symbol` 命令允许我们使用任何符号。其语法为:

```
\symbol[Subset][SymbolName]
```

其中 subset 是我们之前用 `\usesymbols` 加载的任何集合相关联的子集之一。例如, 如果我们想使用与水瓶座相关的占星符号(在 mvs/zodiac 中找到), 我们需要写入:

```
\usesymbols[mvs]
\symbol[zodiac][Aquarius]
```

这将给我们「♈」, 并且出于所有意图和目的, 它将被视为一个「字符」, 因此会受到打印时活动的字体大小的影响。我们还可以使用 `\definecharacter` 将相关符号与命令关联起来。例如:

```
\definecharacter Aries {\symbol[zodiac][Aries]}
```

这将创建一个名为 `\Aries` 的新命令, 该命令将生成字符「♈」。

我们还可以在 itemize 环境中使用这些符号。例如:

```
\usesymbols[mvs]
\definesymbol[1][{\symbol[martinvogel 2][PointingHand]}]
\definesymbol[2][{\symbol[martinvogel 2][CheckedBox]}]
\startitemize[packed]
\item item1 \item item2
\startitemize[packed]
\item item3 \item item4
\stopitemize
\item item5
\stopitemize
```

将产生

- item1
- item2
 - ◊ item3
 - ◊ item4
- item5

第2节 特殊字符格式

严格来说, 是影响所用字体、大小、样式或变体的格式命令。这些命令在 [Chapter 6](#) 中解释。然而, 从更宽泛的角度来看, 我们也可以将那些以某种方式改变其参数所取字符(从而改变其外观)的命令视为格式命令。我们将在本节中探讨其中

一些命令。其他命令,如下划线或文本上方/下方带线的文本(例如,我们想要提供空间来回答问题)将在 [Section 5](#) 中介绍。

2.1 大写、小写与伪小型大写字母

字母本身可以是大写或小写。对于 ConTeXt 来说,大写和小写字母是不同的字符,因此原则上它会按照写入的样子排版字母。然而,有一组命令允许我们确保它们所取的文本始终以大写或小写书写:

- `\word{text}`: 将其所取的文本转换为小写。
- `\Word{text}`: 将其所取文本的第一个字母转换为大写。
- `\Words{text}`: 将其所取的每个单词的第一个字母转换为大写;其余部分为小写。
- `\WORD{text}` 或 `\WORDS{text}`: 将其所取的文本写成大写。

与这些命令非常相似的是 `\cap` 和 `\Cap`。这些命令也会将其所取的文本大写,但随后会应用一个缩放因子,该因子等于字体更改命令中『x』后缀所应用的缩放因子(参见 [Section 4.2](#))。对于大多数字体,这些大写字母的高度将与小写字母相同,从而产生一种伪小型大写字母的效果。与真正的小型大写字母(参见 [Section 5.2](#))相比,它们具有以下优点:

1. `\cap` 和 `\Cap` 适用于任何字体,而真正的小型大写字母仅适用于明确包含它们的字体和样式。
2. 真正的小型大写字母是字体的一种变体,因此与粗体、斜体或倾斜等其他任何变体都不兼容。然而,`\cap` 和 `\Cap` 与任何字体变体完全兼容。

`\cap` 和 `\Cap` 的区别在于,前者将其参数中所有单词的字母都应用缩放因子,而 `\Cap` 不对每个单词的第一个字母应用缩放,从而实现了类似于在小型大写字母文本中使用真正大写字母的效果。如果『caps』所取的文本包含多个单词,则每个单词第一个字母中的大写字母大小将保持不变。

因此,在下面的示例中:

The UN, whose `\Cap{president}` has his
office at `\cap{uN}` headquarters...

The UN, whose PRESIDENT has his office at UN
headquarters...

我们首先需要注意到,第一次写「UN」(大写)和第二次写(小型大写字母,「UN」)之间的大小差异。在示例中,我第二次写了 `\cap{uN}`,以便我们可以看到 `\cap` 的参数是大写还是小写并不重要:该命令将所有字母转换为大写,然后应用缩放因子。这与 `\Cap` 形成对比,后者不缩放第一个字母。

这些命令也可以嵌套,在这种情况下,缩放因子将再次应用,导致进一步缩小,如下例所示,其中第一行的单词「capital」被再次缩放:

```
\cap{People who have amassed their
\cap{capital} at the expense of others
are more often than not
{\bf decapitated} in revolutionary
times}.
```

```
PEOPLE WHO HAVE AMASSED THEIR CAPITAL AT THE
EXPENSE OF OTHERS ARE MORE OFTEN THAN NOT DE-
CAPITATED IN REVOLUTIONARY TIMES.
```

应用于被 `\cap` 作用的文本的 `\nocap` 命令, 会取消其参数文本中的 `\cap` 效果。例如:

```
\cap{When I was One I had just begun,
when I was Two I was \nocap{nearly}
new (A.A. Milne)}.
```

```
WHEN I WAS ONE I HAD JUST BEGUN, WHEN I WAS
TWO I WAS nearly NEW (A.A. MILNE).
```

我们可以使用 `\setupcapitals` 来配置 `\cap` 的工作方式, 并且我们还可以定义该命令的不同版本, 每个版本都有自己的名称和具体配置。我们可以使用 `\definecapitals` 来做到这一点。

这两个命令的工作方式类似:

```
\definecapitals[Name][Configuration]
\setupcapitals[Name][Configuration]
```

`\setupcapitals` 中的「Name」参数是可选的。如果不使用它, 配置将影响 `\cap` 命令本身。如果使用它, 我们需要提供之前用 `\definecapitals` 分配给某个实际配置的名称。

在这两个命令中, 配置允许三个选项:「title」、「sc」和「style」, 前两个允许「yes」和「no」作为值。使用「title」我们指示大写是否也会影响标题(默认情况下会影响), 使用「sc」我们指示命令应使用真正的小型大写字母(「yes」)还是伪小型大写字母(「no」)。默认情况下, 它使用伪小型大写字母, 其优点是即使您使用的字体没有小型大写字母变体, 该命令也能工作。第三个值「style」允许我们指示一个样式命令, 该命令将应用于受 `\cap` 命令影响的文本。

2.2 上标或下标文本

我们已经知道(参见 [Section 1](#)), 在数学模式下, 保留字符「^」和「_」会将紧随其后的字符或组分别转换为上标或下标。为了在数学模式之外实现这种效果, ConT_EXt 包含以下命令:

- `\high{Text}`: 将其所取的文本写成上标, 如下所示: `normalsuperscript`。
- `\low{Text}`: 将其所取的文本写成下标, 如下所示: `normalsubscript`。
- `\lohi{Subscript}{Superscript}`: 将两个参数上下书写: 第一个参数在底部, 第二个在顶部, 这会产生一种奇特的效果:

`\lohi{below}{above}`

above
below

2.3 照排文本

拉丁语表达 *verbatim* (源自 *verbum* = 词 + 后缀 *atim*), 可以翻译为「逐字地」或「原原本本地」, 在像 ConT_EXt 这样的文本处理程序中使用, 指那些根本不应该被处理, 而应该原样转储到最终文件中的文本片段。ConT_EXt 提供了 `\type` 命令, 适用于不超过一行的短文本, 以及 `typing` 环境, 适用于多行文本。这些命令广泛用于计算机书籍中显示代码片段, ConT_EXt 会使用等宽字母 (如打字机或计算机终端) 来格式化这些文本。在这两种情况下, 文本都会被发送到最终文档而不经处理, 这意味着它们可以使用保留字符或特殊字符, 这些字符将原样 转录到最终文件中。同样, 如果 `\type` 的参数或 `\starttyping` 的内容包含命令, 该命令将被写入 最终文档, 但不会执行。

`\type` 命令具有以下特殊性: 其参数可以 包含在花括号内 (如同 ConT_EXt 中的常规做法), 但也可以使用任何其他字符来分隔 (包围) 参数。

当 ConT_EXt 读取 `\type` 命令时, 它假定命令名称后面紧随的第一个非空白字符将作为其参数的分隔符。如果此分隔符是左花括号『{』, `\type` 会将其与右花括号『}』配对。否则, `\type` 认为参数内容从分隔符后面的字符开始, 到该分隔符下一次 出现之前的字符结束。

一些示例将有助于我们更好地理解这一点:

```
\type {Tweedledum and Tweedledee}
\type 1Tweedledum and Tweedledee1
\type |Tweedledum and Tweedledee|
\type zTweedledum and Tweedledeez
\type (Tweedledum and Tweedledee(
```

第一个示例展示了我们可能期望的花括号的普通用法。请注意, 在第二个示例中, 命令名称后的第一个非空白字符是『1』, 第三个示例中是『|』, 第四个示例中是『z』; 因此: 在每种情况下, ConT_EXt 都会认为 `\type` 的参数是该字符与其下一次出现之间的所有内容。最后一个示例也是如此, 它也非常有启发性, 因为原则上我们可能会假设如果参数的开分隔符是『(』, 那么闭分隔符应该是『)』, 但事实并非如此, 因为『(』和『)』是不同的字符, 并且正如我所说, `\type` 会搜索与参数开头使用的字符相同的闭字符分隔符。

只有两种情况下 `\type` 允许开分隔符和闭分隔符不同:

- 如果开分隔符是『{』字符, 它会认为闭分隔符将是『}』。
- 如果开分隔符是『<』, 它会认为闭分隔符将是『>』。这种情况也是唯一的, 因为两个连续的字符被用作分隔符。

然而: `\type` 允许任何分隔符的事实并不意味着我们应该使用「奇怪」的分隔符。从源文件的可读性和可理解性的角度来看, 最好尽可能像 ConT_EXt 的常规做法一样, 用花括号分隔 `\type` 的参数; 当不可能时, 因为 `\type` 的参数中有花括号, 则使用一个符号: 最好不是 ConT_EXt 保留字符的符号。例如: `\type *This is a closing curly bracket: '}'*`。

`\type` 和 `\starttyping` 都可以用 `\setuptype` 和 `\setuptyping` 进行配置。我们还可以使用 `\definetype` 和 `\definetyping` 创建这些命令的自定义版本。关

于这些命令的实际配置选项,我参考「[setup-en.pdf](#)」(位于 ConT_EXt 独立发行版的目录 `tex/texmf-context/doc/context/documents/general/qrcs` 中)。

与 `\type` 非常相似的两个命令是:

- `\typ`: 工作方式类似于 `\type`, 但不禁用断字。
- `\tex`: 一个旨在书写关于 T_EX 或 ConT_EXt 的文本的命令: 它在其所取的文本前添加一个反斜杠。除此之外, 此命令与 `\type` 的不同之处在于它会处理其参数中找到的一些保留字符。特别是, `\tex` 内部的花括号将按照它们在 ConT_EXt 中的通常方式处理。

第3节 字符间距与词间距

3.1 自动设置水平间距

不同字符和单词之间的间距(在 T_EX 中称为水平间距)通常由 ConT_EXt 自动设置:

- 组成单词的字符之间的间距由字体本身定义, 除了等宽字体的情况外, 字体通常会根据要分隔的字符使用或多或少的空白, 例如, 『A』和『V』之间的间距(『AV』)通常小于『A』和『X』之间的间距(『AX』)。然而, 除了这些可能的变化(取决于所涉及的字母组合并由字体预定义)之外, 组成单词的字符之间的间距通常是一个固定且不变的度量。
- 相比之下, 同一行中单词之间的间距可以更具弹性。
 - ◇ 对于一行宽度必须与段落中其余行宽度相同的行中的单词, 单词间距的变化是 ConT_EXt 用来获得等宽行的机制之一, 详见 [Section 3](#)。在这些情况下, ConT_EXt 将在该行的所有单词之间建立完全相同的水平间距(除以下规则外), 同时确保段落不同行中单词之间的间距尽可能相似。
 - ◇ 然而, 除了需要拉伸或收缩单词间距以对齐行之外, 根据活动的语言, ConT_EXt 会考虑某些排版规则, 根据这些规则, 在特定位置, 与该语言相关的排版传统会添加一些额外的空白。例如, 在英文排版传统的某些部分中, 句号后会添加额外的空白。

这种额外间距适用于英语, 也可能适用于其他一些语言(尽管在某些情况下, 如今的英语出版商选择不在句号后添加额外间距), 但不适用于西班牙语, 因为那里的排版传统不同。因此, 我们可以使用 `\setupspacing[broad]` 临时启用此功能, 并使用 `\setupspacing[packed]` 禁用它。我们还可以更改西班牙语(以及任何其他语言, 包括英语)的默认配置, 如 [Section 5.2](#) 中所述。

3.2 改变单词内字符之间的间距

改变组成单词的字符的默认间距被认为是一种非常糟糕的排版实践,除非在标题和 headings 中。然而,ConTeXt 提供了一个命令来改变单词内字符之间的间距¹: `\stretched`,其语法如下:

`\stretched[Configuration]{Text}`

其中 *Configuration* 允许以下任何选项:

- **factor**: 一个整数或小数,表示所需的间距。这个数字不宜过大:0.05 的因子已经肉眼可见。因子为 0 产生的文本字母之间没有额外间距。
- **width**: 指示提交给命令的文本必须具有的总宽度,这样命令本身将计算在该空间中分布字符所需的间距。

根据我的测试,当使用 **width** 选项设定的宽度小于使用 **factor** 等于 0.25 表示文本所需的宽度时,**width** 选项和这个因子会被忽略。我猜测这是因为 `\stretched` 只允许我们增加单词内字符之间的间距,而不是减少它。但我不明白为什么表示文本所需的宽度(因子为 0.25)被用作 **width** 选项的最小度量,而不是文本的自然宽度(即因子为 0)。

- **style**: 应用于文本参数的样式命令或命令。
- **color**: 书写所取文本的颜色。

当使用 **factor** 时,默认情况下文本会被拉伸以适应该行的整个宽度。要仅按请求的因子拉伸文本,请向 `\stretched` 命令添加选项 **width=**。

因此,在下面的示例中,我们可以直观地看到该命令应用于同一句子但宽度不同时的效果:

<code>\stretched[width=4cm]{\bf test text}</code>	t e s t t e x t
<code>\stretched[width=6cm]{\bf test text}</code>	t e s t t e x t
<code>\stretched[width=8cm]{\bf test text}</code>	t e s t t e x t
<code>\stretched[width=9cm]{\bf test text}</code>	t e s t t e x t

不带参数使用时,该命令将使用行的整个宽度。另一方面,在此命令的参数文本中,命令 `\` 被重新定义,不再表示换行,而是插入水平间距。例如:

`\stretched{test\\text}`

产生:

t e s t t e x t

我们可以使用 `\setupstretched` 自定义命令的默认配置。

¹ 包含一个命令来做 ConTeXt 文档本身建议不要做的事情,这非常符合 ConTeXt 的理念。尽管追求排版完美,但目标也是让作者对其文档的外观拥有绝对控制权:无论好坏,归根结底都是作者的责任。



没有 `\definestretched` 命令允许我们设置与命令名称关联的自定义配置,然而,在官方命令列表中(参见 [Section 6](#)),它说 `\setupstretched` 来自 `\setupcharacterkerning`,并且有一个 `\definecharacterkerning` 命令。然而,在我的测试中,我未能通过后者为 `\stretched` 设置任何自定义配置,尽管我必须承认我也没有花太多时间尝试这样做。

3.3 用于在单词之间添加水平间距的命令

我们已经知道,增加单词之间的间距时,添加两个或多个连续的空格是没有用的,因为正如 [Section 2.1](#) 中所述,ConTeXt 会吸收所有连续的空格。如果我们希望增加单词之间的间距,我们需要使用以下允许我们这样做的命令之一:

- `\,` 在文档中插入一个非常小的空白(称为薄间距)。它用于,例如,分隔数字中的千位数(例如 1,000,000),或分隔单引号和双引号。例如:「`1\,473\,451`」将产生「1473451」。
- `\space` 或「`\_`」(反斜杠后跟一个空格,由于它是一个不可见字符,我将其表示为「`_`」)会引入一个额外的空格。
- `\enskip`、`\quad` 和 `\qqquad` 在文档中分别插入半个 *em*、1 *em* 或 2 *em* 的空白。请记住,*em* 是一个依赖于字体大小的度量(通常等于『M』的宽度),通常与字体的磅值大小一致。在这种情况下,使用 12 磅字体,`\enskip` 将提供 6 磅的间距,`\quad` 提供 12 磅,`\qqquad` 提供 24 磅。

我们可以使用 `\setbox0=\hbox{\quad}` 后跟 `\the\wd0` 来发现 *em* 的实际宽度,结果为 10.99396pt。此字面中『M』的实际宽度是 10.14053pt;差异必须由其他人解释。

除了这些提供精确度量空白的命令外,`\hskip` 和 `\hfill` 命令还引入了可变尺寸的水平间距:

`\hskip` 允许我们精确指示要添加多少空白。因此:

<code>This is \hskip 1cm 1 centimetre\\</code>		This is 1 centimetre
<code>This is \hskip 2cm 2 centimetres\\</code>		This is 2 centimetres
<code>This is \hskip 2.5cm 2.5 centimetres\\</code>		This is 2.5 centimetres

指示的间距可以是负数,这将导致一个文本叠加在另一个文本上。因此:

<code>This is farce rather than</code>		This is farce rather than
<code>\hskip -1cm comedy</code>		comedy

`\hfill` 则会根据需要引入尽可能多的空白来占据整行,从而允许我们创建有趣

<code>\hfill On the right\\</code>		On the right
<code>On both\hfill sides\\</code>	On both	sides

在本节中,「复合词」指的是形式上被理解为一个单词的词,而不是仅仅连接在一起的词。这并不总是一个容易区分的概念:「rainbow」明显由两个词组成(「rain + bow」),但英语使用者不会以任何其他方式看待这个组合词,只认为它是一个单词。另一方面,有些单词有时会借助连字符或反斜杠组合在一起。这两个词有不同的含义和用法,但它们被连接在一起(并且在某些情况下可能会变成一个单词,但还没有!)。因此,例如,我们可以找到像「French-Canadian」或「(inter)communication」这样的词(尽管我们也很可能找到「intercommunication」,并发现说大众最终接受了这两个词成为一个单词。语言就是这样演变的)。

「|」命令可用于告诉 ConT_EXt 两个词构成一个复合词。这个命令异常地不以反斜杠开头,并允许两种不同的用法:

- 在这两种情况下, ConT_EXt 都会知道它正在处理一个复合词, 并将应用适用于此类词的适当断词规则。使用两个连续的竖线或在它们之间插入单词分隔符的区别在于, 在第一种情况下, ConT_EXt 将使用预定义为 `\setuphyphenmark` 的分隔符, 换句话说, 即连字符, 默认情况下是「-」。因此, 如果我们写入「`picture||frame`」, ConT_EXt 将生成「`picture~frame`」。

「|」的正常用法是使用连字符,因为这是复合词之间通常使用的。但偶尔分隔符可以是括号,例如,如果我们想要「(inter)space」,或者可以是正斜杠,如「input/

output」。在这些情况下,如果我们希望应用复合词的标准断词规则,我们可以写入「(inter|)|space」或「input|/|output」。如前所述,「|=|」被认为是「|---|」的缩写,并插入一个 em 破折号作为分隔符(—)。

第5节 文本的语言

字符构成单词,这些单词通常属于某种语言。ConT_EXt 知道我们正在使用的书写语言非常重要,因为许多重要事项都取决于此。主要是:

- 单词断字。
- 某些单词的输出格式。
- 与所讨论语言的排版传统相关的某些排版事项。

5.1 设置和更改语言

ConT_EXt 假定语言是英语。有两种过程可以更改这一点:

- 使用 `\mainlanguage` 命令,在前言中使用以更改文档的主要语言。
- 使用 `\language` 命令,旨在更改文档中任何点的活动语言。

两个命令都期望一个由任何语言标识符(或代码)组成的参数。为了标识语言,我们使用 ISO 639-1 中规定的两个字母的国际语言代码,该代码与例如在网络上使用的代码相同,或者使用所讨论语言的英文名称,或者有时使用英文名称的某些缩写。

在 Table 10.5 中,我们找到了 ConT_EXt 支持的语言的完整列表,以及每种语言对应的 ISO 代码。此外,适当的地方还明确提供了某些语言变体的代码¹。

因此,例如,要将西班牙语(卡斯蒂利亚语)设置为文档的主要语言,我们可以使用以下三种中的任何一种:

```
\mainlanguage[es]
\mainlanguage[spanish]
\mainlanguage[sp]
```

要在文档内部启用特定语言,我们可以使用 `\language[Language code]` 命令,或者使用激活该语言的特定命令。因此,例如,`\en` 激活英语,`\fr` 激活法语,`\es` 激活西班牙语,或 `\ca` 激活加泰罗尼亚语。一旦某种实际语言被激活,它

¹ Table 10.5 是使用以下命令获得的列表的摘要:

```
\usemodule[languages-system]
\loadinstalledlanguages
\showinstalledlanguages
```

如果您在此文档上次更新(2026 年)之后很久阅读它,ConT_EXt 可能已经加入了其他语言,因此最好使用这些命令来显示更新的语言列表。

语言	ISO 代码	语言(变体)
南非荷兰语	af, afrikaans	
阿尔巴尼亚语	sq, als, albanian	
古希腊语	agr, ala, ancientgreek	
阿拉伯语	ar, arabic	ar-ac, ar-bh, ar-dz, ar-eg, ar-in, ar-ir, ar-jo, ar-kw, ar-lb, ar-ly, ar-ma, ar-om, ar-qa, ar-sa, ar-sd, ar-sy, ar-tn, ar-ye
亚美尼亚语	hy, hyc, hwy, armenian	
白俄罗斯语	be, belarussian	
缅甸语	my, burmese	
保加利亚语	bg, bulgarian	
孟加拉语	bn, bengali	
加泰罗尼亚语	ca, catalan	
中文	cn, cnr, chinese	cnr-cyrl, cnr-latn
克罗地亚语	hr, croatian	
捷克语	cs, cz, czech	
丹麦语	da, danish	
荷兰语	nl, nld, dutch	
英语	en, eng, english	en-gb, en-ca, uk, ukenglish, en-us, usenglish
世界语	esperanto	
爱沙尼亚语	et, estonian	
波斯语	fi, fa, per, persian, farsi	
芬兰语	fi, finnish	
法语	fr, fra, french	
盖尔语	ga, gaelic	
德语	de, deu, german	de-at, de-ch, de-ed, deo
希腊语	gr, greek	
古希腊语	agr, ancientgreek	
古吉拉特语	gu, gujarati	
希伯来语	he, yi, hebrew	
印地语	hi, hindi	
匈牙利语	hu, hungarian	
印度尼西亚语	id, indonesian	
意大利语	it, italian	
日语	ja, japanese	
卡纳达语	kn, kannada	
韩语	kr, korean	
拉丁语	la, latin	
立陶宛语	lt, lithuanian	
马其顿语	mk, macedonian	
马来语	ms, malay	
马拉雅拉姆语	ml, malayalam	
摩尔多瓦语	mo, moldovan	
挪威语	nb, bokmal, no, norwegian	nn, nynorsk
波斯语	pe, fa, persian	
波兰语	pl, polish	
葡萄牙语	pt, portuguese	pt-br
罗马尼亚语	ro, romanian	
俄语	ru, russian	
梵语	sa, sanskrit	
斯洛伐克语	sk, slovak	
塞尔维亚语	sr, serbian	sr-cyrl, sr-latn
斯洛文尼亚语	sl, slovene, slovenian	
西班牙语	es, sp, spanish	es-es, es-la
瑞典语	sv, swedish	
泰米尔语	ta, tamil	
泰卢固语	te, telugu	
泰语	th, thai	
土耳其语	tr, turkish	tk, turkmen
乌克兰语	ua, ukrainian	
越南语	vi, vietnamese	

表 10.5 ConTeXt 中的语言支持

将保持活动状态,直到我们明确切换到另一种语言,或者激活语言的组被关闭。因此,语言的工作方式类似于字体更改命令。但请注意,由 `\language` 命令或其缩写之一(`\en`、`\fr`、`\de` 等)设置的语言不会影响打印标签所使用的语言(参见 Section 5.3)。

尽管标记文档中所有不属于主要语言的单词和表达式的语言可能很费力,但如果我们希望获得一个排版正确的最终文档(尤其是在专业工作中),这样做很重要。我们不应该标记所有文本,而只标记不属于主要语言的部分。有时可以通过使用宏来自动化语言标记。例如,考虑本文档(最初是用西班牙语写的),其中不断引用 ConT_EXt 命令,而这些命令的原始语言是英语。我设计了一个宏,它除了以适当的格式和颜色书写命令外,还将其标记为英语单词。在我的专业工作中,我需要引用大量法语和意大利语参考文献,我在我的书目数据库中合并了一个字段来提取作品的语言,以便我可以在引用和参考文献列表中自动化语言指示。

如果文档使用两种具有不同字母表的语言(例如,西班牙语和希腊语,或英语和俄语),有一个技巧可以避免我们必须标记使用替代字母表的表达式的语言。具体来说,修改主要语言设置(参见下一节),使其同时加载使用不同字母表的语言的默认断字模式。例如,如果我们想使用英语和古希腊语,以下命令将使我们免于指定希腊文本的语言:

```
\setuplanguage[en][patterns={en, agr}]
```

这之所以有效,是因为英语和希腊语使用不同的字母表,因此两种语言的断字模式不可能发生冲突;因此我们可以同时加载它们。但是对于使用相同字母表的两种语言,同时加载断字模式将不可避免地导致错误的断字。

5.2 配置语言

ConT_EXt 将某些实用程序的功能与任何时候活动的特定语言相关联。可以使用 `\setuplanguage` 更改默认关联,其语法为:

```
\setuplanguage[Language][Configuration]
```

其中 `Language` 是我们要配置的语言的语言代码,`Configuration` 包含我们想要为该语言设置(或更改)的具体配置。具体来说,最多允许 32 个不同的配置选项,但我只讨论那些似乎适合本入门指南的选项。

- **date**: 允许我们配置默认日期格式。关于此的更多信息,请参见 [page 160](#)。
- **lefthyphenmin, righthyphenmin**: 支持单词断字时左侧或右侧必须有的最小字符数。例如 `\setuplanguage[en][lefthyphenmin=4]` 将不会以任何可能连字符左侧少于 4 个字符的方式断字。
- **spacing**: 此选项的可能值是「`broad`」或「`packed`」。在第一种情况下(`broad`),将应用英语的单词间距规则,这意味着在句号后且后面跟有其他字符时,将添加一定量的额外空白。另一方面,「`spacing=packed`」将阻止应用这些规则。对于英语,`broad` 是默认值。
- **leftquote, rightquote**: 分别指示 `\quote` 将用于其参数文本左侧和右侧的字符(或命令)(关于此命令,请参见 [page 161](#))。
- **leftquotation, rightquotation**: 分别指示 `\quotation` 将用于其参数文本左侧和右侧的字符(或命令)(关于此命令,请参见 [page 161](#))。

5.3 与特定语言关联的标签

ConT_EXt 的许多命令会自动生成某些文本(或标签),例如, `\placetable` 命令在插入的表格下方写入标签「Table xx」,或 `\placefigure` 插入标签「Figure xx」。

这些标签 对用 `\mainlanguage` 设置的语言敏感(但对用 `\language` 设置的语言不敏感),我们可以通过以下方式更改它们:

```
\setuplabeltext[Language][Key=Label]
```

其中 **Key** 是 ConT_EXt 识别标签的术语, **Label** 是我们希望 ConT_EXt 生成的文本。因此,例如,

```
\setuplabeltext[es][figure=Imagen~]
```

将确保当主要语言是西班牙语时,用 `\placefigure` 插入的图像不被称作「Figure x」,而是「Imagen x」。请注意,在标签文本本身之后,必须留一个空格以确保标签不会附着到下一个字符上。在示例中,我使用了保留字符「~」;我也可以写成「`[figure=Imagen{}]`」,将空格括在花括号中,以确保 ConT_EXt 不会将其去除。

我们可以用 `\setuplabeltext` 重新定义哪些标签? ConT_EXt 的文档在这方面没有人们希望的那么完整。2013 年的参考手册(对这条命令解释得最多)提到了「chapter」、「table」、「figure」、「appendix」...,并补充了「其他类似的文本元素」。我们可以假设这些名称将是所讨论元素的英文名称。



自由开源软件的一个优点是源文件可供用户使用;因此我们可以查看它们。我这样做了,通过窥探 ConT_EXt 的源文件,我发现了文件「`lang-txt.lua`」,位于 `tex/texmf-context/tex/context/base/mkiv1`,我认为它包含了预定义的标签及其不同的翻译。如果任何时候 ConT_EXt 生成了我们想要更改的默认文本,要查看与该文本关联的标签名称,我们可以打开相关文件并找到我们想要更改的文本。这样我们就可以看到与之关联的标签名称。

例如,如果我们在正在撰写的论文中看到标签「Master's thesis」,查看 `lang-txt.lua` 我们会看到标签的名称是「`mastersthesis`」。这对于说英语的人来说可能很容易猜到,但一个阿尔巴尼亚人想要知道「Tema e diplomës」的标签是什么,可能更难想象出「`mastersthesis`」;搜索这个文件将提供答案。

如果我们想在文档的某处插入与某个标签关联的文本,我们可以使用 `\labeltext` 命令。因此,例如,如果我想引用一个表格,以确保我以与 ConT_EXt 在 `\placetable` 命令中调用它相同的方式来命名它,我可以写入:「Just as shown in the `\labeltext{table}` on the next page.」。在 `\mainlanguage` 为英语的文档中,此文本将产生:「Just as shown in the 表 on the next page.」

您可能已经注意到示例中单词「on」之前没有空格。回想一下,标签文本(默认情况下)是定义为后面带一个空格的。如果「on」前面有一个空格,我们将看到这是一个在输出中出现两个连续空格的情况。(这是因为在输入文本中空格彼此不相邻;确切的规则有些棘手。)

¹ 您可能会惊讶地看到在 ConT_EXt LMTX 的入门指南中引用了 ConT_EXt Mark IV 的文件。事实证明,LMTX 在某些情况下会从 Mark IV「借用」数据和代码,即当 Mark IV 和 LMTX 之间没有变化时。可以预见,将来 LMTX 的更新和更改越多,它使用的 Mark IV 文件就越少。

一些可以用 `\setuplabeltext` 重新定义的标签默认是空的；例如「chapter」或「section」。这是因为默认情况下 ConT_EXt 不会向章节命令添加标签。如果我们想更改此默认操作，我们只需要在文档前言中重新定义这些标签，因此，例如，`\setuplabeltext[chapter=Chapter~]` 将确保章节前面带有单词「Chapter」和一个空格。

最后，重要的是要指出，尽管在 ConT_EXt 中，通常允许将多个逗号分隔的选项作为参数，最后一个选项可以以逗号结尾而不会出错。但在 `\setuplabeltext` 中，这会在编译时产生错误。

5.4 一些与语言相关的命令

5.4.1 与日期相关的命令

ConT_EXt 有三个与日期相关的命令，它们在运行时以活动语言生成输出。它们是：

- `\currentdate`：在主要语言为英语的文档中不带参数运行时，它以「Day Month Year」的格式返回系统日期。例如：「11 September 2020」。但我们也可以告诉它使用不同的格式（就像在美国和英语世界的其他一些地区那样，它们遵循将月份放在日期之前的系统，因此是臭名昭著的日期 9/11），或者包含星期几的名称（weekday），或者只包含日期的某些元素（day, month, year）。

要指示不同的日期格式，请使用「dd」或「day」表示月份的日期，「mm」表示月份数字，「month」表示月份名称，以及「MONTH」表示大写的月份名称。关于年份，「yy」只写最后两位数字，而「year」或「y」写全部四位。例如

`\currentdate[weekday, day, month]`

如果在 2020 年 9 月 9 日运行，将输出「Wednesday 9 September」。

如果您正在查看本文档的 PDF（而不是阅读 ConT_EXt 源代码），您会看到此 PDF 编译于 23 四月 2026。要像这样显示日期，有必要在月份和年份之间添加一个空格（否则 23 四月 2026 会显示出来），这是通过以下方式完成的：`\currentdate[day, month, {}, y]`。如果我们希望日期组成部分之间有某些分隔元素，我们必须显式地写入它们。

- `\date`：此命令不带任何参数运行时，产生与 `\currentdate` 完全相同的输出，即标准格式的实际日期。但是，可以给一个特定的日期作为参数。为此提供两个参数：使用第一个参数，我们可以指示要表示的日期对应的日（「d」）、月（「m」）和年（「y」），而使用第二个参数（可选），我们可以指示要表示的日期的格式。例如，如果我们想知道约翰·列侬和保罗·麦卡特尼相遇的那天是星期几，根据维基百科，这一事件发生在 1957 年 7 月 6 日，我们可以写

`\date[d=6, m=7, y=1957][weekday]`

这样我们就会发现这一历史性事件发生在一个星期六。

- `\month` 接受一个数字作为参数,并返回与该数字对应的月份名称。

5.4.2 `\translate` 命令

`translate` 命令支持一系列与特定语言关联的短语,这样,根据任何时候活动的语言,最终文档中将插入其中一个短语。在以下示例中,`translate` 命令用于将四个短语与西班牙语和英语关联起来,这些短语被保存在一个内存缓冲区中(关于 `buffer` 环境,请参见 [Section 6](#)):

```
\language[es]
\startbuffer
\starttabulate[|*{4}{lw(.25\textwidth)}|]
  \NC \translate[es=Su carta de fecha, en=Your letter dated]
  \NC \translate[es=Su referencia, en=Your reference]
  \NC \translate[es=Nuestra referencia, en=Our reference]
  \NC \translate[es=Fecha, en=Date] \NC\NR
\stoptabulate
\stopbuffer
```

因此,如果在文档中激活西班牙语的位置插入缓冲区,则会显示西班牙语短语;但如果插入缓冲区的位置激活了英语,则会插入英语短语。因此:

```
\language[es]
\getbuffer
  将生成:
Your letter dated      Your reference      Our reference      Date
而:
\language[en]
\getbuffer
  将生成
Your letter dated      Your reference      Our reference      Date
```

5.4.3 `\quote` 和 `\quotation` 命令

文本文档中最常见的排版错误之一是引号(单引号或双引号)被打开但没有明确关闭。为避免这种情况发生,ConTeXt 提供了 `\quote` 和 `\quotation` 命令,它们将作为其参数的文本加上引号;`\quote` 将使用单引号,`\quotation` 将使用双引号。

这些命令对语言敏感,因为它们使用为所讨论语言设置的默认字符或命令集来打开和关闭引号(参见 [Section 5.2](#));因此,例如,如果我们想使用西班牙语作为双引号的默认样式——即西班牙语、意大利语和法语典型的角引号(尖括号)——我们将写:

```
\setuplanguage[es][leftquotation=«, rightquotation=»].
```

然而,这些命令不管理嵌套引号;尽管我们可以利用 `\quote` 和 `\quotation` 是 ConT_EXt 所谓的 *delimitedtext* 的实际应用这一事实来创建实现此功能的实用程序,并且可以使用 `\definedelimitedtext` 定义进一步的应用。因此,以下示例:

```
\definedelimitedtext
[CommasLevelA]
[left=«, right=»]

\definedelimitedtext
[CommasLevelB]
[left=„, right=“]

\definedelimitedtext
[CommasLevelC]
[left=`, right=']
```

将创建三个命令,允许最多三个不同级别的引用。第一级使用「side」引号,第二级使用双引号,第三级使用单引号。

当然,如果我们使用英语作为主要语言,那么默认的单引号和双引号(弯引号,如本文档中所见,而不是直引号!)将自动使用。

第11章 段落、行与垂直间距

Table of Contents: 第1节 段落及其特性; 1.1 自动缩进段落首行; 1.2 特殊段落缩进; 第2节 段落之间的垂直间距; 2.1 `\setupwhitespace`; 2.2 之间没有额外垂直间距的段落; 2.3 在文档特定位置添加额外的垂直间距; 2.4 `\setupblank` 和 `\defineblank`; 2.5 获得更多垂直间距的其他方法; 第3节 ConTeXt 如何构建构成段落的行; 3.1 保留字符『~』的使用; 3.2 单词断字; 3.3 换行的容忍度级别; 3.4 在特定点强制换行; 第4节 行间距; 第5节 与行相关的其他事项; 5.1 将源文件中的换行转换为最终文档中的换行; 5.2 行号; 第6节 水平与垂直对齐; 6.1 水平对齐; 6.2 垂直对齐;

文档的整体外观主要由页面大小和布局(我们在 [Chapter 5](#) 中已经了解)、我们选择的字体(在 [Chapter 6](#) 中讨论过)以及行间距、段落对齐方式和段落间距等其他因素决定。本章重点讨论这些其他因素。

第1节 段落及其特性

段落是 ConTeXt 的基本文本单位。开始一个新段落有两种方式:

1. 在源文件中插入一个或多个连续的空行。
2. 使用 `\par` 或 `\endgraf` 命令。

第一种方式是通常使用的方法,因为它更简单,并且生成的源文件更易于阅读和理解。使用显式命令插入段落分隔符通常只在宏内部(参见 [Section 7.1](#))或表格单元格中(参见 [Section 3](#))进行。

在一个排版良好的文档中,从排版角度来看,段落之间在视觉上相互区分是很重要的。有两种典型的方法可以实现这一点:稍微缩进每个段落的第一行,或者稍微增加段落之间的空白。有时会结合使用这两种方法,尽管在某些地方不推荐这样做,因为被认为在排版上显得冗余。

我并不完全同意。单纯的首行缩进并不总能够足够地在视觉上突出段落之间的分隔;但不伴随缩进而增加间距会在段落从页面顶部开始时带来问题:我们可能不确定这是一个新段落,还是上一页的延续。结合使用两种方法可以消除疑虑。

首先,让我们看看如何使用 ConTeXt 实现行和段落的缩进。

1.1 自动缩进段落首行

段落首行自动插入小缩进的功能默认是禁用的。我们可以根据需要启用和禁用它。启用时,缩进的程度可以通过 `\setupindenting` 命令来指示,该命令允许使用以下值来指示是否应启用缩进:

- **always**: 所有段落都将缩进, 无一例外。
- **yes**: 启用正常 段落缩进。某些前面有额外垂直间距的段落, 如章节的第一个段落, 或某些环境之后的段落, 将不会缩进。
- **no, not, never, none**: 禁用段落首行自动缩进。

在启用自动缩进的情况下, 我们还可以通过相同的命令指示缩进多少。为此, 我们可以直接使用一个尺寸(例如 1.5cm)或象征性的词「small」、「medium」和「big」, 它们分别表示我们想要小、中或大的缩进。

在某些排版传统中(包括西班牙语), 默认缩进是两个 quad。在排版术中, quad(最初是 *quadrat*)是一种用于活字排版的金属隔片, 其高度和宽度等于所用字体的磅值大小。因此, 对于 12 磅的字体, quad 的尺寸将是宽 12 磅、高 12 磅。这个术语后来被用作排版术中两种常见空格大小的通用名称, 无论使用何种排版形式。ConT_EXt 有两个 quad 命令: `\quad` 生成一个上述类型的空格, `\qqquad` 生成两倍于此的空格, 但同样基于所使用的字体。对于 11 磅的字母, 两个 quad 的缩进将为 22 磅; 对于 12 磅的字母, 则为 24 磅。

当缩进启用时, 如果我们不想让某个段落缩进, 需要使用 `\noindentation` 命令。

通常, 我会在我的文档中使用 `\setupindenting[yes, big]` 启用自动缩进。但在本文档中, 我没有这样做, 因为如果启用了缩进, 大量的短句和示例会导致页面外观视觉上不整洁。

1.2 特殊段落缩进

突出段落的一种图形化方法是缩进段落的右侧或左侧(或两侧)。例如, 这用于块引用。

ConT_EXt 有一个环境允许我们改变段落缩进以突出段落中的文本。这就是「narrower」环境:

```
\startnarrower [Options] ... \stopnarrower
```

其中 Options 可以是:

- **left**: 缩进左边距。
- **Num*left**: 缩进左边距, 将正常 缩进量乘以 Num(例如 `2*left`)。
- **right**: 缩进右边距。
- **Num*right**: 缩进右边距, 将正常 缩进量乘以 Num(例如 `3*right`)。
- **middle**: 缩进两边距。这是默认值。
- **Num*middle**: 缩进两边距, 将正常 缩进量乘以 Num。

在解释选项时, 我提到了正常缩进; 这指的是「narrower」默认应用的左右缩进量。这个量 可以通过 `\setupnarrower` 配置, 该命令允许以下配置选项:

- **left**: 应用于左边距的缩进量。
- **right**: 应用于右边距的缩进量。
- **middle**: 应用于两边距的缩进量。
- **before**: 进入环境前要运行的命令。
- **after**: 退出环境后要运行的命令。

如果我们想在文档中使用 narrower 环境的不同配置, 我们可以使用 `\definewidth[Name] [Configuration]` 为每个配置分配不同的名称, 其中 Name 是与该配置关联的名称, Configuration 允许与 `\setupnarrower` 相同的选项。

第2节 段落之间的垂直间距

2.1 `\setupwhitespace`

正如我们已经从 (Section 2.2) 中知道的, 源文件中有多少个连续的空行对 ConTeXt 来说并不重要: 一个或多个空行只会在最终文档中插入一个段落分隔符。要增加段落之间的间距, 在源文件中添加额外的空行是无济于事的。相反, 此功能由 `\setupwhitespace` 命令控制, 该命令允许以下值:

- **none**: 表示段落之间不会有额外的垂直间距。
- **small, medium, big**: 分别插入小、中或大的垂直间距。这些值插入的实际间距大小取决于字体大小。
- **line, halfline, quarterline**: 根据行高来度量额外的空白, 分别插入额外的一行、半行或四分之一行的间距。
- **DIMENSION**: 为段落之间的间距设定一个特定的尺寸, 与字体大小无关。例如, `\setupwhitespace[5pt]`。

作为一般规则, 不建议为 `\setupwhitespace` 设置精确的尺寸值。最好使用象征性的值 small、medium、big、line、halfline 或 quarterline。这样做的原因有两个:

- 象征性值是所谓的弹性尺寸 (参见 Section 8.2), 这意味着它们有正常尺寸, 但允许该值有一定程度的增加或减少 (「拉伸」或「收缩」), 以帮助 ConTeXt 排版页面, 使得段落分隔在美学上相似。然而, 段落之间的固定分隔尺寸使得实现良好的文档分页更加困难。
- 象征性值 small、medium、big 等是基于字体大小计算的, 因此如果在某些部分字体大小发生变化, 段落之间的垂直间距量也会发生变化, 最终结果将始终是和谐的。相反, 垂直间距的固定值不会受到字体大小变化的影响, 这通常会转化为文档中空白分布不佳 (从美学角度而言) 且不符合排版调整规则的文档。

当为段落垂直间距设置了某个值后,还有两个额外的命令可用:`\nowhitespace`,它消除特定段落之间的任何额外间距;以及`\whitespace`,它执行相反的操作。然而,这些命令很少需要,因为 ConT_EXt 本身就能很好地管理段落之间的垂直间距;尤其是如果输入的是根据当前活动字体大小和间距计算的预定义尺寸之一作为值。

`\nowhitespace` 的含义是显而易见的。但 `\whitespace` 本身就不一定了,因为既然已经为所有段落普遍设置了垂直间距(ConT_EXt 猜想),那么为特定段落命令垂直间距有什么意义呢?然而,在编写高级宏时,`\whitespace` 在需要根据某个条件的值做出决策的循环上下文中可能很有用。这或多或少属于高级编程,我在此不作深入讨论。

2.2 之间没有额外垂直间距的段落

如果我们希望文档的特定部分中的段落之间没有额外的垂直间距,当然可以修改 `\setupwhitespace` 的通用配置,但这在某种程度上违背了 ConT_EXt 的理念,即通用配置命令应仅放在源文件的前言中,以实现文档一致且易于修改的通用外观。因此有了「packed」环境,其通用语法为:

```
\startpacked[Space] ... \stoppacked
```

其中 `Space` 是一个可选参数,指示环境内段落之间所需的垂直间距量。如果省略 `Space`,则不会应用额外的垂直间距。

2.3 在文档特定位置添加额外的垂直间距

如果在文档的某个特定位置,段落之间的正常垂直间距不够,我们可以使用 `\blank` 命令。不带参数使用时,`\blank` 和 `\blank[big]` 都将插入与 `\setupwhitespace` 设置的相同的垂直间距量。但我们可以在方括号之间指示一个特定的尺寸,或者根据字体大小计算的象征性值之一:small、medium 或 big。我们还可以将这些尺寸乘以某个整数,等等,例如,`\blank[3*medium]` 将插入相当于三个 medium 换行的间距。我们还可以将两个尺寸放在一起。例如,`\blank[2*big, medium]` 将插入两个 large 和一个 medium 的间距。

由于 `\blank` 旨在增加段落之间的垂直间距,如果在应该增加间距的两个段落之间插入了分页符,则 `\blank` 无效;如果我们连续插入两个或更多 `\blank` 命令,则只有一个会生效(即要插入间距最大的那个)。放置在分页符之后的 `\blank` 命令也没有任何效果。然而,在这些情况下,我们可以使用象征性词「force」作为命令选项来强制插入垂直间距。因此,例如,如果我们希望文档中的章节标题在页面上更靠下,使得该页的总长度小于其他页面(一种相对常见的排版实践),在配置 `\chapter` 命令时,我们应该写入类似于以下内容:

```
\setuphead
[chapter]
[
  page=yes,
```

```

before={\blank[4cm, force]},
after={\blank[3*medium]}
]

```

这一系列命令将确保章节总是在新页开始,并且章节标签向下移动 4 厘米。如果不使用「`force`」选项,则不会出现额外的间距。

2.4 `\setupblank` 和 `\defineblank`

之前,我说过不带参数使用的 `\blank` 等同于 `\blank[big]`。但是,我们可以使用 `\setupblank` 更改 `\blank` (不带参数)输出的间距量,例如将其设置为 `\setupblank[0.5cm]` 或 `\setupblank[medium]`;这不会影响 `\blank[big]` (或 `medium` 或 `small`)输出的间距量。不带参数使用时,`\setupblank` 将根据当前字体大小调整该值。

与 `\setupwhitespace` 一样,当 `\blank` 插入的空白值是预定义的象征性值之一时,它是一个允许进行一定调整的弹性尺寸。我们可以使用「`fixed`」来改变这一点,并且稍后可以使用 (「`flexible`」) 恢复默认值。因此,例如,对于双栏文本,建议设置 `\setupblank[fixed, line]`,当回到单栏时,设置 `\setupblank[flexible, default]`。

使用 `\defineblank`,我们可以将某个配置与一个名称关联起来。此命令的一般格式为:

```
\defineblank[Name] [Configuration]
```

一旦定义了我们的空白配置,我们就可以使用 `\blank[ConfigurationName]` 来调用它。

2.5 获得更多垂直间距的其他方法

在 $\text{T}_{\text{E}}\text{X}$ 中,插入额外垂直间距的命令是 `\vskip`。这个命令,像几乎所有 $\text{T}_{\text{E}}\text{X}$ 命令一样,也可以在 $\text{ConT}_{\text{E}}\text{Xt}$ 中工作,但强烈建议不要使用它,因为它会干扰 $\text{ConT}_{\text{E}}\text{Xt}$ 某些宏的内部功能。建议使用 `\godown` 来代替,其语法为:

```
\godown[Dimension]
```

其中 *Dimension* 需要是一个带或不带小数的数字,后跟一个度量单位。例如,`\godown[5cm]` 将指示 $\text{ConT}_{\text{E}}\text{Xt}$ 添加一个 5 cm 的间隙。如果当前页面剩余空间小于 5 cm, `\godown` 将导致分页 (并且「剩余」空间不会出现在下一页的顶部)。类似地, `\godown` 在页面开头无效,尽管我们可以通过写入例如「`\_ \godown[3cm]`」¹ 来欺骗它,这将首先插入一个空白,意味着我们不再处于页面开头,然后向下移动三

¹ 请记住,在本文档中,当空白字符对我们很重要时,我们使用『`\_`』字符来表示它。

厘米。(注意:这会向下移动略多于 3 cm,因为现在页面顶部有一个(没有文本的)行。这可以避免,但如何避免的解释对于本入门指南来说有点复杂。)

正如我们所知,`\blank` 也允许精确尺寸作为参数。因此,从用户的角度来看,写入 `\blank[3cm]` 或 `\godown[3cm]` 几乎是一样的。然而,它们之间存在一些细微的差别。例如,两个连续的 `\blank` 命令不会累积;当这种情况发生时,只有施加更大距离的那个被应用。而两个或更多 `\godown` 命令则可以完美地累积。

起初,`\blank` 的「非累积」行为可能看起来很奇怪。但考虑一个命令,它被设计为在其产生的任何输出之后留出一点空间,而第二个命令被设计为在其输出之前留出一点空间。如果在某个文档中,第二个命令紧跟在第一个命令之后,我们(通常)不希望这两个间距量相加:`\blank` 会注意只使用两者中较大的间距量,而不是两个量的和。

另一个相当有用的 $\text{T}_{\text{E}}\text{X}$ 命令,在 $\text{ConT}_{\text{E}}\text{Xt}$ 中使用没有问题,是 `\vfill`。此命令插入一个灵活的垂直空白,一直延伸到页面底部。就好像该命令将其后书写的内容向下推。这允许产生有趣的效果,例如如何将某个段落放置在页面底部,只需在其前面加上 `\vfill` 即可。然而,如果不与强制分页结合使用,`\vfill` 的效果很难体现,因为如果一个段落向上增长,那么将其向下推的意义不大。

因此,例如,为确保一行文本放置在页面底部,我们可以写:

```
\
\vfill
Line at the bottom
\page[yes]
```

像所有其他插入垂直间距的命令一样,`\vfill` 在页面开头无效。但我们可以通过在它前面加上一个强制空白来欺骗它。因此,例如:

```
\page[yes]
\vfill
Centre line
\vfill
\page[yes]
```

将在页面上垂直居中短语「centre line」。

在第一个 `\vfill` 示例中,添加空格是为了防止有人试图只使用底部三行来创建一个示例 $\text{ConT}_{\text{E}}\text{Xt}$ 文件。但也可以让页面顶部有一个段落,后跟 `\vfill`、「Line at the bottom」和 `\page[yes]`。

第 3 节 $\text{ConT}_{\text{E}}\text{Xt}$ 如何构建构成段落的行

排版系统的主要任务之一是将一长串单词分割成适当大小的单独行。例如,本文中的每个段落都被分割成了宽度为 15 厘米的行,但作者不必担心这些细节,因为 $\text{ConT}_{\text{E}}\text{Xt}$ 会在整体考虑每个段落之后选择断点,这样甚至一个段落的最后几个词也会影响第一行的分割。其结果是,整个段落中单词之间的间距尽可能均匀。

这是最能体现文字处理器工作方式差异以及使用 ConT_EXt 等系统获得更高质量的一个方面。因为文字处理器在到达行尾并换到下一行时,会调整刚结束的那一行的空白以填满该行。它对每一行都这样做,最后,段落中的每一行通常都会有不同词间距。这可能会导致非常糟糕的效果(例如,词间距大的行与词间距小的行相邻会在视觉上产生冲突)。而 ConT_EXt 则整体处理段落,并为每一行计算有多少可接受的断点以及每种断行方式会产生的词间距量。由于一行的断点会影响后续行的潜在断点,因此可能性的总数可能非常高;但这对于 ConT_EXt 来说不是问题。它将基于整个段落做出最终决定,确保每一行中单词之间的间距尽可能相似,从而产生比粗糙系统生成的段落更美观、更少分散注意力的段落。

为了做到这一点,ConT_EXt 会测试不同的替代方案,并根据其内部参数为每个方案分配一个所谓的「劣度」值。(这些参数是在深入研究排版艺术后建立的。)最后,在探索了所有可能性之后,ConT_EXt 会选择最不合适的选项(劣度值最小的那个)。通常,这效果很好,但不可避免地会出现选择的断点不是最佳,或者在我们看来不是最佳的情况。因此,有时我们希望告诉程序某些位置不是好的断点。相反,在其他时候,我们希望在特定点强制断行。以下小节探讨了一些我们可能影响断行的方法。

3.1 保留字符『~』的使用

行断点的主要候选者显然是单词之间的空格。为了指示某个空格绝不应该被换行代替,我们使用(正如我们已经知道的)『~』保留字符,T_EX 称之为 *tie*,它将两个单词绑定在一起。(某些系统使用「不间断空格」这一表达。)

通常建议在以下情况下使用此不间断空格:

- 缩写与其所指术语之间。例如,Dr.~Anne Ruben 或 p.~45。
- 数字与其伴随的术语之间。例如,Elizabeth~II、45~volumes。
- 数字与它们之前或之后的符号之间,只要这些符号不是上标。例如,73~km、53~€;但不包括像 35' 这样的情况。
- 用文字表示的百分比中。例如,twenty~per~cent。
- 由空白分隔的数字组中。例如,5~357~891。尽管在这些情况下,最好使用所谓的薄间距,在 ConT_EXt 中用 \, 命令实现,因此写成 5\,357\,891。
- 避免缩写成为该行的唯一项目。例如:

```
There are sectors such as entertainment, communications media,  
commerce,~etc.
```

对于这些情况,KNUTH(T_EX 之父)补充了以下建议:

- 在不在句末的缩写之后。
- 在引用文档的某些部分(如章节、附录、图表等)时。例如 Chapter~12。
- 在人的名字和姓氏首字母之间,或名字首字母和姓氏之间。例如,Donald~E. Knuth、A.~Einstein。
- 在并置于名称的数学符号之间。例如,dimension~\$d\$、width~\$w\$。
- 在序列中的符号之间。例如 {1,~2, \dots,~\$n\$}。
- 当一个数字与一个介词严格绑定时。例如 from 0 to~1。
- 当数学表达式用文字表达时。例如,equal~to~\$n\$。
- 在段落内的列表中。例如:(1)~green, (2)~red, (3)~blue。

情况很多?毫无疑问,排版的完美需要付出额外的努力。很明显,如果我们不想,就不必应用这些规则,但了解它们并没有坏处。此外——这里我根据经验说——一旦我们习惯了应用它们(或其中任何一个),这样做就会变得自动化。这就像我们在书写时给单词加重音一样(就像我们在西班牙语中需要做的那样):对于我们这些习惯自动加重音的人来说,写一个带重音的单词并不比写一个不带重音的单词花费更多时间。

3.2 单词断字

除了主要由单音节词组成的语言外,如果行断点仅限于单词之间的空格,很难获得最佳结果。因此,ConT_EXt 还会分析在单词的两个音节之间插入换行的可能性;要做到这一点,ConT_EXt 必须知道文本的语言,因为每种语言的断字规则都不同。这就是 `\mainlanguage` 命令在文档前言中如此重要的原因。

可能会发生 ConT_EXt 无法适当地断字的情况。有时这可能是因为它自己的单词分割规则妨碍了这项任务(例如,ConT_EXt 永远不会将一个单词分成两部分,如果任一部分没有特定的最小字母数),其他时候则是因为单词有歧义。毕竟,ConT_EXt 会对「unionised」这个词做什么呢?这个词可能出现在像「the unionised workforce」这样的短语中,但也可能出现在化学文本中,如「an unionised atom」(即未离子化的)。如果 ConT_EXt 必须在分页前处理作为页面最后一个词的「manslaughter」呢?它可能会将这个单词拆分为 man-slaughter(正确),但也可能拆分为 mans-laughter(有歧义或完全错误)。

无论出于何种原因,如果我们对单词的拆分方式不满意,或者拆分不正确,我们可以通过使用 `\-` 控制符号明确指示单词可以拆分的潜在位置来更改它。因此,例如,如果「unionised」给我们带来任何问题,我们可以在源文件中将其写为「union`\-`ised」;或者如果我们对「manslaughter」有疑问,我们可以写「man`\-`slaughter」。

如果有问题的单词在文档中多次使用,那么更倾向于在前言中使用 `\hyphenation` 命令来显示应该如何断字:该命令旨在包含在源文件的前言中,它接受一个或多个单词(逗号分隔)作为参数,指示可以用连字符分割的位置。例如:

```
\hyphenation{union-ised, man-slaughter}
```

如果作为此命令参数的单词不包含连字符,其效果将是该单词永远不会被断字。使用 `\hbox` 命令也可以达到同样的效果,该命令会在单词周围创建一个不可见但不可分割的水平盒子,或者使用 `\unhyphenated`,它可以防止其参数中的一个或多个单词被断字。但是,`\hyphenation` 是全局起作用的,而 `\hbox` 和 `\unhyphenated` 是局部起作用的,这意味着 `\hyphenation` 命令会影响文档中其参数包含的单词的所有出现;而 `\hbox` 或 `\unhyphenated` 只在源文件中遇到它们的位置起作用。

在内部,断字的工作方式部分受 T_EX 变量 `\pretolerance` 和 `\tolerance` 控制。第一个变量控制仅基于空白的拆分的可接受性。默认值是 100,但如果我们更改它,例如改为

10 000,那么 ConT_EXt 总是认为任何需要断字的换行都是可接受的,这意味着实际上,我们正在取消基于音节的断字。相反,如果我们将 `\pretolerance` 值设置为 `-1`,我们将强制 ConT_EXt 每次都考虑在行末使用断字。

我们可以直接在文档中赋值来为 `\pretolerance` 设置一个任意值。例如:

```
\pretolerance=10000
```

但我们也可以使用 `\setupalign` 中的「`lesshyphenation`」和「`morehyphenation`」值来操作这个值;详细信息请参见 [Section 6.1](#)。

虽然 `\pretolerance` 控制 ConT_EXt 在首次尝试将段落分解为行时是否会尝试断字,但 `\tolerance` 控制 ConT_EXt 对最终段落的挑剔程度。设置 `\tolerance=10000` 告诉 ConT_EXt 它可以输出非常「松散」的段落;通常这是不希望的。

3.3 换行的容忍度级别

在寻找可能的换行点时,ConT_EXt 通常非常严格,这意味着它更倾向于允许一个单词超出右边距,因为它无法对其进行断字,并且如果导致该行词间距增加过大,它也不愿在该单词之前插入换行。这种默认行为通常能提供最佳结果,只有在极少数情况下,某些行会在右侧稍微突出。其理念是作者(或排版员)在文档完成后检查这些例外情况,然后就如何处理问题做出适当的决定。这可能意味着在超出边界的单词前插入一个 `\break` 命令,或者也可能意味着以不同的方式措辞段落,使该单词移到别处。稍微改写通常足以解决此类问题。

然而,在某些情况下,ConT_EXt 的低容忍度可能是个问题。在这些情况下,我们可以告诉它对行中的空白更加宽容。我们有 `\setuptolerance` 命令用于此目的,它允许我们改变计算换行时的容忍度级别,ConT_EXt 称之为「水平容忍度」(因为它影响水平间距)以及在计算分页时的「垂直容忍度」。我们将在 [Section 6.2](#) 中讨论这一点。

影响换行的水平容忍度,默认设置为「`verystRICT`」值。我们可以通过设置以下任何值来改变它:「`strict`」、「`tolerant`」、「`verytolerant`」或「`stretch`」。因此,例如,

```
\setuptolerance[horizontal, verytolerant]
```

将使得一行几乎不可能超出右边距,即使这意味着在该行的单词之间建立非常大且难看的间距。

3.4 在特定点强制换行

要在特定点强制换行,我们使用 `\break`、`\crlf` 或 `\\` 命令。第一个命令 `\break` 在其所在位置输入一个换行。这很可能会导致放置该命令的行在美学上变形,该行单词之间出现巨大的空白。这可以在以下示例中看到,其中第三行(左侧源片段中)的 `\break` 命令导致第二行(右侧格式化文本中)非常难看。

On the corner of the old quarter I saw
 him `\emph{swagger}` along like
 the`\break` tough guys do when they walk,
 hands always in their overcoat pockets,
 so no one can know which of them carries
 the dagger.

On the corner of the old quarter I saw him *swa-*
gger along like the
 tough guys do when they walk, hands always in their
 overcoat pockets, so no one can know which of them
 carries the dagger.

为了避免这种效果,我们可以使用 `\\` 或 `\crlf` 命令,它们也插入强制换行,但它们会用足够的空白填充原始行,使其左对齐:

On the corner of the old quarter I saw
 him `\emph{swagger}` along like
 the`\\` tough guys do when they walk,
 hands always in their overcoat pockets,
 so no one can know which of them carries
 the dagger.

On the corner of the old quarter I saw him *swa-*
gger along like the
 tough guys do when they walk, hands always in their
 overcoat pockets, so no one can know which of them
 carries the dagger.

在正常行上,据我所知,`\\` 和 `\crlf` 之间没有区别;但在章节标题中,有一个区别:

- `\\` 在文档正文中生成换行,但当章节标题传输到目录时不会生成换行。
- `\crlf` 生成的换行既适用于文档正文,也适用于当章节标题传输到目录时。

换行不应与段落分隔符混淆。换行只是结束当前行并开始下一行,但使我们保持在同一个段落中,因此原始行和新行之间的分隔将由段落内的正常间距决定。因此,只有三种情况可能建议强制换行:

- 在非常特殊的情况下,当 ConT_EXt 无法找到合适的换行点,导致一行在右侧突出时。在这些情况下(极少发生,主要发生在行包含不可分割的盒子或照排文本时[参见 [Section 2.3](#)]),在突出到右边距的单词之前使用 `\break` 强制换行可能会有帮助。
- 在实际上由单独行组成的段落中,每行包含的信息与前面行的信息无关,例如,信件的信头,其中第一行可能包含发件人姓名,第二行包含收件人,第三行包含日期;或者在谈论作品作者身份的文本中,一行是作者姓名,另一行是其职务或学术职位,也许第三行是日期,等等。在这些情况下,应使用 `\\` 或 `\crlf` 命令强制换行。这些类型的段落也通常是右对齐的。
- 在写诗或类似类型的文本时,用于分隔诗的一行与另一行。尽管在后一种情况下,最好使用 [Section 5.1](#) 中解释的 `lines` 环境。

第4节 行间距

行间距是分隔构成段落的行的距离。ConTeXt 会根据所使用的实际字体,尤其是其基本大小(通过 `\setupbodyfont` 或 `\switchtobodyfont` 设置),自动计算行间距。

我们可以使用 `\setupinterlinespace` 命令来影响行间距,该命令允许三种不同的语法:

- `\setupinterlinespace [..Interline space..]`, 其中 *Interline space* 是一个精确值或一个分配预定义行间距的象征性词:
 - 当它是一个特定值时,可以是一个尺寸(例如,15pt),或者一个简单的整数或小数(例如,1.2)。在后一种情况下,该数字根据 ConTeXt 的默认行距被解释为「行数」。将行间距设置为与字体大小无关的尺寸(例如 15pt)的一个问题是,如果文档的字体大小更改,15pt 可能不再合适。如果确实需要带单位的尺寸(第一种情况),通常使用(例如)3ex 这样的单位更好,因为「ex」是当前字体大小的函数。
 - 当它是一个象征性词时,可以是「small」、「medium」或「big」,分别应用小、中或大的行间距,但始终基于 ConTeXt 会应用的默认行间距。
- `\setupinterlinespace [...=...,...]`。在这种模式下,通过显式更改 ConTeXt 用来计算适当行间距的度量来设置行间距。我之前说过行间距是基于特定字体及其大小计算的;但那只是为了把事情说得非常简单:实际上,字体及其大小会建立某些度量,行间距是从这些度量计算出来的。通过这种 `\setupinterlinespace` 方法,这些度量被修改,因此行间距也被修改。可以通过此过程操作的实际度量和值(其含义我将不解释,因为这超出了简单入门指南的范围)包括:line、height、depth、minheight、mindepth、distance、top、bottom、stretch 和 shrink。
- `\setupinterlinespace [Name]`。使用此命令,我们建立或配置一种特定且自定义的行间距类型,该类型先前已使用 `\defineinterlinespace` 定义。

使用:

```
\defineinterlinespace[Name] [Configuration]
```

我们可以将某个行间距配置与一个特定名称关联起来,然后只需在文档中的某个点使用 `\setupinterlinespace[Name]` 即可触发该配置。要恢复到正常的行间距,我们可以写 `\setupinterlinespace[reset]`。

第5节 与行相关的其他事项

5.1 将源文件中的换行转换为最终文档中的换行

正如我们已经知道的(参见 [Section 2.2](#)),默认情况下,ConTeXt 会忽略源文件中它认为是简单空格的换行,除非有两个或更多连续的换行(即一个或多个空行),在这种情况下会插入一个段落分隔符。然而,在某些情况下,我们可能希望尊重原始源文件中的换行,例如在写诗时。为此,ConTeXt 为我们提供了「`lines`」环境,其格式为:

```
\startlines[Options] ... \stoplines
```

其中选项可以是以下任意值(以及其他):

- **space**: 当此选项设置为「on」时,除了尊重源文件中的换行外,环境还将尊重源文件中的空格,暂时忽略「吸收」规则。
- **before**: 进入环境前要插入的文本或要运行的命令。
- **after**: 退出环境后要插入的文本或要运行的命令。
- **inbetween**: 在「段落」之间要插入的文本或要运行的命令。
- **indenting**: 指示是否缩进环境中的段落的值(参见 [Section 1.1](#))。
- **align**: 环境中行的对齐方式(参见 [Section 6](#))。
- **style**: 在环境中应用的样式命令。
- **color**: 在环境中应用的颜色。

因此,例如:

<pre>\startlines One-one was a race horse. Two-two was one too. One-one won one race. Two-two won one too. \stoplines</pre>	<pre>One-one was a race horse. Two-two was one too. One-one won one race. Two-two won one too.</pre>
---	--

我们还可以使用 `\setuplines` 修改环境的默认工作方式,并且与许多 ConTeXt 命令一样,也可以为此环境的特定配置分配一个名称。我们使用 `\definelines` 命令来执行此操作,其语法为:

`\definelines[Name] [Configuration]`

其中,作为配置,我们可以包含通常为此环境解释的相同选项。一旦我们定义了自己的自定义行环境,要插入它,我们应该写:

`\startlines[Name] ... \stoplines`

5.2 行号

在某些类型的文本中,建立某种行号是很常见的。例如,在计算机编程文本中,作为示例提供的代码片段通常带有行号;诗歌和校勘本是另外两个例子,但还有很多其他例子。对于所有这些情况,ConT_EXt 提供了 `linenumbering` 环境,其格式为:

`\startlinenumbering[Options] ... \stoplinenumbering`

可用的选项有:

- **continue**: 在文档中有多个片段需要编号行的情况下,此选项控制是否对每个片段重新开始编号。使用「`continue=no`」(默认值),此片段的行号将重置。但如果行号希望从上一个片段结束的地方继续,我们选择「`continue=yes`」。
- **start**: 指示第一行的编号,以防我们不想让它从『1』开始,也不希望它从上一个片段继续。
- **step**: 此选项使得行号以一定的间隔打印(尽管环境中包含的所有行都会被计数)。例如,对于诗歌,通常行号只出现在 5 的倍数行(第 5、10、15 行.....)。

所有这些选项都可以通过 `\setuplinenumbering` 为文档中的所有 `linenumbering` 环境进行全局设置。此命令还允许我们配置行号的其他方面:

- **conversion**: 行号编号类型。它可以是 [page 102](#) 上关于章节编号解释的任何一种。
- **style**: 决定行号样式的命令(字体、大小、变体.....)。
- **color**: 打印行号的颜色。
- **location**: 行号放置的位置。可以是以下任意值: `text`、`begin`、`end`、`default`、`left`、`right`、`inner`、`outer`、`inleft`、`inright`、`margin` 或 `inmargin`。
- **distance**: 行号与行本身之间的距离。
- **align**: 编号对齐方式。可以是: `inner`、`outer`、`flushleft`、`flushright`、`left`、`right`、`middle` 或 `auto`。
- **command**: 在打印之前,行号将作为参数传递给的命令。

- **width**: 为打印行号保留的宽度。
- **left, right, margin**:

我们还可以使用 `\definelinenumbering` 创建不同的自定义行号配置,以便将配置与名称关联:

```
\definelinenumbering[Name] [Configuration]
```

一旦定义了特定配置并将其与名称关联,我们就可以使用以下命令调用它:

```
\startlinenumbering[Name] ... \stoplinenumbering
```

第6节 水平与垂直对齐

通常控制文本对齐的命令是 `\setupalign`。此命令用于控制水平对齐和垂直对齐。

6.1 水平对齐

当一行的精确 宽度没有占满所有可能的宽度时,就产生了如何处理由此产生的空白的问题。¹ 在这方面,我们基本上可以做三件事:

1. 将其累积在行的两侧之一:如果我们将其累积在左侧,行看起来会有点向右推,而如果我们将其累积在右侧,行则保持在左侧。前者是右对齐,后者是左对齐。默认情况下,ConTeXt 对段落的最后一行应用左对齐。

当几个连续的行左对齐时,右侧是不规则的;但当右对齐时,看起来不均匀的是左侧。为了实现左对齐文本,使用 `flushleft` 选项;类似地,对于右对齐文本,使用 `flushright` 选项。²

在准备双面打印的文档中,除了参考右边距和左边距,还可以参考内边距和外边距(内边距是页面之间的边距)。值 `flushinner`(或简称 `inner`)和 `flushouter`(或简称 `outer`)在这些情况下建立相应的对齐方式。

2. 将额外空间分布到两个边距。结果将是行居中。执行此操作的 `\setupalign` 选项是 `middle`。
3. 将其分布到组成行的所有单词中,必要时通过增加词间距,使行的宽度正好与其可用空间相同。在这些情况下,我们称之为两端对齐的行。这也是 ConTeXt 的

¹ 所谓精确 宽度,我指的是在 ConTeXt 调整词间距以实现两端对齐之前 的行的宽度。

² ConTeXt 的一个历史趣闻是,还有 `left` 和 `right` 选项,它们的作用可能与您想的相反。也就是说,`left` 并不执行左对齐,而是导致文本设置为右对齐,因为右对齐文本的另一个名称是「参差不齐的左边缘」。为避免混淆,建议不要使用 `left` 或 `right` 作为 `\setupalign` 的选项。

默认值,这就是为什么 `\setupalign` 中没有特殊的选项来设置它。但是,如果我们更改了默认的两端对齐,可以使用 `\setupalign[reset]` 恢复它。

我们刚刚看到的 `\setupalign` 的值(`right`、`flushright`、`left`、`flushleft`、`inner`、`flushinner`、`outer`、`flushouter` 和 `middle`)可以与 `broad` 结合使用,这会导致稍微粗糙一些的对齐。

影响水平对齐的另外两个可能的 `\setupalign` 值与行末单词的断字有关。是否发生断字会影响空白的量。

具体来说,`\setupalign` 接受 `morehyphenation` 选项,它使 ConT_EXt 更加努力地寻找基于断字的断点,而 `lesshyphenation` 则产生相反的效果。使用 `\setupalign[horizontal, morehyphenation]`,行中剩余的空白通常会减少,因此为实现对齐而添加的额外空白将不那么明显。相反,使用 `\setupalign[horizontal, lesshyphenation]`,将会有更多剩余空白,对齐将更加明显。

`\setupalign` 旨在包含在前言中并影响整个文档,或者包含在特定点并影响从该点到结束的所有内容。如果我们只想更改一行或几行的对齐方式,可以使用:

- 「alignment」环境,旨在影响多行。其一般格式为:

`\startalignment[Options] ... \stopalignment`

其中 `Options` 是 `\setupalign` 允许的任何选项。

- `\leftaligned`、`\midaligned` 或 `\rightaligned` 命令,分别导致左对齐、居中对齐或右对齐;如果我们希望段落中的最后一个词(但仅限于此,而不是该行的其余部分)右对齐,我们可以使用 `\wordright`。所有这些命令都需要将受影响的文本放在花括号之间。

另一方面,请注意,如果 `\setupalign` 中的词「right」和「left」导致与名称所暗示的相反的对齐,那么 `\leftaligned` 和 `\rightaligned` 命令则不会发生同样的情况,它们带来的正是其名称所暗示的对齐方式:left 左对齐,right 右对齐。

6.2 垂直对齐

水平对齐在一行的宽度没有占满其所有可用空间时发挥作用,而垂直对齐则影响整个页面的高度。如果一页的精确文本高度没有占满其所有可用高度,我们如何处理剩余的空白?我们可以将其堆积在顶部(「height」),这意味着页面上的文本将被向下推;我们可以将其堆积在底部(「bottom」),或者将其分布在段落之间(「line」)。垂直对齐的默认值是「bottom」。

垂直容忍度级别

就像我们可以使用 `\setuptolerance` 改变 ConT_EXt 对于一行中允许的水平间距的容忍度级别(水平容忍度)一样,我们也可以改变其垂直容忍度,即对于段落之

间的间距大于 ConT_EXt 默认认为排版良好的页面合理的容忍度。垂直容忍度的可能选项与水平容忍度相同: `verystRICT`、`strict`、`tolerant` 和 `verytolerant`。默认值是 `\setuptolerance [vertical, strict]`。

控制孤行和寡行

间接影响垂直对齐的一个方面是孤行和寡行的控制。这两种现象都指分页导致段落的一行与段落的其余部分被隔离在不同的页面上。这在排版上被认为是不理想的。如果与段落其余部分分离的那一行是页面上的第一行,我们称之为寡行;如果与其段落分离的那一行是页面上的最后一行,我们称之为孤行(有些人称孤行为「clubs」)。

默认情况下,ConT_EXt 不实施控制来确保这些行不出现。但我们可以通过更改 ConT_EXt 的一些内部变量来改变这一点: `\widowpenalty` 控制寡行, `\clubpenalty` 控制孤行。因此,在文档前言中执行以下语句将确保执行此控制:

```
\widowpenalty=10000  
\clubpenalty=10000
```

执行此控制意味着 ConT_EXt 将避免插入将段落的第一行或最后一行与找到其余行的页面分开的分页符。这种避免的严格程度取决于我们分配给变量的值。使用示例中的 10 000 这样的值,控制将是绝对的;使用例如 150 这样的值,控制将不那么严格,偶尔可能会出现一些孤行或寡行,当从排版角度来看替代方案更糟时。

第12章 特殊构造与段落

Table of Contents: 第1节 脚注与尾注; 1.1 ConTeXt 中的注释类型及其相关命令; 1.2 仔细审视脚注和尾注; 1.3 局部注释; 1.4 创建和使用自定义类型的注释; 1.5 配置注释; 1.6 编译时临时排除注释; 第2节 多列段落; 2.1 `\startcolumns` 环境; 2.2 并行段落; 第3节 结构化列表; 3.1 选择列表类型和条目之间的分隔符; 3.1.1 无序列表; 3.1.2 有序列表; 3.2 输入列表的条目; 3.3 基本列表配置; 3.4 附加列表配置; 3.5 使用 `\items` 命令的简单列表; 3.6 预设列表行为并创建我们自己的列表类型; 第4节 描述与枚举; 4.1 描述; 4.2 枚举; 第5节 线条与框架; 5.1 简单线条; 5.2 与文本关联的线条; 5.3 带框架的单词或文本; 第6节 其他感兴趣的环境和构造;

第1节 脚注与尾注

注释是“用于各种目的的次级文本元素,例如阐明或扩展正文、提供来源的参考文献、包含引文、引用其他文档或陈述文本的含义”[*Libro de Estilo de la Lengua española* (西班牙语语言指南), 第195页]。它们在学术性文本中尤为重要。它们可以放置在页面或文档的不同位置。如今,最广泛使用的是位于页面底部的注释(因此称为脚注);有时也位于页边空白处(边注)、每章或每节末尾,或文档末尾(尾注)。在特别复杂的文档中,还可能在不同系列的注释:作者注、译者注、更新注等。特别是在评注版中,注释装置可能变得相当复杂,只有少数排版系统能够支持;ConTeXt 就是其中之一。有许多命令可用于建立和配置不同类型的注释。

为了解释这一点,首先指出注释可能涉及的几个要素是有用的:

- 标记或注释锚点:放置在正文中的符号,表示存在与之链接的注释。并非所有类型的注释都有相关联的锚点,但当有锚点时,该锚点出现在两个地方:在正文中注释所指的位置,以及在注释文本本身的开头。同一参考标记在这两个地方的出现,使得注释能够与正文关联起来。
- 注释 ID 或标识符:标识注释并使其区别于其他注释的字母、数字或符号。有些注释,例如边注,可以没有 ID。当没有 ID 时,ID 通常与锚点一致。

如果我们只考虑脚注,我们看不出我刚才所说的参考标记和 id 之间的区别。我们在其他类型的注释中能清楚地看到区别:例如行注有一个 id,但没有参考标记。

- 注释的文本或内容,总是位于页面或文档中与生成注释并指示其内容的命令不同的位置。
- 与注释关联的标签:与注释关联的标签或名称,不会显示在最终文档中,但允许我们在文档的其他地方引用它并检索其 ID。

1.1 ConTeXt 中的注释类型及其相关命令

我们在 ConTeXt 中有多种类型的注释。目前我只列出它们,用一般术语描述并提供生成它们的命令的信息。稍后我将详细说明前两种:

- **脚注:**无疑是最流行的,以至于所有类型的注释通常都被泛称为脚注。脚注在文档中找到命令的位置引入带有注释 *id* 的标记,并将注释文本本身插入到标记出现页面的底部。它们通过 `\footnote` 命令创建。
- **尾注:**这些注释通过 `\endnote` 命令创建,在文档中放置 `\endnote` 命令的位置插入带有注释标识符的标记。然而,注释的内容被插入到文档的另一个位置,并且插入是由另一个命令(`\placenames`)产生的。
- **边注:**顾名思义,它们写在文本的页边空白处,在文档正文中没有 ID 或自动生成的标记或锚点。创建它们的主要命令(但不是唯一的)是 `\inmargin` 和 `\margintext`。
- **行注:**一种典型于对行进行编号的环境的注释类型,例如在 `\startlinenumbers` ... `\stoplinenumbers` 的情况下(参见 [Section 5.2](#))。注释通常写在底部,指的是特定的行号。它们由 `\linenote` 命令生成,并通过 `\setuplinenote` 进行配置。该命令在正文中不打印标记,但在注释本身中打印注释所指的行号(用作 ID)。

我现在将专门阐述前两种类型的注释,因为:

- 边注在其他地方处理([Section 7](#));以及
- 行注具有高度专业化的用途(尤其是在评注版中),我认为在像这样的一篇入门文档中,读者只需知道它们存在就足够了。

1.2 仔细审视脚注和尾注

脚注和尾注命令的语法以及它们所具有的配置和定制机制非常相似,因为实际上,这两种注释都是更通用构造(注释)的特定实例,其他实例可以使用 `\definernote` 命令创建(参见 [Section 1.4](#))。

创建每种注释的命令语法如下:

```
\footnote[Label]{Text}  
\endnote[Label]{Text}
```

其中:

- `Label` 是一个可选参数,为注释分配一个标签,允许我们在文档的其他地方引用它。

- **Text** 是注释的内容。它可以任意长,并包含特殊的段落和设置,但应注意,对于脚注,在注释丰富且过长的文档中,正确的页面布局相当困难。

原则上,任何可以在正文中使用的命令都可以在注释文本中使用。然而,我已经能够验证,某些在正文中不构成任何问题的构造和字符,当出现在注释文本中时确实会产生编译错误。这些情况是我在测试时发现的,但我没有以任何方式整理它们。

当使用 **Label** 参数为注释设置了标签时, **\note** 命令允许我们检索该注释的 ID。此命令在文档上打印与其作为参数的标签相关联的注释的 ID。因此,例如:¹

```
Humpty Dumpty\footnote[humpty]
{Probably the best-known English
nursery rhyme character}
sat on a wall,\
Humpty Dumpty\note[humpty]
had a great fall.\
All the king's horses and
all the king's men\
Couldn't put Humpty\note[humpty]
together again.
```

Humpty Dumpty¹ sat on a wall,
Humpty Dumpty¹ had a great fall.
All the king's horses and all the king's men
Couldn't put Humpty¹ together again.

¹ Probably the best-known English nursery rhyme character

\footnote 和 **\endnote** 之间的主要区别在于注释出现的位置:

\footnote 通常,它在命令所在页面的底部打印注释文本,因此注释标记及其文本(或文本的开头,如果要跨两页)将出现在同一页上。为此,ConTeXt 将通过计算页面底部注释位置所需的空間来对页面排版进行必要的调整。

历史注:在 ConTeXt MkIV 中,在某些环境下, **\footnote** 不会将注释文本插入页面底部本身,而是插入环境的“脚部”。例如,这发生在表格中,也发生在 **columns** 环境中。在这种情况下,如果希望环境内的注释放在页面底部,可以使用 **\footnotetext** 命令与前面提到的 **\note** 命令组合使用,而不是 **\footnote**。前者也接受一个可选参数作为标签,但只打印注释文本而不打印标签。由于 **\note** 只打印标签而不打印文本,将两者结合起来可以让我们将注释放在任何想要的位置。所以,例如,我们可以在表格或多列环境内写 **\note[MyLabel]**,然后,在该环境之外,写 **\footnotetext[MyLabel]{Note text}**。

\endnote 仅在源文件中命令所在的位置打印注释锚点。注释的实际内容通过另一个命令 (**\placenotes[endnote]**) 插入到文档的另一个位置,该命令在其所在位置将插入文档(或相关章节或节)中所有尾注的内容。

1.3 局部注释

\startlocalfootnotes 环境意味着其中包含的脚注被视为局部注释,这意味着它们的编号将被重置,并且注释的内容不会与其他注释一起自动插入,而只会插入到文档中找到 **\placelocalfootnotes** 命令的位置,该命令可能在该环境内,也可能不在。

¹ 因为脚注位于页面底部,此示例在右侧排版版本周围使用一个矩形框,以示意脚注在(横向)页面上是如何排列的。

1.4 创建和使用自定义类型的注释

我们可以使用 `\definenote` 命令创建特殊类型的注释。这在复杂文档中很有用,这些文档中有来自不同作者或用于不同目的的注释,通过不同的格式和不同的编号在图形上区分我们文档中的每种注释类型。

`\definenote` 的语法如下:

```
\definenote[Name][Model][Configuration]
```

其中:

- **Name** 是我们分配给新注释类型的名称。
- **Model** 是初始使用的注释模型。它可以是 `footnote` 或 `endnote`;在前一种情况下,我们的注释模型将像脚注一样工作,在后一种情况下像尾注一样工作,尽管要将它们插入文档中,我们不会使用 `\placenames[endnote]`,而是使用 `\placenames[Name]`(我们分配给这些注释类型的名称)。

理论上,此参数是可选的,尽管在我的测试中,一些没有它创建的注释不可见,我没有耐心找出原因是什么。

- **Configuration** 是第二个可选参数,允许我们将新类型的注释与其模型区分开来:通过设置不同的格式,或不同的编号类型,或两者兼有。

根据 ConTeXt 命令的官方列表(参见 [Section 6](#)),创建新注释类型时可以提供的设置基于稍后可以用 `\setupnote` 提供的设置。然而,正如我们稍后将看到的,实际上有两个可能的命令用于设置注释:`\setupnote` 和 `\setupnotation`。因此,我认为在创建注释类型时最好省略此参数,然后使用适当的命令设置我们的新注释。至少这样更容易解释。

例如,以下内容将创建一个名为「BlueNote」的新注释类型,它将类似于脚注,但其内容将以粗体和蓝色打印:

```
\definenote [BlueNote] [footnote]
\setupnotation
[BlueNote]
[color=blue, style=bf]
```

一旦我们创建了一个新的注释类型,例如 `BlueNote`,允许我们使用它的命令就可用。在我们的示例中,这将是 `\BlueNote`,其语法类似于 `\footnote`:

```
\BlueNote[Label]{Text}
```

1.5 配置注释

注释(脚注或尾注、使用 `\definenote` 创建的注释以及使用 `\linenote` 设置的行注)的配置通过两个命令实现:`\setupnote` 和 `\setupnotation`¹。两者的语法相似:

`\setupnote[NoteType][Configuration]`
`\setupnotation[NoteType][Configuration]`

其中 `NoteType` 指的是我们正在配置的注释类型(`footnote`、`endnote` 或我们自己创建的某种注释类型的名称), 而 `Configuration` 包含该命令的特定配置选项。

问题是这两个命令的名称并没有很清楚地表明它们之间的区别或每个命令配置什么; 此外, 这些命令的许多选项没有文档记录这一事实也无济于事。经过大量测试, 我无法得出任何结论让我理解为什么某些东西是用一个命令配置的, 而另一些是用另一个命令配置的,¹ 也许除了, 因为我所做的选择使其工作, `\setupnotation` 总是影响注释文本或与注释文本一起打印的 ID, 而 `\setupnote` 有一些影响插入到正文中的注释标记的选项。

我现在将尝试整理我在对两个命令的不同选项进行一些测试后发现的内容。我将大部分选项放在一边, 因为它们没有文档记录, 我也无法就它们的用途或在什么条件下使用它们得出任何结论:

• 用于标记的 ID:

注释总是通过一个数字来标识。我们可以在这里配置的是:

- 起始数字: 由 `\setupnotation` 中的 `start` 控制。其值必须是一个整数, 它使用此值开始计数注释。
- 编号系统: 取决于 `\setupnotation` 中的 `numberconversion` 选项。其值可以是:
 - * 阿拉伯数字: `n`、`N` 或 `numbers`。
 - * 罗马数字: `I`、`R`、`Romannumerals`、`i`、`r`、`romannumerals`。前三个是大写罗马数字, 后三个是小写。

¹ `\setupnote` 命令有 35 个直接配置选项和从 `\setupframed` 继承的 45 个附加选项; `\setupnotation` 有 45 个直接配置选项和从 `\setupcounter` 继承的另外 23 个选项。由于这些选项没有文档记录, 尽管对于其中许多选项我们可以从名称猜出它们的用途, 但我们需要验证我们的直觉是否正确。另外, 考虑到这些选项中有许多允许许多值, 并且都必须测试... 您会看到, 为了写出这个解释, 我不得不进行相当多的测试; 虽然做一次测试很快, 但做很多测试既慢又无聊。因此, 如果我说, 除了我在正文中提到的两个通用注释配置命令(下面的解释将重点介绍它们)之外, 还有四个其他可能的配置命令我将从我的解释中省略, 希望读者能谅解我:

- `\setupnotes` 和 `\setupnotations`: 换句话说, 与上面相同的名称, 但是复数形式。wiki 说命令的单数和复数版本是同义的, 我相信它。
- `\setupfootnotes` 和 `\setupendnotes`: 这些应该分别是脚注和尾注的具体应用。也许基于这些命令来解释注释设置会更容易; 然而, 因为我用 `\setupfootnotes` 尝试的第一个选项(`numberconversion`) 没有起作用, 尽管我知道这些命令的其他选项确实有效, 但我懒得(在我已经不得不进行的众多测试中)添加必要的测试来写下以下内容。
我确实相信(从一些随机测试来看), 在这两个命令中有效的所有内容(但我的解释中省略了)在我确实给出解释的命令中也有效。

¹ 在 [ConTeXt wiki](#) 上有一个页面, 我是偶然发现的(因为它不是专门针对注释的), 它表明区别在于 `\setupnotation` 控制要插入的注释文本, 而 `\setupnote` 控制将要放置注释的环境(?) 但这与事实不一致, 例如, 注释文本的宽度(与其插入有关)由 `\setupnote` 的 `width` 选项控制, 而不是由 `\setupnotation` 的同名选项控制。这里控制的是标记和注释文本之间的空间宽度。

- * 字母编号: A、Character、Characters、a、character、characters, 取决于我们想要大写字母(前三个选项)还是小写字母(其余选项)。
 - * 单词编号: 换句话说, 我们写出表示数字的单词, 例如, 『3』变成「three」。有两种方法可能; Words 选项以大写形式写出单词, words 以小写形式。
 - * 符号编号: 我们可以使用四组不同的符号, 具体取决于所选选项: set 0、set 1、set 2 或 set 3。在 [page 102](#) 上有一个每个选项使用的符号示例。
- ◇ 决定重新开始注释编号的事件: 这取决于 `\setupnotation` 中的 way 选项。当值为 bytext 时, 文档中的所有注释将按顺序编号, 而不会重置编号。当值为 bychapter、bysection、bysubsection 等时, 每次更改章、节或小节时, 注释计数器将重置, 而当值为 byblock 时, 每次更改文档宏结构中的块时, 它会重置编号(参见 [Section 6](#))。bypage 值使注释计数器在每次更改页面时重新启动。
- 配置注释标记:
- ◇ 是否显示它: `\setupnotation` 中的 number 选项。
 - ◇ 标记相对于注释文本的位置: `\setupnotation` 中的 alternative 选项: 它可以采用以下任何值: left、inleft、leftmargin、right、inright、rightmargin、inmargin、margin、innermargin、outermargin、serried、hanging、top、command。
 - ◇ 注释本身中标记的格式: `\setupnotation` 中的 numbercommand 选项。
 - ◇ 正文中标记的格式: `\setupnote` 中的 textcommand 选项。
- numbercommand 和 textcommand 选项必须由将标记内容作为参数的命令组成。这可以是我們自己定义命令。然而, 我发现简单的格式化命令(`\bf`、`\it` 等)也有效, 即使它们不是带参数的命令。
- ◇ 标记和文本之间的距离(在注释本身中): `\setupnotation` 中的 distance 和 width 选项。我无法发现使用一个选项或另一个选项之间的区别(如果确实有区别的话)。
 - ◇ 是否存在允许在正文中的标记和注释本身的标记之间跳转的超链接: `\setupnote` 中的 interaction 选项。值为 yes 时将有链接, 值为 no 时将没有链接。

• 配置注释文本本身。

我们可以影响以下方面:

- ◇ 位置: 这取决于 `\setupnote` 中的 location 选项。

原则上我们已经知道脚注位于页面底部(location=page), 尾注位于找到 `\placenames[endnote]` 命令的位置(location=text)。但是, 我们可以调整此行为

并设置例如 `location=text`, 这将使脚注类似于尾注, 因此它们出现在文档中找到 `\placenames[footnote]` 命令(或特定于脚注的命令 `\placefootnotes`)的位置。使用这种方法, 我们可以例如将注释打印在它们所在的段落下方。

- ◇ 注释之间的段落分隔: 默认情况下, 每个注释都打印在自己的段落中, 但我们可以通过将 `\setupnote` 中的 `paragraph` 选项设置为「yes」来使同一页面上的所有注释打印在同一段落中。
- ◇ 注释文本本身的书写样式: `\setupnotation` 中的 `style` 选项。
- ◇ 字体大小: `\setupnote` 中的 `bodyfont` 选项。

此选项仅适用于我们想要手动设置脚注字体大小的情况。这样做几乎从来都不是一个好主意, 因为默认情况下, ConT_EXt 会调整脚注的字体大小, 使其比正文小, 但大小与正文的字体大小成比例。

- ◇ 注释文本的左边距: `\setupnotation` 中的 `margin` 选项。
- ◇ 最大宽度: `\setupnote` 中的 `width` 选项。
- ◇ 列数: `\setupnote` 中的 `n` 选项指定注释文本将位于两列或更多列中。『`n`』值必须是一个整数。

• **注释之间或注释与文本之间的间距:** 这里, 我们有以下选项:

- ◇ `rule`, 在 `\setupnote` 中, 确定注释区域和带有正文的页面区域之间是否有一条线(规则)。其可能值为 `yes`、`on`、`no` 和 `off`。前两个启用规则, 后两个禁用它。
- ◇ `before`, 在 `\setupnotation` 中: 在插入注释文本之前要运行的命令。用于插入额外的间距、注释之间的分割线等。
- ◇ `after`, 在 `\setupnotation` 中: 在插入注释文本之后要运行的命令。

1.6 编译时临时排除注释

`\notesenabledfalse` 和 `\notesenabledtrue` 命令分别告诉 ConT_EXt 启用或禁用注释的编译。当文档有大量冗长的注释时, 如果我们想获得一个不带注释的版本, 此功能可能很有用。根据我个人的经验, 例如, 当我在校对博士论文时, 我更喜欢第一次一口气读完, 不带注释, 然后进行第二次阅读, 加入注释。

第2节 多列段落

将文本排版成多于一列是一种可以建立的可能性:

- a. 作为页面布局的一般特性。
- b. 作为某些构造的特性,例如结构化列表、脚注或尾注。
- c. 作为应用于文档中特定段落的特性。

在任何这些情况下,即使我们处理多于一列,大多数命令和环境也能完美工作。然而,存在一些限制,主要与浮动对象(参见 [Section 1](#))尤其是表格([Section 3](#))有关,即使它们不是浮动对象。

关于允许的列数,ConT_EXt 没有理论限制。但是,有两个物理限制必须考虑:

- 纸张宽度:无限数量的列需要无限的纸张宽度(如果文档要打印)或屏幕宽度(如果是打算在屏幕上显示的文档)。实际上,考虑到市售和用于制作书籍的纸张尺寸以及计算机设备屏幕的正常宽度,由超过四或五列组成的文本很难合适地排布。
- 计算机内存大小:ConT_EXt 参考手册指出,在正常系统(既不是特别强大也不是特别资源有限)中,可以处理 20 到 40 列。

在本节中,我将重点介绍多列机制在特殊段落或片段中的使用,因为:

- 多列作为页面布局选项已经讨论过(在 [Section 3.4](#) 的 [Subsection 3.4.2](#) 中)。
- 某些构造(例如结构化列表或脚注)提供的将文本排版成多于一列的可能性,将在相关构造或环境的讨论中介绍。

2.1 `\startcolumns` 环境

将多列片段插入文档的标准过程是使用 `columns` 环境,其格式为:

```
\startcolumns[Configuration] ... \stopcolumns
```

其中 `Configuration` 允许我们控制环境的许多方面。我们可以在每次调用环境时指示所需的配置,或者为所有对环境调用调整环境的默认操作;后者通过以下方式实现:

```
\setupcolumns[Configuration]
```

在这两种情况下,配置选项都是相同的。最重要的选项按其功能排序如下:

- **控制列数和列间距的选项:**

- ◇ `n`: 控制列数。如果省略, 将生成两列。
 - ◇ `nleft`、`nright`: 这些选项用于双面文档布局 (参见 [Section 3.4](#) 的 [Subsection 3.4.1](#)), 分别用于设置左(偶数)页和右(奇数)页上的列数。
 - ◇ `distance`: 指定列之间的间距。
 - ◇ `separator`: 确定用什么标记列之间的分隔。可以是 `space` (默认值) 或 `rule`, 在这种情况下, 将在列之间生成一条线(规则)。如果在列之间建立了规则, 则此规则又可以由以下两个选项配置:
 - * `rulecolor`: 线条的颜色。
 - * `rulethickness`: 线条的粗细。
 - ◇ `maxwidth`: 指定列及其间间距可能占据的最大宽度。
- **控制列中文本分布的选项:**
- ◇ `balance`: 默认情况下, ConT_EXt 平衡列, 意思是它会在列之间分布文本, 使它们具有大致相同的文本量。但是, 我们可以将此选项设置为「no」, 则文本不会在上一列填满之前开始新的一列。
 - ◇ `direction`: 确定文本在列之间的分布方向。默认情况下, 遵循自然阅读顺序(从左到右), 但将此选项设置为 `reverse` 值会导致从右到左。
- **影响此环境内文本排版的选项:**
- ◇ `tolerance`: 用多于一列书写文本意味着列内的行宽更小, 并且正如在描述 ConT_EXt 用于构建行的机制时解释的那样 ([Section 3](#)), 这使得找到插入换行的最佳点变得困难。此选项允许我们临时更改环境中的水平容差 (参见 [Section 3.3](#)), 以便于文本的排版。
 - ◇ `align`: 控制环境内行的水平对齐。它可以采用以下任何值: `right`、`flushright`、`left`、`flushleft`、`inner`、`flushinner`、`outer`、`flushouter`、`middle` 或 `broad`。有关这些选项的含义, 请参见 [Section 6.1](#)。
 - ◇ `color`: 指定环境内文本的书写颜色。

另一种影响文本在列间分布的方式是 `\column` 命令。在正文中的任何位置使用此命令会指示 ConT_EXt 停止当前列并继续在下一列中排版。

例如, 下面的示例演示了如何将一些西班牙语文本 (来自 ANSS 西班牙语原版) 与英语翻译并排排版。英语部分被故意截断, 以演示 `\column` 命令如何阻止正常的列平衡。(在这种情况下, 完整的英语翻译将占用与西班牙语相同的行数。)

```

\startnarrower
\startcolumns[distance=1cm,separator=rule]
    纸张宽度:无限数量的列需要无限的纸张宽度(如果文档要打印)或屏幕宽度(如果是打算在屏幕上
    显示的文档)。
\column
    The width of the paper: An unlimited number of columns requires a
    unlimited width of paper \unknown
\stopcolumns
\stopnarrower

```

纸张宽度:无限数量的列需要无限的纸张宽度(如果文档要打印)或屏幕宽度(如果是打算在屏幕上显示的文档)。

The width of the paper: An unlimited number of columns requires a unlimited width of paper ...

2.2 并行段落

多列排版的一个特定版本是并行段落。在这种类型的构造中,文本分布在两列(通常,尽管有时超过两列)中,但不允许在它们之间自由流动,而是严格控制每列中显示的内容。这非常有用,例如,用于生成对比文本两个版本的文档,例如新修订的法律的新旧版本,或双语版本;或者用于为特定文本定义编写词汇表,其中要定义的文本出现在左侧,定义出现在右侧等。

通常,表格机制用于处理这些类型的段落,但这是因为大多数文本处理器不如 ConT_EXt 强大,ConT_EXt 具有 `\defineparagraphs` 和 `\setupparagraphs` 命令。这些命令允许使用列机制构建并行段落,该机制虽然有局限性,但比表格机制更灵活。

据我所知,这些段落没有特殊名称。我称它们为「并行段落」,因为在我看来,这比 ConT_EXt 用来指代它们的术语更具描述性:「*paragraphs*」。

这里的基本命令是 `\defineparagraphs`,其语法为:

```
\defineparagraphs[Name][Configuration]
```

其中 `Name` 是我们给此构造的名称,`Configuration` 指定它将具有的特性,这些特性也可以稍后通过以下方式设置

```
\setupparagraphs[Name][Column][Configuration]
```

其中 `Name` 是创建时给定的名称,`Column` 是一个可选参数,允许我们配置特定的列,而 `Configuration` 允许我们确定它在实践中的工作方式。

例如:

```

\defineparagraphs
[MurciaFacts]
[n=3, before={\blank},after={\blank}]

\setupparagraphs

```

```
[MurciaFacts][1]
[width=.1\textwidth, style=bold]

\setupparagraphs
[MurciaFacts][2]
[width=.4\textwidth]
```

上面的片段将创建一个名为 MurciaFacts 的三列环境,然后将第一列设置为占据行宽的 10

一旦创建了环境,我们就可以用它来写一个关于穆尔西亚的简短历史:

```
\startMurciaFacts
825
\MurciaFacts
City of Murcia founded.
\MurciaFacts
The origins of the city of Murcia are uncertain, but there is evidence
that it was ordered to be founded under the name of Madina (or Medina)
Mursiya in the year 825 by the Emir of al-Andalus Abderramán II,
probably built over a much earlier settlement.
\stopMurciaFacts
```

825	穆尔西亚市建立。	穆尔西亚市的起源尚不确定,但有证据表明,安达卢斯埃米尔阿卜杜勒拉赫曼二世于 825 年下令以 Madina(或 Medina)Mursiya 的名义建立它,很可能是在一个更早的定居点上建造的。
-----	----------	--

如果我们想继续讲述穆尔西亚的故事,下一个事件将需要一个新的环境实例(`\startMurciaFacts`),因为使用此机制无法包含多个行。(也就是说,这个特定的环境不允许我们继续另一行讲述例如 830 年发生的事情。)

从刚刚给出的例子中,我想强调以下细节:

- 一旦使用例如命令 `\defineparagraphs[MaryPoppins]` 创建了一个环境,它就变成了一个正常的环境,以 `\startMaryPoppins` 开始,以 `\stopMaryPoppins` 结束。
- 还会创建一个 `\MaryPoppins` 命令,在环境内部使用它来指示何时更改列。

上面「普通」`\startcolumns` 环境的例子本可以作为本「并行段落」节的例子。然而,介绍 `\column` 命令似乎是在正确的地方。ConTeXt wiki 指出 `\column` 是 `\columnbreak` 命令的别名,这甚至可能是一个更容易记住的名称。请注意,该命令不是 `\columns`,与 `\defineparagraphs[...]` 创建的类似命令相比,这是一个轻微的不一致。

至于并行段落的配置选项(`\setupparagraphs`),据我所知,在本入门文档的这个阶段,并考虑到刚刚给出的例子,读者已经准备好弄清楚每个选项的用途。因此,下面我将仅指示选项的名称和类型,并在适当时给出可能的值。但是请记住,`\setupparagraphs [Name] [Configuration]` 设置影响整个环境的配置,而

`\setupparagraphs [Name] [NumColumn] [Configuration]` 将配置专门应用于所指示的列。

- | | | |
|------------------------------|---|-----------------------------------|
| • <code>n</code> : 数字 | • <code>align</code> : 派生自 <code>\setupalign</code> | • <code>rulethickness</code> : 尺寸 |
| • <code>before</code> : 命令 | | • <code>style</code> : 样式命令 |
| • <code>after</code> : 命令 | • <code>inner</code> : 命令 | • <code>color</code> : 颜色 |
| • <code>width</code> : 尺寸 | • <code>rule</code> : on 或 off | |
| • <code>distance</code> : 尺寸 | • <code>rulecolor</code> : 颜色 | |

上面的选项列表并不完整;我从列表中排除了那些我通常不会在这里解释的选项。我还利用了我们在专门讨论列的这一节这一事实,以三列显示选项列表,尽管我没有用本节解释的任何命令来做到这一点;相反,使用了 `itemize` 环境的 `columns` 选项,下一节将专门介绍它。

第3节 结构化列表

当信息以有序的方式呈现时,读者更容易掌握。但如果这种排列在视觉上也是可感知的,那么它就会向读者突出显示这里有一个结构化的文本。这就是为什么有某些构造或机制试图在视觉上组织文本,从而有助于其结构化。在 ConTeXt 为此目的向作者提供的工具中,最重要的是 `itemize` 环境,它用于开发我们可以称之为结构化列表的内容。

列表由一系列文本元素(我将称之为条目)组成,其中每个条目前面都有一个有助于通过将其与其余部分区分开来突出显示它的字符,我称之为「分隔符」。分隔符可以是数字、字母或符号。通常(但并非总是)条目是段落,并且列表被格式化以确保每个元素的分隔符的可见性;通常通过应用悬挂缩进来实现,这使得第一个单词或字符突出¹。在嵌套列表的情况下,每个子列表的缩进逐渐增加。在 HTML 中,分隔符是数字或顺序递增的字符的列表称为有序列表,这意味着列表的每个条目将有一个不同的分隔符,允许我们通过其编号或标识符来引用每个元素;而对于列表中的每个条目都使用相同字符或符号的列表,则称为无序列表。

ConTeXt 自动管理编号列表中分隔符的编号或字母顺序,以及嵌套列表所需的缩进。此外,在嵌套无序列表的情况下,ConTeXt 还会根据其前面的符号,选择不同的字符或符号,以便一眼就能区分列表条目的级别。

参考手册说列表的最大嵌套级别是四,但我猜那是在 2013 年,手册编写的时候。根据我的测试,对于有序列表似乎没有限制(在我的测试中,我达到了 15 级嵌套)。而对于无序列表,似乎也没有限制,即无论我们包含多少嵌套,都不会产生错误;但是,对于无序列表,ConTeXt 只对前九个嵌套级别应用默认符号。

无论如何,应该指出的是,在列表中过度使用嵌套可能会产生与我们的意图相反的效果,即读者会感到迷失,无法将每个条目定位在列表的总体结构中。因此,我个人认为,虽然列表是构建文本的强大工具,但几乎从来都不是一个好主意超过第三级嵌套;即使是第三级也只应在某些我们可以证明其合理性的情况下使用。

在 ConTeXt 中编写列表的通用工具是 `\itemize` 环境,其语法如下:

¹ 在排版中,应用于段落除第一行之外的所有行的缩进称为悬挂缩进,它使段落的第一个单词或字符易于找到。

`\startitemize[Options][Configuration] ... \stopitemize`

其中两个参数都是可选的。第一个允许使用已被 ConT_EXt 赋予明确含义的符号名称作为内容;第二个参数很少使用,允许为影响环境功能的某些变量分配特定值。

3.1 选择列表类型和条目之间的分隔符

3.1.1 无序列表

默认情况下, `itemize` 生成的列表是无序列表,其中分隔符将根据嵌套级别自动选择:

- | | |
|------------|------------|
| • 对于第一级嵌套。 | ○ 对于第六级嵌套。 |
| – 对于第二级嵌套。 | ○ 对于第七级嵌套。 |
| ★ 对于第三级嵌套。 | □ 对于第八级嵌套。 |
| ▷ 对于第四级嵌套。 | ✓ 对于第九级嵌套。 |
| ◦ 对于第五级嵌套。 | |

然而,我们可以明确指示想要使用与特定级别关联的符号,只需将级别编号作为参数传递即可。因此, `\startitemize[4]` 将生成一个无序列表,其中将使用 ▷ 字符作为分隔符,而不管列表的嵌套级别如何。

我们还可以使用 `\definesymbol` 修改每个级别的预定符号:

`\definesymbol[Level]{Symbol associated with the level}`

例如

`\definesymbol[1][\diamond]`

将使无序列表的第一级使用 ◇ 符号。使用相同的命令,我们可以为高于九的嵌套级别分配一些符号。因此,例如

`\definesymbol[10][\copyright]`

将为第 10 级嵌套分配国际版权符号:©。

3.1.2 有序列表

要生成有序列表,我们需要告诉 `\startitemize` 我们想要的排序类型。它可以是:

n	1, 2, 3, 4, ...	g	$\alpha, \beta, \gamma, \delta, \dots$	a	a, b, c, d, ...
m	1, 2, 3, 4, ...	G	A, B, Γ , Δ , ...	A	A, B, C, D, ...

KA A, B, C, D, ...

r i, ii, iii, iv, ...

KR I, II, III, IV, ...

R I, II, III, IV, ...

`n` 和 `m` 之间的区别在于用于表示数字的数字样式:`n` 使用当前字体的数字, 而 `m` 使用的数字是通过 `\os`(「旧式」)命令获得的;`{\os 0 1 2 3 4 5 6 7 8 9}` 产生 `o 1 2 3 4 5 6 7 8 9`。

3.2 输入列表的条目

通常, 使用 `\startitemize` 创建的列表中的条目使用 `\item` 命令引入; 但是, `\item` 也有一个“环境版本”形式: `\startitem ... \stopitem`。因此, 以下示例:

```
\startitemize[a, packed]
\startitem 第一个元素 \stopitem
\startitem 第二个元素 \stopitem
\startitem 第三个元素 \stopitem
\stopitemize
```

- a. 第一个元素
- b. 第二个元素
- c. 第三个元素

产生的结果与以下完全相同

```
\startitemize[a, packed]
\item 第一个元素
\item 第二个元素
\item 第三个元素
\stopitemize
```

- a. 第一个元素
- b. 第二个元素
- c. 第三个元素

`\item` 或 `\startitem` 是用于将条目引入列表的通用命令。除此之外, 还有以下附加命令, 用于当我们想要实现特殊结果时:

\head 当我们想要避免在相关条目之后插入分页时, 应使用此命令代替 `\item`。

一种常见的构造是在列表元素下方立即包含一个嵌套列表或文本块, 这样列表元素在某种意义上就充当了子列表或文本块的标题。在这些情况下, 该元素和后续段落之间的分页是不可取的。如果我们使用 `\head` 而不是 `\item` 来开始这些元素, `ConTeXt` 将尽力(尽可能)不将此元素与下一个块分开。

\sym `\sym{Text}` 命令开始一个条目, 其中 `\sym` 的参数文本用作分隔符, 而不是通常的数字或符号。例如, 此列表的条目以 `\sym` 而不是 `\item` 开始。

\sub 此命令应仅在有序列表中使用(其中每个条目前面都有一个数字或按字母顺序排列的字母), 它使其条目保留前一个条目的编号, 并且为了指示编号是重复的并且这不是一个错误, 在左侧打印『+』符号。如果我们引用一个以前的列表, 并对其提出修改建议, 但为了清晰起见, 应保持原始列表的编号, 这可能很有用。

\mar 此命令保持条目的编号, 但在页边距中添加一个字母或字符(作为参数, 放在花括号中传递给它)。我不太确定它有多大用处。

还有两个用于引入条目的附加命令,它们的组合会产生非常有趣的效果,并且如果我可以这么说的话,我认为最好用一个例子来解释它们。它们是 `\ran`(*range* 的缩写)和 `\its`(*items* 的缩写)。第一个接受一个参数(放在花括号中),第二个重复列表中用作分隔符的符号 *x* 次(默认四次,但我们可以通过使用 `\startitemize` 的 `items=` 选项来改变它)。以下示例显示了这两个命令如何协同工作以创建一个模拟问卷的列表。注意如何使用 `width` 指定「No Yes」列的宽度,以及如何使用 `distance` 指定复选框和问题之间的间隙。这在 [Section 3.4](#) 中有更详细的解释。

阅读以下介绍后,回答以下问题:

```
\startitemize[8,packed][width=8em,distance=2em,items=5]
\ran{No \hss Yes}
\its 我永远不会使用 \ConTeXt,它太难了。
\its 我只会用它来写大书。
\its 我会一直使用它。
\its 我非常喜欢它,以至于我将给我的下一个孩子起名叫 \quotation{Hans},以向 Hans Hagen 致敬。
\stopitemize
```

阅读本介绍后,回答以下问题:

No	Yes
☐ ☐ ☐ ☐ ☐	我永远不会使用 ConT _E Xt,它太难了。
☐ ☐ ☐ ☐ ☐	我只会用它来写大书。
☐ ☐ ☐ ☐ ☐	我会一直使用它。
☐ ☐ ☐ ☐ ☐	我非常喜欢它,以至于我将给我的下一个孩子起名叫「Hans」,以向 Hans Hagen 致敬。

可以通过给 `\its` 一个参数来更改单个条目的分隔符数量,如下例所示。

```
\startitemize[5,packed][width=8em,distance=1.5em,items=4]
\ran{A bit\hss A lot}
\its 我喜欢蛋糕。
\its 我喜欢馅饼。
\its[3] 我喜欢饼干。
\stopitemize
```

A bit	A lot
◦ ◦ ◦ ◦	我喜欢蛋糕。
◦ ◦ ◦ ◦	我喜欢馅饼。
◦ ◦ ◦	我喜欢饼干。

3.3 基本列表配置

回想一下,`\startitemize` 允许两个参数。我们已经看到第一个参数如何让我们选择我们想要的列表类型。但我们也可以用它来指示列表的其他特性;这是通过 `\startitemize` 在其第一个参数中的以下选项完成的:

- **columns**: 此选项确定列表由两列或更多列组成。在 **columns** 选项之后, 必须将所需的列数写成用逗号分隔的单词: two、three、four、five、six、seven、eight 或 nine。如果 **columns** 后面没有跟任何数字, 则生成两列。
- **intro**: 此选项尝试不通过换行将列表与其前面的段落分开。
- **continue**: 在有序列表(数字或字母)中, 此选项使列表延续上一个编号列表的编号。如果使用了 **continue** 选项, 则不必指示我们想要什么类型的列表, 因为假定它将与最后一个编号列表相同。
- **packed**: 是最常用的选项之一。它的使用会导致列表不同条目之间的垂直空间尽可能减小。
- **nowhite**: 产生类似于 **packed** 的效果, 但更剧烈: 它不仅减少了条目之间的垂直空间, 还减少了列表与周围文本之间的垂直空间。
- **broad**: 增加条目分隔符和条目文本之间的水平空间。可以通过将一个数字乘以 **broad** 来增加空间, 例如 `\startitemize[2*broad]`。
- **serried**: 删除条目分隔符和文本之间的水平空间。
- **intext**: 删除悬挂缩进。
- **text**: 删除悬挂缩进并减少条目之间的垂直空间。
- **repeat**: 在嵌套列表中, 使子级别的编号重复与前一级别相同的级别。这样, 在第一级上, 我们将有: 1, 2, 3, 4; 在第二级上: 1.1, 1.2, 1.3 等。该选项必须在内部列表上指示, 而不是在外部列表上。
- **margin, inmargin**: 默认情况下, 列表分隔符打印在左侧, 但在文本区域本身内(「atmargin」)。选项 **margin** 和 **inmargin** 将分隔符移动到页边距。

3.4 附加列表配置

`\startitemize` 中的第二个参数(也是可选的)允许对列表进行更详细和彻底的配置。

- **before, after**: 分别在开始或结束 **itemize** 环境之前或之后要运行的命令。
- **inbetween**: 在两个 **items** 之间运行的命令。
- **beforehead, afterhead**: 在使用 `\head` 命令输入的条目之前或之后要运行的命令。
- **left, right**: 要打印在分隔符左侧或右侧的字符。例如, 要获得字母两边有括号的字母列表, 我们可以写:

`\startitemize[a][left=(, right=)]`

- **stopper**: 指示要写在分隔符后面的字符。仅在有序列表中有效。
- **width, maxwidth**: 为分隔符保留的空间宽度, 因此也是悬挂缩进的宽度。
- **factor**: 表示分隔符和文本之间分隔因子的数字。
- **distance**: 分隔符和文本之间距离的度量。
- **leftmargin, rightmargin, margin**: 要添加到条目左侧(leftmargin)或右侧(rightmargin)的边距。
- **start**: 项目编号开始的数字。
- **symalign, itemalign, align**: 条目的对齐方式。允许与 `\setupalign` 相同的值; `symalign` 控制分隔符的对齐, `itemalign` 控制条目文本的对齐, `align` 控制两者的对齐。
- **indenting**: 环境内段落第一行的缩进。参见 [Section 1.1](#)
- **indentnext**: 指示环境之后的段落是否应该缩进。值为 `yes`、`no` 和 `auto`。
- **items**: 在使用 `\its` 输入的条目中, 指示必须重复分隔符的次数。
- **style, color; headstyle, headcolor; marstyle, marcolor; symstyle, symcolor**: 这些选项对根据条目是分别使用 `\item`、`\head`、`\mar` 还是 `\sym` 命令引入的, 来控制条目的样式和颜色。

3.5 使用 `\items` 命令的简单列表

对于非常简单的未编号列表(其中条目不太大), `itemize` 环境的一种替代方案是 `\items` 命令, 其语法为:

`\items[Configuration]{Item 1, Item 2, ..., item n}`

不同的列表项用逗号分隔。例如:

图形文件可以具有
(除其他外)以下
扩展名:

`\items{png, jpg, tiff, gif}`

图形文件可以具有(除其他外)以下扩展名:

- `png`
- `jpg`
- `tiff`
- `gif`

此命令支持的配置选项是 `itemize` 选项的子集, 除了两个特定于此命令的选项:

- `symbol`: 此选项确定将生成的列表类型。它支持与用于选择某种列表类型的 `itemize` 相同的值。
- `n`: 此选项指示从哪个项目编号开始有分隔符。

3.6 预设列表行为并创建我们自己的列表类型

在前面的部分中,我们已经看到了如何指示我们想要的列表类型以及它应该具有的特性。但是每次调用列表时都这样做是低效的,并且通常会产生一个不连贯的文档,其中每个列表都有自己的外观,但不同的外观不符合任何标准。

更可取的结果是:

- 在文档前言中预设 `itemize` 环境和 `\items` 的正常行为。
- 创建我们自己的定制列表。例如:一个按字母顺序编号的列表(我们可以称之为 `ListAlpha`),一个用罗马数字编号的列表(`ListRoman`)等。

我们通过 `\setupitemize` 和 `\setupitems` 命令实现第一点。第二点需要使用 `\defineitemgroup` 或 `\defineitems` 命令。第一个将创建一个类似于 `itemize` 的列表环境,第二个将创建一个类似于 `\items` 的命令。

第4节 描述与枚举

描述和枚举是两种构造,允许对展开、描述或定义某个短语或单词的段落或段落组进行一致的排版。

4.1 描述

对于描述,我们区分标题及其解释或展开。我们可以使用以下方式创建新的描述:

```
\definedescription[Name] [Configuration]
```

其中 `Name` 是此新构造的名称,`Configuration` 控制我们的新结构的外观。在前述声明之后,我们将有一个新命令和一个使用我们选择的名称的环境。因此:

```
\definedescription[Concept]
```

将创建以下命令:

```
\Concept{Title}
\startConcept {Title} \stopConcept
```

当标题的解释文本仅包含一个段落时,我们将使用该命令;对于标题描述占据多个段落的情况,则使用环境。当使用该命令时,紧随其后的段落将被视为标题的解释文本。但是当使用环境时,所有内容都将以适当的缩进进行格式化,以明确其与标题的关系。

例如:

```
\definedescription
[Concept]
[alternative=left, width=1cm, headstyle=bold]

\Concept{Contextualise}

Place something in a certain context, or typeset a text with the typesetting
system called \ConTeXt. The ability to correctly contextualise in any situation
is considered a sign of intelligence and good sense.
```

这将产生以下结果:

语境化 将某物置于特定语境中,或使用名为 ConTeXt 的排版系统排版文本。在任何情况下正确语境化的能力被视为智慧和良好判断力的标志。

与 ConTeXt 通常的情况一样,我们的新构造将具有的外观可以在创建时通过 Configuration 参数指示,或者稍后通过 `\setupdescription` 指示:

```
\setupdescription[Name] [Configuration]
```

其中 Name 是我们新描述的名称,Configuration 决定其外观。在不同的可能配置选项中,我将强调:

- **alternative**: 此选项从根本上控制构造的外观。它确定标题相对于其描述的位置。其可能值为 `left`、`right`、`inmargin`、`inleft`、`inright`、`margin`、`leftmargin`、`rightmargin`、`innermargin`、`outermargin`、`serried` 和 `hanging`。它们的名称足够清晰,可以让人了解结果,但如果有疑问,最好进行测试看看效果。
- **width**: 控制用于书写标题的框的宽度。根据 **alternative** 的值,该距离也将成为解释文本缩进的一部分。
- **distance**: 控制标题和解释之间的距离。
- **headstyle**, **headcolor**, **headcommand**: 这些影响标题本身的外观。使用 **headcommand**,我们可以指示一个命令,标题文本将作为参数传递给该命令。例如: `headcommand=\WORD` 将确保标题文本大写。
- **style**, **color**: 控制描述文本的外观。

4.2 枚举

枚举是在多个级别上构建的编号文本元素。每个元素以一个标题开始,默认情况下,该标题由结构名称及其编号组成,尽管我们可以使用 `text` 选项更改标题。它们与描述非常相似,但不同之处在于:

- 枚举中的所有元素共享相同的标题。
- 因此它们通过编号相互区分。

这种结构非常有用,例如,用于编写教科书中的公式、问题或练习,确保它们被正确编号并以一致的方式格式化。

我们使用以下方式创建枚举

```
\defineenumeration[Name] [Configuration]
```

其中 *Name* 是新构造的名称, *Configuration* 控制其外观。

因此,在以下示例中:

```
\defineenumeration  
[Exercise]  
[alternative=top, before=\blank, after=\blank, between=\blank]
```

我们创建了一个名为 *Exercise* 的新结构,并且与枚举一样,我们将有以下新命令可用:

```
\Exercise  
\startExercise  
\stopExercise
```

该命令用于单段落的练习,而环境用于多段落的练习。但是由于枚举最多可以有四层深度,还将创建以下命令和环境:

```
\subExercise  
\startsubExercise  
\stopsubExercise  
\subsubExercise  
\startsubsubExercise  
\stopsubsubExercise  
\subsubsubExercise  
\startsubsubsubExercise  
\stopsubsubsubExercise
```

并且,为了控制编号,还有以下附加命令:

- `\setEnumerationName`: 设置当前编号值。
- `\resetEnumerationname`: 将枚举计数器重置为零。
- `\nextEnumerationName`: 将枚举计数器增加一。

枚举的外观可以在创建时确定,或者稍后使用 `\setupenumeration` 确定,其格式为:

`\setupenumeration[Name] [Configuration]`。

对于每个枚举,我们可以分别配置其每个级别。因此,例如,`\setupenumeration [subExercise] [Configuration]` 将影响名为「Exercise」的枚举的第二级。

可以使用 `\setupenumeration` 配置的选项和值与 `\setupdescription` 中的类似。

第5节 线条与框架

ConT_EXt 参考手册中说,T_EX 具有强大的文本管理能力,但在管理图形信息方面非常弱。我不敢苟同:确实,在处理线条和框架方面,ConT_EXt(实际上是 T_EX)的可能性不像排版文本那样强大。但是要说系统在这方面很弱,我认为有点夸张。我不知道其他排版系统能为文档做的关于线条和框架的任何功能是 ConT_EXt 无法生成的。而且如果我们将 ConT_EXt 与 MetaPost 或 TiKZ(ConT_EXt 有一个用于此的扩展模块)结合起来,那么可能性只受限于我们的想象力。

然而,在以下部分中,我将仅限于解释如何生成简单的水平线和垂直线,以及围绕单词、句子或段落的框架。

5.1 简单线条

绘制水平线的最简单方法是使用 `\hairline` 命令,它生成一条占据正常文本行所有宽度的水平线。

因为 `\hairline` 占据文本行的整个宽度,所以同一行上没有空间容纳任何文本。为了生成能够与文本共存的水平线,我们需要 `\thinrule` 命令。如果在段落之间使用,这第二个命令将使用该行的整个宽度;在这种情况下,它将产生与 `\hairline` 相同的效果。当在段落中使用时,`\thinrule` 将产生与 `\hfill` 相同的水平扩展(参见 [Section 3.3](#)),但它不是用空白填充水平空间(如 `\hfill` 所做的的那样),而是用一条线填充它。

```
左边\thinrule\\
\thinrule 右边\\
两边\thinrule 都有\\
\thinrule 居中\thinrule
```

```
左边_____
_____右边
两边_____都有
_____居中_____
```

使用 `\thinrules` 命令,我们可以生成多条线。例如,`\thinrules[n=2]` 将生成两条连续的线,每条线都占据正常行的宽度。使用 `\thinrules` 生成的线也可以配置,要么在实际调用命令时,将配置作为其参数之一指示,要么通常使用

`\setupthinrules`。配置包括线条的粗细(`rulethickness`)、颜色(`color`)、背景色(`background`)、行间距(`interlinespace`)等。

我将省略一些选项的解释。读者可以在 `setup-en.pdf` 中查阅它们(参见 [Section 6](#))。有些选项仅在细微差别上有所不同(即它们之间几乎没有区别),我认为让读者尝试去看到区别,比我自己试图用语言表达要快。例如:我刚才说的线条粗细取决于 `rulethickness` 选项。但它也受到 `height` 和 `depth` 选项的影响。

可以使用 `\h1` 和 `\v1` 命令生成较小的线。第一个生成水平线,第二个生成垂直线。两者都接受一个数字作为参数,允许我们指定线的长度。在 `\h1` 中,数字以 *ems* 为单位测量长度(在此命令中无需指定测量单位),而在 `\v1` 中,参数乘以当前文本行的高度以获得线长。

因此 `\h1[3]` 生成一条 3 *ems* 长的水平线,而 `\v1[3]` 生成一条对应于三行高度的垂直线。请记住,线测量指示符必须放在方括号之间,而不是花括号之间。在这两个命令中,参数都是可选的。如果不输入,则假定值为 1。这是 `\v1[1]:|`。

`\fillinline` 是另一个用于创建精确长度水平线的命令。它支持更多配置,我们可以在其中指示(或使用 `\setupfillinlines` 预设)宽度(`width` 选项)以及其他特性。

此命令的一个特点是,写在它右侧的文本将被放置在线的左侧,通过必要的空白将该文本与线分开以占据整行。例如:

```
\fillinline[width=6cm] 姓名
```

将生成

姓名

我怀疑这种奇怪的操作是由于此宏被设计用于编写表格,其中文本后面有一条水平线,必须要在上面写东西。



除了线的宽度,我们还可以配置边距(`margin`)、距离(`distance`)、粗细(`rulethickness`)和颜色(`color`)。

与 `\fillinline` 几乎相同的是 `\fillinrules`,尽管此命令允许我们插入多于一条线(「`n`」选项):

```
\fillinrules[Configuration] {Text before rule} {Text after rule}
```

其中三个参数都是可选的。(要获得线后的文本,请使用两组花括号,其中第一组为空。)例如,行

```
\fillinrules[distance=1cm, separator=?, n=2]{To be} {or not to be}
```

生存?

还是毁灭

警告:在闭合方括号和第一个开放花括号之间放置一个空格会导致完全不同的结果:

生存还是毁灭

5.2 与文本关联的线条

尽管我们刚刚看到的一些命令可以生成与同一行上的文本共存的线条,但这些命令实际上专注于绘制线条。要编写与特定文本关联的线条,ConTeXt 有以下命令:

- 生成文本之间的线条。
- 生成文本下方的线条(下划线)、文本上方的线条(上划线)或穿过文本的线条(删除线)。

要生成文本之间的线条,通常的命令是 `\textrule`。此命令绘制一条穿过页面整个宽度的线,并将它作为参数接受的文本写在左侧(但不在页边距处)。例如:

`\textrule{示例文本}`

—— 示例文本 ————

假定 `\textrule` 允许一个可选的第一参数,具有三个可能的值:top、middle 和 bottom。但是,经过一些测试,我无法找出这些选项产生什么效果。



类似于 `\textrule` 的是 `\starttextrule` 环境,除了在环境开头插入带文本的线之外,它还在结尾插入一条水平线。此命令的格式为:

`\starttextrule[Configuration]{Text on the line} ... \stoptextrule`

`\starttextrule{引言文本}`

一些文本将被设置在两条线之间。底线横跨页面。

`\stoptextrule`

—— 引言文本 ————

一些文本将被设置在两条线之间。底线横跨页面。

`\textrule` 和 `\starttextrule` 都可以使用 `\setuptextrule` 进行配置。

要绘制文本下方、上方或穿过文本的线条,使用以下命令:

`\underbar{文本单词}`

`\underbars{文本单词}`

`\overbar{文本单词}`

`\overbars{文本单词}`

`\overstrike{文本单词}`

`\overstrikes{文本单词}`

文本单词

文本单词

文本单词

文本单词

文本单词

文本单词

如我们所见,对于每种类型的线条(文本下方、上方或穿过),都有两个命令。命令的单数版本在作为参数接受的所有文本下方、上方或穿过绘制一条线,而命令的复数版本仅在线条下方、上方或穿过单词绘制,而不穿过空白。

这些命令彼此不兼容;也就是说,其中两个不能应用于同一文本。如果我们尝试,最后一个将始终占上风。另一方面,`\underbar` 可以重复,为已经加下划线的文本加下划线:`\underbar{\underbar{two underbars}}` 输出 two underbars。

参考手册指出,`\underbar` 会禁用构成其参数的文本中单词的断字。我不清楚该陈述是仅指 `\underbar` 还是指我们正在检查的六个命令。

5.3 带框架的单词或文本

要用框架(即一个框)包围文本,我们使用:

- `\framed` 或 `\inframed` 命令,如果文本相对简短且不超过一行。
- `\startframedtext` 环境用于较长的文本。

`\framed` 和 `\inframed` 的区别在于绘制框架的起始点。在 `\frame` 中,框架从一条称为基线的理想线向上绘制,字母坐落在基线上,但某些字母会向下延伸(像『j』和『p』这样的字母具有下行字母,即延伸到基线以下的笔画)。在 `\inframed` 中,框架也从线上可能的最低点向上绘制。例如:

这里有 `\framed{两个}` 不错的
`\inframed{框架}`。

这里有 两个 不错的 框架。

`\framed` 和 `\inframed` 文本都可以使用 `\setupframed` 进行定制,而 `\startframedtext` 使用 `\setupframedtext` 进行定制。这两个命令的定制选项非常相似。它们允许我们指示框架的尺寸(`height`、`width`、`depth`)、角部形状(`framecorner`),可以是 `rectangular` 或圆角(`round`)、框架颜色(`framecolor`)、线条粗细(`framethickness`)、内容对齐(`align`)、文本颜色(`foregroundcolor`)、背景颜色(`background` 和 `backgroundcolor`)等。

对于 `\startframedtext`,还有一个看似奇怪的选项,`frame=off`,它导致框架不被绘制(尽管它仍然存在,但不可见)。此选项在段落(或其他应保持在一起的材料)内不希望有分页的情况下很有用。由于段落周围的框架是不可分割的,将段落包含在 `framedtext` 环境中(即使关闭了框架绘制选项)可确保段落内不插入分页。

我们还可以使用 `\defineframed` 和 `\defineframedtext` 创建这些命令的定制版本。

第6节 其他感兴趣的环境和构造

在 ConT_EXt 中还有许多环境我甚至没有提到,或者只是顺便提了一下。例如:

- **buffer**: 缓冲区是存储在内存中供以后重用的文本片段。一个缓冲区在文档中的某处使用 `\startbuffer[BufferName] ... \stopbuffer` 定义, 并且可以在文档的其他地方根据需要多次使用 `\getbuffer[BufferName]` 检索。如果您检查本章的「源代码」, 您会看到缓冲区经常在并排示例中使用, 以便使用 `\typebuffer[BufferName]` 显示和使用 `\getbuffer[BufferName]` 排版的文本只需输入一次。
- **chemical**: 此环境允许我们在其中放置化学公式。如果说 $\text{T}_{\text{E}}\text{X}$ 在许多其他事情中因其能够正确排版带有数学公式的文本而脱颖而出, 那么 $\text{ConT}_{\text{E}}\text{Xt}$ 从一开始就试图将这种能力扩展到化学公式, 并且它拥有这个环境, 其中启用了用于编写化学公式的命令和结构。
- **combination**: 此环境允许我们将多个浮动元素(参见 [Section 1](#))组合在同一页面上。这对于在我们的文档中对齐不同的连接外部图像特别有用。
- **formula**: 这是一个旨在排版数学公式的环境。
- **hiding**: 存储在此环境中的文本将不会被编译, 因此不会出现在最终文档中。这对于暂时禁用源文件中某些片段的编译很有用。通过将一行或多行标记为注释也可以实现同样的效果。但是, 当我们要禁用的片段相对较长时, 比将源文件的数十行或数百行标记为注释更有效的是在片段开头插入 `\starthiding` 命令, 在结尾插入 `\stophiding`。
- **legend**: 在数学上下文中, $\text{T}_{\text{E}}\text{X}$ 应用不同的规则, 使得不能书写任何普通文本。然而, 有时公式会附带对其中所用元素的描述。为此, 有一个 `\startlegend` 环境, 允许我们在数学上下文中放置普通文本。
- **linecorrection**: 通常, $\text{ConT}_{\text{E}}\text{Xt}$ 可以正确管理行之间的垂直空间, 但偶尔一行可能包含导致其无法正确显示的内容。这主要发生在带有使用 `\framed` 框架的片段的行上。在这种情况下, 此环境会调整行距, 以便段落正确显示。
- **mode**: 此环境旨在包含源文件中的片段, 这些片段仅在激活相应模式时才会被编译。模式的使用不是本入门文档的主题, 但如果我们希望能够从单个源文件生成具有不同格式的文档的多个版本(例如, 要打印的版本或针对屏幕阅读优化的版本), 这是一个非常有趣的工具。与此环境互补的是 `\startnotmode`, 其中包含在 `\startnotmode[someMode]` 和 `\stopnotmode` 之间的源材料在模式 `someMode` 未被激活时包含在文档中。
- **opposite**: 此环境用于在左右页面的内容相关时排版文本。
- **quotation**: 一个与 `narrower` 非常相似的环境(参见 [Section 1.2](#)), 旨在插入中等长度的字面引用。此环境确保内部的文本被引用, 并且增加边距, 以便带有引用的段落能在页面上视觉上突出。但应注意, 根据英文常见的块引用样式, 不应有开头的引号和结尾的引号——这使得这个命令或环境的用处可能不如预期。
- **standardmakeup**: 此环境旨在生成带有文档标题的页面, 这在有一定长度的学术文档中相对常见, 例如博士论文、硕士论文等。

要了解这些环境(或我未提及的其他环境)中的任何一个,我建议执行以下步骤:

1. 在 ConT_EXt 参考手册中查找有关该环境的信息。此手册并未提及所有环境;但它确实对上面列表中的每个项目都有所说明。
2. 编写一个使用该环境的测试文档。
3. 查阅 ConT_EXt 的官方命令列表(参见 [Section 6](#)),了解相关环境的配置选项,然后测试它们以确切了解它们的作用。

第13章 图像、表格及其他浮动对象

Table of Contents: 第1节 什么是浮动对象,它们有什么作用?; 第2节 外部图像; 2.1 直接插入图像; 2.2 使用 `\placefigure` 插入图像; 2.3 插入集成到文本块中的图像; 2.4 插入图像的配置与变换; 2.4.1 导致图像某些变换的插入命令选项; 2.4.2 用于变换图像的特定命令; 第3节 表格; 3.1 关于表格及其在文档中放置的一般概念; 3.2 使用 `tabulate` 环境创建简单表格; 3.2.1 `tabulate` 环境的表格列布局; 3.2.2 `tabulate` 环境的列条目; 第4节 图像、表格及其他浮动对象的共同方面; 4.1 浮动对象插入选项; 4.2 配置浮动对象标题; 4.3 两个或多个对象的组合插入; 4.4 浮动对象的一般配置; 第5节 定义额外的浮动对象;

本章主要讨论浮动对象(floats)。但顺着这个概念,我们借此机会解释两种不一定是浮动对象、但通常被配置为浮动对象的对象类型:外部图像和表格。查看本章的目录,人们可能会认为这非常杂乱:它首先讨论浮动对象,然后讨论图像和表格,最后再次讨论浮动对象。这种杂乱的缘由是教学法上的:图像和表格可以在不过分强调它们通常是浮动对象的情况下进行解释;然而,当我们开始审视它们时,发现我们已经了解了两种浮动对象,这会大有帮助。

第1节 什么是浮动对象,它们有什么作用?

如果一个文档只包含普通文本,分页将相对容易:知道页面文本区域的最大高度就足以测量不同段落的高度,从而知道在哪里插入分页符。问题在于,许多文档中存在对象、片段或不可分割的文本块,例如图像、表格、公式、带框段落等。

有时这些对象会占据页面的大部分空间,这反过来又带来了一个问题:如果必须将其插入文档中的特定位置,它可能无法适应当前页面。在这种情况下,可能不得不突然中断页面,在底部留下大片空白,从而将相关对象及其后面的文本移到下一页。然而,良好的排版规则表明,除了章节的最后一页外,每页的文本量应该相同。

因此,建议避免出现大的垂直空白;浮动对象是实现这一目标的主要机制。「浮动对象」是指不必位于文档中特定位置,而是可以在文档中移动或浮动的对象。其理念是允许 ConTeXt 从分页的角度决定放置这些对象的最佳位置,甚至允许它们移动到另一页;但始终确保它们不会偏离源文件中插入点太远。

因此,没有对象本身必须是浮动的。但有些对象偶尔需要成为浮动对象。无论如何,决定权在于作者(或负责排版的人,如果不是同一个人的话)。

毫无疑问,允许改变不可分割对象的精确位置,极大地促进了排版出平衡美观的页面。但随之而来的问题是,由于我们在编写原文时不知道这样的对象最终会出现

在哪里,因此很难引用它。例如,如果我刚刚在文档中放入了一个插入图像的命令,并在下一段中想要描述它并写下类似「从上图可以看出」的内容,那么当图像浮动时,它很可能被放置在我刚刚写的内容之后,结果就会产生不一致:读者在引用它的文本之前寻找图像,却找不到,因为浮动之后,图像最终出现在该引用之后。

这个问题通过给浮动对象编号(将它们分类后)来解决,这样我们就不会将图像称为「上图」或「下图」,而是(例如)「图 1.3」,因为我们可以使用 ConT_EXt 的内部引用机制来确保图像编号始终保持最新(参见 Section 2)。此外,对这些对象进行编号使得创建它们的索引(表格索引、图形索引、图像索引、公式索引等)变得容易。具体操作方法请参见(Section 2)。

ConT_EXt 中处理浮动对象的机制相当复杂,并且达到了很高的抽象层次,可能不适合初学者。因此,在本章中,我将首先使用两个特例来解释它:图像和表格。然后,我会尝试进行一些概括,以便我们能够理解如何将该机制扩展到其他类型的对象。

第2节 外部图像

正如读者在这个阶段所知(因为在 Section 5 中已经解释过),ConT_EXt 与 MetaPost 完美集成,可以生成以与文本转换编程相同的方式编程的图像和图形。还有一个用于 ConT_EXt 的扩展模块¹,允许它使用 TikZ.²但此类图像不在本入门介绍中讨论(因为那可能会使其长度加倍)。这里我指的是使用外部图像,这些图像位于我们计算机上的文件中,或者由 ConT_EXt 直接从互联网下载。

2.1 直接插入图像

要直接插入图像(不作为浮动对象),我们使用 `\externalfigure` 命令,其语法为

`\externalfigure[Name] [Configuration]`

其中:

- **Name** 可以是包含图像的文件名,也可以是互联网上找到的图像的网址,或者是先前使用 `\useexternalfigure` 命令与图像关联的符号名称;该命令的格式与 `\externalfigure` 类似,不过它的第一个参数是符号名称,用于与相关图像关联。
- **Configuration** 是一个可选参数,允许我们在将图像插入文档之前对其应用某些变换。我们将在 Section 2.4 中更仔细地研究这个参数。

¹ ConT_EXt 扩展模块提供了额外的实用程序,但其描述未包含在本入门介绍中。

² 这是一种图形编程语言,旨在与基于 T_EX 的系统一起工作。TikZ 是一个“递归缩写”,源自德语短语「TikZ ist keinen Zeichenprogramm」,翻译过来意思是:「TikZ 不是一个绘图程序」。递归缩写是程序员的一种玩笑。撇开 MetaPost(我不知道如何使用它)不谈,我相信 TikZ 是一个很棒的图形编程系统。

允许的图像格式有 pdf、jpg、jp2(JPEG 2000)、jbig、jbig2、jb2、png、gif、tif、svg、mps 和 eps。ConTeXt 可以直接管理其中八种格式,而其余的(svg、eps、gif 和 tif)需要使用外部工具(因操作系统而异)进行转换,因此必须安装在系统上,以便 ConTeXt 能够处理这些类型的文件。

MkIV 注:\externalfigure 支持的格式中还包括一些视频格式。特别是:QuickTime(扩展名.mov)、Flash Video(扩展名.flv)和 MPEG 4(扩展名.mp4)。但大多数 PDF 阅读器不知道如何处理带有嵌入视频的 PDF 文件。对此我无法多言,因为我还没有做过任何测试。

LMTX 更新:测试表明,.mp4 和 .mov 文件可以成功嵌入到 ConTeXt 的输出中,并且(在 Linux 上)PDF 阅读器 evince 可以播放这些影片。其他成功案例如果被发现并报告,将在此处添加。

无需指明文件扩展名:ConTeXt 将搜索具有指定名称和已知图像格式扩展名的文件。如果有多个候选文件,则首先使用 PDF 格式(如果存在),如果不存在则使用 MPS 格式(由 MetaPost 生成的图形)。如果这两者都不存在,则遵循以下顺序:jpg、png、jp2、jbig、svg、eps、tif 以及最后的 gif。

如果图像的实际格式与其存储文件的扩展名不符,ConTeXt 将无法打开它,除非我们使用 method 选项指明图像的实际格式。

如果图像不是单独放在段落之外,而是集成到文本段落中,并且其高度大于行距,则会调整行距以防止图像与前面的行重叠,如本行  所附的例子所示。

默认情况下,ConTeXt 在工作目录、其父目录以及该目录的父目录中搜索图像。我们可以使用 \setupexternalfigures 命令的 directory 选项来指示包含我们将使用的图像的目录位置,这会将那个目录添加到搜索路径中。如果我们希望搜索仅在图像目录中进行,则还必须设置 location 选项。因此,例如,为了让 ConTeXt 在「img」目录中查找所有需要的图像,我们应该写:

```
\setupexternalfigures
[directory=img, location=global]
```

在 \setupexternalfigures 的 directory 选项中,我们可以包含多个目录,用逗号分隔;但在这种情况下,我们需要将目录括在花括号中。例如「directory={img, ~/images}」。

对于 directory 选项,我们始终使用『/』字符作为目录之间的分隔符,即使对于使用『\』作为目录分隔符的 Microsoft Windows 也是如此。

\externalfigure 也能够使用托管在互联网上的图像。因此,例如,以下片段将指示 ConTeXt 从互联网直接插入 CervanTeX 徽标到文档中。CervanTeX 是西班牙语 TeX 用户组:¹

¹ 互联网地址往往很长,并且没有太多空间来显示并排示例。因此,为了让左列中的命令合适地排布,我在网址内插入了一个换行。如果您复制并粘贴示例的 URL,如果不删除此换行,它可能无法工作(取决于您的操作系统、复制后的 URL 如何发送到您的 Web 浏览器,以及可能还有您的 Web 浏览器本身)。

```
\externalfigure
[http://www.cervantex.es/files/
cervantex/cervanTeXcolor-small.jpg]
```

```
name: http://www.cervantex.es/files/
cervantex/cervanTeXcolor-small.jpg
file: http://www.cervantex.es/files/
cervantex/cervanTeXcolor-small.jpg
state: unknown
```

当包含远程文件的文档首次编译时,它会从服务器下载并存储在 LuaMetaTeX 缓存目录中,对于独立的 ConTeXt 发行版,该目录位于您的系统上安装 ConTeXt 的位置的子目录 `tex/texmf-cache/luametateX-cache/context/` 中。如果您使用的是 T_EXlive,则缓存目录位于子目录 `texmf-var/luametateX-cache/context/` 中。缓存的文件在后续编译中使用。通常,如果缓存中的图像超过一天,则会重新下载远程图像。要更改此阈值,请参阅 ConTeXt wiki 上的[此页面](#)。

如果 ConTeXt 找不到应该插入的图像,不会产生错误,但会插入一个文本块来代替图像,其中包含关于应该放在那里的图像的信息。该块的大小将是图像大小(如果 ConTeXt 知道的话),否则为标准大小。[Section 4.3](#) 中有一个这样的例子。

2.2 使用 `\placefigure` 插入图像

正如我们已经看到的,图像可以直接使用 `\externalfigure` 插入。但最好使用 `\placefigure` 插入图像。此命令使 ConTeXt:

- 知道正在插入一个图像,该图像必须被纳入文档的图像列表中,以便后续我们可以根据需要生成图像索引;
- 为图像分配一个编号,从而方便对其的内部引用;
- 为图像添加标题,在图像及其标题之间创建一个文本块(这意味着它们不能被分开);
- 自动设置正确查看图像所需的空白(水平和垂直);
- 将图像放置在指示的位置,如有必要使文本围绕它流动;以及
- 如果可能,根据其大小和位置规范,将图像转换为浮动对象。¹

此命令的语法如下:

```
\placefigure[Options][Label] {Title} {Image}
```

各个参数的含义如下:

- **Options** 是一组指示,通常涉及放置图像的位置。由于这些选项在此命令和其他命令中相同,我将在后面一起解释它们(在 [Section 4.1](#) 中)。目前,我将在示例中

¹ 后者是我的结论,因为在放置选项中,有些像 `force` 或 `split` 这样的选项是违背浮动对象的真正概念的。

使用 `here` 选项。它告诉 ConT_EXt, 尽可能将图像精确地放置在文档中找到插入命令的位置。

- `Label` 是一个文本字符串, 用于内部引用此对象, 以便我们可以引用它 (参见 [Section 2](#))。
- `Title` 是要添加到图像的标题文本。
- `Image` 是插入图像的命令。

例如:

```
\placefigure
[here]
[fig:texknuth]
{\TeX\ 徽标和 {\sc Knuth} 的照片}
{
  \externalfigure
  [https://i.ytimg.com/vi/8c5Rrfabr9w/maxresdefault.jpg]
  [scale=600]
}
```

正如我们在示例中看到的, 通过插入图像 (顺便说一下, 这是直接从互联网上托管的图像完成的), 与直接使用 `\externalfigure` 命令时相比, 发生了一些变化。添加了垂直空间, 图像居中, 并添加了标题。这些是乍看之下显而易见的外部变化。从内部角度来看, 该命令也产生了其他同样重要的效果:

- 首先, 图像已被插入到 ConT_EXt 为插入到文档中的对象内部维护的「图像列表」中。这反过来意味着该图像将出现在可以用 `\placelist[figure]` 生成的图像索引中 (参见 [Section 2](#)), 尽管有两个特定的命令用于生成图像索引, 它们是 `\placelistoffigures` 或 `\completelistoffigures`。
- 其次, 图像已链接到作为 `\placefigure` 命令第二个参数添加的标签, 这意味着从现在起我们可以使用该标签对其进行内部引用 (参见 [Section 2](#))。例如, 「`\in{Figure}[fig:texknuth]`」的结果是「参见 Figure ??」。
- 最后, 图像变成了一个浮动对象, 这意味着如果出于排版需要 (分页) 需要移动, ConT_EXt 会改变其位置。

实际上, `\placefigure` 尽管名称如此, 但不仅用于插入图像。我们可以用它插入任何东西, 包括文本。然而, 使用 `\placefigure` 插入到文档中的文本或其他项目, 将被视为图像一样对待, 即使它们不是; 它们将被添加到 ConT_EXt 内部管理的图像列表中, 并且我们可以应用类似于用于图像的变换, 例如缩放或旋转、添加框架等。因此, 以下示例:

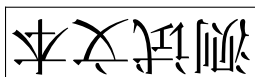


图 13.1 使用 `\placefigure` 进行文本变换

这是通过以下方式实现的：

```
\placefigure
[here, force]
[fig:testtext]
{使用 \backslashplacefigure 进行文本变换}
{\rotate[rotation=180]{\framed{\tfd 测试文本}}}
```

2.3 插入集成到文本块中的图像

除了非常小的图像(可以集成到一行中而不会过多干扰段落间距)外,图像通常被插入到只包含它们自己的段落中(或者换句话说,图像可以被视为一个独立的段落)。如果图像是使用 `\placefigure` 插入的,并且其尺寸允许,根据我们关于其放置位置的指示(参见 [Section 4.1](#)),ConTeXt 将允许前后段落的文本围绕图像流动。但是,如果我们想确保某个图像不会与某个文本分开,我们可以使用 `figuretext` 环境,其语法如下:

```
\startfiguretext
[Options]
[Label]
{Title}
{Image}

... 文本

\stopfiguretext
```

该环境的参数与 `\placefigure` 完全相同,并且具有相同的含义。但这里的选项不再是浮动对象放置位置的选项,而是关于将图像集成到段落中的指示;因此,例如,「`left`」在此表示图像将放置在左侧,而文本将流向右侧,而「`left, bottom`」将表示图像必须放置在与它关联的文本的左下侧。

在环境内编写的文本将是围绕图像流动的文本。

2.4 插入图像的配置与变换

2.4.1 导致图像某些变换的插入命令选项

`\externalfigure` 命令中的最后一个参数允许我们对插入的图像执行某些调整。我们可以进行这些调整：

- 一般情况下,对于要插入到文档中的所有图像;或者对于在某个点之后插入的所有图像。在这种情况下,我们使用 `\setupexternalfigures` 命令进行调整。
- 对于要多次插入文档中的特定图像。在这种情况下,调整是在 `\useexternalfigure` 命令的最后一个参数中进行的,该命令将外部图形与符号名称关联起来。
- 在插入特定图像的确切时刻。在这种情况下,调整是在 `\externalfigure` 命令本身中进行的。

通过这种方式可以实现的图像更改如下:

更改图像的大小 我们可以这样做:

- 通过指定精确的宽度或高度,分别使用 `width` 和 `height` 选项;如果只调整两个值中的一个,另一个会自动调整以保持图像的纵横比。

我们可以指定特定的高度或宽度,也可以将其表示为页面高度或行宽的百分比。例如:

```
width=.4\textwidth
```

将确保图像的宽度等于行宽的 40%。

- 缩放图像:`xscale` 选项将水平缩放图像;`yscale` 将垂直缩放,而 `scale` 将同时水平和垂直缩放。这三个选项需要一个代表缩放因子的数字乘以 1000。也就是说:`scale=1000` 将保持图像原始大小,而 `scale=500` 将其缩小一半,`scale=2000` 将其大小加倍。

条件缩放,仅在图像超过特定尺寸时应用,通过 `maxwidth` 和 `maxheight` 选项实现,它们接受一个尺寸。例如,`maxwidth=0.2\textwidth` 将仅在图像宽度超过行宽的 20% 时进行缩放。

旋转图像。 要旋转图像,我们使用 `orientation` 选项,它接受一个指定将应用的旋

转度数的值(例如,`orientation=90`)。旋转是逆时针方向进行的。

wiki 暗示使用此选项可以实现的旋转必须是 90 的倍数(90、180 或 270),要实现不同的旋转,我们必须使用 `\rotate` 命令。使用 ConT_EXt LMTX 的测试表明,其他值也可以接受,但这样做可能会损坏图像。因此,请谨慎使用其他值。

为图像添加框 我们还可以使用 `frame=on` 选项用框架包围图像,并配置框架颜色

(`framecolor`)、框架与图像之间的距离(`frameoffset`)、绘制框架的线条粗

细(`rulethickness`)或其角部形状(`framecorner`),可以是圆角(`round`)或矩形。

图像的其他可配置方 除了已经看到的涉及要插入图像变换的方面外,使面。

用 `\setupexternalfigures` 我们还可以配置其他方面,例如在哪里查找图像(`directory` 选项),是仅应在指定目录中搜索图像(`location=global`)还是也应包括工作目录及其父目录(`location=local`),以及图像是否将是交互式的(`interaction`)等。

2.4.2 用于变换图像的特定命令

ConTeXt 中有三个命令可用于变换图像,并且可以与 `\externalfigure` 结合使用。它们是:

- 镜像图像:通过 `\mirror` 命令实现。
- 裁剪:通过 `\clip` 命令实现,其中给出裁剪的宽度(`width`)、高度(`height`)、水平偏移(`hoffset`)和垂直偏移(`voffset`)尺寸。例如,比较 `minidoubt.jpg` 图像与裁剪版本:

```
\externalfigure[minidoubt]
[width=2cm,frame=on,
framecolor=red]
```



```
\clip [width=2cm, height=1cm,
hoffset=3mm, voffset=2mm]
{\externalfigure[minidoubt]
[width=2cm,frame=on,
framecolor=red]}
```



- 旋转。第三个用于对图像应用变换的命令是 `\rotate` 命令。这是一头倾斜 30 度的牛:

```
\rotate[rotation=-30]
{\externalfigure[cow]
[width=2cm]}
```



这些命令的典型用途是与图像一起使用,但它们实际上作用于盒子。这就是为什么我们可以将它们应用于任何文本,只需将其包围在一个盒子中(命令会自动执行),这将产生如下奇特的效果:

第3节 表格

3.1 关于表格及其在文档中放置的一般概念

表格是结构化的对象,包含文本、公式甚至图像,排列在一系列单元格中,使我们能够直观地看到每个单元格内容之间的关系。为此,信息被组织成行和列:同一行中的所有数据(或条目)彼此之间具有某种关系,同一列中的所有数据(或条目)也是如此。单元格是行与列的交点,如 Figure 13.2 所示。

表格非常适合显示相互关联的文本或数据项,因为每个项都被封闭在自己的单元格中,即使其内容或其他单元格的内容发生变化,一个单元格相对于其他单元格的相对位置也不会改变。

在所有涉及文本排版的任务中,在我看来,表格的创建是唯一一个在图形程序(文字处理器类型)中比在 ConT_EXt 中更容易完成的任务。因为绘制表格(在文字处理程序中执行的操作)比描述表格(我们使用 ConT_EXt 时所做的事情)更容易。每次行更改和列更改都需要一个命令的存在,这意味着实现表格需要花费更长的时间,而不是简单地说出我们想要多少行和列。

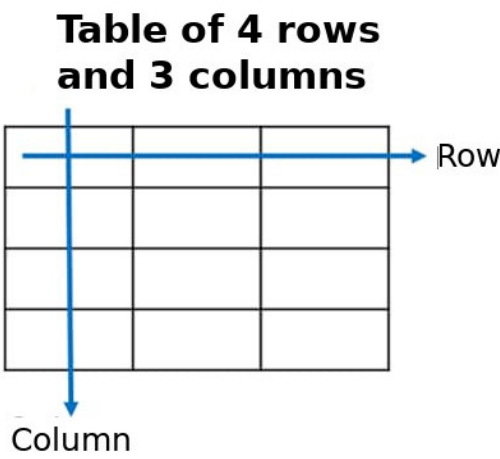


图 13.2 简单表格的图像

ConT_EXt 有三种不同的机制来生成表格;一种是 `tabulate` 环境,推荐用于简单表格,并且是直接受 T_EX 表格启发的。另一种是所谓的自然表格,受 HTML 表格启发,适用于具有特殊设计需求的表格,例如,并非所有行都具有相同数量的列。第三种选择是所谓的极端表格,明显基于 XML,推荐用于占据一页以上的中等或长表格。在这三种中,我将只解释第一种。自然表格在「ConT_EXt Mark IV an excursion」中有相当好的解释(但也可以在您的 ConT_EXt 发行版中查找名为 `examples-naturaltables.pdf` 的文件),而对于极端表格,在「ConT_EXt Standalone」文档中有一篇关于它们的文档(查找名为 `xtables-mkiv.pdf` 的文件)。

与图像类似的情况也发生在表格上:我们可以在文档中的某个点简单地编写必要的命令来生成表格,并将其精确地插入到那个点,或者我们可以使用 `\placetable` 命令来插入表格。后一种方法有一些优点:

- ConT_EXt 将为表格编号并将其添加到表格列表中,允许对表格进行内部引用(通过其编号),并将其包含在(最终的)表格索引中。
- 将获得在文档中放置表格的灵活性,从而简化分页任务。

`\placetable` 的格式类似于我们看到的 `\placefigure`(参见 [Section 2.2](#)):

```
\placetable[Options][Label] {Title} {table}
```

关于与表格放置和标题配置相关的选项,我请读者参考 [Sections 4.1](#) 和 [4.2](#)。然而,其中有一个选项似乎是专为表格设计的。这就是「`split`」选项,当设置时,它授权 ConT_EXt 将表格扩展到两页或更多页,在这种情况下,表格不能再是浮动对象。

一般来说,我们可以使用 `\setuptables` 命令设置表格的配置。此外,与图像一样,可以使用 `\placelistoftables` 或 `\completelistoftables` 生成表格索引。有关打印此类列表的更多信息,请参见 [Section 2.2](#)。

3.2 使用 `tabulate` 环境创建简单表格

最简单的表格是使用 `tabulate` 环境实现的,其格式为:

```
\starttabulate[表格列布局]
... % 表格内容
...
...
\stoptabulate
```

其中方括号中的参数(以代码形式)描述了表格将具有的列数,并且(有间接地)指示了它们的宽度。我说该参数以代码形式描述了设计,因为乍一看它似乎非常神秘:它由一系列字符组成,每个字符都有特殊的含义。我将一步一步地解释它,因为我认为这样更容易理解。

这是典型的情况,我们可以配置的方面数量巨大,这意味着需要大量的文本来描述它。这看起来极其困难。事实上,对于实践中构建的大多数表格,第 1 点和第 2 点就足够了。其余的是额外的可能性,知道它们的存在是有用的,但并非排版大多数表格所必需知道的。

3.2.1 `tabulate` 环境的表格列布局

我们首先解释如何在 `tabulate` 环境中描述列。完成此操作后,我们将描述如何将实际数据输入到表格中([Subsubsection 3.2.2](#))。

1. **列分隔符:** 使用「|」字符来分隔表格列。因此,例如,「`[lT|rB]`」将描述一个两列的表格,其中第一列将具有与指示符「l」和「T」(我们马上会看到)相关联的

特性,第二列将具有与「r」和「B」相关联的特性。例如,一个简单的三列表格,其中每列中的数据左对齐,将被描述为「[l|l|l]」。

2. **确定列中单元格的基本性质:**当我们构建表格时,首先要决定的是我们想要给定单元格的内容始终写在一行上,还是相反,给定单元格的文本可以分布在两行或更多行上(如果文本太长而无法容纳在一行上)。在 `tabulate` 环境中,这个问题不是逐个单元格决定的,而是被视为列的特性。

- a. **单行单元格:**如果一列中的单元格内容,无论其长度如何,都要写在一行上,我们必须指定该列中文本的对齐方式,可以是左对齐(「l」)、右对齐(「r」)或居中对齐(「c」)。

原则上,这些列的宽度将根据需要容纳最宽的单元格。但是我们可以使用「w(Width)」说明符来限制列的宽度。例如,「[|rw(2cm)|c|c|]」将描述一个两列的表格,第一列右对齐且精确宽度为2厘米,另外两列居中且没有宽度限制。

需要注意的是,单行列的宽度限制可能会导致一列中的文本与下一列中的文本重叠。因此,我的建议是,当需要固定宽度的列时,始终使用多行单元格列。

- b. **如果需要可以占用多行的单元格:**「p」说明符(「段落」)生成的列中,每个单元格的文本将根据需要占用尽可能多的行。如果我们只指定「p」,列的宽度将是可用的全部宽度(考虑到其他列所需的宽度)。但也可以指示「p(Width)」,在这种情况下,宽度将是指定的宽度。考虑以下示例:

```
\starttabulate[|l|r|p|]
\starttabulate[|l|p(4cm)|]
\starttabulate[|r|p(.6\textwidth)|]
\starttabulate[|p|p|p|]
```

第一个示例将创建一个三列表格,第一列和第二列为单行,分别左对齐和右对齐,第三列将占据剩余宽度以及容纳其所有内容所需的高度。在第二个示例中,第二列的宽度正好是四厘米,无论其内容如何;但如果内容不适合该空间,它将占用多行。第三个示例根据行的最大宽度按比例计算第二列的宽度,在最后一个示例中,将有三列平均分配可用宽度。

注意,实际上,如果一个单元格是一个四边形,「p」说明符所做的就是根据文本的长度授权列中单元格具有可变高度。

3. **在列的描述中添加关于要使用的字体样式和变体的指示:**一旦决定了列的基本性质(宽度和高度,自动或固定),我们仍然可以在列内容的描述中添加一个代表格式的字符,该字符表示必须书写该列的格式。这些字符可以是「B」表示粗体,「I」表示斜体,「S」表示倾斜,「R」表示罗马字体,或「T」表示打字机字体。

4. **可以在表格列描述中指定的其他附加方面:**

- **包含数学公式的列:**「m」和「M」说明符启用列中的数学模式,而无需在其每个单元格中指定。此列中的单元格将无法容纳普通文本。

尽管 ConT_EXt 的前身 T_EX 是为排版任何类型的数学而生的,但到目前为止,我几乎没有说过关于编写数学的内容。在数学模式中(我不会解释),ConT_EXt 会改变我们的正常规则,甚至使用不同的字体。数学模式有两种变体:一种我们可以称之为线性,公式包含在包含普通文本的行内(「m」指示符),另一种是完整数学模式,在一种没有普通文本的环境中显示公式。在表格中,这两种模式的主要区别基本上是公式书写的大小以及其周围的水平和垂直空间。

- 调整列单元格内容周围的水平空白量。默认情况下,tabulate 环境会在列之间添加一点额外的空间,以确保两个相邻单元格的内容不会相互接触。(请注意,它不会在最左侧列的左侧或最右侧列的右侧添加额外空间,如果您的表格设计在左侧或右侧使用垂直线,这一点最为明显。)可以使用类似于 `\setuptabulate{unit=10mm}` 的命令来设置此空间量,这将设置列间空间为 10mm,在两列之间均匀分布。我们可以使用「in」、「jn」和「kn」指示符来删除此额外空间或增加它。每个指示符分别用 n 乘以 unit 空间(默认为 1 cm)来替换通常在列的左侧、右侧或两侧(分别)提供的空间量。因此,例如,「|j2r|」指定了一列,该列将右对齐,并且我们希望在该内容之后有一个宽度为 2 cm 的空白。如果需要 unit 空间的小数倍数,请将小数括在花括号中,如本例所示: `j{1.5}`。
- 在列的每个单元格的内容之前或之后添加文本。`b{Text}` 和 `a{Text}` 说明符导致花括号之间的文本在单元格内容之前(「b」,来自 *before*)或之后(「a」,来自 *after*)写入。
- 将格式命令应用于整个列。我们之前提到的「B」、「I」、「S」、「R」、「T」指示符并未涵盖所有格式可能性:例如,没有小型大写字母、无衬线或影响字体大小的指示符。使用「f\Command」指示符,我们可以指定一个格式命令,该命令会自动应用于列中的所有单元格。例如,「|lf\sc|」将以小型大写字母排列的内容。
- 将任何命令应用于列中的所有单元格。最后,「h\Command」指示符将指定的命令应用于列中的所有单元格。

在 Table 13.1 中显示了一些表格格式规范字符串的示例。

格式说明符	含义
l	生成宽度自动、左对齐的列。
rB	生成宽度自动、右对齐且为粗体的列。
cIm	生成启用数学内容的列。居中且为斜体。
j4cb{---}	此列的内容将居中,以长破折号(—)开头,右侧有 4 units 的空白。
l p(.7\textwidth)	生成两列:第一列左对齐且宽度自动。第二列占据行总宽度的 70%。

表 13.1 指定 tabulate 中列格式的一些示例

3.2.2 `tabulate` 环境的列条目

一旦设计好表格,就需要输入其内容。为了解释如何做到这一点,我将首先描述如何填充一个有行分隔线和列分隔线的表格:

- 我们通常先绘制一条水平线。在表格中,这是通过 `\HL` 命令(来自 *Horizontal Line*)完成的。
- 然后我们写第一行:在每个单元格的开头,我们必须指示一个新的单元格开始。这可以通过 `\NC` 命令(来自 *Next Column*)完成,或者如果需要垂直线,可以使用 `\VL` 命令(来自 *Vertical Line*)。所以我们以此命令开始,然后写入每个单元格的内容。每次更改单元格时,我们使用 `\NC` 或 `\VL` 命令(根据需要)。
- 在一行的末尾,我们明确指示将使用 `\NR` 命令(来自 *Next Row*)开始新的一行。在 `\NR` 之后,我们重复 `\HL` 以绘制新的水平线(如果需要)。
- 就这样,一行一行地写出表格的所有行。完成后,我们添加一个 `\NR` 命令,如果需要,再添加一个 `\HL` 以用底部水平线关闭网格。

一旦掌握了窍门,这并不特别困难,尽管当我们查看表格的源代码时,很难想象它会是什么样子。在 Table 13.2 中,我们看到了可以在表格内使用(且必须使用)的命令。有一些我没有解释,但我认为我给出的描述已经足够了。ConT_EXt wiki 上关于 `\starttabulate` 的页面有一些此处未解释的选项。

命令	含义
<code>\HL</code>	插入一条水平线
<code>\NC</code>	开始新的一列
<code>\NR</code>	开始新的一行
<code>\VL</code>	插入分隔列的垂直线(用于代替 <code>\NC</code>)
<code>\NN</code>	在数学模式下开始一列(用于代替 <code>\NC</code>)
<code>\TB</code>	在两行之间添加一些额外的垂直空间
<code>\NB</code>	指示下一行开始一个不可分割的块,在该块内不能有分页

表 13.2 表格内使用的命令

现在,作为示例,我将转录编写 Table 13.2 的代码。

```
\placetable
[here]
[tbl:tablecommands]
{表格内使用的命令}
{\starttabulate[1|p(.6\textwidth)]}
\HL
\NC {\bf 命令}
\NC {\bf 含义}
```

```

\NR
\HL
\NC \tex{HL}
\NC 插入一条水平线
\NR
\NC \tex{NC}
\NC 开始新的一列
\NR
\NC \tex{NR}
\NC 开始新的一行
\NR
\NC \tex{VL}
\NC 插入分隔列的垂直线(用于代替 \tex{NC})
\NR
\NC \tex{NN}
\NC 在数学模式下开始一列(用于代替 \tex{NC})
\NR
\NC \tex{TB}
\NC 在两行之间添加一些额外的垂直空间
\NR
\NC \tex{NB}
\NC 指示下一行开始一个不可分割的块,在该块内不能有分页
\NR
\HL
\stoptabulate}

```

读者会注意到,我通常为每个单元格使用一行(或两行)文本。在真实的源文件中,我会为每个单元格只使用一行文本;在示例中,我拆分了太长的行。为每个单元格使用一行使我更容易编写表格,因为我所做的是写入每个单元格的内容,没有行或列分隔命令。写完所有内容后,我选择表格中的文本,并要求我的文本编辑器在每行的开头插入「`\NC`」。之后,每两行(因为表格有两列),我插入一行添加 `\NR` 命令,因为每两列开始新的一行。最后,我手动在我希望出现水平线的位置插入 `\HL` 命令。我描述它所用的时间几乎比实际做它还要长!

但也要注意,在表格中,我们可以使用 ConT_EXt 的普通命令。特别是在此表格中,我们不断使用 `\tex`,这在 [Section 2.3](#) 中有解释。

第4节 图像、表格及其他浮动对象的共同方面

我们已经知道,图像和表格不一定是浮动对象,但它们是很好的候选对象,尽管它们必须通过 `\placefigure` 或 `\placetable` 命令插入到文档中。除了这两个命令之外,在 ConT_EXt 中,我们还有 `\placechemical` 命令(用于插入化学公式)、`\placegraphic` 命令(用于插入图形)和 `\placeintermezzo` 命令,用于插入 ConT_EXt 称之为间奏曲的结构。¹ 所有这些命令又是一个更通用命令的具体应用,该命令称为 `\placefloat`,其语法如下:

¹ 在书籍中,间奏曲是分隔较长段落的短小插曲。

`\placefloat[Name] [Options] [Label] {Title} {Contents}`

请注意, `\placefloat` 与 `\placefigure` 和 `\placetable` 相同, 除了第一个参数, 在 `\placefloat` 中它接受浮动对象的名称。这是因为每种类型的浮动对象都可以通过两个不同的命令插入到文档中: `\placefloat[TypeName]` 或 `\placeTypeName`。换句话说: `\placefloat[figure]` 和 `\placefigure` 是完全相同的命令, 就像 `\placefloat[table]` 与 `\placetable` 是相同的命令一样。

因此, 从现在起, 我将讨论 `\placefloat`, 但请注意, 我所说的一切也将适用于 `\placefigure` 或 `\placetable`, 它们是该命令的具体应用。

`\placefloat` 的参数是:

- **Name**: 指的是相关的浮动对象。它可以是一些预定义的浮动对象 (`figure`, `table`, `chemical`, `intermezzo`) 或由我们自己使用 `\definefloat` 创建的浮动对象 (参见 [Section 5](#))。
- **Options**: 一系列符号词, 告诉 ConT_EXt 应该如何插入对象。其中绝大多数涉及在哪里插入它。我们将在下一节中看到这一点。
- **Label**: 用于内部引用此对象的标签。
- **Title**。要添加到对象的标题文本。关于其配置, 请参见 [Section 4.2](#)。
- **Contents**。这当然取决于对象的类型。对于图像, 通常是一个 `\externalimage` 命令; 对于表格, 是创建表格的命令; 对于间奏曲, 是一个 (可能带有框架的) 文本片段; 等等。

前三个参数在方括号中引入, 是可选的。最后两个 (在花括号之间引入) 是必需的, 尽管它们可以为空。因此, 例如: `\placefloat{}{}` 将在文档中插入:



图 13.3

(此处框架的大小显示了当选项或内容未指定大小时 ConT_EXt 的默认大小。)

注意: 我们看到 ConT_EXt 认为要插入的对象是图像, 因为它被编号为图像并包含在 图像列表中。这让我假设图像是默认的浮动对象。



4.1 浮动对象插入选项

`\placefigure`、`\placetable` 和 `\placefloat` 中的 `Options` 参数控制与插入这些类型对象相关的不同方面,主要是对象将插入到页面上的位置。这里支持几个值,每个值具有不同的性质:

- 一些插入位置是相对于页面元素建立的(`top`、`bottom`、`inleft`、`inright`、`inmargin`、`margin`、`leftmargin`、`rightmargin`、`leftedge`、`rightedge`、`innermargin`、`inneredge`、`outeredge`、`inner`、`outer`)。当然,它必须是一个能够适应其预期放置区域的对象,并且必须在页面布局中为该元素预留了空间。关于这一点,请参见章节 2 和 3。
- 其他可能的插入位置更与对象周围的文本相关,并且是关于对象应放置在哪里以便文本围绕它流动的指示。基本上,这些是 `left` 和 `right` 值。
- `here` 选项被解释为一种建议,即将对象保留在源文件中所在的位置。如果分页要求不允许,则此建议将不会被尊重。如果我们添加 `force` 选项,此指示会得到加强,其确切含义是:强制在该点插入对象。请注意,通过强制在特定点插入,该对象将不再是浮动的。
- 其他可能的选项与要插入对象的页面有关:「`page`」将其插入新页面;「`opposite`」将其插入当前页的对面页;「`leftpage`」将其插入偶数页;「`rightpage`」将其插入奇数页。

有一些选项与对象的位置无关。其中包括:

- `none`: 此选项抑制标题。
- `split`: 此选项允许对象扩展到一页以上。当然,它必须是一个本质上可分割的对象,例如表格。当使用此选项并且对象被分割时,它不能再被称为浮动对象。

4.2 配置浮动对象标题

除非我们在 `\placefloat` 中使用「`none`」选项,否则默认情况下,浮动对象会关联一个由三个元素组成的标题:

- 相关对象类型的名称。该名称与对象类型的名称完全相同;因此,例如,如果我们定义了一个名为「`sequence`」的新浮动对象,并将一个「`sequence`」作为浮动对象插入,标题将是「Sequence 1」。这仅仅是对象名称的首字母大写。

尽管刚刚说了,但如果文档的主要语言不是英语,预定义对象(例如「`figure`」或「`table`」对象)的英文名称将被翻译。因此,例如,在西班牙语文档中,「`figure`」对象被称

为「Figura」,而「table」对象被称为「Tabla」。可以使用 `\setuplabeltext` 更改预定义对象的这些西班牙语名称,如 [Section 5.3](#) 中所述。

- 其编号。默认情况下,对象按章节编号,因此第3章的第一个表格将是「Table 3.1」。
- 其内容。这是作为 `\placefloat` 的一个参数引入的。

使用 `\setupcaptions` 或 `\setupcaption[Object]`,我们可以更改变号系统和标题本身的外观。第一个命令将影响所有对象的所有标题,第二个命令将仅影响特定类型对象的标题:

- 至于编号系统,这由 `number`、`way`、`prefixsegments` 和 `numberconversion` 选项控制:
 - ◊ `number` 可以采用 `yes`、`no` 或 `none` 值,并控制是否会有编号。
 - ◊ `way` 指示编号是贯穿整个文档连续编号(`way=bytext`),还是在每章开头重新开始(`way=bychapter`)或每节开头重新开始(`way=bysection`)。在重新开始的情况下,将此选项的值与 `prefixsegments` 选项协调是适当的。
 - ◊ `prefixsegments` 指示编号是否会有前缀,以及前缀是什么。因此,`prefixsegments=chapter` 导致对象编号始终以章节编号开头,而 `prefixsegments=section` 将在对象编号前加上节编号。
 - ◊ `numberconversion` 控制编号的类型。此选项的值可以是:阿拉伯数字(「numbers」)、小写字母(「a」、「characters」)、大写字母(「A」、「Characters」)、小型大写字母(「KA」)、大写罗马数字(「I」、「R」、「Romannumerals」)、小写罗马数字(「i」、「r」、「romannumerals」)或小型大写罗马数字(「KR」)。
- 标题本身的外观由众多选项控制。我将列出它们,但关于每个选项含义的详细解释,我请读者参考 [Section 4.4](#),其中解释了节命令外观的控制,因为这些选项基本相同。相关选项是:
 - ◊ 控制标题所有元素的格式:`style`、`color`、`command`。
 - ◊ 仅控制对象类型的名称格式:`headstyle`、`headcolor`、`headcommand`、`headseparator`。
 - ◊ 仅控制编号格式:`numbercommand`。
 - ◊ 仅控制标题本身的格式:`textcommand`。
- 我们还可以控制其他方面,例如构成标题的不同元素之间的距离、标题的宽度、其相对于对象的位置等。关于可以使用此命令配置的选项,我请读者参考 [ConTeXt wiki](#) 中的信息。

4.3 两个或多个对象的组合插入

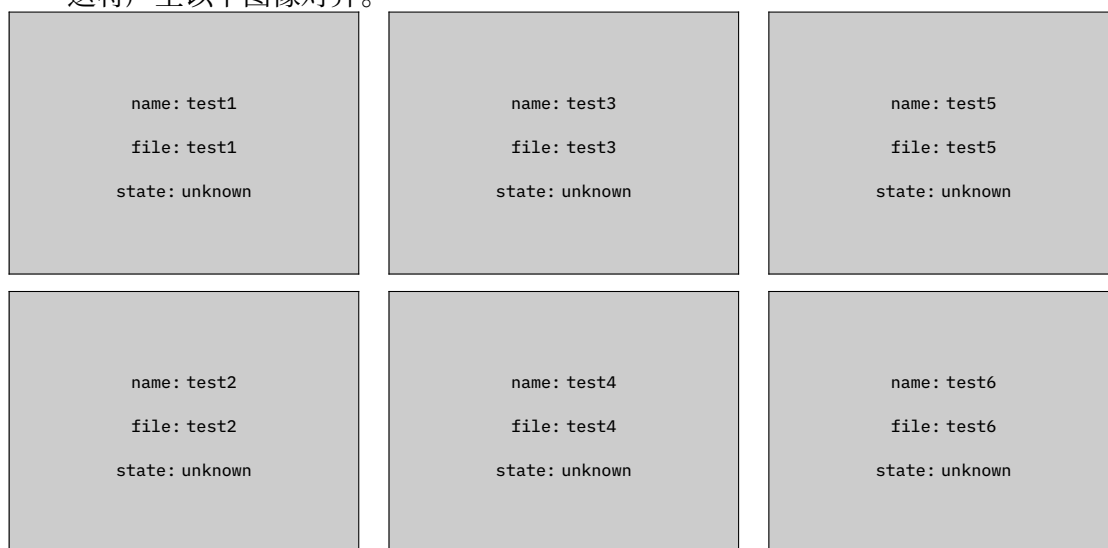
要在文档中插入两个或多个不同的对象,使得 ConT_EXt 将它们保持在一起并视为单个对象,我们使用 `\startcombination` 环境,其语法为:

```
\startcombination[Ordering] ... \stopcombination
```

其中 `Ordering` 指示对象应如何排序:如果它们都需要水平排序,`Ordering` 仅指示要组合的对象数量。但是,如果我们想将对象组合成两行或更多行,我们将不得不指示每行的对象数量,然后是行数,并用『*』字符分隔这两个数字。例如:

```
\startcombination[3*2]
{\externalfigure[test1]}
{\externalfigure[test2]}
{\externalfigure[test3]}
{\externalfigure[test4]}
{\externalfigure[test5]}
{\externalfigure[test6]}
\stopcombination
```

这将产生以下图像对齐。



在前面的示例中,我组合的图像实际上并不存在(在本文档的源代码中),这就是为什么 ConT_EXt 生成了带有它们信息的文本框来代替图像。

另一方面,请注意,`\startcombination` 中的每个要组合的元素都被括在花括号内。

实际上,`\startcombination` 不仅允许我们连接和对齐图像,还允许任何类型的盒子,例如 `\startframedtext` 环境中的文本、表格等。要配置组合,我们可以使用 `\setupcombination` 命令,并且还可以使用 `\definecombination` 创建预配置的组合。

4.4 浮动对象的一般配置

我们已经看到,使用 `\placefloat` 我们可以控制正在插入的浮动对象的位置和一些其他细节。也可以配置:

- 特定类型浮动对象的全局特性。这是通过 `\setupfloat[Name of type of floating object]` 完成的。
- 我们文档中所有浮动对象的全局特性。这是通过 `\setupfloats` 完成的。

请记住,正如 `\placefloat[figure]` 等同于 `\placefigure` 一样,`\setupfloat[figure]` 等同于 `\setupfigures`,而 `\setupfloat[table]` 等同于 `\setuptables`。

关于这些命令的可配置选项,我请读者参考 ConT_EXt 官方命令列表([Section 6](#))。

第5节 定义额外的浮动对象

`\definefloat` 命令允许我们定义自己的浮动对象。其语法为:

`\definefloat[Singular name] [Plural name] [Configuration]`

其中 `Configuration` 参数是一个可选参数,允许我们在创建时指示此新对象的配置。我们也可以稍后使用 `\setupfloat[Name in the singular]` 进行配置。

由于我们以本节结束本入门介绍,我将借此机会更深入地探讨 ConT_EXt 命令看似丛林的现象,一旦理解了,它就不再是那么一个丛林,实际上是非常合理的。

让我们首先问问自己,对于 ConT_EXt 来说,浮动对象到底是什么,答案是它是一个具有以下特征的对象:

- 相对于其在页面上的位置,它有一定的自由空间。
- 它有一个关联的列表,允许对这些对象进行编号,并最终生成它们的索引。
- 它有一个标题。
- 当对象确实可以浮动时,它必须被视为一个不可分割的单位,即(用 T_EX 术语来说)封闭在一个盒子中。

换句话说,浮动对象实际上由三个元素组成:对象本身、与其关联的列表以及标题。要控制对象本身,我们只需要一个命令来设置其位置,另一个命令将对象插入文档;要设置列表方面,通用列表控制命令就足够了;要设置标题方面,通用标题控制命令就足够了。

而这正是 ConT_EXt 的精妙之处:它本来可以设计成一个简单的命令来控制浮动对象(`\setupfloats`), 以及一个简单的命令来插入浮动对象:`\placefloat`; 但 ConT_EXt 所做的是:

1. 设计一个命令来将名称链接到特定的浮动对象配置。这就是 `\definefloat` 命令, 它实际上不链接一个名称, 而是链接两个名称, 一个单数形式和一个复数形式。
2. 与全局浮动对象配置命令一起, 创建一个允许仅配置特定类型对象的命令:`\setupfloat[Object]`。
3. 在浮动对象位置命令(`\placefloat`)中添加一个参数, 允许区分对象类型:(`\placefloat[Object]`)。
4. 为浮动对象的所有操作创建包含对象名称的命令。其中一些命令(实际上是其他更通用命令的克隆)将使用对象的单数名称, 而其他命令将使用其复数名称。

因此, 当我们创建一个新的浮动对象并告诉 ConT_EXt 其单数和复数名称时, ConT_EXt:

- 在内存中保留一个空间来存储该对象类型的特定配置。
- 使用该对象类型的单数名称创建一个新列表, 因为浮动对象与列表相关联。
- 创建一个链接到此新对象类型的新「标题」类型, 以维护这些标题的定制配置。
- 最后, 它创建一组特定于该新对象类型的新命令, 其名称实际上是更通用命令的同义词。

在 Table 13.3 中, 我们可以看到当我们定义一个新的浮动对象时自动创建的命令, 以及它们同义的更通用命令:

命令	同义词	示例
<code>\completelistof<复数名称></code>	<code>\completelist[复数名称]</code>	<code>\completelistoffigures</code>
<code>\place<单数名称></code>	<code>\placefloat[单数名称]</code>	<code>\placefigure</code>
<code>\placelistof<复数名称></code>	<code>\placelist[复数名称]</code>	<code>\placelistoffigures</code>
<code>\setup<单数名称></code>	<code>\setupfloat[单数名称]</code>	<code>\setupfigure</code>

表 13.3 创建新浮动对象时自动创建的命令

实际上, 还会创建一些与前述命令同义的附加命令, 由于我没有将它们包含在本章的解释中, 因此我从 Table 13.3 中省略了它们, 它们是: `\start<单数名称>`、`\start<单数名称>text` 和 `\startplace<单数名称>`。

我使用了用于图像的命令(「figures」)作为定义新浮动对象时创建的命令的示例; 我这样做是因为图像、表格以及 ConT_EXt 预定义的其余浮动对象都是 `\definefloat` 的实际案例:

```
\definefloat[chemical][chemicals]
```

```
\definefloat[figure][figures]
\definefloat[table][tables]
\definefloat[intermezzo][intermezzi]
\definefloat[graphic][graphics]
```

最后,我们看到,实际上 ConT_EXt 绝不控制包含在每个特定浮动对象中的任何类型的材料;它假定这是作者的工作。这就是为什么我们也可以使用 `\placefigure` 或 `\placetable` 命令插入文本。然而,使用 `\placefigure` 输入的文本被包含在图像列表中,如果使用 `\placetable` 输入,则被包含在表格列表中。

Appendices

第 A 章 安装、配置和更新 ConT_EXt

T_EX 的主要发行版 (T_EX Live、MikTeX、MacTeX 等) 都包含一个 ConT_EXt 版本。然而,这并不总是最新的版本。在本附录中,我将解释安装 ConT_EXt LMTX 的过程。此安装也包含 ConT_EXt MkIV,但不包含现已过时的 MkII。

安装过程在任何操作系统上都遵循相同的步骤;但细节因系统而异。不过,我们可以简化事情,在下面的内容中,我将区分两大系统类别:

- **Unix 类型系统:** 这包括 Unix 本身,以及 GNU Linux、Mac OS、FreeBSD、OpenBSD 或 Solaris。所有这些系统的过程基本相同;有一些非常小的差异,我会在适当的地方指出。
- **Windows 系统:** 包括该操作系统的不同版本:Windows 11(撰写本文时的最新版本)、Windows 10、Windows 8、Windows 7、Windows Vista、Windows XP、Windows NT 等。

关于 Microsoft Windows 系统安装过程的重要说明:

ConT_EXt 与所有 T_EX 系统一样,被设计为在终端中工作;安装程序和过程也同样被设计。在 Windows 中,这也完全可能,并且不会造成任何重大困难。问题在于,一方面 Windows 用户并不总是习惯这样做,另一方面,由于 Windows 诞生时就带有(错误的)幻觉,认为计算机系统中的一切都可以通过图形界面完成,通常该操作系统的版本不会过多地“宣传”如何使用终端。此外,该系统的每个版本都常常更改运行终端的程序名称以及打开方式。据我所知,Windows 终端仿真程序有过许多名称:「DOS 窗口」、「命令提示符」、「cmd」等。该程序在 Windows 应用程序菜单中的位置也因 Windows 版本而异。

我在 2004 年就不再使用基于 Windows 的系统了,所以在这里我几乎无法帮助读者。读者必须自己弄清楚如何在其特定版本的操作系统中打开终端;这应该不会太难。

第 1 节 安装和配置「ConT_EXt Standalone」

被称为「Standalone」的 ConT_EXt 发行版,也称为「ConT_EXt Suite」,是一个完整且更新的 ConT_EXt 发行版,它从互联网下载必要的文件,占用磁盘空间不大,易于更新,最重要的是——因此得名 *Standalone*——它包含在一个单独的目录中,该目录可以放在我们硬盘上的任何位置。甚至一台计算机可以有多个版本的 ConT_EXt,每个版本都在自己的目录中。该发行版包含运行 ConT_EXt LMTX(这意味着 LuaMetaTeX 引擎)以及 ConT_EXt Mark IV(这意味着 LuaTeX 引擎)所需的字体、二进制文件和文档。

有关 $\text{T}_\text{E}\text{X}$ 引擎的信息,请参见 [Section 4.1](#);关于与 $\text{ConT}_\text{E}\text{Xt}$ 相关的 $\text{T}_\text{E}\text{X}$ 引擎,以及称为 Mark II 和 Mark IV 的版本,请参见 [Section 5.1](#)。

以下各节说明如何在我们的系统上安装、运行、更新和恢复「 $\text{ConT}_\text{E}\text{Xt}$ Standalone」。这里提供的数据和过程是对 [ConT_EXt wiki](#) 中更广泛信息的总结,并添加了一些来自托管在 [wikibooks](#) 上的 $\text{ConT}_\text{E}\text{Xt}$ 维基教科书的额外细节。如果安装有任何问题,或者你想扩展任何细节,你应该直接查阅这些资源(尽管后者是法语的)。

第2节 安装 LMTX

LMTX 是最新版本的 $\text{ConT}_\text{E}\text{Xt}$ 。这个名字是所使用的 $\text{T}_\text{E}\text{X}$ 引擎名称的首字母缩写:LuaMeta $\text{T}_\text{E}\text{X}$ 。该版本于 2019 年发布,大约从 2020 年 5 月起,它已成为推荐的默认 $\text{ConT}_\text{E}\text{Xt}$ 发行版,正如 [ConT_EXt wiki](#) 所建议的那样。

2019 年 LMTX 的开发相当密集,版本可能每周变化数次。然而,到 2026 年,LMTX 早已成为一个稳定且相当成熟的产品。尽管 LMTX 仍在积极开发,但开发主要涉及添加新功能或改进现有能力,更新极不可能“破坏”现有的 LMTX 文档。

2.1 安装过程本身

安装非常简单:

- **第 1 步:** 决定要安装 LMTX 的位置,如果需要,在该位置创建一个目录。对于这些说明,我将假设安装是在我们主目录下某个名为「`context`」的目录中完成的。
- **第 2 步:** (在安装目录中)从 [ConT_EXt wiki](#) 下载与您的操作系统和处理器相对应的 zip 文件。截至 2026 年 4 月,可以是以下任何一种:
 - ◇ GNU/Linux
 - * X86/AMD64 处理器
 - * [32 位版本](#)。
 - * [64 位版本](#)。
 - * ARM 处理器
 - * [32 位版本](#)。
 - * [64 位版本](#)。
 - ◇ Mac OS,
 - * [AMD64 位版本](#)
 - * [ARM 64 位版本](#)
 - ◇ FreeBSD 14.0
 - * [AMD64 版本](#)。
 - ◇ OpenBSD 7.4
 - * [AMD64 版本](#)。
 - ◇ Microsoft Windows

- * [X86\(32 位\)版本](#)
- * [AMD64 版本](#)
- * [ARM 64 位版本](#)

如果您不知道您的系统是 32 位还是 64 位,除非您的计算机非常 老旧,否则很可能是 64 位。如果您不知道您的处理器是 X86/AMD64 还是 ARM,对于 Linux 和 Windows 用户来说,很可能是 X86/AMD64,而对于 2020 年后制造的 Apple 计算机用户来说,很可能是 ARM 64。

- **第 3 步:**将上一步下载的文件解压到安装目录中。这将创建一个名为「bin」的目录,其中包含安装所需的三个文件。它还将创建两个(Unix 类系统)或三个(Windows 系统)文件,一个名为「`installation.pdf`」,另一个是实际的安装程序,称为「`install.sh`」(Unix 类型系统)或「`install.bat`」(Windows 系统),并且在 Windows 系统上还有另一个名为「`setpath.bat`」的文件。`installation.pdf` 文件包含有关 ConT_EXt 文件组织方式、如何更新安装以及一些获取帮助的地方的有用信息。
- **第 4 步:**运行安装程序(「`install.sh`」或「`install.bat`」)。该程序需要互联网连接,因为它会通过互联网下载 ConT_EXt 发行版文件。

- ◊ 在 Unix 类型系统上,位于安装目录中的安装程序从终端运行,可以使用 `bash` 或 `sh`:

```
sh install.sh
```

不需要管理员权限,除非安装目录位于用户主目录之外。

在大多数情况下,最好不要 使用管理员权限安装 ConT_EXt;如果您想将其安装到主目录之外的某个位置(例如 `/usr/local/context`),最好先使用管理员权限创建目录,然后使用管理员权限将所选目录的所有权更改为您的用户 ID。例如,以下两个命令

```
sudo mkdir /usr/local/context
sudo chown $USER /usr/local/context
```

将创建目录 `/usr/local/context` 并将其所有权更改为您的用户 ID,这样从那时起您就可以在该目录中创建和删除文件,而无需调用管理员权限。

- ◊ 在 Windows 类型系统上,您必须打开一个终端,移动到安装目录,然后从终端运行 `install.bat`。这里也不要求以系统管理员身份运行安装程序,但建议这样做,以便可以使用文件的符号链接,从而节省磁盘空间。

- **第 5 步:**让 LMTX 的位置可知:

在 Windows 系统中,安装 ConT_EXt 会创建一个名为「`setpath.bat`」的文件,该文件会更新所有必要的配置文件,让 Windows 知道您已在系统中安装了 LMTX 以及安装位置。

在 GNU Linux 系统、FreeBSD、OpenBSD 或 Mac OS 中,不会生成自动化此任务的脚本,因此我们必须将 ConT_EXt 二进制文件的地址加入到我们的 PATH 变量中,我们可以通过在终端中运行以下命令(修改后包含正确的信息!)来实现:

```
export PATH="InstallationDir/tex/texmf-Platform/bin:$PATH"
```

其中 *InstallationDir* 是选择的安装目录, *texmf-Platform* 将根据我们安装的 LMTX 版本而变化。例如,如果我们在 AMD64 Linux 系统上安装了 ConT_EXt 在 `/home/user/context` 中,那么 *texmf-Platform* 将是 *texmf-linux-64*,因此我们应该从终端运行以下命令:

```
export PATH="/home/user/context/texmf-linux-64/bin:$PATH"
```

运行此命令会将 LMTX 包含在命令解释器的路径中,但仅限于运行它的终端保持打开状态。如果我们希望每次打开终端时都自动设置 `context` 程序的位置(在 99.9% 的情况下,我们都希望如此),我们必须将此命令包含在我们用户使用的 *shell*¹ 程序的配置文件中。此文件的名称根据我们使用的特定 shell(`bash`, `sh`, `zsh`, `ksh`, `tcsh`, `csh`, ...)而变化。在大多数 Linux 系统上,默认 shell 是 `bash`;对于 `bash` 用户,要更新的文件称为「`.bashrc`」,因此我们应该运行以下命令:

```
echo export PATH="/home/user/context/texmf-linux-64/bin:$PATH" >> ~/.bashrc
```

对于使用 `zsh` 的用户,类似的命令是

```
echo export PATH="/home/user/context/texmf-linux-64/bin:$PATH" >> ~/.zshenv
```

• 第 6 步:最终调整(主要针对 GNU Linux)

ConT_EXt 附带大量字体选择,但您可能希望能够使用「系统」字体和/或您已获得并存储在主目录下某个位置的字体。(如果您没有这种需求,可以完全跳过此步骤。)

在撰写本文时,在 Windows 系统上,ConT_EXt standalone 发行版在以下位置查找字体:

```
C:/windows/fonts//
```

在 Apple 系统上,它在以下位置查找:

```
$HOME/Library/Fonts//;/Library/Fonts//;/System/Library/Fonts//
```

在 GNU Linux 系统中,有许多可以安装字体的目录,并且(在撰写本文时)ConT_EXt 没有设置默认位置来查找系统或用户字体。因此,如果我们希望 ConT_EXt 能够使用这些字体,我们必须告诉它在哪里找到它们。为此,我们可以

¹ 在 Unix 类型系统中,命令解释器通常被称为「shell」。

修改 ConT_EXt 安装中的文件 `.../tex/texmf-linux-64/bin/mtxrun.lua`, 但我们的更改可能会在下次 ConT_EXt 更新时被覆盖。除了这个问题之外, 对于大多数非专家用户来说, 修改这个关键的 ConT_EXt 文件也不是一个好主意。

相反, 非 Windows、非 Apple 系统的用户可以设置一个环境变量 `OSFONTDIR`, 列出 ConT_EXt 应搜索字体的目录。例如,

```
export OSFONTDIR=~/.fonts//:/usr/share/fonts//:/usr/share/texmf/fonts//
```

告诉 ConT_EXt 在三个额外的目录(以及它们下面的子目录!)中查找字体: 我们主目录下的 `.fonts` 目录、系统目录 `/usr/share/fonts` 和系统目录 `/usr/share/texmf/fonts/opentype`。每个用户必须决定希望 ConT_EXt 搜索的特定目录列表。

`/usr/share/texmf/fonts` 目录只有在我们的操作系统中安装了其他 T_EX (或基于 T_EX 的其他系统)时才会存在;在这种情况下, 此目录应包含在 `OSFONTDIR` 路径中, 以便我们可以使用此类安装可能包含的 `opentype` 字体。如果您有任何希望 ConT_EXt 使用的商业字体, 您必须确保包含这些字体的目录(或多个目录)的路径是 `OSFONTDIR` 中包含的路径之一。例如, 我看到有时字体安装在 `/usr/local/fonts` 而不是(或除了)`/usr/share/fonts`。

虽然您可以每次打开终端仿真器窗口运行 ConT_EXt 时都键入 `export OSFONTDIR=...` 命令, 但一旦您决定了所需的目录列表, 将该命令放在您的命令解释器(「shell」)启动时读取的初始化文件之一(例如, `~/.zshenv`、`~/.bashrc`、`~/.cshrc`、`~/.profile`, ...)中是有意义的。

如果您从桌面环境或窗口管理器启动编辑器, 并从编辑器内部运行 ConT_EXt, 您还必须确保 `OSFONTDIR` 环境变量对您的编辑器是已知的。如何做到这一点取决于您的特定桌面环境或窗口管理器, 如果您不知道如何在编辑器的环境中获得 `OSFONTDIR`, 最好的办法可能是在处理您特定类型系统的网络论坛或邮件列表上提问。

最后, 让 ConT_EXt 使用通过 `OSFONTDIR` 添加的字体文件更新其字体数据库可能是个好主意。这可以通过在已知并导出了 `OSFONTDIR` 的终端中运行以下命令来完成:

```
context --generate
context --make
```

2.2 运行「ConT_EXt Standalone」

「ConT_EXt Standalone」被设计为能够与其他 T_EX 系统的安装共存, 这是一个优点, 因为它允许我们在同一操作系统上安装多个不同版本。(这个概念在本章 [Section 3](#) 中有更全面的描述。)

为了利用这个优点, 我们有两种选择来指定要运行的特定 `context`:

- 在 Unix 类型系统上,我们可以简单地给出要运行的特定 context 的「完整路径名」。例如,我们可能已经将 PATH 变量设置为指向(例如)/usr/local/context/tex/texmf-linux-64/bin:\$PATH(如上面的第 5 步所述)。如果我们想使用另一个 context 程序,也许是在目录 /usr/local/texlive/current/bin/x86_64-linux 中找到的,我们可以执行(有点长的)命令

```
/usr/local/texlive/current/bin/x86_64-linux/context mydoc.tex
```

来使用 T_EXlive 的 context 可执行文件编译 mydoc.tex。

- 在 Unix 类型系统上,我们可以选择在每次想要尝试不同的 context 程序时更改 PATH 环境变量。但是,创建 shell 别名或 shell 函数来调用替代的 context 程序可能更容易。(创建 shell 别名的说明见下面的 Section 3;创建 shell 函数的说明超出了本 ConT_EXt LMTX 入门指南的范围,但通过简单的互联网搜索很容易找到说明。)
- 在 Windows 系统上,我们可以(在终端中)从我们想要使用的 context 版本的特定安装目录运行 `tex\setuptex.bat` 命令。运行此命令后,运行 context 命令将引用此特定安装目录中的 context。

如果我们的系统中没有其他 T_EX 或其任何衍生品的安装,我们可以通过每次打开终端时自动化指定 context 和 m_TXrun 的位置来避免上述工作:

- 在 Unix 类系统上,通过将其包含在包含通用终端启动脚本的文件中来完成(例如,「.bashrc」、「.zshenv」,...)。

终端的配置文件取决于终端默认使用的 shell 程序。如果是 bash(GNU Linux 系统中最常用的),则在开头读取的文件是 .bashrc。sh 和 ksh shells 使用名为 .profile 的文件,zsh 使用 .zshenv,而 tcsh 或 csh 读取 .cshrc 文件。某些具体实现可能会更改这些文件的名称,因此,例如,.bashrc 有时被称为 .bash_profile。

- 在 Windows 类型系统上,我们可以在桌面上创建一个运行 cmd.exe 的快捷方式,然后编辑它,将双击时运行的命令设置为:

```
C:\WINDOWS\System32\cmd.exe /k C:\Programs\context\tex\setuptex.bat
```

另一种可能性是,如果我们不希望每次想使用 ConT_EXt 时都运行此脚本,也不希望永久设置运行它所需的环境变量,那么可以从文本编辑器本身执行此操作,而不是从终端运行 ConT_EXt。如何做到这一点取决于您使用的特定文本编辑器。ConT_EXt wiki 提供了有关如何设置各种常见编辑器的信息:emacs、LEd、Notepad++、Scite、TeXnicCenter、TeXworks、vim 以及其他一些。

2.3 在 LMTX 中安装扩展模块

ConT_EXt LMTX 包含一个用于安装或升级 ConT_EXt 扩展模块的过程。在 ConT_EXt wiki 中有安装和更新模块的说明;请参阅[此页面](#),以了解从终端运行的命

令。我亲自验证过这在 GNU Linux 系统上有效。我不确定它是否能在 Windows 系统上工作,因为我没有任何版本的该操作系统可以用来测试。

以下命令(至少在 ConT_EXt 版本 2026.03.25 中)列出所有已知模块:

```
mtxrun --script install-modules --list
```

2.4 更新 LMTX

ConT_EXt LMTX 正在持续开发中,因此「ConT_EXt Standalone」经常更新。更新 LMTX 就像再次运行安装程序(「install.sh」或「install.bat」)一样简单:它会将已安装的文件与 Web 服务器上的文件进行比较,并根据需要更新。

在极不可能的情况下,如果获取文件的网站发生变化,我们可能需要从头重新安装 ConT_EXt。或者,我们可以下载新的 zip 文件,将其解压到现有的 ConT_EXt 安装目录中,然后执行「install.sh」脚本(对于 Windows 用户,则是「install.bat」脚本)。

第3节 在同一系统上使用多个版本的 ConT_EXt (仅限 Unix 类型系统)

Unix 类系统的大多数或所有命令解释器中,称为 `alias` 的 shell 功能允许我们将不同的名称与不同版本的 ConT_EXt 关联起来。因此,我们可以使用例如 T_EX Live 中包含的 ConT_EXt 版本;或者 LMTX 的 *Standalone* 版本。

例如,如果我们将 2020 年 1 月和 8 月下载的 LMTX 版本存储在不同的目录中,我们可以在 `~/.bashrc`(或您选择的 shell 在打开终端时读取的等效文件,如 `~/.zshenv` 或 `~/.profile`)中写入以下两条指令:

```
alias context-01="/home/user/context/202001/tex/texmf-linux-64/bin/context"
alias context-08="/home/user/context/202008/tex/texmf-linux-64/bin/context"
```

请注意, `csh` 和 `tcsh` 用户必须使用以下语法,并且可能希望将这些命令输入到他们的 `~/.cshrc` 或 `~/.tcshrc` 文件中:

```
alias context-01"/home/user/context/202001/tex/texmf-linux-64/bin/context"
alias context-08"/home/user/context/202008/tex/texmf-linux-64/bin/context"
```

这些指令将名称 `context-01` 与安装在「context/202001」目录中的 LMTX 版本关联,将 `context-08` 与安装在「context/202008」目录中的版本关联。在将这些别名输入到适当的文件(并打开一个新的终端模拟器使其生效)之后,可以通过键入以下命令使用 2020 年 1 月的 ConT_EXt 版本处理 `mydoc.tex`:

```
context-01 mydoc.tex
```


第 B 章 生成数学和非数学符号的命令

在下面的表格中,您将找到生成各种符号的命令,这些命令都经过了本人的逐一验证;其中大部分(但不是全部)适用于数学环境。

我知道这些排列方式很可能有待改进。问题在于,我本人更偏向文科背景,不知道其中许多符号在数学中的用途;有时我甚至不确定它们是否真的是数学符号。这就是为什么我将一组我相当确定不用于数学的符号单独列出,而对于其余的符号,我根据某些可识别的形状(三角形、正方形、星号、菱形、箭头、点)进行分组。其余的符号,大概是用于数学的,我按生成它们的命令的字母顺序排列。

此外,这些表格并不详尽。我相信还有更多可以用 ConT_EXt 生成的符号,尽管我还没有找到任何收集了所有这些符号的文档或网页。这里收集的符号,大部分是在 T_EX 或 L^AT_EX 中有效,并且我已经验证在 ConT_EXt 中也有效的符号。其中许多符号在 L^AT_EX 或 plain T_EX 中仅在数学模式下有效(例如,放在一对『\$』之间)。在 ConT_EXt 中,如下列表格所示,只有在极少数情况下才需要这样做。

货币及法律用途符号

©	<code>\copyright</code>	®	<code>\registered</code>	¢	<code>\textcent</code>
®	<code>\textcircledP</code>	¤	<code>\textcurrency</code>	\$	<code>\textdollar</code>
Ⓓ	<code>\textdong</code>	€	<code>\texteuro</code>	f	<code>\textflorin</code>
£	<code>\textsterling</code>	¥	<code>\textyen</code>	™	<code>\trademark</code>

三角形、圆形、正方形及其他形状

△	<code>\triangle</code>	○	<code>\bigcirc</code>	□	<code>\square</code>
◁	<code>\triangleleft</code>	◦	<code>\circ</code>	■	<code>\blacksquare</code>
▷	<code>\triangleright</code>	●	<code>\bullet</code>	◻	<code>\boxdot</code>
▽	<code>\triangledown</code>	⊗	<code>\circledast</code>	⊖	<code>\boxminus</code>
▼	<code>\blacktriangledown</code>	⊙	<code>\circledcirc</code>	⊕	<code>\boxplus</code>
◀	<code>\blacktriangleleft</code>	⊖	<code>\circleddash</code>	⊗	<code>\boxtimes</code>
▶	<code>\blacktriangleright</code>	⊕	<code> \$\bigoplus\$ </code>	*	<code>\ast</code>
▲	<code>\blacktriangle</code>	⊗	<code> \$\bigotimes\$ </code>	✕	<code>\maltese</code>
△	<code>\triangleq</code>	⊕	<code>\oplus</code>	*	<code>\star</code>
◇	<code>\diamond</code>	⊖	<code>\ominus</code>	♣	<code>\clubsuit</code>
◊	<code>\lozenge</code>	⊗	<code>\otimes</code>	♥	<code>\heartsuit</code>
◆	<code>\blacklozenge</code>	⌘	<code>\oslash</code>	♠	<code>\spadesuit</code>
◇	<code>\diamondsuit</code>	○	<code>\odot</code>	∅	<code>\varnothing</code>

箭头:

$\leftarrow$	<code>\leftarrow, \gets</code>	$\rightarrow$	<code>\rightarrow, \to</code>	$\leftrightarrow$	<code>\leftrightharrow</code>
$\nleftarrow$	<code>\nleftarrow</code>	$\nrightarrow$	<code>\nrightarrow</code>	$\Leftrightarrow$	<code>\Leftrightarrow</code>
$\longleftarrow$	<code>\longleftarrow</code>	$\longrightarrow$	<code>\longrightarrow</code>	$\longleftrightarrow$	<code>\longleftrightarrow</code>
$\Lleftarrow$	<code>\Lleftarrow</code>	$\Rrightarrow$	<code>\Rrightarrow</code>	$\Longleftrightarrow$	<code>\Longleftrightarrow</code>
$\nLleftarrow$	<code>\nLleftarrow</code>	$\nRrightarrow$	<code>\nRrightarrow</code>	$\leftrightharpoons$	<code>\leftrightharpoons</code>
$\lsh$	<code>\Lsh</code>	$\rsh$	<code>\Rsh</code>	$\leftrightsquigarrow$	<code>\leftrightsquigarrow</code>
$\mapsto$	<code>\mapsto</code>	$\mapsto$	<code>\mapsto</code>	$\nLeftrightarrow$	<code>\nLeftrightarrow</code>
$\longmapsto$	<code>\longmapsto</code>	$\longmapsto$	<code>\longmapsto</code>	$\nletrightarrow$	<code>\nletrightarrow</code>
$\Mapsfrom$	<code>\Mapsfrom</code>	$\Mapsto$	<code>\Mapsto</code>	$\rightleftarrows$	<code>\rightleftarrows</code>
$\Longmapsfrom$	<code>\Longmapsfrom</code>	$\Longmapsto$	<code>\Longmapsto</code>	$\rightleftharpoons$	<code>\rightleftharpoons</code>
$\leftarrowtail$	<code>\leftarrowtail</code>	$\rightarrowtail$	<code>\rightarrowtail</code>	$\updownarrow$	<code>\updownarrow</code>
$\twoheadleftarrow$	<code>\twoheadleftarrow</code>	$\twoheadrightarrow$	<code>\twoheadrightarrow</code>	$\Updownarrow$	<code>\Updownarrow</code>
$\circlearrowleft$	<code>\circlearrowleft</code>	$\circlearrowright$	<code>\circlearrowright</code>	$\updownarrows$	<code>\updownarrows</code>
$\curvearrowleft$	<code>\curvearrowleft</code>	$\curvearrowright$	<code>\curvearrowright</code>	$\uparrow$	<code>\uparrow</code>
$\hookleftarrow$	<code>\hookleftarrow</code>	$\hookrightarrow$	<code>\hookrightarrow</code>	$\Uparrow$	<code>\Uparrow</code>
$\leftharpoondown$	<code>\leftharpoondown</code>	$\rightharpoondown$	<code>\rightharpoondown</code>	$\upuparrows$	<code>\upuparrows</code>
$\leftharpoonup$	<code>\leftharpoonup</code>	$\rightharpoonup$	<code>\rightharpoonup</code>	$\twoheaduparrow$	<code>\twoheaduparrow</code>
$\leftleftarrows$	<code>\leftleftarrows</code>	$\rightrightarrows$	<code>\rightrightarrows</code>	$\upharpoonleft$	<code>\upharpoonleft</code>
$\looparrowleft$	<code>\looparrowleft</code>	$\looparrowright$	<code>\looparrowright</code>	$\upharpoonright$	<code>\upharpoonright</code>
$\swarrow$	<code>\swarrow</code>	$\searrow$	<code>\searrow</code>	$\downarrow$	<code>\downarrow</code>
$\nwarrow$	<code>\nwarrow</code>	$\nearrow$	<code>\nearrow</code>	$\Downarrow$	<code>\Downarrow</code>
$\leftsquigarrow$	<code>\leftsquigarrow</code>	$\rightsquigarrow$	<code>\rightsquigarrow</code>	$\downdownarrows$	<code>\downdownarrows</code>
$\iff$	<code>\iff</code>	$\twoheaddownarrow$	<code>\twoheaddownarrow</code>	$\downharpoonleft$	<code>\downharpoonleft</code>
$\implies$	<code>\implies</code>			$\downharpoonright$	<code>\downharpoonright</code>

标点符号

$\because$	<code>\because</code>	$\cdot$	<code>\cdot</code>	$\cdotp$	<code>\cdotp</code>
$\cdots$	<code>\cdots</code>	$:$	<code>\colon</code>	$\ddots$	<code>\ddots</code>
$\dots$	<code>\dots</code>	$\ldotp$	<code>\ldotp</code>	$\ldots$	<code>\ldots</code>
ellipses	<code>\text{ellipses}</code>	$\therefore$	<code>\therefore</code>	$\vdots$	<code>\vdots</code>
quotedblbase	<code>\text{quotedblbase}</code>	quotedbl	<code>\text{quotedbl}</code>		

主要用于数学的符号:

$\aleph$	<code>\aleph</code>	$\amalg$	<code>\amalg</code>	$\angle$	<code>\angle</code>
$\approx$	<code>\approx</code>	$\approx$	<code>\approx</code>	$\asymp$	<code>\asymp</code>
$\backsimeq$	<code>\backsimeq</code>	$\backslash$	<code>\backslash</code>	$\barwedge$	<code>\barwedge</code>
$\between$	<code>\between</code>	$\bigcap$	<code>\bigcap</code>	$\bigcup$	<code>\bigcup</code>
$\bigsqcup$	<code>\bigsqcup</code>	$\biguplus$	<code>\biguplus</code>	$\bigvee$	<code>\bigvee</code>
$\bigwedge$	<code>\bigwedge</code>	$\bot$	<code>\bot</code>	$\bowtie$	<code>\bowtie</code>
$\bumpeq$	<code>\bumpeq</code>	$\cap$	<code>\cap</code>	$\Cap$	<code>\Cap</code>
$\circeq$	<code>\circeq</code>	$\complement$	<code>\complement</code>	$\cong$	<code>\cong</code>
$\coprod$	<code>\coprod</code>	$\cup$	<code>\cup</code>	$\Cup$	<code>\Cup</code>
$\curlyeqprec$	<code>\curlyeqprec</code>	$\curlyeqsucc$	<code>\curlyeqsucc</code>	$\curlyvee$	<code>\curlyvee</code>
$\curlywedge$	<code>\curlywedge</code>	$\dashv$	<code>\dashv</code>	$\dagger$	<code>\dagger, dag</code>
$\ddagger$	<code>\ddagger, dag</code>	$\diamondsuit$	<code>\diamondsuit</code>	$\div$	<code>\div</code>

$\div$	<code>\divideontimes</code>	$\dot{=}$	<code>\doteq</code>	$\dot{=}$	<code>\doteqdot</code>
$\dot{+}$	<code>\dotplus</code>	ℓ	<code>\ell</code>	$\emptyset$	<code>\emptyset</code>
$\equiv$	<code>\eqcirc</code>	$\gtrsim$	<code>\eqslantgtr</code>	$\lesssim$	<code>\eqslantless</code>
$\equiv$	<code>\equiv</code>	$\eth$	<code>\eth</code>	$\exists$	<code>\exists</code>
$\exists!$	<code>\exists!</code>	$\fallingdotseq$	<code>\fallingdotseq</code>	$\flat$	<code>\flat</code>
$\forall$	<code>\forall</code>	$\frown$	<code>\frown</code>	$\geq$	<code>\geq, \ge</code>
$\gtrsim$	<code>\geqslant</code>	$\gg$	<code>\gg</code>	$\ggg$	<code>\ggg</code>
$\gtrsim$	<code>\gnapprox</code>	$\gneq$	<code>\gneq</code>	$\gnsim$	<code>\gnsim</code>
$\gtrsim$	<code>\gtrapprox</code>	$\gtrdot$	<code>\gtrdot</code>	$\gtreqless$	<code>\gtreqless</code>
$\gtrsim$	<code>\gtreqqless</code>	$\gtrless$	<code>\gtrless</code>	$\gtrsim$	<code>\gtrsim</code>
$\hbar$	<code>\hbar</code>	$\heartsuit$	<code>\heartsuit</code>	$\hslash$	<code>\hslash</code>
$\int$	<code>\int</code>	$\Im$	<code>\Im</code>	$\imath$	<code>\imath</code>
$\in$	<code>\in</code>	∞	<code>\infty</code>	$\int$	<code>\int</code>
$\intercal$	<code>\intercal</code>	$\jmath$	<code>\jmath</code>	$\wedge$	<code>\wedge</code>
$\times$	<code>\leftthreetimes</code>	$\leq$	<code>\leq, \le</code>	$\leq$	<code>\leq</code>
$\leq$	<code>\leqslant</code>	$\lessapprox$	<code>\lessapprox</code>	$\lessdot$	<code>\lessdot</code>
$\leq$	<code>\lesseqgtr</code>	$\lesseqqgtr$	<code>\lesseqqgtr</code>	$\lessgtr$	<code>\lessgtr</code>
$\lesssim$	<code>\lesssim</code>	$\ll$	<code>\ll</code>	$\lll$	<code>\lll</code>
$\lesssim$	<code>\lnapprox</code>	$\lneq$	<code>\lneq</code>	$\lneq$	<code>\lneq</code>
$\lesssim$	<code>\lnsim</code>	$\lor$	<code>\lor</code>	$\ltimes$	<code>\ltimes</code>
$\measuredangle$	<code>\measuredangle</code>	$\models$	<code>\models</code>	$\mp$	<code>\mp</code>
$\multimap$	<code>\multimap</code>	$\nVDash$	<code>\nVDash</code>	∇	<code>\nabla</code>
$\natural$	<code>\natural</code>	$\ncong$	<code>\ncong</code>	$\neq$	<code>\neq</code>
$\neg$	<code>\neg\$ 或 \$\neg\$</code>	$\nexists$	<code>\nexists</code>	$\ngeq$	<code>\ngeq</code>
$\ngtr$	<code>\ngtr</code>	$\ni$	<code>\ni</code>	$\nleq$	<code>\nleq</code>
$\nless$	<code>\nless</code>	$\nmid$	<code>\nmid</code>	$\not\approx$	<code>\not\approx</code>
$\not\equiv$	<code>\not\equiv</code>	$\not\sim$	<code>\not\sim</code>	$\not\sim$	<code>\not\sim</code>
$\notin$	<code>\notin</code>	$\nparallel$	<code>\nparallel</code>	$\nprec$	<code>\nprec</code>
$\nsim$	<code>\nsim</code>	$\nsubseteq$	<code>\nsubseteq</code>	$\nsucc$	<code>\nsucc</code>
$\supseteq$	<code>\nsupseteq</code>	$\ntriangleleft$	<code>\ntriangleleft</code>	$\ntrianglelefteq$	<code>\ntrianglelefteq</code>
$\ntriangleright$	<code>\ntriangleright</code>	$\ntrianglerighteq$	<code>\ntrianglerighteq</code>	$\nvDash$	<code>\nvDash</code>
$\nVDash$	<code>\nVDash</code>	$\oint$	<code>\oint</code>	$\parallel$	<code>\parallel</code>
∂	<code>\partial</code>	$\perp$	<code>\perp</code>	$\permil$	<code>\permil</code>
$\pm$	<code>\pm</code>	$\prec$	<code>\prec</code>	$\preccurlyeq$	<code>\preccurlyeq</code>
$\preceq$	<code>\preceq</code>	$\precnsim$	<code>\precnsim</code>	$\precsim$	<code>\precsim</code>
$\prime$	<code>\prime</code>	$\prod$	<code>\prod</code>	$\propto$	<code>\propto</code>
$\Re$	<code>\Re</code>	$\rightthreetimes$	<code>\rightthreetimes</code>	$\risingdotseq$	<code>\risingdotseq</code>
$\rtimes$	<code>\rtimes</code>	$\sharp$	<code>\sharp</code>	$\sim$	<code>\sim</code>
$\simeq$	<code>\simeq</code>	$\smile$	<code>\smile</code>	$\sphericalangle$	<code>\sphericalangle</code>
$\sqcap$	<code>\sqcap</code>	$\sqcup$	<code>\sqcup</code>	$\sqsubset$	<code>\sqsubset</code>
$\sqsubseteq$	<code>\sqsubseteq</code>	$\sqsupset$	<code>\sqsupset</code>	$\sqsupseteq$	<code>\sqsupseteq</code>
$\subset$	<code>\subset</code>	$\Subset$	<code>\Subset</code>	$\subseteq$	<code>\subseteq</code>
$\subsetneq$	<code>\subsetneq</code>	$\succ$	<code>\succ</code>	$\succcurlyeq$	<code>\succcurlyeq</code>
$\succeq$	<code>\succeq</code>	$\succsim$	<code>\succsim</code>	$\succsim$	<code>\succsim</code>
$\sum$	<code>\sum</code>	$\supset$	<code>\supset</code>	$\Supset$	<code>\Supset</code>
$\supseteq$	<code>\supseteq</code>	$\supsetneq$	<code>\supsetneq</code>	$\surd$	<code>\surd</code>
pm	<code>\text{pm}</code>	$\times$	<code>\times</code>	$\top$	<code>\top</code>
$\triangle$	<code>\triangle</code>	$\uplus$	<code>\uplus</code>	$\Vdash$	<code>\Vdash</code>
$\Vdash$	<code>\Vdash</code>	$\vee$	<code>\vee o \lor</code>	$\veebar$	<code>\veebar</code>
$\Vdash$	<code>\Vdash</code>	$\Vdash$	<code>\Vdash</code>	$\wedge$	<code>\wedge o \land</code>
$\wp$	<code>\wp</code>	$\wr$	<code>\wr</code>		

其他符号

¶	<code>\P</code>	§	<code>\S</code>	0°C	<code>\celsius{0}</code>
✓	<code>\checkmark</code>	Ω	<code>\mho</code>	Ω	<code>\ohm</code>
°	<code>\textdegree</code>	№	<code>\textnumero</code>	␣	<code>\textvisiblespace</code>

第 C 章 命令索引

本索引列出了本入门中讨论过的命令。有些命令仅仅是顺带提及,在这种情况下,索引中出现的页码指示了它们被提及的位置。但其他命令则是更详细解释的主题。在后一种情况下,索引中只列出详细解释开始的位置,尽管该命令也可能在本入门中的其他地方被引用。

索引中不包括:

- 关闭先前用 `\startSomething` 打开的结构的 `\stopSomething` 命令,除非文本对 `\stop` 命令有特别说明,或者在与相应的 `\start` 命令不同的位置进行了处理。
- 旨在生成符号的命令,所有这些命令都可在[附录 B](#)中找到。
- 对于生成变音符号或字母的命令,并且有大写和小写版本(分别用于生成大写或小写字母),只包含小写版本。

a	<code>\atpage</code> 132
<code>\aa</code> 142	<code>\atrightmargin</code> 75
<code>\aacute</code> 142	
<code>\about</code> 130	b
<code>\abreve</code> 142	<code>\backslash</code> 25
<code>\acircumflex</code> 142	<code>\</code> 171
<code>\adaptlayout</code> 66	<code>\begingroup</code> 39
<code>\adaptpapersize</code> 62	<code>\beta</code> 144
<code>\adiaeresis</code> 142	<code>\bf</code> 84
<code>\ae</code> 143	<code>\bfa</code> 85
<code>\aeligature</code> 143	<code>\bfb</code> 85
<code>\agrave</code> 142	<code>\bfc</code> 85
<code>\alpha</code> 144	<code>\bfd</code> 85
<code>\amacron</code> 142	<code>\bfx</code> 85
<code>\aring</code> 142	<code>\bfxx</code> 85
<code>\at</code> 130	<code>\bgroup</code> 38, 39
<code>\atilde</code> 142	<code>\bi</code> 84
<code>\atleftmargin</code> 75	<code>\bia</code> 85

`\bib` [85](#)
`\bic` [85](#)
`\bid` [85](#)
`\bigbodyfont` [87](#)
`\bix` [85](#)
`\bixx` [85](#)
`\blank` [47, 166](#)
`\bold` [84](#)
`\bolditalic` [84](#)
`\boldslanted` [84](#)
`\break` [171](#)
`\bs` [84](#)
`\bsa` [85](#)
`\bsb` [85](#)
`\bsc` [85](#)
`\bsd` [85](#)
`\bsx` [85](#)
`\bsxx` [85](#)
`\buildtextaccent` [145](#)
`\buildtextbootomcomma` [145](#)
`\buildtextbottomdot` [145](#)
`\buildtextcedilla` [145](#)
`\buildtextgrave` [145](#)
`\buildtextmacron` [145](#)
`\buildtexttognek` [145](#)

c

`\Cap` [149](#)
`\c` [142](#)
`\ca` [156](#)
`\calligraphic` [84](#)
`\cap` [149](#)
`\ccedilla` [142](#)
`\cf` [84](#)
`\chapter` [96](#)
`\chi` [144](#)
`\clip` [212](#)
`\clubpenalty` [178](#)
`\color` [91](#)
`\colored` [91](#)
`\completecontent` [110](#)
`\completelist` [119](#)

`\completelistofchemicals` [120](#)
`\completelistoffigures` [120, 209](#)
`\completelistofgraphics` [120](#)
`\completelistofintermezzi` [120](#)
`\completelistoftables` [120, 214](#)
`\completindex` [123](#)
`\crlf` [171](#)
`\currentdate` [160](#)

d

`\date` [160](#)
`\de` [158](#)
`\define` [35](#)
`\definealternativestyle` [88](#)
`\defineblank` [167](#)
`\definebodyfontenvironment` [86](#)
`\definebodyfontswitch` [88](#)
`\definecapitals` [150](#)
`\definecharacter` [144](#)
`\definecharacterkerning` [154](#)
`\definecolor` [93](#)
`\definecombination` [222](#)
`\definecombinedlist` [120](#)
`\defineconversionset` [70](#)
`\definedelimitedtext` [162](#)
`\definedescription` [196](#)
`\defineenumeration` [198](#)
`\definefloat` [223](#)
`\definefontfeature` [143](#)
`\definefontstyle` [88](#)
`\defineframed` [202](#)
`\defineframedtext` [202](#)
`\definehead` [106](#)
`\defineinterlinespace` [173](#)
`\defineitemgroup` [196](#)
`\defineitems` [196](#)
`\definelayou` [67](#)
`\definelinenumbering` [176](#)
`\definelines` [174](#)
`\definelist` [118](#)
`\definemargindata` [77](#)
`\definennarrower` [165](#)

`\definernote` [182](#)
`\definepapersize` [61](#)
`\defineparagraphs` [188](#)
`\defineregister` [124](#)
`\definesectionblock` [108](#)
`\definestartstop` [37](#)
`\definestretched` [154](#)
`\definesymbol` [191](#)
`\definetext` [74](#)
`\definetype` [151](#)
`\defintyping` [151](#)
`\delta` [144](#)
`\dontleavehmode` [84](#)

e

`\eacute` [142](#)
`\ebreve` [142](#)
`\ecircumflex` [142](#)
`\ediaeresis` [142](#)
`\egrave` [142](#)
`\egroup` [38, 39](#)
`\em` [89](#)
`\emacron` [142](#)
`\emdash` [49](#)
`\en` [156](#)
`\enableregime` [45](#)
`\endash` [49](#)
`\endgraf` [163](#)
`\endgroup` [39](#)
`\endnote` [180](#)
`\enskip` [154](#)
`\environment` [54](#)
`\epsilon` [144](#)
`\es` [156](#)
`\eta` [144](#)
`\etilde` [142](#)
`\externalfigure` [206](#)

f

`\fillinline` [200](#)
`\footnote` [180](#)
`\fr` [156](#)

`\framed` [202](#)
`\from` [137](#)

g

`\gamma` [144](#)
`\getbuffer` [203](#)
`\getmarking` [74](#)
`\godown` [167](#)
`\goto` [137](#)

h

`\H` [142](#)
`\HL` [217](#)
`\hairline` [199](#)
`\handwritten` [84](#)
`\hbox` [170](#)
`\head` [192](#)
`\hfill` [154](#)
`\high` [150](#)
`\hl` [200](#)
`\hskip` [154](#)
`\hw` [84](#)
`\hyphen` [49](#)
`\hyphenatedurl` [136](#)
`\hyphenatedurlseparator` [137](#)
`\hyphenation` [170](#)

i

`\i` [142](#)
`\iacute` [142](#)
`\ibreve` [142](#)
`\icircumflex` [142](#)
`\idiaeresis` [142](#)
`\igrave` [142](#)
`\imacron` [142](#)
`\in` [130](#)
`\index` [122](#)
`\inframed` [202](#)
`\ininner` [75](#)
`\ininneredge` [75](#)
`\ininnermargin` [75](#)
`\inleft` [75](#)

`\inleftedge` [75](#)
`\inleftmargin` [75](#)
`\inmargin` [75](#)
`\inothet` [75](#)
`\inouter` [75](#)
`\inouteredge` [75](#)
`\inoutermargin` [75](#)
`\input` [51](#)
`\inright` [75](#)
`\inrightedge` [75](#)
`\inrightmargin` [75](#)
`\iota` [144](#)
`\it` [84](#)
`\ita` [85](#)
`\italic` [84](#)
`\italicbold` [84](#)
`\itb` [85](#)
`\itc` [85](#)
`\itd` [85](#)
`\item` [192](#)
`\items` [195](#)
`\itilde` [142](#)
`\its` [193](#)
`\itx` [85](#)
`\itxx` [85](#)

j
`\j` [142](#)

k
`\kappa` [144](#)
`\kcedilla` [142](#)

l
`\l` [142](#)
`\labeltext` [159](#)
`\lambda` [144](#)
`\language` [156](#)
`\lastpagenumber` [70](#)
`\lastrealpagenumber` [70](#)
`\lastuserpagenumber` [70](#)
`\lcedilla` [142](#)

`\leftaligned` [177](#)
`\letterbackslash` [136](#)
`\letterescape` [136](#)
`\letterhash` [136](#)
`\letterhat` [25](#)
`\letterpercent` [136](#)
`\lettertilde` [25](#)
`\linenote` [180](#)
`\loadinstalledlanguages` [156](#)
`\lohi` [150](#)
`\low` [150](#)

m
`\mainlanguage` [156](#)
`\mar` [192](#)
`\margintext` [75](#)
`\mediaeval` [84](#)
`\midaligned` [177](#)
`\minus` [49](#)
`\mirror` [212](#)
`\mono` [84](#)
`\month` [160](#)
`\mu` [144](#)

n
`\NB` [217](#)
`\NN` [217](#)
`\NR` [217](#)
`\ncedilla` [142](#)
`\noheaderandfooterlines` [74](#)
`\noindentation` [164](#)
`\nolist` [98, 100](#)
`\nomarking` [98, 100](#)
`\normal` [84](#)
`\note` [181](#)
`\notesenabledfalse` [185](#)
`\notesenabledtrue` [185](#)
`\nowhitespace` [166](#)
`\nu` [144](#)

o
`\o` [142](#)

`\oacute` [142](#)
`\obreve` [142](#)
`\ocircumflex` [142](#)
`\odiaeresis` [142](#)
`\oe` [143](#)
`\oeligature` [143](#)
`\ograve` [142](#)
`\omacron` [142](#)
`\omega` [144](#)
`\omicron` [144](#)
`\os` [84](#)
`\otilde` [142](#)
`\overbar` [201](#)
`\overbars` [201](#)
`\overstrike` [201](#)
`\overstrikes` [201](#)

p
`\page` [71](#)
`\pagenumber` [70, 73](#)
`\pagereference` [128](#)
`\par` [163](#)
`\parindent` [41](#)
`\parskip` [41](#)
`\part` [96](#)
`\phi` [144](#)
`\pi` [144](#)
`\placebookmarks` [138](#)
`\placechemical` [218](#)
`\placecontent` [110](#)
`\placefigure` [208](#)
`\placefloat` [218](#)
`\placegraphic` [218](#)
`\placeindex` [123](#)
`\placeintermezzo` [218](#)
`\placelist` [119](#)
`\placelistofchemicals` [119](#)
`\placelistoffigures` [119, 209](#)
`\placelistofgraphics` [119](#)
`\placelistofintermezzi` [119](#)
`\placelistoftables` [119, 214](#)
`\placelocalfootnotes` [181](#)

`\placenotes` [181, 182](#)
`\placetable` [214](#)
`\pretolerance` [170](#)
`\product` [55](#)
`\project` [56](#)
`\psi` [144](#)

q
`\qqquad` [154](#)
`\quad` [154](#)
`\quotation` [161](#)
`\quote` [161](#)

r
`\ReadFile` [53](#)
`\r` [142](#)
`\ran` [193](#)
`\rcedilla` [142](#)
`\readfile` [53](#)
`\realpagenumber` [70](#)
`\ref` [131](#)
`\reference` [128](#)
`\regular` [84](#)
 reserved characters
`\{` [25](#)
`\}` [25](#)
`\$` [25](#)
`\backslash` [25](#)
`\letterhat` [25](#)
`\lettertilde` [25](#)
`\#` [25](#)
`\%` [25](#)
`\&` [25](#)
`\_` [25](#)
`\|` [25](#)
`\rho` [144](#)
`\rightaligned` [177](#)
`\rm` [84](#)
`\rma` [85](#)
`\rmb` [85](#)
`\rmc` [85](#)
`\rmd` [85](#)

`\rmx` [85](#)
`\rmxx` [85](#)
`\roman` [84](#)
`\rotate` [212](#)

s

`\sans` [84](#)
`\sansserif` [84](#)
`\sc` [84](#)
`\scedilla` [142](#)
`\section` [96](#)
`\seeindex` [123](#)
`\serif` [84](#)
`\sethyphenatedurlafter` [136](#)
`\sethyphenatedurlbefore` [136](#)
`\sethyphenatedurlnormal` [136](#)
`\setupalign` [176](#)
`\setuparranging` [66](#)
`\setupbackgrounds` [90](#)
`\setupblank` [167](#)
`\setupbodyfont` [81, 83](#)
`\setupbottomtexts` [75](#)
`\setupcapitals` [150](#)
`\setupcaption` [221](#)
`\setupcaptions` [221](#)
`\setupcharacterkerning` [154](#)
`\setupcolors` [90](#)
`\setupcolumns` [186](#)
`\setupcombinedlist` [111](#)
`\setupcounter` [183](#)
`\setupdescription` [197](#)
`\setupendnotes` [183](#)
`\setupenumeration` [199](#)
`\setupexternalfigures` [210](#)
`\setupfillinlines` [200](#)
`\setupfloat` [223](#)
`\setupfloats` [223](#)
`\setupfooter` [74](#)
`\setupfootertexts` [72, 75](#)
`\setupfootnotes` [183](#)
`\setupframed` [202](#)
`\setupframedtext` [202](#)

`\setuphead` [99](#)
`\setupheader` [74](#)
`\setupheadertexts` [72, 75](#)
`\setupheadnumber` [102](#)
`\setupheads` [99](#)
`\setupheadtext` [111](#)
`\setuphyphenmark` [155](#)
`\setupindenting` [163](#)
`\setupinteraction` [133](#)
`\setupinterlinespace` [173](#)
`\setuplabeltext` [159](#)
`\setuplanguage` [158](#)
`\setuplayout` [65](#)
`\setuplinenote` [180](#)
`\setuplinenumbering` [175](#)
`\setuplines` [174](#)
`\setuplist` [113](#)
`\setupmargindata` [76](#)
`\setupnarrower` [164](#)
`\setupnotation` [182](#)
`\setupnotations` [183](#)
`\setupnote` [182](#)
`\setupnotes` [183](#)
`\setuppagenumbering` [69](#)
`\setuppapersize` [59](#)
`\setupparagraphs` [188](#)
`\setupregister` [124](#)
`\setupsectionblock` [108](#)
`\setupspacing` [152](#)
`\setupstretched` [153](#)
`\setuptables` [214](#)
`\setuptextrule` [201](#)
`\setuptolerance` [171, 177](#)
`\setuptoptexts` [75](#)
`\setuptype` [151](#)
`\setuptyping` [151](#)
`\setupurl` [136](#)
`\setupuserpagenumber` [69](#)
`\setupwhitespace` [165](#)
`\showbodyfontenvironment` [87](#)
`\showcolor` [92](#)
`\showcolorcomponents` [93](#)

<code>\showfont</code>	83	<code>\startcolumns</code>	186
<code>\showframe</code>	65	<code>\startcombination</code>	203, 222
<code>\showinstalledlanguages</code>	156	<code>\startcomponent</code>	55
<code>\showlayout</code>	65	<code>\startenvironment</code>	54
<code>\showsetups</code>	65	<code>\startfiguretext</code>	210
<code>\showsymbolset</code>	147	<code>\startformula</code>	203
<code>\sigma</code>	144	<code>\startframedtext</code>	202
<code>\sl</code>	84	<code>\startfrontmatter</code>	107
<code>\sla</code>	85	<code>\starthiding</code>	203
<code>\slanted</code>	84	<code>\startitem</code>	192
<code>\slantedbold</code>	84	<code>\startitemize</code>	191
<code>\slb</code>	85	<code>\startlegend</code>	203
<code>\slc</code>	85	<code>\startlinecorrection</code>	203
<code>\sld</code>	85	<code>\startlinenumbers</code>	175
<code>\slx</code>	85	<code>\startlines</code>	174
<code>\slxx</code>	85	<code>\startlocalfootnotes</code>	181
<code>\smalcaps</code>	84	<code>\startMPpage</code>	62
<code>\smallbodyfont</code>	87	<code>\startmode</code>	203
<code>\smallbold</code>	87	<code>\startnarrower</code>	164
<code>\smallbolditalic</code>	87	<code>\startnotmode</code>	203
<code>\smallboldslanted</code>	87	<code>\startopposite</code>	203
<code>\smallitalicbold</code>	87	<code>\startpacked</code>	166
<code>\smallslanted</code>	87	<code>\startpagefigure</code>	62
<code>\smallslantedbold</code>	87	<code>\startpart</code>	96
<code>\somewhere</code>	131	<code>\startproduct</code>	55
<code>\space</code>	154	<code>\startproject</code>	56
<code>\ss</code>	84, 142	<code>\startquotation</code>	203
<code>\ssa</code>	85	<code>\startsection</code>	96
<code>\ssb</code>	85	<code>\startsetups</code>	37
<code>\ssc</code>	85	<code>\startstandardmakeup</code>	203
<code>\ssd</code>	85	<code>\startsubject</code>	96
<code>\ssx</code>	85	<code>\startsubsection</code>	96
<code>\ssxx</code>	85	<code>\startsubsubsection</code>	96
<code>\start</code>	39	<code>\startsubsubsubject</code>	96
<code>\startalignment</code>	177	<code>\startsubsubsubsection</code>	96
<code>\startappendices</code>	107	<code>\startsubsubsubsubject</code>	96
<code>\startbackmatter</code>	107	<code>\startTEXpage</code>	62
<code>\startbodymatter</code>	107	<code>\starttabulate</code>	214
<code>\startbuffer</code>	203	<code>\starttext</code>	50
<code>\startchapter</code>	96	<code>\starttextrule</code>	201
<code>\startchemical</code>	203	<code>\starttitle</code>	96
<code>\startcolor</code>	92	<code>\starttyping</code>	151

`\stop` [39](#)
`\stoptext` [50](#)
`\stretched` [153](#)
`\structureuservariable` [98](#)
`\sub` [192](#)
`\subject` [96](#)
`\subsection` [96](#)
`\subsubject` [96](#)
`\subsubsection` [96](#)
`\subsubsubject` [96](#)
`\subsubsubsection` [96](#)
`\subsubsubsubject` [96](#)
`\support` [84](#)
`\switchtobodyfont` [83](#)
`\sym` [192](#)
`\symbol` [148](#)

t

`\TB` [217](#)
`\TeX` [5, 6](#)
`\tau` [144](#)
`\tcedilla` [142](#)
`\teletype` [84](#)
`\tex` [152](#)
`\textheight` [66](#)
`\textreference` [128](#)
`\textrule` [201](#)
`\_` [154](#)
`\textwidth` [66](#)
`\tf` [84](#)
`\tfa` [85](#)
`\tfb` [85](#)
`\tfc` [85](#)
`\tfd` [85](#)
`\tfx` [85](#)
`\tfxx` [85](#)
`\theta` [144](#)
`\thinrule` [199](#)
`\thinrules` [199](#)
`\title` [96](#)
`\tolerance` [170](#)
`\translate` [161](#)

`\tt` [84](#)
`\tta` [85](#)
`\ttb` [85](#)
`\ttc` [85](#)
`\ttdd` [85](#)
`\tttx` [85](#)
`\ttxx` [85](#)
`\tx` [85](#)
`\txx` [85](#)
`\typ` [152](#)
`\type` [151](#)

u

`\u` [142](#)
`\uacute` [142](#)
`\ubreve` [142](#)
`\ucircumflex` [142](#)
`\udiaeresis` [142](#)
`\ugrave` [142](#)
`\umacron` [142](#)
`\underbar` [201](#)
`\underbars` [201](#)
`\unhyphenated` [170](#)
`\upsilon` [144](#)
`\usecolors` [91](#)
`\useexternalfigure` [206](#)
`\usemodule` [156](#)
`\usereregime` [45](#)
`\userpagenumber` [70](#)
`\usesymbols` [147](#)
`\useURL` [135](#)
`\utilde` [142](#)

v

`\VL` [217](#)
`\varepsilon` [144](#)
`\varkappa` [144](#)
`\varphi` [144](#)
`\varpi` [144](#)
`\varrho` [144](#)
`\varsigma` [144](#)
`\vartheta` [144](#)

`\vbox` [72](#)
`\vfill` [168](#)
`\vl` [200](#)
`\vskip` [167](#)

w

`\WORD` [149](#)
`\WORDS` [149](#)
`\Word` [149](#)
`\Words` [149](#)
`\whitespace` [166](#)

`\widowpenalty` [178](#)
`\word` [149](#)
`\wordright` [177](#)
`\writebetweenlist` [117](#)
`\writetolist` [116](#)

x

`\xi` [144](#)

z

`\zeta` [144](#)